VOLUME 4

Spring & Spring Boot Internals

IoC, AOP, Boot auto-configuration, MVC, WebFlux, Data JPA, transactions, security

Java 21 · Spring Boot 3 · SQL · Microservices · AI integration
Interview preparation for 12+ years experience · real scenarios from the RefinePoint platform

Contents

15. Spring Core: IoC Container & Dependency Injection

15.1 IoC vs DI
15.2 BeanFactory vs ApplicationContext
15.3 Container startup: what `refresh()` really does
15.4 The bean lifecycle
15.5 Constructor, setter and field injection
15.6 How `@Autowired` resolves a dependency
15.7 Bean scopes
15.8 Circular dependencies and the three-level cache
15.9 `@Component`, `@Bean`, `@Configuration`
15.10 `@Conditional`, profiles, Environment
15.11 Application events
15.12 `FactoryBean`, `BeanPostProcessor`, lazy init, thread safety
15.13 Interview questions and answers

16. Spring AOP, Proxies & How @Transactional Really Works

16.1 AOP terminology
16.2 Spring AOP vs AspectJ
16.3 JDK dynamic proxy vs CGLIB
16.4 How the proxy gets created
16.5 What happens inside the proxy on each call
16.6 Pointcut expressions
16.7 Writing custom aspects
16.8 Spring features implemented as proxies
16.9 How `@Transactional` really works
16.10 Every way to bypass the proxy
16.11 Proxies, `equals`, `getClass` and debugging
16.12 Performance and design guidance
16.13 Propagation as the interceptor sees it
16.14 Declarative vs programmatic transactions
16.15 Reactive and virtual-thread considerations
16.16 Interview questions and answers

17. Spring Boot Internals

17.1 What Boot adds over plain Spring
17.2 `@SpringBootApplication` decomposed
17.3 `SpringApplication.run()` step by step
17.4 The auto-configuration mechanism
17.5 Writing a custom starter — RefinePoint's `ryfn-platform`
17.6 Starters and dependency management
17.7 The embedded server
17.8 Externalized configuration
17.9 Actuator
17.10 Logging
17.11 Graceful shutdown
17.12 Fat jar layout and layered jars
17.13 Boot 2 → 3 migration
17.14 AOT, native images and CRaC
17.15 DevTools and startup optimisation
17.16 Interview questions and answers

18. Spring MVC & REST APIs

18.1 The request flow
18.2 Filters vs interceptors vs AOP
18.3 Controllers, mapping and arguments
18.4 HttpMessageConverters, Jackson and content negotiation
18.5 Validation
18.6 Global exception handling
18.7 REST design
18.8 File upload, download and streaming
18.9 CORS
18.10 Outbound HTTP clients
18.11 OpenAPI, security basics and multi-tenant paths
18.12 Rate limiting
18.13 Testing with MockMvc

18.14 Interview questions and answers

19. Reactive Programming & Spring WebFlux

19.1 Why reactive
19.2 Reactive Streams: the specification
19.3 Project Reactor: `Mono` and `Flux`
19.4 Cold vs hot
19.5 Schedulers: `publishOn` vs `subscribeOn`
19.6 The WebFlux stack and the Netty event loop
19.7 WebClient
19.8 R2DBC
19.9 Context propagation
19.10 Testing and debugging
19.11 SSE and streaming, e.g. LLM tokens
19.12 Spring Cloud Gateway is WebFlux
19.13 WebFlux vs MVC + virtual threads
19.14 Interview questions and answers

20. Spring Data JPA & Hibernate Internals

20.1 JPA vs Hibernate vs Spring Data JPA
20.2 The persistence context
20.3 Entity lifecycle
20.4 Spring Data repositories
20.5 Pagination
20.6 Relationships and mapping
20.7 Fetching and the N+1 problem
20.8 Second-level cache and query cache
20.9 ID generation
20.10 Batch inserts and updates
20.11 Locking
20.12 Inheritance mapping
20.13 Auditing, soft delete and JSON columns
20.14 Multi-tenancy approaches
20.15 When to use JdbcTemplate instead
20.16 Flyway migrations
20.17 Interview questions and answers

21. Transactions in Depth

21.1 ACID — what each letter really costs
21.2 How a Spring local transaction actually works
21.3 Propagation — all seven levels
21.4 Isolation levels and the anomalies they prevent
21.5 readOnly, timeout and rollback rules
21.6 Exceptions and the rollback-only trap
21.7 Programmatic transactions
21.8 Transaction-bound events
21.9 Transactions, threads, virtual threads and pool exhaustion
21.10 Locking: optimistic vs pessimistic
21.11 Distributed transactions: the dual-write problem, 2PC, sagas and the outbox
21.12 Interview questions and answers

22. Spring Security, OAuth2, JWT & Keycloak

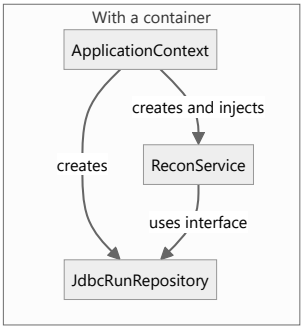
22.1 Architecture: from servlet container to your controller
22.2 Authentication flow
22.3 Authorization
22.4 CSRF, CORS, sessions
22.5 OAuth2 and OpenID Connect
22.6 JWT
22.7 Keycloak
22.8 Multi-tenant authorization
22.9 API keys, service-to-service auth, webhooks
22.10 OWASP Top 10 for Java developers
22.11 Testing security
22.12 Interview questions and answers

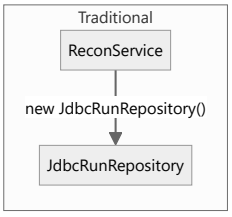
15. Spring Core: IoC Container & Dependency Injection

Every Spring interview eventually becomes a container interview. "What happens when the application starts?", "Why did this bean get a proxy?", "Why does this circular dependency fail?". A candidate who can walk through `refresh()`, explain where a `BeanPostProcessor` fits, and reason about the three-level cache can answer almost any "why did Spring do that?" question from first principles. This chapter builds that model; Chapter 16 continues with proxies and Chapter 17 with Boot's auto-configuration.

15.1 IoC vs DI

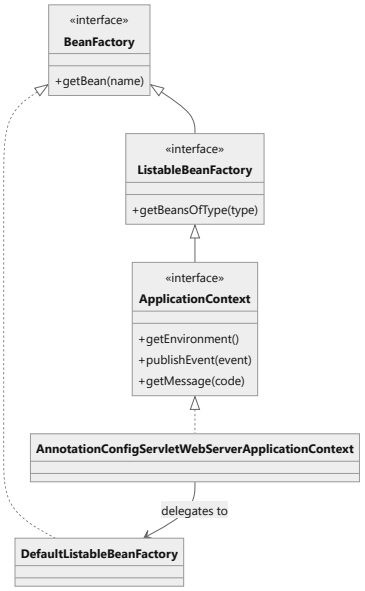
Inversion of Control is the general principle: a framework calls your code and controls the flow, instead of your code calling a library. **Dependency Injection** is one form of IoC: an object receives its dependencies instead of creating or finding them. **DIP** (Chapter 1) is the design principle that makes the injected type an abstraction. They are three different ideas.





Without DI you cannot swap the repository in a test without changing the class. With DI, `new ReconService(fakeRepo)` is a plain constructor call, and the container also gets a single place to apply cross-cutting behaviour: it can hand out a proxy instead of the bean, and the service never knows.

15.2 BeanFactory vs ApplicationContext

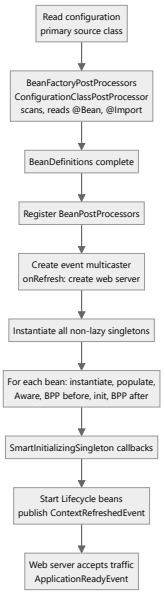


Feature	`BeanFactory`	`ApplicationContext`
Bean creation and DI	Yes	Yes, by delegating to a `DefaultListableBeanFactory`
Singleton instantiation	Lazy, on first `getBean`	**Eager** at startup, so errors surface early
`BeanPostProcessor` registration	Manual	Automatic
`BeanFactoryPostProcessor` run	Manual	Automatic
Events	No	`publishEvent`
`Environment`, profiles, properties	No	Yes
i18n (`MessageSource`) and resources	No	Yes

In a servlet Boot application the concrete class is `AnnotationConfigServletWebServerApplicationContext`; in a WebFlux one, such as our api-gateway, it is the reactive equivalent. You only touch `BeanFactory` directly in framework-level code.

15.3 Container startup: what `refresh()` really does

`SpringApplication.run` prepares the `Environment`, creates the context, then calls `AbstractApplicationContext.refresh()`. This is the most valuable sequence in Spring to memorise.

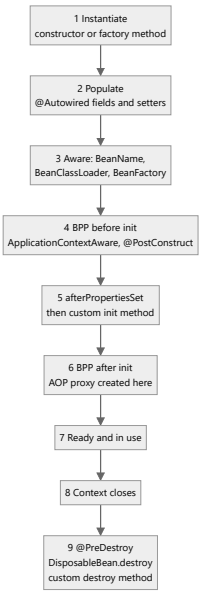



```
refresh()
 1. prepareRefresh                  init property sources, validate required properties
 2. obtainFreshBeanFactory          get the DefaultListableBeanFactory
 3. prepareBeanFactory              class loader, resolvable deps (BeanFactory, ApplicationContext...)
 4. postProcessBeanFactory          subclass hook (web scopes)
 5. invokeBeanFactoryPostProcessors ConfigurationClassPostProcessor runs here: all definitions appear
 6. registerBeanPostProcessors      instantiate BPPs early, sorted by Ordered
 7. initMessageSource
 8. initApplicationEventMulticaster
 9. onRefresh                       Boot creates embedded Tomcat here
10. registerListeners
11. finishBeanFactoryInitialization preInstantiateSingletons: your beans are built here
12. finishRefresh                   lifecycle start, ContextRefreshedEvent, web server start
```

The key separation: first Spring produces `BeanDefinition`s, which are metadata (class, scope, constructor args, init method, lazy flag). No application object exists yet. Then it creates objects from them. That is why a `BeanFactoryPostProcessor` can change a definition before any bean is built, and why auto-configuration can decide "no bean of this type is registered yet".

> **RefinePoint:** `PlatformDefaults` in `ryfn-platform` is an `EnvironmentPostProcessor`, which runs even earlier, before the context exists. It adds shared Kafka settings as the **lowest-priority** property source (producer `acks=all`, idempotence, serializers, SASL from `KAFKA_*` variables), so every service gets them and any service's `application.yml` or the environment still wins. Understanding that ordering is what makes this trick possible.

15.4 The bean lifecycle



Details that interviewers probe:

- `@PostConstruct` **runs inside "BPP before init"**, handled by `CommonAnnotationBeanPostProcessor`, so it precedes `afterPropertiesSet`, which precedes the `@Bean(initMethod=...)` method.
- **Field injection is itself done by a BPP**: `AutowiredAnnotationBeanPostProcessor.postProcessProperties`, during populate.
- **Only some `Aware` interfaces are direct.** `BeanNameAware`, `BeanClassLoaderAware` and `BeanFactoryAware` are invoked by the factory. `EnvironmentAware`, `ApplicationEventPublisherAware` and `ApplicationContextAware` come from `ApplicationContextAwareProcessor`, a BPP.
- **The proxy appears at step 6.** During `@PostConstruct`, `this` is the raw target, so a `@Transactional` method called from `@PostConstruct` runs **without a transaction**.
- **Destroy order is the reverse of dependency order:** dependents go first, so a bean can still use its dependencies in `@PreDestroy`.
- **Spring does not call destroy methods for prototypes.** It hands them over and forgets them.
- `@Bean` **infers a destroy method:** a public `close()` or `shutdown()` is called automatically. Disable it with `@Bean(destroyMethod = "")`.
- `SmartInitializingSingleton.afterSingletonsInstantiated` runs after all singletons exist, which is a safe place for "all beans are ready" work.
- `kill -9` **runs nothing;** SIGTERM triggers the shutdown hook, which closes the context.

```
@Component
public class DemoBean implements BeanNameAware, InitializingBean, DisposableBean {
    public DemoBean(Dep dep)             { log("1 constructor"); }
    @Autowired void setOther(Other o)    { log("2 populate"); }
    public void setBeanName(String n)    { log("3 aware"); }
    @PostConstruct void pc()             { log("4 @PostConstruct"); }
    public void afterPropertiesSet()     { log("5 afterPropertiesSet"); }
    @PreDestroy void pd()                { log("9a @PreDestroy"); }
    public void destroy()                { log("9b destroy"); }
}
```

> **RefinePoint:** `RunWorker.stop()` is a real `@PreDestroy` with business meaning. On SIGTERM it sets `stopping`, runs `UPDATE recon_runs SET status='QUEUED', instance=NULL ... WHERE instance=? AND status='RUNNING'` so another instance picks those runs up immediately, removes itself from `run_workers`, then shuts its pools down. A hard kill skips all of it, and that is why a heartbeat reaper exists too: it fails runs whose instance has been silent for two minutes. Always design for both paths.

15.5 Constructor, setter and field injection

	Constructor	Setter	Field
Immutability (`final`)	Yes	No	No
Required dependencies are explicit	Yes	No	No
Object valid after `new`	Yes	Not until set	No
Testable without Spring	`new X(mock)`	Yes	Needs reflection
Too many dependencies are visible	Painfully, which is good	Somewhat	Hidden
Detects cycles early	Yes, fails at startup	No	No
Good for	**Everything by default**	Optional, re-configurable	Tests only

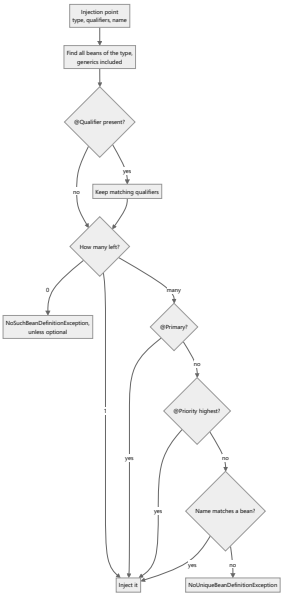
```
@Service
@RequiredArgsConstructor                    // Lombok generates the single constructor
public class PaddleWebhookProcessor {
    private final OrgSubscriptionRepository subscriptionRepo;
    private final PaddleApi paddle;
    private final AuditTrail audit;
}
```

Since Spring 4.3, a class with a **single constructor needs no** `@Autowired`. With several constructors you must mark one, or provide a no-arg one for Spring to use.

```
// ryfn-expression: three constructors, one for the container
public ExpressionEngine() { this((Supplier<LookupSource>) () -> null); }   // tests and tools
@Autowired public ExpressionEngine(ObjectProvider<LookupSource> lookups) { this((Supplier<LookupSource>) lookups::getIfAvailable); }
public ExpressionEngine(Supplier<LookupSource> lookups) { this.lookups = lookups; }
```

Trap: Lombok's `@RequiredArgsConstructor` does **not** copy `@Qualifier` or `@Value` from fields to constructor parameters unless `lombok.config` has `lombok.copyableAnnotations += org.springframework.beans.factory.annotation.Qualifier`. Without it the qualifier is silently ignored. Our `EmailSender` uses `@Value` on a non-final field for this reason.

15.6 How `@Autowired` resolves a dependency



Order: by type, narrowed by `@Qualifier`, then `@Primary`, then `@Priority`, and finally the **field or parameter name** as the fallback. Spring 6.2, which Boot 3.4 uses, adds `@Fallback`, the mirror of `@Primary`: a bean used only if no other candidate exists.

```
@Bean public ApiKeyAuthenticationFilter apiKeyFilter(ApiKeyVerifier v)         { return new ApiKeyAuthenticationFilter(v, true); }
@Bean public ApiKeyAuthenticationFilter internalApiKeyFilter(ApiKeyVerifier v) { return new ApiKeyAuthenticationFilter(v, false); }

@Bean
FilterRegistrationBean<ApiKeyAuthenticationFilter> reg(@Qualifier("apiKeyFilter") ApiKeyAuthenticationFilter f) { /* ... */ }
```

That is real code from `PlatformSecurityAutoConfiguration`: two beans of one type, and an explicit qualifier for each. Its bean also has `setEnabled(false)`, since a `Filter` bean is auto-registered with the servlet container unless told otherwise.

Optional and multiple dependencies:

```
public X(Optional<Foo> foo) { }            // absent = Optional.empty()
public X(@Nullable Foo foo) { }            // absent = null
@Autowired(required = false) Foo foo;      // absent = left alone
public X(ObjectProvider<Foo> foo) { }      // deferred: getIfAvailable, getIfUnique, stream
public X(List<Foo> all) { }                // all beans, in @Order / Ordered order
public X(Map<String, Foo> byName) { }      // bean name -> bean
```

An injected empty `List` is not given to you when there are no beans: resolution fails unless the injection is optional or an `ObjectProvider` is used. `@Order` affects **the order in collections only**, not which bean is chosen for a single injection point. Generics are honoured: `Repo<Run>` will not receive a `Repo<Break>`.

Trap: the name fallback relies on the parameter name being in the bytecode. Spring 6.1 removed the debug-info (`LocalVariableTable`) based lookup, so you need `-parameters`, which `spring-boot-starter-parent` sets by default. A custom compiler setup can break working injection after an upgrade.

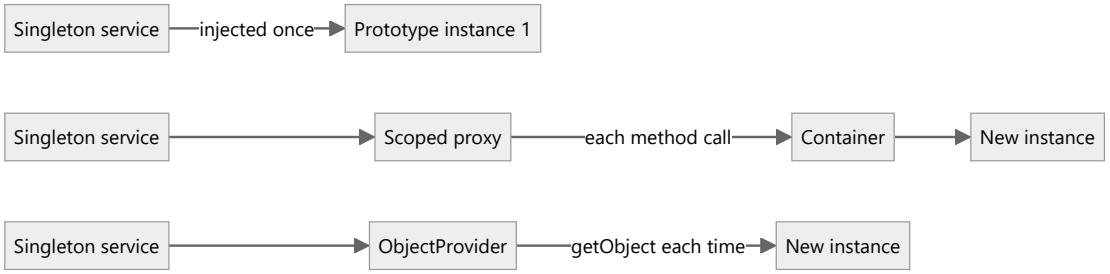
> **RefinePoint:** `EmailSender` takes `ObjectProvider<JavaMailSender>` because mail is optional: Boot creates that bean only if `spring.mail.host` is set. `configured()` is simply `mailSender.getIfAvailable() != null`, which is how the "send test" button can report "Email: not configured on this server". `Optional` would work too, but the provider re-reads on every call and never fails.

15.7 Bean scopes

Scope	Instances	Destroy callback	Notes
`singleton` (default)	One per container per definition	Yes	Created at startup
`prototype`	New per lookup or injection	**No**	Container forgets it
`request`	One per HTTP request	Yes	Bound to the request thread
`session`	One per HTTP session	Yes	Serializable if sessions replicate
`application`	One per `ServletContext`	Yes	Can span contexts, visible as an attribute
`websocket`	One per WebSocket session	Yes	STOMP message scope

Injecting a prototype into a singleton

The classic bug: the singleton is built once, so its dependency is resolved once. You get the same "prototype" forever.




```
// 1. ObjectProvider: the recommended approach
@Service @RequiredArgsConstructor
class Exporter {
    private final ObjectProvider<CsvBuffer> buffers;
    void export() { CsvBuffer b = buffers.getObject(); /* a new one each call */ }
}

// 2. @Lookup: the container generates a CGLIB subclass that overrides this method
@Service abstract class Exporter2 {
    @Lookup abstract CsvBuffer newBuffer();
}

// 3. Scoped proxy: the injected object is a proxy that resolves per call
@Component @Scope(value = "prototype", proxyMode = ScopedProxyMode.TARGET_CLASS)
class CsvBuffer { }
```

Option	Good for	Watch out
`ObjectProvider`	Default choice, explicit	Spring API in your class; `jakarta.inject.Provider` is the neutral form
`@Lookup`	Keeping the class free of Spring API calls	Needs a subclassable class, and it doesn't apply on `@Bean`-created beans
Scoped proxy	Request and session beans	With a prototype **each method call** gets a new instance, which surprises people

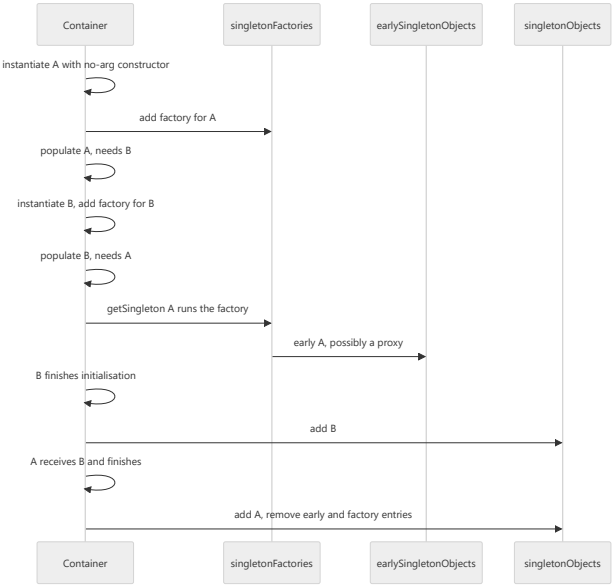
A **request-scoped bean in a singleton** needs a proxy, which is why `@RequestScope` sets `proxyMode = TARGET_CLASS` for you. The proxy looks up the current request through `RequestContextHolder`, a `ThreadLocal`. On an `@Async` thread it fails with "No thread-bound request found".

> **RefinePoint:** we use no custom scopes at all, and that is a defensible design. Everything is a singleton. Request data is passed as method parameters, or read from `RequestContextHolder` at one controlled point: `AuditTrail.currentRequest()` checks `instanceof ServletRequestAttributes` and returns null otherwise, so the same audit call works in a controller, in a scheduled job and in a Kafka listener.

15.8 Circular dependencies and the three-level cache

`DefaultSingletonBeanRegistry` holds three maps:

Level	Map	Holds
1	`singletonObjects`	Fully initialised beans
2	`earlySingletonObjects`	Early references already handed to someone: not yet fully initialised
3	`singletonFactories`	`ObjectFactory` lambdas that produce an early reference on demand



Why three levels, not two? The early reference might have to be a **proxy**. If B injected the raw A while A later got wrapped at step 6, B would hold the wrong object. The factory at level 3 calls `getEarlyBeanReference`, letting `AbstractAutoProxyCreator` make the proxy early, but **only if a cycle actually occurs**; otherwise the normal after-init path runs. Level 2 stores the result so every other bean in the cycle gets the same object.

Why constructor cycles fail: an early reference needs an instance to exist. In `new A(new B(new A(...)))` no instance exists at the time, so Spring throws `BeanCurrentlyInCreationException`. Setter and field injection work because the object exists after its no-arg constructor has run. **Prototype cycles always fail,** since they aren't cached.

Boot 2.6+ prohibits all cycles by default (`spring.main.allow-circular-references=false`), so even field-injected cycles fail on startup with a report drawing the loop. You can turn it back on, but that treats a design smell as a config problem.

How to fix one, in order of preference:

1. **Redesign.** Extract the shared logic into a third bean; cycles usually mean mixed responsibilities.
2. **Use an event** when one side only needs to notify the other.
3. **`ObjectProvider`** resolves the dependency on use, so construction has no cycle.
4. **`@Lazy` on the constructor parameter** injects a lazy-resolving proxy.

> **RefinePoint:** we have a real example. In lookup-service `RuleFunctionService` depends on `LocalFunctionSource` to forget cached definitions when one changes, and `LocalFunctionSource` must call back to `RuleFunctionService` to read definitions. That is a cycle, and `LocalFunctionSource` breaks it with `ObjectProvider<RuleFunctionService>`, calling `functions.getObject().definitions(org)` on use. No setting was changed, both classes keep constructor injection, and the dependency is visible right in the signature.

> **Trap:** with `allow-circular-references=true` and `@Async` on a bean in a cycle, startup fails with "has been injected into other beans in its raw version". `AsyncAnnotationBeanPostProcessor` wraps at step 6 without supporting early references, so the raw early reference and the final proxy differ. `@Transactional` does not have this problem because its auto-proxy creator supports `getEarlyBeanReference`.

15.9 `@Component`, `@Bean`, `@Configuration`

	`@Component` family	`@Bean`
Declared on	Class	Method in a config class
Discovery	Component scanning	Reading the configuration class
Control over creation	Container calls the constructor	You write the code
Third-party classes	No, cannot annotate	**Yes**
Conditional wiring	On class	On method, more fine-grained
Multiple beans of one class	Not directly	Yes

Stereotypes: `@Component` is the base. `@Service` adds only meaning. `@Repository` adds **exception translation** (`PersistenceExceptionTranslationPostProcessor` converts to `DataAccessException`). `@Controller` and `@RestController` make a class a handler for `RequestMappingHandlerMapping`. `@Configuration` is a meta-annotated `@Component` with bean-method handling.

Scanning: `ClassPathBeanDefinitionScanner` reads class files with **ASM, without loading the classes**, and creates definitions. Boot scans from the package of the `@SpringBootApplication` class downward. Default bean name is the decapitalised simple class name. Two beans with one name fail in Boot, since `spring.main.allow-bean-definition-overriding` defaults to `false`.

`@Configuration` full mode vs lite mode




```
@Configuration
public class AppConfig {
    @Bean DataSource dataSource() { return new HikariDataSource(); }
    @Bean JdbcTemplate jdbc() { return new JdbcTemplate(dataSource()); }   // returns the singleton: CGLIB intercepts
}

@Configuration(proxyBeanMethods = false)
public class LiteConfig {
    @Bean DataSource dataSource() { return new HikariDataSource(); }
    @Bean JdbcTemplate jdbc(DataSource ds) { return new JdbcTemplate(ds); } // inject as a parameter, always correct
}
```

Full mode needs the class to be non-final and its `@Bean` methods non-private, non-final. **Lite mode** is faster to start and is what Boot's own auto-configurations use (`@AutoConfiguration` is `proxyBeanMethods = false`). If you always declare dependencies as method parameters, it is also safe. Our `PlatformEventsAutoConfiguration` is written like this: each `@Bean` method takes its dependencies as parameters.

> **Trap:** `@Bean` methods returning a `BeanFactoryPostProcessor` must be `static`. Otherwise creating that bean forces the configuration class to be instantiated very early, before BPPs are registered, so `@Autowired` and `@Value` inside that config break, with a warning in the log.

15.10 `@Conditional`, profiles, Environment

`@Conditional` takes a `Condition` that is evaluated while definitions are registered. Boot's versions are built on it:

Annotation	Registers the bean when
`@ConditionalOnProperty`	A property has the value
`@ConditionalOnClass`	A class is on the class path
`@ConditionalOnMissingBean`	No bean of that type is defined yet
`@ConditionalOnBean`	A bean of that type is defined
`@ConditionalOnWebApplication`	Running as a web app
`@Profile`	A profile is active (it is itself `@Conditional(ProfileCondition.class)`)

```
// ryfn-platform's PlatformEventsAutoConfiguration
@Bean @ConditionalOnMissingBean
public AuditTrail auditTrail(EventPublisher events) { return new AuditTrail(events); }

@Bean @ConditionalOnProperty(name = "rp.kafka.enabled", havingValue = "true", matchIfMissing = true)
public OutboxRelay outboxRelay(EventStore store, KafkaTemplate<String, String> kafka, TopicNames topics, TransactionTemplate tx) { ... }
```

`@ConditionalOnMissingBean` lets a service override any platform default by defining its own bean. It depends on ordering, so it is only reliable in auto-configurations, which are processed after user configuration (`DeferredImportSelector`).

> **RefinePoint:** we use **no `@Profile` in Java code**. Behaviour differs per environment through properties, not by naming environments: `rp.kafka.enabled=false` in tests lets events wait in the outbox, and `ai.digest.enabled` turns the `DigestScheduler` on. A property answers "what does this environment do?", which stays true when someone adds a fourth environment. Profile-coded logic tends to rot.

Environment and property sources

`Environment` is a chain of `PropertySource`s searched in order, with first match winning.

```
Highest priority (Boot)
  1. Test properties (@SpringBootTest, @TestPropertySource)
  2. Command-line arguments             --server.port=9000
  3. SPRING_APPLICATION_JSON
  4. Servlet parameters
  5. JNDI
  6. Java system properties             -Dserver.port=9000
```

```
  7. OS environment variables          SERVER_PORT=9000
  8. random.* values
  9. Profile-specific files, outside the jar then inside
 10. application.yml, outside the jar then inside
 11. @PropertySource on @Configuration
 12. SpringApplication.setDefaultProperties, and sources added last
Lowest priority
```

Our `PlatformDefaults` adds its source at the end of the chain, which is why a service's `application.yml` overrides it.

`@Value` vs `@ConfigurationProperties`

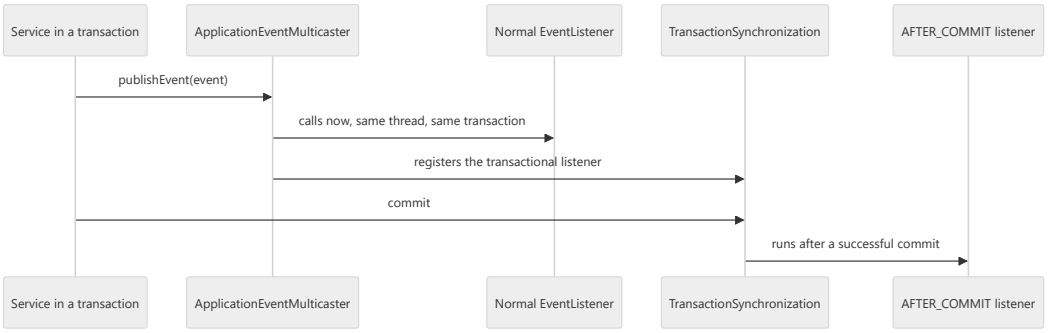
	`@Value`	`@ConfigurationProperties`
Binding	One value	A typed tree
Relaxed binding (`max-size` , `MAX_SIZE`)	Limited	Full
Validation	No	`@Validated` with JSR-380
SpEL	`#{...}`	No
IDE metadata	No	Yes, with the processor
Immutable	With constructor	Records, constructor binding
Lists and maps	Awkward	Natural
Best for	One or two values	Anything grouped

```java
// Real: ryfn-platform, every service shares this under rp.*
@Getter @Setter
@ConfigurationProperties(prefix = "rp")
public class PlatformProperties {
    private String internalToken = "";
    private Map<String, String> services = new LinkedHashMap<>();
    private Duration cacheTtl = Duration.ofSeconds(30);      // "30s" in YAML binds to Duration
}

// The modern immutable form
@ConfigurationProperties("export") @Validated
public record ExportProps(@Min(1) int workers, @NotNull Duration timeout) {}
```

Register them with `@EnableConfigurationProperties` or `@ConfigurationPropertiesScan` . `AiConfig` does the first for `AiProperties` .

15.11 Application events



Kind	When it runs	Thread	Exception effect
`@EventListener`	During `publishEvent`	Publisher's	Propagates, can roll back
`@EventListener` + `@Async`	Queued	Executor	Goes to the `AsyncUncaughtExceptionHandler`
`@TransactionalEventListener`	Transaction phase, default `AFTER_COMMIT`	Publisher's	Cannot undo the commit
Both `@Async` and `@TransactionalEventListener`	After commit, queued	Executor	Isolated

The phases are `BEFORE_COMMIT` , `AFTER_COMMIT` , `AFTER_ROLLBACK` , `AFTER_COMPLETION` .

Key mechanics:

- `EventListenerMethodProcessor` finds `@EventListener` methods and wraps each in an `ApplicationListenerMethodAdapter` .
- The default `SimpleApplicationEventMulticaster` is **synchronous**. Use a custom executor on it only with care, since every listener becomes asynchronous.
- **If no transaction is active, a `@TransactionalEventListener` is dropped**, unless `fallbackExecution = true` .
- **Writing inside `AFTER_COMMIT` :** the old transaction has committed, and its resources are still bound, so a plain `save` will not be committed. Use `REQUIRES_NEW` or `@Async` . Since Spring 6.1 a `@Transactional` on such a listener must be `REQUIRES_NEW` or `NOT_SUPPORTED` ; otherwise startup fails.
- `@EventListener` can have a condition (`condition = "#e.orgId != null"`) and can return a new event to publish.

RefinePoint: `NotificationService.onAlertRaised` combines `@Async` with `@TransactionalEventListener(phase = AFTER_COMMIT, fallbackExecution = true)`. It runs only after the alert is really committed, on another thread, so a slow SMTP server never holds a transaction, and a failing webhook never affects the save. `fallbackExecution = true` is there because an alert may also be raised outside any transaction. `RunWorker.onQueued` uses the same listener to wake the poller, so the poll sees the committed row and picks up a new run at once, rather than waiting for the next scheduled tick.

15.12 `FactoryBean`, `BeanPostProcessor`, lazy init, thread safety

`FactoryBean`

A bean that produces another bean. `getBean("x")` returns the **product**, `getBean("&x")` the factory itself. It is how complex setups plug into the container: `LocalContainerEntityManagerFactoryBean`, `JpaRepositoryFactoryBean` (creates your Spring Data repository proxy), `ProxyFactoryBean`.

`FactoryBean` vs `BeanFactory`: `BeanFactory` is the container. A `FactoryBean` is one bean in it, which makes a product.

`BeanFactoryPostProcessor` vs `BeanPostProcessor`

	`BeanFactoryPostProcessor`	`BeanPostProcessor`
Works on	**Definitions**, before beans exist	**Instances**, one by one
Runs	Once, at refresh step 5	For every bean, around init
Examples	`ConfigurationClassPostProcessor`, `PropertySourcesPlaceholderConfigurer`	`AutowiredAnnotationBeanPostProcessor`, `CommonAnnotationBeanPostProcessor`, `AnnotationAwareAspectJAutoProxyCreator`, `AsyncAnnotationBeanPostProcessor`, `ScheduledAnnotationBeanPostProcessor`, `ConfigurationPropertiesBindingPostProcessor`

```
@Component
public class SlowInitDetector implements BeanPostProcessor, PriorityOrdered {
    private final Map<String, Long> started = new ConcurrentHashMap<>();
    @Override public Object postProcessBeforeInitialization(Object bean, String name) {
        started.put(name, System.nanoTime());
        return bean;
    }
    @Override public Object postProcessAfterInitialization(Object bean, String name) {
        Long t = started.remove(name);
        if (t != null && System.nanoTime() - t > 1_000_000_000L) log.warn("Slow init: {}", name);
        return bean;                                  // returning a different object replaces the bean
    }
    @Override public int getOrder() { return Ordered.HIGHEST_PRECEDENCE; }
}
```

Trap: a `BeanPostProcessor` is created early. Anything it injects is built before other BPPs are ready, so those dependencies miss AOP and more. Look for the log line "is not eligible for getting processed by all BeanPostProcessors". Keep BPPs dependency-free, and take an `ObjectProvider` if you need one.

Lazy initialisation

`@Lazy` on a bean defers creation to first use; on an injection point it injects a proxy. `spring.main.lazy-initialization=true` applies it globally. Faster startup, but **config errors surface at the first request, and the first request is slow**. So use it in development or for rarely used heavy beans, and not broadly in production. Our services use it nowhere.

Are Spring singletons thread-safe?

Not automatically. Spring guarantees one instance and safe publication, but one instance serves all threads. Thread safety depends on the state you put in it. Stateless services with `final` dependencies are safe. Any mutable field needs a thread-safe type.

```
// Bug: one SimpleDateFormat, not thread-safe, shared across request threads
@Service class Bad { private final SimpleDateFormat f = new SimpleDateFormat("yyyy-MM-dd"); }

// Bug: per-request data in a singleton field
@Service class AlsoBad { private UUID currentOrg; }
```

RefinePoint: `RunWorker` is a singleton with real mutable state, and each field shows the right tool: `private volatile boolean stopping`, `AtomicBoolean polling` (CAS guard so only one poll runs at a time), `Set<UUID> working = ConcurrentHashMap.newKeySet()`, and final executor pools. `RateLimitFilter` holds its buckets in a `ConcurrentHashMap`. That is a good answer to "have you ever had state in a singleton?".

15.13 Interview questions and answers

Q1. What is IoC, and how does it differ from DI?

IoC means a framework controls the flow and calls your code. DI is one form of it, where dependencies are given to an object rather than created or located by it. Event callbacks and template methods are IoC without DI.

Q2. What is a Spring bean?

An object that is instantiated, wired and managed by the container from a `BeanDefinition`. An object you `new` yourself is not a bean, so it receives no injection, proxying or lifecycle callbacks.

Q3. BeanFactory vs ApplicationContext?

`BeanFactory` is the core: definitions, lazy `getBean`, DI. `ApplicationContext` extends it with eager singleton creation, automatic post-processor registration, events, `Environment` and i18n. A context delegates bean work to an internal `DefaultListableBeanFactory`.

Q4. Walk me through what happens when a Spring Boot app starts.

`SpringApplication.run` prepares the `Environment`, running `EnvironmentPostProcessor`s, and creates the context. `refresh()` then runs `ConfigurationClassPostProcessor`, which scans and reads `@Bean`, `@Import` and auto-configurations to produce definitions. It registers `BeanPostProcessor`s, creates the embedded server in `onRefresh`, instantiates all non-lazy singletons, starts lifecycle beans, publishes `ContextRefreshedEvent` and opens the port. `CommandLineRunner`s run, and `ApplicationReadyEvent` fires.

Q5. What is a BeanDefinition?

Metadata describing how to build a bean: class or factory method, scope, constructor and property values, lazy flag, primary flag, init and destroy methods, dependencies. It is what makes conditional registration and post-processing possible, since definitions exist before objects.

Q6. Give the full bean lifecycle.

Instantiate → populate (`@Autowired`) → `BeanNameAware`, `BeanClassLoaderAware`, `BeanFactoryAware` → BPP before-init (`ApplicationContextAware`, `@PostConstruct`) → `afterPropertiesSet` → custom init → BPP after-init (AOP proxy) → in use → `@PreDestroy` → `DisposableBean.destroy` → custom destroy.

Q7. Order of `@PostConstruct`, `afterPropertiesSet` and `initMethod`?

`@PostConstruct` first, as it is run by a BPP before init; then `afterPropertiesSet`; then the `initMethod`. If all three name the same method, it runs once.

Q8. Can I use injected dependencies in a constructor? In `@PostConstruct`?

With constructor injection, yes: those dependencies are parameters. Field-injected values are still null in the constructor. In `@PostConstruct` all of them are set. But proxies aren't applied yet, so `@Transactional` on a self-call doesn't apply.

Q9. How do you run code after the app has fully started?

`ApplicationRunner` or `CommandLineRunner`, or listen to `ApplicationReadyEvent`. Use `SmartInitializingSingleton` for "all singletons exist" work, and `SmartLifecycle` for start and stop with ordered phases. For a transactional startup task, avoid `@PostConstruct`.

Q10. `ContextRefreshedEvent` vs `ApplicationReadyEvent`?

`ContextRefreshedEvent` is a Spring event on each refresh, and it can fire more than once with child contexts. `ApplicationReadyEvent` is Boot's, fired once after runners complete, when the app is ready for traffic. Prefer the latter.

Q11. Why is constructor injection recommended?

Dependencies can be `final`, the object is valid once constructed, tests can call `new`, required dependencies are visible, an overlong parameter list exposes a design smell, and cycles fail fast. Field injection hides all of these and needs reflection in tests.

Q12. When would you use setter injection?

For a genuinely optional dependency with a sensible default, or one re-assignable after creation. It's also how a few older frameworks must be configured. It's rare in modern code.

Q13. Is `@Autowired` required on a constructor?

Not if the class has one constructor (Spring 4.3+). With several, annotate one, or Spring looks for a no-arg constructor and fails if there is none. Our `ExpressionEngine` has three constructors, with `@Autowired` on the `ObjectProvider` one.

Q14. How does Spring resolve when two beans match a type?

Filter by `@Qualifier`, then pick `@Primary`, then highest `@Priority`, then a bean whose name matches the field or parameter name. Otherwise `NoUniqueBeanDefinitionException`. In 6.2 `@Fallback` beans are considered only if no regular bean qualifies.

Q15. `@Primary` vs `@Qualifier`?

`@Primary` sets a default at the definition. `@Qualifier` is an explicit choice at the injection point, and it wins over `@Primary`. Use `@Primary` when one bean is usually right, such as in tests, and `@Qualifier` when callers genuinely differ.

Q16. How do you create a custom qualifier?

Define an annotation meta-annotated with `@Qualifier`, such as `@ReadOnly`, and put it on both bean and injection point. It's type-safe and avoids string names.

Q17. How do you inject all implementations of an interface?

`List<T>` in `@Order` order or `Map<String, T>` by bean name. Useful for strategy dispatch. For zero-or-more semantics use `ObjectProvider<T>.stream()`, as a plain `List` fails when there are none.

Q18. Does `@Order` affect which bean is injected?

No, it affects only collection order, and things like filter and aspect order. To choose a single bean use `@Primary` or `@Qualifier`.

Q19. `ObjectProvider` vs `Optional` vs `@Autowired(required=false)`?

`Optional` and `required=false` resolve once at injection. `ObjectProvider` resolves on each call, supports `getIfAvailable`, `getIfUnique`, `stream`, and defers creation, so it also handles cycles and prototypes. It's the most flexible.

Q20. What are the bean scopes?

Singleton, prototype, request, session, application, websocket, plus custom ones via `Scope` and `registerScope`. Web scopes need a web-aware context.

Q21. How does Spring handle a prototype bean's destruction?

It doesn't. After creating and initialising it, the container keeps no reference, so `@PreDestroy` never runs. The caller must clean up, or use a custom BPP to track them.

Q22. Inject a prototype into a singleton. What happens? How do you fix it?

It's resolved once when the singleton is built, so it stays one instance. Fix with `ObjectProvider.getObject()`, `@Lookup`, or a scoped proxy. Injecting the `ApplicationContext` also works but it's service locator style.

Q23. How does `@Lookup` work?

The container generates a CGLIB subclass overriding the method to call `getBean`. So the class must be non-final, and it doesn't work for beans from `@Bean` factory methods, because the container doesn't control their instantiation.

Q24. Singleton scope vs application scope?

Singleton is per `ApplicationContext`; application is per `ServletContext`, can be shared across contexts in one web app, and is exposed as a servlet attribute. With a single Boot context they behave nearly the same.

Q25. What is a scoped proxy?

A proxy bean of the same type injected in place of the short-lived bean. Each method call resolves the actual target from the current scope. Needed because a singleton gets wired once while the request bean doesn't exist at startup. `@RequestScope` and `@SessionScope` set it up by default.

Q26. Why does a request-scoped bean fail in an `@Async` method?

Request attributes live in a `ThreadLocal` on the request thread. The async thread has none, hence "No thread-bound request found". Pass values explicitly as parameters, or use a `TaskDecorator` that copies the attributes, while remembering the request may finish first.

Q27. How does Spring resolve circular dependencies?

For singleton field or setter injection, A is instantiated and a factory for its early reference is placed in `singletonFactories`. When B needs A it gets that early reference and finishes; then A finishes. The three caches keep this consistent. Boot 2.6+ disallows this by default.

Q28. Why three caches? Wouldn't two do?

Two would suffice with no proxies. Level 3 allows lazy creation of an early **proxy** via `getEarlyBeanReference` only when a cycle actually occurs, so non-cyclic beans still get proxied at the normal stage. Level 2 ensures all beans in the cycle get the same early object.

Q29. Why do constructor cycles fail?

Early exposure requires an instance. Each constructor needs the other's finished instance, so no object can exist first, hence `BeanCurrentlyInCreationException`. `@Lazy` on one parameter breaks it by injecting a proxy.

Q30. How did you deal with a circular dependency in your project?

"`RuleFunctionService` calls `LocalFunctionSource` to forget cached definitions, while `LocalFunctionSource` reads definitions from `RuleFunctionService`. I kept constructor injection on both and gave the source an `ObjectProvider<RuleFunctionService>`, resolved at use. Startup has no cycle, and no global setting changed."

Q31. `@Component` vs `@Bean`?

`@Component` goes on your class and is found through scanning. `@Bean` goes on a factory method and is used for third-party classes, conditional or multiple instances, or construction logic. `AiConfig` uses `@Bean` because it chooses which `LlmClient` to return.

Q32. `@Component`, `@Service`, `@Repository`, `@Controller`: any real difference?

All are components. `@Repository` adds exception translation; `@Controller` makes it a web handler. `@Service` carries only intent, though aspects can target it.

Q33. What does `@Configuration` do that `@Component` with `@Bean` doesn't?

It's enhanced by CGLIB, so an inter-bean call returns the container's singleton. In a `@Component` a `@Bean` method call is a plain Java call that creates another object, so a second, unmanaged instance appears.

Q34. What is `proxyBeanMethods = false`, and when use it?

It turns off CGLIB enhancement, giving lite mode: faster startup, less memory, compatible with native images. Use it whenever bean methods take dependencies as parameters and don't call each other. It's what auto-configurations do.

Q35. Why must a `BeanFactoryPostProcessor` `@Bean` method be static?

So the container can call it without creating the config instance. A non-static one forces the config class to be created during step 5, before BPPs exist, breaking its `@Autowired` and `@Value`.

Q36. BeanPostProcessor vs BeanFactoryPostProcessor?

BFPP edits definitions before instantiation; BPP handles instances around initialisation and may replace them. `@Value` placeholders in definitions are resolved through `PropertySourcesPlaceholderConfigurer`, a BFPP, and `@Autowired` is executed by a BPP.

Q37. Why might a bean not get proxied though it has `@Transactional`?

It may be created too early, as a dependency of a BPP or BFPP ("not eligible for getting processed by all BeanPostProcessors"). Other causes: it isn't a Spring bean, or the call is a self-invocation. Check with `AopUtils.isAopProxy(bean)`.

Q38. What is `FactoryBean`? How do you get the factory itself?

A bean that creates another object, exposed under the factory's name. Prefix with `&`: `ctx.getBean("&myFactory")`. Spring Data repositories are built by `JpaRepositoryFactoryBean`.

Q39. How does component scanning work, and what if my class is outside the base package?

ASM reads class metadata in the base packages without loading classes, filters by annotation, and registers definitions. Classes outside aren't found. Widen with `scanBasePackages`, use `@Import`, or better for libraries, register an auto-configuration in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, as `ryfn-platform` does.

Q40. How do profiles work?

`spring.profiles.active` activates them. `@Profile` filters definitions, and `application-{profile}.yml` adds property sources. Use profile groups for bundles. Keep logic out of them and use feature properties to describe behaviour; we use no `@Profile` in our Java code.

Q41. What is the order of property sources? Give examples.

Command line > `SPRING_APPLICATION_JSON` > system properties > OS environment > external profile file > packaged profile file > external `application.yml` > packaged one > `@PropertySource` > defaults. So `SERVER_PORT=9000` in the environment overrides the packaged YAML, and `--server.port` beats both.

Q42. Why can a `@Value` be null or the literal `${x}`?

Null: the object was created with `new`, or read in a constructor under field injection. A literal `${x}`: no `PropertySourcesPlaceholderConfigurer` is active (Boot adds it), or a non-static BFPP disturbed it. A missing key without a default fails startup: `${x:default}` fixes that.

Q43. `@Value` vs `@ConfigurationProperties`?

`@Value` is for one or two values and supports SpEL. `@ConfigurationProperties` gives typed, validated, relaxed-bound groups with IDE metadata. Use records for immutability. Ours are `PlatformProperties` for `rp.*` and `AiProperties` for `ai.*`.

Q44. How does relaxed binding work, and where does it fail?

`app.max-size`, `app.maxSize`, `APP_MAXSIZE` and `app.max_size` all bind to `maxSize`. `@ConfigurationProperties` is fully relaxed. `@Value` needs the canonical kebab-case form in the placeholder to benefit from any relaxed matching at all.

Q45. Default event listener behaviour: sync or async?

Synchronous, on the publisher's thread and inside its transaction. An exception propagates to the publisher. Add `@Async` per listener, or configure a task executor on the multicaster for all of them.

Q46. `@EventListener` vs `@TransactionalEventListener`?

The first runs immediately and joins the transaction. The second is held as a `TransactionSynchronization` and runs at a phase, by default after commit. So it sees committed data and won't act on rolled-back work. Without a transaction it's dropped unless `fallbackExecution = true`.

Q47. Why might a `@TransactionalEventListener` not fire, or its writes not be saved?

Not fired: no active transaction, the transaction rolled back (for `AFTER_COMMIT`), or `publishEvent` was on another thread. Not saved: after commit the old transaction is finished, so writes need `REQUIRES_NEW`. Since 6.1 a plain `@Transactional` on it fails at startup.

Q48. How is an in-process event different from the outbox?

An in-process event dies with the JVM: `AFTER_COMMIT` listeners aren't called if the instance crashes right after commit. The outbox stores the event in the same transaction, so it survives. We use `@TransactionalEventListener` for local, losable side effects such as waking the poller (the scheduled poll catches anything missed), and the outbox for events that must arrive.

Q49. Are singleton beans thread-safe?

Not inherently. Creation is safe, but one instance serves all threads, so mutable fields need a thread-safe approach. Keep services stateless; where state is required use `volatile`, atomics, or concurrent collections, like `RunWorker`.

Q50. Does `@Lazy` speed up startup? What are the downsides?

Yes, but errors appear at first use, and the first request is slow. Probes may report healthy although a broken bean exists. Use it on selected heavy beans, or in local development.

Q51. How can I detect or debug bean wiring problems?

Read Boot's `FailureAnalyzer` report. Use `/actuator/beans` for the graph, `/actuator/conditions` for why auto-configs did or didn't apply, and `--debug`. `logging.level.org.springframework.beans=DEBUG` shows creation, and `ApplicationStartup` with `BufferingApplicationStartup` shows slow beans.

Q52. How do you speed up slow context startup?

Measure first using `/actuator/startup`. Exclude unused auto-configurations, narrow scanning, avoid I/O in `@PostConstruct`, and defer heavy work. Spring 6.2 offers `@Bean(bootstrap = BACKGROUND)` for parallel initialisation, CDS archives help, and AOT and native images are the extreme.

Q53. Can two contexts share beans? What is a parent context?

A child context sees its parent's beans, not the reverse. Old web apps had a root context plus a servlet context; Boot usually has one. Spring Cloud's bootstrap context and management-port child contexts still use the idea. BPPs and AOP don't cross contexts.

Q54. Why is `ctx.getBean()` in business code bad?

It's service locator: hidden dependencies, runtime failures, harder tests. Acceptable in framework-level code, bootstrap, or factories when the bean type is decided at runtime, and `ObjectProvider` or a `Map` injection usually does better.

Q55. How do you override a bean from a shared library?

Define a bean of that type in your service. If the library's auto-configuration has `@ConditionalOnMissingBean`, it backs off, as with `AuditTrail`, `EventStore` and `EventPublisher` in `ryfn-platform`. Without that condition you get a name clash or `NoUniqueBeanDefinitionException`, so library authors should always use it.

Q56. Design question: how would you build a plug-in point with Spring?

Define an interface, inject `List<Plugin>` or `ObjectProvider<Plugin>`, and have each plug-in be a bean. Library plug-ins register through auto-configuration, with `@ConditionalOnMissingBean` defaults. For dropping jars in at runtime Spring isn't enough: we use `ServiceLoader` in `Plugins.load` and pass the results into beans.

Q57. A bean's `@PreDestroy` doesn't run in Kubernetes. Why?

The pod got SIGKILL: `terminationGracePeriodSeconds` expired, the OOM killer struck, or the process wasn't PID 1 and SIGTERM never reached it (a shell-form `ENTRYPOINT`). Set `server.shutdown=graceful`, an adequate grace period, and exec-form entrypoints. And always design for the crash path too, like our heartbeat reaper.

Q58. Tricky: two `@Bean` methods have the same name in two config classes. What happens?

Boot fails with `BeanDefinitionOverrideException`, since overriding is off by default. In plain Spring, or with `spring.main.allow-bean-definition-overriding=true`, the later silently replaces the earlier, which is a nasty, order-dependent bug.

16. Spring AOP, Proxies & How @Transactional Really Works

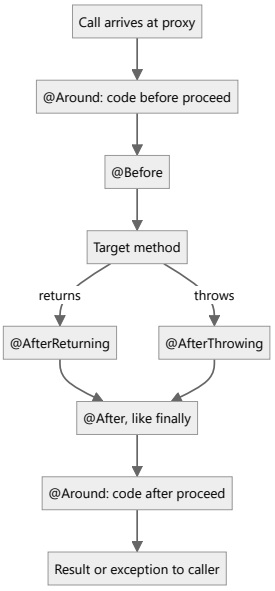
More production bugs come from misunderstood proxies than from any other part of Spring. Typical ones: a `@Transactional` method that never opened a transaction, an `@Async` method that ran on the caller's thread, a `@Cacheable` that never cached, a checked exception that committed half a payment. Each is explained by one sentence: **the annotation is implemented by a proxy, and only calls that go through the proxy get the behaviour.** This chapter makes that sentence concrete: how the proxy is created, what runs inside it, and every way to accidentally bypass it.

16.1 AOP terminology

Aspect-Oriented Programming modularises **cross-cutting concerns**: logic needed in many places that is not the business purpose of any of them, such as transactions, security, auditing, metrics, retries and caching.

Term	Meaning	Example
Aspect	A module holding a cross-cutting concern	`TimingAspect` class
Join point	A point in execution where advice can run. In Spring AOP: **only method execution on a bean**	`ExportService.export(..)` being called
Pointcut	A predicate selecting join points	`execution(* com.ryfnflow..service.*.*(..))`
Advice	The code to run at the join point, and when	`@Around` , `@Before` , ...
Advisor	Spring's pairing of one pointcut with one advice	`BeanFactoryTransactionAttributeSourceAdvisor`
Target	The original object being advised	Your `JournalService` instance
Proxy	The object created to apply advice; callers hold this	`JournalService$$SpringCGLIB$$0`
Weaving	Linking aspects with targets	Spring: at runtime, by proxy creation
Introduction	Adding an interface to an advised object	`@DeclareParents` (rare)

Advice types



Advice	Can stop the call?	Can change arguments or result?	Use
`@Before`	Only by throwing	No	Validation, access checks
`@AfterReturning`	No	Can read the result; changing a mutable result works but is bad form	Audit on success
`@AfterThrowing`	No, cannot swallow	Can rethrow a different exception	Exception translation, alerting
`@After`	No	No	Cleanup, like `finally`
`@Around`	**Yes** (don't call `proceed`)	**Yes**	Transactions, timing, retry, caching

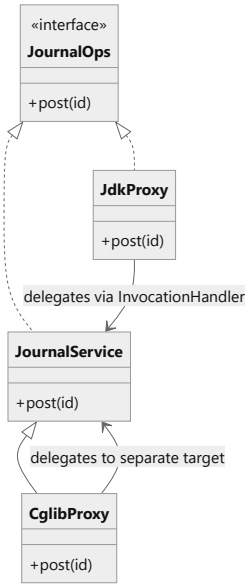
Use the weakest advice that does the job: an `@Around` that forgets to call `proceed()` or forgets to return its result silently breaks the method.

16.2 Spring AOP vs AspectJ

	Spring AOP	AspectJ
Mechanism	Runtime proxies (JDK or CGLIB)	Bytecode weaving
When	At bean creation	Compile time (ajc), post-compile, or load time (LTW, a Java agent)
Join points	Public (and with CGLIB, protected/package) method execution on Spring beans	Method call and execution, constructors, field get/set, static initialisers, any object
Self-invocation advised	**No**	Yes, the code itself is modified
Objects created with `new`	No	Yes
Final or private methods	No	Yes
Performance	One proxy hop plus an interceptor chain per call	Near zero, inlined into the code
Setup	None: `spring-boot-starter-aop`	Compiler plugin or `-javaagent`
Syntax	Uses AspectJ's annotations and pointcut language	Full AspectJ language

Spring AOP **borrows AspectJ's annotations and pointcut parser** (`aspectjweaver`), but weaving is pure Spring proxies. That's why `@Aspect` works without the AspectJ compiler. You can switch transactions to AspectJ mode with `@EnableTransactionManagement(mode = AdviceMode.ASPECTJ)` , which solves self-invocation, at the cost of build or agent setup. Few teams need it.

16.3 JDK dynamic proxy vs CGLIB



	JDK dynamic proxy	CGLIB proxy
How	`java.lang.reflect.Proxy` creates a class implementing the bean's **interfaces**	Generates a **subclass** of the bean's class
Requires	At least one interface	Non-final class; a constructor Spring can call (Objenesis usually bypasses it)
Inject by concrete class	**Fails**: proxy is only the interface type	Works
Advises	Interface methods only	Any non-final, non-private, non-static method
`getClass()`	`jdk.proxy2.$Proxy87`	`JournalService$$SpringCGLIB$$0`
Default in Spring Framework	When interfaces exist	When no interfaces exist
Default in Spring Boot	Only if `spring.aop.proxy-target-class=false`	**Always** (`proxy-target-class=true` since Boot 2.0)

Why Boot made CGLIB the default: with JDK proxies, injecting `JournalService` (the class) instead of `JournalOps` (the interface) failed with `BeanNotOfRequiredTypeException` , a constant source of confusion. Since Spring 4, CGLIB is repackaged inside spring-core (`org.springframework.cglib`), and Objenesis creates the proxy instance **without calling your constructor**, so the old "constructor runs twice" problem is gone.

```
// What a JDK proxy does, by hand
JournalOps proxy = (JournalOps) Proxy.newProxyInstance(
    JournalOps.class.getClassLoader(), new Class<?>[]{ JournalOps.class },
    (p, method, args) -> {
        long t = System.nanoTime();
```

```
        try { return method.invoke(target, args); }                // delegate to the real object
        catch (InvocationTargetException e) { throw e.getCause(); } // unwrap
        finally { log.info("{} took {} µs", method.getName(), (System.nanoTime() - t) / 1000); }
    });
```

Important structural fact: a Spring CGLIB proxy is a subclass, but it does **not** use `super.post()`. It holds a reference to a separate, fully initialised **target** instance and delegates to it. The proxy's own fields (inherited from your class) are never initialised and stay null/zero. Consequences come up below: field access through the proxy, `final` methods, and `this` inside the target.

16.4 How the proxy gets created



Key points:

- `AbstractAutoProxyCreator` **is a** `BeanPostProcessor`**.** Boot registers `AnnotationAwareAspectJAutoProxyCreator` (via `AopAutoConfiguration`) which collects **all** `Advisor` beans: your `@Aspect`s (each advice method becomes an advisor) plus infrastructure advisors for transactions, caching, method security.
- **One proxy per bean, however many concerns.** A bean that is `@Transactional` and has `@PreAuthorize` gets **one** proxy whose interceptor chain contains both interceptors.
- **Not all proxies come from the auto-proxy creator.** `@Async` uses `AsyncAnnotationBeanPostProcessor`, and `@Validated` uses `MethodValidationPostProcessor`. If the bean is already a proxy, these add their advisor to the existing proxy rather than wrapping it again.
- **The decision is per bean, the matching is per method.** At call time, the proxy looks up (and caches) the interceptor chain for that `Method`. Methods with no matching advisor go straight to the target.

16.5 What happens inside the proxy on each call



The chain is a `ReflectiveMethodInvocation`. Each interceptor receives the invocation and calls `invocation.proceed()`, which moves an index to the next interceptor, and finally reflectively calls the target. This is the **Chain of Responsibility** pattern: any interceptor can stop the call (security throws `AccessDeniedException`), and each sees the result on the way out.

Advice ordering

- Between aspects: `@Order` or `Ordered` on the **aspect class**. **Lower value = higher precedence = outermost** (runs first on the way in, last on the way out).
- Infrastructure order: `@EnableTransactionManagement(order = ...)`, `@EnableCaching(order = ...)`. Both default to `Ordered.LOWEST_PRECEDENCE`, so their relative order is undefined unless you set it.
- Inside one aspect, advice types run in a fixed order since Spring 5.2.7: `@Around`, `@Before`, then after the method `@AfterReturning` / `@AfterThrowing`, `@After`, then `@Around` completes. Two advices of the same type in one aspect have **no guaranteed order** — split them into two aspects with `@Order`.
- Method security (`@EnableMethodSecurity`) uses fixed ordered interceptors, with `@PreAuthorize` before the transaction interceptor, so a denied call never opens a transaction.

Why ordering matters: a **retry** aspect must be *outside* the transaction aspect. If it's inside, every retry runs in the same (already rollback-only, or with a stale persistence context) transaction. If it's outside, each attempt gets a fresh transaction.

```
@Aspect @Component @Order(1)          // outermost
class RetryAspect { /* ... */ }
// @EnableTransactionManagement(order = 100)  → transaction inside the retry
```

16.6 Pointcut expressions

Designator	Matches	Example
`execution`	Method signature	`execution(public * com.ryfnflow..*Service.*(..))`
`within`	All methods of types in a package or type	`within(com.ryfnflow.crm.journal..*)`
`@annotation`	Methods carrying an annotation	`@annotation(com.ryfnflow.Timed)`
`@within`	Methods of types carrying an annotation	`@within(org.springframework.stereotype.Service)`
`args`	Runtime argument types	`args(java.util.UUID, ..)`
`@args`	Runtime argument's type has an annotation	`@args(com.x.Sensitive)`
`this`	Proxy is an instance of type	`this(com.x.Auditable)`
`target`	Target is an instance of type	`target(com.x.JournalService)`
`bean`	Spring bean name (Spring-only)	`bean(*Service)`

`execution` syntax: `execution(modifiers? return-type declaring-type? name(params) throws?)`

```
execution(* *(..))                              any method
execution(* com.ryfnflow..service.*.*(..))      any method of any class in any 'service' package below com.ryfnflow
execution(* *..JournalService.post(java.util.UUID, ..))   first parameter UUID, then anything
execution(java.util.List *.find*(*))            returns List, name starts with find, exactly one parameter
..  in a package = any sub-package;   ..  in parameters = zero or more arguments;   *  = exactly one
```

Combine with `&&`, `||`, `!`, and name reusable pointcuts:

```
@Aspect @Component
class Pointcuts {
    @Pointcut("within(com.ryfnflow.crm..*)") void ours() {}
    @Pointcut("@within(org.springframework.stereotype.Service)") void services() {}
    @Pointcut("ours() && services() && execution(public * *(..))") void publicServiceMethods() {}
}
```

Performance tip: `within` and `execution` with a narrow package are evaluated statically at proxy-creation time. `args`, `this`, `target`, `@args` may need a runtime check on each call. Always narrow with `within(...)` first, and never write a pointcut like `execution(* *(..))` without a package scope — it tries to advise every bean, including Spring's own.

16.7 Writing custom aspects

Timing with Micrometer

```
@Target(ElementType.METHOD) @Retention(RetentionPolicy.RUNTIME)
public @interface Timed { String value(); }

@Aspect @Component @RequiredArgsConstructor
class TimingAspect {
    private final MeterRegistry meters;

    @Around("@annotation(timed)")                       // binds the annotation instance
    public Object time(ProceedingJoinPoint pjp, Timed timed) throws Throwable {
        Timer.Sample sample = Timer.start(meters);
        String outcome = "ok";
        try {
            return pjp.proceed();                       // must return the result!
        } catch (Throwable t) {
            outcome = t.getClass().getSimpleName();
            throw t;                                    // never swallow
        } finally {
            sample.stop(meters.timer(timed.value(), "outcome", outcome));
        }
    }
}
```

(Micrometer already ships `@Timed` / `@Observed` with `ObservedAspect`; write your own only when you need different tags.)

Audit trail

```
@Target(ElementType.METHOD) @Retention(RetentionPolicy.RUNTIME)
public @interface Audited { String action(); String entityType(); }

@Aspect @Component @RequiredArgsConstructor
class AuditAspect {
    private final AuditTrail audit;

    @AfterReturning(pointcut = "@annotation(a) && args(orgId, id, ..)", argNames = "a,orgId,id")
    public void record(Audited a, UUID orgId, UUID id) {
```

```
        audit.record(orgId, a.action(), a.entityType(), id, Map.of());
    }
}
```

RefinePoint: we deliberately do **not** audit with an aspect. Services call `audit.record(orgId, "JOURNAL_POSTED", "JOURNAL", id, Map.of("number", j.number(), "target", t, "externalId", externalId))` explicitly. The reasons are worth saying in an interview: the details map needs values computed in the middle of the method (the external id returned by the accounting system), some actions are audited only on certain branches, and an explicit call is visible in code review — nobody can remove auditing by renaming a method so the pointcut stops matching. `AuditTrail` writes through the outbox, so the entry commits or rolls back with the business change. Aspects shine for *uniform* concerns (timing, tracing); audit in a finance product is business data.

Retry

```
@Aspect @Component @Order(Ordered.HIGHEST_PRECEDENCE)   // outside the transaction
class OptimisticRetryAspect {
    @Around("@annotation(RetryOnConflict)")
    public Object retry(ProceedingJoinPoint pjp) throws Throwable {
        for (int attempt = 1; ; attempt++) {
            try { return pjp.proceed(); }
            catch (ObjectOptimisticLockingFailureException e) {
                if (attempt == 3) throw e;
                Thread.sleep(ThreadLocalRandom.current().nextLong(20, 50L * attempt));  // jittered backoff
            }
        }
    }
}
```

Retry only **idempotent** work, cap attempts, back off with jitter, and retry only exceptions that are transient. Spring Retry's `@Retryable` and Resilience4j's `@Retry` are this aspect, packaged.

Rate limit and tenant check

```
@Aspect @Component @RequiredArgsConstructor
class TenantGuardAspect {
    private final WorkspaceAccess access;

    @Before("within(com.ryfnflow.crm..controller..*) && args(orgCode, ..)")
    public void check(String orgCode) {
        access.resolve(orgCode, OrgRole.VIEWER);   // throws AccessDeniedException
    }
}
```

RefinePoint: our tenant check is **not** an aspect but a `HandlerInterceptor` — `OrgAccessInterceptor.preHandle` matches `/api/v1/organizations/{code}/...`, requires VIEWER for GET/HEAD and MEMBER for writes, and checks workspace locks. An interceptor sees the URL and HTTP method, which is exactly what the rule depends on, and it applies to every endpoint by path without each method opting in. Finer rules sit on controller methods as `@PreAuthorize("@orgAuth.can('journals.post')")` — which *is* AOP, via method security. Rate limiting lives in the gateway's `RateLimitFilter`, before any service is reached. Choosing filter vs interceptor vs aspect by **what information the rule needs** is the senior answer.

16.8 Spring features implemented as proxies

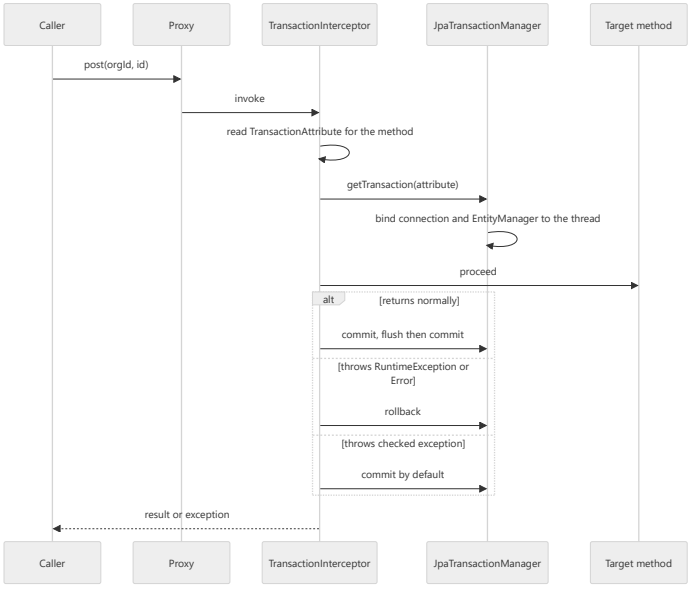
Annotation	Enabled by	Created by	Interceptor
`@Transactional`	`@EnableTransactionManagement` (Boot auto)	Auto-proxy creator + `BeanFactoryTransactionAttributeSourceAdvisor`	`TransactionInterceptor`
`@Cacheable`, `@CachePut`, `@CacheEvict`	`@EnableCaching`	Auto-proxy creator + `BeanFactoryCacheOperationSourceAdvisor`	`CacheInterceptor`
`@Async`	`@EnableAsync`	`AsyncAnnotationBeanPostProcessor`	`AnnotationAsyncExecutionInterceptor`
`@Retryable`	`@EnableRetry`	Spring Retry advisor	`RetryOperationsInterceptor`
`@PreAuthorize`, `@PostAuthorize`	`@EnableMethodSecurity`	Auto-proxy creator + method security advisors	`AuthorizationManagerBeforeMethodInterceptor`
`@Validated` + JSR-380 on params	`MethodValidationPostProcessor` (Boot auto)	That BPP	`MethodValidationInterceptor`
`@Lookup`, scoped proxy, `@Configuration`	Container	CGLIB directly	Method interceptors in the generated class
Spring Data repository interface	`@EnableJpaRepositories`	`JpaRepositoryFactoryBean`	Query method interceptors around `SimpleJpaRepository`

`@Async` in short: the interceptor wraps the call in a `Callable`, submits it to the `TaskExecutor` (Boot's `applicationTaskExecutor`, virtual threads with `spring.threads.virtual.enabled=true`), and returns immediately: nothing for `void`, or a `CompletableFuture` that completes when the task does. Any other return type is unsupported (older versions handed the caller `null`; current ones reject it), so declare `void` or `CompletableFuture<T>`. Exceptions from `void` methods go to the `AsyncUncaughtExceptionHandler` (logged by default) — the caller never sees them.

`@Cacheable` in short: build the key (`SimpleKeyGenerator`: the single argument, or `SimpleKey` of all args), look in the `Cache`; hit → return without calling the target; miss → call, store, return. `sync = true` makes concurrent misses on one key compute once. Caches need a `CacheManager`; with none configured Boot uses a `ConcurrentMapCacheManager` with **no eviction or TTL**.

> **RefinePoint:** we have no `@Cacheable` — cross-service answers (access decisions, plan limits, API-key checks) are cached by `TtlCache` in `ryfn-platform` for `rp.cache-ttl` (30 s by default), called explicitly from the remote clients. A plain TTL cache is easy to reason about and is never bypassed by self-invocation. `@Async` is used where it pays: `NotificationService.onAlertRaised` and `ChatDispatcher` deliver email, webhooks and chat posts off the request thread, and several services have `@EnableAsync` on their application class. `@PreAuthorize("@orgAuth.can('...')")` is on most controller methods — `OrgAuth` is a bean named `orgAuth` that evaluates the caller's permission *in the workspace of the request*, because Keycloak realm roles cannot express per-workspace owners and admins.

16.9 How `@Transactional` really works



Step by step, `TransactionAspectSupport.invokeWithinTransaction`:

1. **Find the attribute.** `AnnotationTransactionAttributeSource` reads `@Transactional` from the method, then the class (method wins), caching the result per method. No attribute → call target directly.
2. **Pick the manager.** `transactionManager` qualifier on the annotation, else the single `PlatformTransactionManager` bean (`JpaTransactionManager` with JPA on the class path).
3. **`getTransaction(attr)`** applies propagation: join the existing transaction (`REQUIRED`), suspend it and start a new one (`REQUIRES_NEW`), create a savepoint (`NESTED`), and so on. A new transaction obtains a connection, sets `autoCommit=false`, applies isolation, timeout and read-only, and **binds resources to the thread** in `TransactionSynchronizationManager` (a set of `ThreadLocal`s). That's why `JdbcTemplate` and the `EntityManager` in other beans join the same transaction transparently.
4. **Proceed** through the rest of the chain to your method.
5. **On exception:** `completeTransactionAfterThrowing` asks `rollbackOn(ex)`. Default: `RuntimeException` or `Error` → rollback; checked exception → **commit**. Rollback rules from `rollbackFor` / `noRollbackFor` are checked first (the closest matching exception type in the hierarchy wins).
6. **On success:** `commitTransactionAfterReturning` → `beforeCommit` synchronizations (Hibernate flush), commit, `afterCommit` synchronizations (`@TransactionalEventListener(AFTER_COMMIT)`), then unbind and release.
7. If an inner participant marked the transaction **rollback-only**, commit by the outer one throws `UnexpectedRollbackException`.

Because state lives in `ThreadLocal`s, **a transaction does not cross threads**: work submitted to an executor, `@Async` methods, `parallelStream()` and `CompletableFuture.supplyAsync` run outside the caller's transaction.

Rollback rules

```
@Transactional                                // rolls back on RuntimeException and Error only
@Transactional(rollbackFor = Exception.class)   // also on checked exceptions
@Transactional(noRollbackFor = BusinessWarning.class)
```

Why checked exceptions commit by default: the rule copies EJB's convention that checked exceptions are "expected business outcomes" the caller handles. In practice this surprises people: `throws IOException` from a file upload step commits everything written before it.

RefinePoint: we use the rule *on purpose* in `JournalService.post`, annotated `@Transactional(noRollbackFor = IllegalArgumentException.class)`. When posting a journal to the external accounting system fails, the method records the failure — `UPDATE journals SET post_error = ?, target = ?, connection_id = ?` — and then throws `IllegalArgumentException("Posting failed: ...")` so the user sees a 400. Without `noRollbackFor`, the throw would roll back the very row that explains the failure. With it, the error message is committed and the caller still gets an exception. A nice example of rollback rules being a design tool, not just a gotcha.

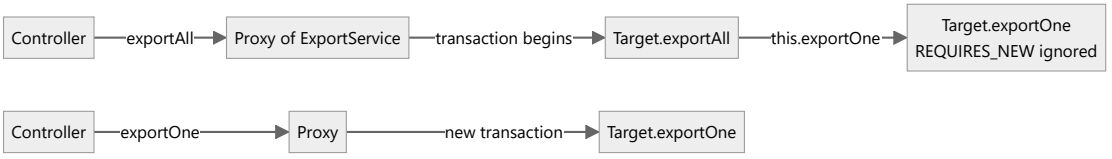
The swallowed exception

```
@Transactional
public void importAll(List<Row> rows) {
    for (Row r : rows) {
        try { rowImporter.importRow(r); }          // a @Transactional(REQUIRED) bean → joins the outer tx
        catch (RuntimeException e) { log.warn("skip {}", r, e); }   // swallowed
    }
}   // commit → UnexpectedRollbackException: Transaction silently rolled back because it has been marked as rollback-only
```

The inner proxy's `TransactionInterceptor` saw a `RuntimeException` and, being a *participant*, could not roll back — so it marked the shared transaction **rollback-only**. Catching the exception in the outer method doesn't clear the flag. Fixes: `REQUIRES_NEW` for each row (its own transaction, committed or rolled back independently), `NESTED` (savepoint per row, JDBC only), or validate before writing so nothing throws. The opposite bug — catching an exception *inside* the transactional method without rethrowing — commits partial work because the interceptor never sees an exception.

16.10 Every way to bypass the proxy

Self-invocation



Inside the target, `this` is the target, not the proxy. `this.exportOne()` is a plain Java call, so `@Transactional(propagation = REQUIRES_NEW)`, `@Async`, `@Cacheable` and `@PreAuthorize` on `exportOne` are all ignored.

Fixes, best first:

```
// 1. Move the method to another bean (cleanest; usually reveals a real responsibility)
@Service @RequiredArgsConstructor
class ExportService {
    private final ExportStep step;                              // separate bean → called through its proxy
    @Transactional public void exportAll(List<UUID> ids) { ids.forEach(step::exportOne); }
}

// 2. Programmatic transaction — no proxy needed at all
private final TransactionTemplate tx;
public void exportAll(List<UUID> ids) {
    ids.forEach(id -> tx.executeWithoutResult(s -> exportOne(id)));
}

// 3. Self-injection (works, but a smell)
@Autowired @Lazy private ExportService self;   // the proxy
self.exportOne(id);

// 4. AopContext.currentProxy() with @EnableAspectJAutoProxy(exposeProxy = true) — couples code to AOP
// 5. AspectJ weaving mode — the call itself is woven
```

RefinePoint: `OutboxRelay.relay()` loops `tx.execute(status -> sendBatch())` until a batch comes back short. A `@Transactional sendBatch()` called from `relay()` would be self-invocation and silently run with *no* transaction — the `FOR UPDATE SKIP LOCKED` locks would be released immediately and two instances could send the same events. `TransactionTemplate` makes each batch its own transaction explicitly. `RunWorker.claim()` and `WorkspaceDataService.purge()` use `TransactionTemplate` for the same reason: the transaction boundary is a *part* of the method, not the whole of it.

Other bypasses

Situation	What happens	Why
`private` method with `@Transactional`	Ignored, no error	Proxies can't override private methods; `AnnotationTransactionAttributeSource` only considers public by default
`protected` / package-private	**Applied since Spring 6.0** for class-based (CGLIB) proxies	`publicMethodsOnly` default changed; JDK proxies still interface-only
`final` method on a CGLIB proxy	Not advised; the call runs on the **proxy instance itself**, whose fields are null → often a `NullPointerException`	Subclass can't override final; the proxy's own fields were never initialised
`final` class	Proxy creation fails at startup (CGLIB)	Can't subclass
`static` method	Never advised	Not dispatched through an instance
Object created with `new`	No proxy at all	Not a Spring bean
Called in `@PostConstruct`	No transaction	Proxy is created after init (BPP after-init)
`@Transactional` on interface method, CGLIB proxy	Works in Spring 6	Attribute source checks interfaces too; older versions were inconsistent
Checked exception thrown	Commits	Default rollback rule
Exception caught inside the method	Commits	Interceptor never sees it
Work on another thread	Outside the transaction	ThreadLocal-bound resources
Wrong transaction manager (two DataSources)	Transaction on the other database	Specify `@Transactional("billingTx")`

Trap: Kotlin classes are final by default — the `kotlin-spring` (all-open) plugin opens Spring-annotated classes. In Java, Lombok or records can bite: **records are final**, so a record can never be a CGLIB-proxied bean.

16.11 Proxies, `equals`, `getClass` and debugging

- `getClass()` on a CGLIB proxy returns the generated subclass (`JournalService$$SpringCGLIB$$0`). Use `AopUtils.getTargetClass(bean)` or `ClassUtils.getUserClass(bean)` for the real class. Code doing `bean.getClass().getAnnotation(...)` misses annotations unless they are `@Inherited` ; use `AnnotationUtils.findAnnotation` / `MergedAnnotations` .
- `equals / hashCode :` proxies delegate `equals` to the target only if the target implements `equals` ; JDK proxies treat two proxies as equal when their interfaces, advisors and targets are equal. Don't put beans in hash-based sets keyed by identity expectations.
- **Field access through a proxy:** `proxy.someField` (package-private access from the same package) reads the *proxy's* uninitialised field — always null. Always go through methods.
- `instanceof :` a JDK proxy is `instanceof` its interfaces only, not the class.
- **Hibernate proxies** are a different family (lazy loading of entities, `HibernateProxy`), with the same `getClass()` trap in `equals` — use `Hibernate.getClass(entity)` or `instanceof` .

Debugging a proxy problem

```
AopUtils.isAopProxy(bean);          // any Spring AOP proxy?
AopUtils.isCglibProxy(bean);
AopUtils.isJdkDynamicProxy(bean);
((Advised) bean).getAdvisors();      // which advisors, in what order
AopTestUtils.getTargetObject(bean);  // unwrap in tests
TransactionSynchronizationManager.isActualTransactionActive();   // inside the method: is there really a tx?
TransactionSynchronizationManager.getCurrentTransactionName();
```

```
# see transactions begin, join, commit and roll back
logging.level.org.springframework.transaction.interceptor=TRACE
logging.level.org.springframework.orm.jpa.JpaTransactionManager=DEBUG
# see which beans get proxied and why
logging.level.org.springframework.aop=DEBUG
```

In a debugger, a stack trace through a proxy shows `CglibAopProxy$DynamicAdvisedInterceptor.intercept` → `ReflectiveMethodInvocation.proceed` → `TransactionInterceptor.invoke` → your method. If `TransactionInterceptor` is missing from the trace, the call didn't go through the proxy.

16.12 Performance and design guidance

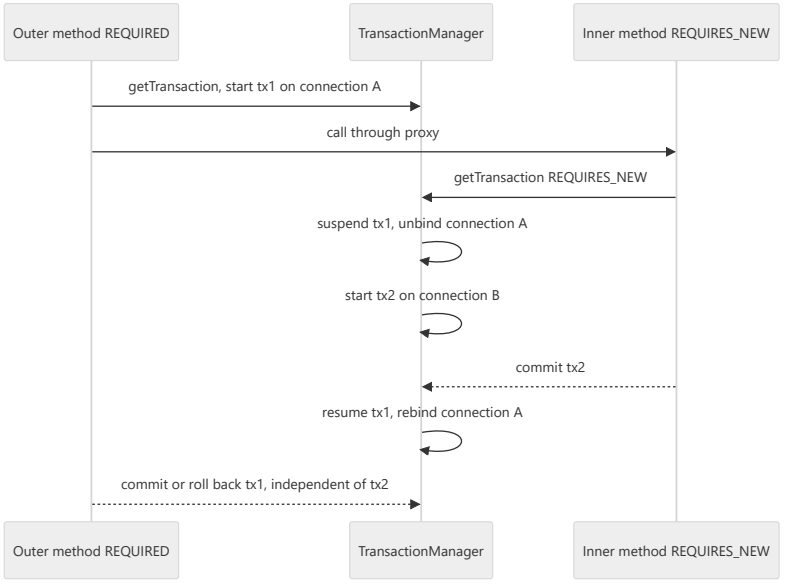
- A proxy hop plus a short interceptor chain costs nanoseconds to low microseconds — irrelevant next to a database round trip. Thousands of aspects with runtime-checked pointcuts (`args` , `target`) on hot paths is where it shows.
- Keep transactions **short**: no HTTP calls, no file uploads, no waiting on Kafka inside them. A transaction pins a pooled connection for its whole duration; ten slow external calls inside `@Transactional` can exhaust a Hikari pool of ten.

- Put `@Transactional` on **service methods**, not repositories (they already have their own) and not controllers (mixes web and transaction lifecycles).
- `@Transactional(readOnly = true)` on query services: Hibernate skips dirty checking and flush (`FlushMode.MANUAL`), and with read replicas a routing data source can send it to a replica. Several of our query services are annotated `@Transactional(readOnly = true)` at class level.
- Prefer explicit code (`TransactionTemplate` , explicit `audit.record`) when the boundary is part of an algorithm; prefer annotations when the boundary *is* the method.

16.13 Propagation as the interceptor sees it

Propagation is decided inside `getTransaction(attr)` — the interceptor asks "is there a transaction bound to this thread?" and acts per the attribute. (Chapter 21 covers isolation levels and database behaviour; here is what the *proxy* does.)

Propagation	Existing transaction on thread	No transaction	Typical use
`REQUIRED` (default)	Join it (participant)	Start new	Almost everything
`REQUIRES_NEW`	**Suspend** it, start a new physical one, resume after	Start new	Audit log that must survive rollback, per-item work in a batch
`NESTED`	Savepoint inside it (JDBC `DataSourceTransactionManager` ; JPA only with JDBC savepoint support)	Start new	Partial rollback of one step
`SUPPORTS`	Join	Run without	Read methods usable both ways
`NOT_SUPPORTED`	Suspend, run without	Run without	Long non-transactional work called from a transaction
`MANDATORY`	Join	**Throw** `IllegalTransactionStateException`	Methods that must never be the boundary
`NEVER`	**Throw**	Run without	Assert no transaction (rare)

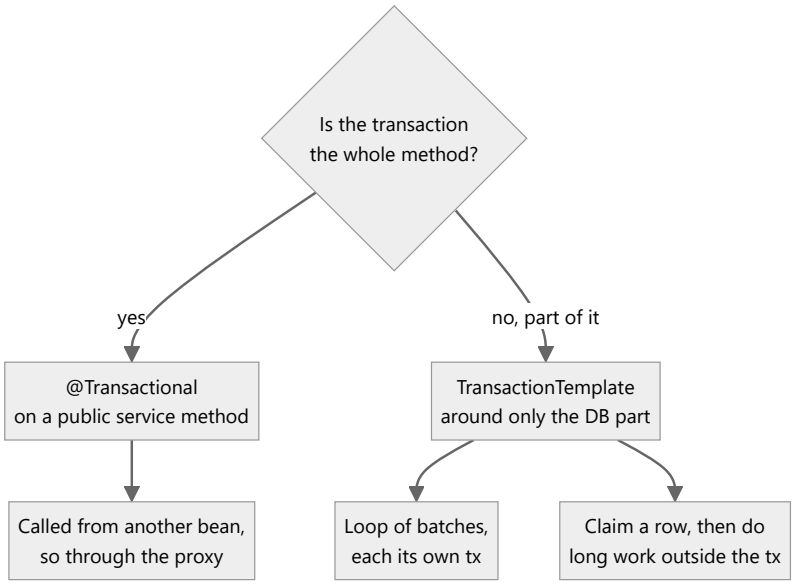


Three consequences seniors mention:

1. `REQUIRES_NEW` **needs a second pooled connection** while the first is suspended. Under load, if every request holds one connection and waits for a second, a pool of N connections deadlocks at N concurrent requests — the pool waits forever for itself. Size pools with this in mind, or avoid `REQUIRES_NEW` on hot paths.
2. **The inner commit is final.** If the outer transaction later rolls back, the inner work stays — which is exactly what you want for "record the failure" and exactly what you don't want for business data.
3. **Propagation only applies through the proxy.** A self-invoked `REQUIRES_NEW` method silently joins the outer transaction (Q56).

Participants and rollback-only: only the interceptor that *started* a physical transaction commits or rolls it back. A participant that sees an exception marks the shared transaction rollback-only (`globalRollbackOnParticipationFailure` , default `true`). That's the mechanism behind `UnexpectedRollbackException` .

16.14 Declarative vs programmatic transactions




```java
// Programmatic: the boundary is explicit and cannot be bypassed by self-invocation
@Component @RequiredArgsConstructor
class Claimer {
    private final TransactionTemplate tx;
    private final JdbcTemplate jdbc;

    UUID claimNext() {
        return tx.execute(status -> {
            List<UUID> ids = jdbc.queryForList(
                "SELECT id FROM recon_runs WHERE status = 'QUEUED' ORDER BY queued_at LIMIT 1 FOR UPDATE SKIP LOCKED", UUID.class);
            if (ids.isEmpty()) return null;
            jdbc.update("UPDATE recon_runs SET status = 'RUNNING', started_at = now() WHERE id = ?", ids.get(0));
            return ids.get(0);
        });   // committed here: the lock is released, the run is ours
    }
}
// The long engine work then runs outside any transaction, with its own short transactions for results.
```

	`@Transactional`	`TransactionTemplate`	`PlatformTransactionManager` directly
Boundary	Whole method	A lambda	Explicit begin/commit/rollback
Bypassed by self-invocation	Yes	No	No
Mixed with non-transactional work in one method	No	Yes	Yes
Boilerplate	None	Low	High, error-prone
Rollback	Rules on exception types	Exception in lambda, or `status.setRollbackOnly()`	Manual
Reactive	`@Transactional` on `Mono` / `Flux` with `ReactiveTransactionManager`	`TransactionalOperator`	—

RefinePoint: `RunWorker.claim()` is the real version of the code above: `tx.execute(...)` takes a PostgreSQL advisory transaction lock (`pg_advisory_xact_lock(hashtext('run-queue'))`) so one claim at a time holds the per-workspace cap exactly, selects the next run with `FOR UPDATE OF r SKIP LOCKED` , marks it `RUNNING` with this instance's name and commits — all in milliseconds. The engine then runs the reconciliation on a pool thread outside that transaction. If the whole `poll()` were `@Transactional` , the advisory lock and the row lock would be held for the entire reconciliation and every other instance would stall. This is the clearest example in our code of "the transaction boundary is part of the method, not the method".

16.15 Reactive and virtual-thread considerations

- **Reactive (WebFlux):** `@Transactional` works on methods returning `Mono` / `Flux` with a `ReactiveTransactionManager` (R2DBC). Transaction state lives in the Reactor `Context` , not a `ThreadLocal` , because a reactive pipeline hops threads. Blocking JPA inside a reactive pipeline gets no reactive transaction. Our api-gateway is reactive (Spring Cloud Gateway) and has no database transactions at all, which keeps it simple.
- **Virtual threads (`spring.threads.virtual.enabled=true`):** each request still runs on one (virtual) thread from start to finish, so `ThreadLocal` -based transactions, security context and request attributes work unchanged. What changes: `@Async` and `@Scheduled` executors use virtual threads too, and `synchronized` blocks around I/O can pin carrier threads on Java 21 — prefer `ReentrantLock` in aspects or interceptors that lock.

- **Context propagation:** for `@Async`, register a `TaskDecorator` (or Micrometer's context-propagation library) to copy MDC, security context and tracing into the task. It never copies the transaction — by design.

16.16 Interview questions and answers

Q1. What is AOP and why use it?

A way to modularise cross-cutting concerns — transactions, security, metrics, caching, retries — so they are written once and applied declaratively instead of being copy-pasted into every method. Business methods stay focused on business logic; the concern can be changed in one place.

Q2. Define aspect, join point, pointcut, advice, weaving.

Aspect: the module containing the concern. Join point: a point in execution — in Spring AOP only a bean method execution. Pointcut: a predicate selecting join points. Advice: the code run at a selected join point (before, after, around). Weaving: linking aspects to targets — Spring does it at runtime by creating proxies.

Q3. What advice types exist, and which is most powerful?

`@Before`, `@AfterReturning`, `@AfterThrowing`, `@After` (finally) and `@Around`. `@Around` is the most powerful: it decides whether to call `proceed()`, can change arguments and the result, and can catch exceptions. Use the weakest type that works, because `@Around` bugs (forgotten `proceed` or `return`) silently break methods.

Q4. Can `@AfterThrowing` swallow an exception?

No. It runs when the method throws and can throw a different exception, but the original propagates otherwise. To suppress or convert an exception into a return value you need `@Around` with a try/catch.

Q5. Spring AOP vs AspectJ?

Spring AOP uses runtime proxies, advises only public (or with CGLIB, non-private) method executions on Spring beans, and needs no build changes. AspectJ weaves bytecode at compile or load time and can advise constructors, field access, private and static methods, objects created with `new`, and self-invocation. Spring uses AspectJ's annotation syntax and pointcut parser but not its weaver.

Q6. When would you actually choose AspectJ weaving?

When you need advice on self-invocation, domain objects created with `new` (`@Configurable`), private methods or field access, or when per-call proxy overhead on an extremely hot path matters. Otherwise the build and agent complexity isn't worth it.

Q7. JDK dynamic proxy vs CGLIB?

JDK proxies implement the target's interfaces using `java.lang.reflect.Proxy` and an `InvocationHandler`; they need interfaces and the bean is only usable as the interface type. CGLIB generates a subclass at runtime, so it works without interfaces, can be injected as the concrete class, but cannot advise `final` methods or proxy `final` classes.

Q8. Which one does Spring Boot use by default?

CGLIB, for all beans: `spring.aop.proxy-target-class=true` has been Boot's default since 2.0. Plain Spring Framework uses JDK proxies when interfaces are present. Set the property to `false` to get JDK proxies for interface-based beans.

Q9. Why did Boot switch the default to CGLIB?

Injecting a bean by its class type failed with JDK proxies (`BeanNotOfRequiredTypeException`) whenever the class happened to implement any interface, which confused developers constantly. CGLIB proxies are type-compatible with the class, and modern CGLIB with Objenesis doesn't call the constructor twice.

Q10. Does a CGLIB proxy call `super.method()`?

No. Spring's CGLIB proxy is a subclass instance that holds a reference to a separate, fully initialised target and delegates to it. The proxy's inherited fields are never initialised — which is why final methods (executed on the proxy) see null fields.

Q11. When is the proxy created in the bean lifecycle?

In `postProcessAfterInitialization` of `AbstractAutoProxyCreator` (a `BeanPostProcessor`), after `@PostConstruct` and init methods. Earlier only if the bean is part of a circular reference, via `getEarlyBeanReference`. So code in `@PostConstruct` runs on the raw target.

Q12. A bean has `@Transactional`, `@PreAuthorize` and a custom aspect. How many proxies?

One. The auto-proxy creator collects all matching advisors into one proxy with an ordered interceptor chain. Post-processors like `AsyncAnnotationBeanPostProcessor` add their advisor to an existing proxy instead of wrapping again.

Q13. What is the self-invocation problem?

A call from one method of a bean to another method of the same bean via `this` goes directly to the target, bypassing the proxy, so annotations on the called method (`@Transactional` , `@Async` , `@Cacheable` , `@PreAuthorize`) have no effect. No error is reported.

Q14. How do you fix self-invocation?

Best: move the method to a separate bean. Or use programmatic control (`TransactionTemplate`). Or inject the bean's own proxy (`@Lazy` self-injection) or use `AopContext.currentProxy()` with `exposeProxy = true` — both work but couple the class to AOP. Or switch to AspectJ weaving.

Q15. Give a real case where you avoided self-invocation by design.

"Our `OutboxRelay` loops batches with `tx.execute(status -> sendBatch())` . Had I written `@Transactional sendBatch()` and called it from `relay()` , it would run with no transaction, the `FOR UPDATE SKIP LOCKED` row locks would release immediately, and two instances could send the same events. `TransactionTemplate` makes each batch its own transaction explicitly."

Q16. Does `@Transactional` work on private methods?

No — proxies cannot intercept private methods, and nothing warns you. Since Spring 6.0, `protected` and package-private methods *are* supported for class-based (CGLIB) proxies; earlier versions supported only public methods.

Q17. What happens with `@Transactional` on a `final` method?

The CGLIB subclass can't override it, so the call executes on the proxy instance directly — no transaction, and since the proxy's own fields are uninitialised, injected dependencies are null, typically causing a `NullPointerException` . Spring logs a warning at startup for final methods in proxied classes.

Q18. Can a record or final class be a proxied bean?

Not with CGLIB — creation fails because it can't be subclassed. A final class implementing an interface can be proxied with a JDK proxy if `proxy-target-class=false` , and then must be injected by the interface.

Q19. How is advice ordered across aspects?

By `@Order` / `Ordered` on the aspect class: lowest value = highest precedence = outermost, running first before and last after. Without an order the sequence is undefined. Infrastructure advisors have their own order (`@EnableTransactionManagement(order=...)`).

Q20. And within one aspect?

Since Spring 5.2.7: `@Around` , `@Before` , the method, `@AfterReturning` or `@AfterThrowing` , `@After` , end of `@Around` . Two advices of the same type in the same aspect have no defined order — split them into separate ordered aspects.

Q21. Why must a retry aspect be outside the transaction?

If the retry is inside, every attempt reuses the same transaction — which may be rollback-only already and has a persistence context holding the stale entity, so retries can never succeed. Outside, each attempt starts a fresh transaction with fresh data. Give the retry aspect higher precedence (lower order value).

Q22. Explain the `execution` pointcut syntax.

`execution(modifiers? return-type declaring-type? method-name(params) throws?)` . `*` matches one element, `..` in a package means any sub-package, `..` in parameters means any number of arguments. Example: `execution(public * com.ryfnflow..service.*Service.find*(java.util.UUID, ..))` .

Q23. `within` vs `execution` vs `@annotation` vs `@within` ?

`within` selects by type or package location. `execution` by method signature. `@annotation` by an annotation on the method. `@within` by an annotation on the declaring class. `@annotation(x)` can also bind the annotation instance to an advice parameter.

Q24. `this` vs `target` pointcuts?

`this(T)` matches when the **proxy** is an instance of T; `target(T)` when the **target** is. They differ with JDK proxies: the proxy implements only interfaces, so `this(ConcreteClass)` doesn't match while `target(ConcreteClass)` does.

Q25. How do you access method arguments and the annotation in advice?

Use `JoinPoint.getArgs()` and `getSignature()` , or bind them: `@Around("@annotation(timed) && args(orgId, ..)") Object x(ProceedingJoinPoint p, Timed timed, UUID orgId)` . Binding is type-safe and also filters by type. Compile with `-parameters` or specify `argNames` .

Q26. Write an `@Around` timing aspect — what are the common mistakes?

Call `pjp.proceed()` exactly once (unless retrying deliberately), return its result, rethrow exceptions (`throws Throwable`), and record in `finally` . Mistakes: returning `null` or void, swallowing exceptions, forgetting `@Component` so the aspect is never registered, pointcuts that are too broad.

Q27. Why might my aspect not apply?

The aspect isn't a bean (missing `@Component`), `spring-boot-starter-aop` isn't on the class path, the pointcut doesn't match (package typo, wrong annotation import), the target isn't a Spring bean, the call is a self-invocation, the method is private/final/static, or the bean was created too early to be proxied.

Q28. Can an aspect advise another aspect?

No. Classes annotated with `@Aspect` are excluded from auto-proxying by other aspects in Spring AOP. AspectJ weaving can, but it's rarely a good idea.

Q29. Where would you use an aspect vs a filter vs an interceptor?

Filter: raw HTTP concerns before Spring MVC — authentication, CORS, rate limits, request logging. Interceptor: MVC-level concerns needing the handler or path — our `OrgAccessInterceptor` checks the workspace role from the URL. Aspect: method-level concerns on any bean, web or not — transactions, metrics, method security.

Q30. Why don't you implement the audit trail as an aspect?

"Audit details need values computed mid-method (the external id returned by the accounting system), some actions are audited only on some branches, and an explicit `audit.record(...)` is visible in code review — nobody can silently disable it by renaming a method so the pointcut stops matching. It writes through the outbox in the same transaction, so audit and change commit together."

Q31. How does `@Transactional` work internally?

The bean is proxied; `TransactionInterceptor` reads the method's `TransactionAttribute` , asks the `PlatformTransactionManager` for a transaction according to propagation, binds the connection and `EntityManager` to the thread via `TransactionSynchronizationManager` , calls the method, then commits or rolls back according to rollback rules, running synchronization callbacks around commit.

Q32. How does a repository called inside a service join the same transaction?

Through the thread-bound resources. `JpaTransactionManager` binds an `EntityManagerHolder` and a `ConnectionHolder` to the current thread. Spring's shared `EntityManager` proxy and `DataSourceUtils.getConnection` (used by `JdbcTemplate`) look there first, so they use the same connection.

Q33. Which exceptions roll back by default?

Unchecked ones: `RuntimeException` and `Error` . Checked exceptions commit. Change with `rollbackFor` / `noRollbackFor` ; the most specific matching rule wins.

Q34. Why don't checked exceptions roll back?

Spring inherited the EJB convention: checked exceptions represent expected, recoverable business outcomes, unchecked ones represent failures. Many teams set `rollbackFor = Exception.class` as a convention, or avoid checked exceptions in service code.

Q35. Give an example where `noRollbackFor` was the correct design.

"In `JournalService.post` , if pushing a journal to the accounting system fails, we save the failure (`post_error` , target, connection) and throw `IllegalArgumentException("Posting failed: ...")` so the user gets a clear 400. The method is `@Transactional(noRollbackFor = IllegalArgumentException.class)` — otherwise the throw would roll back the very row that records why it failed."

Q36. What is `UnexpectedRollbackException` ?

Thrown when an outer transaction tries to commit but an inner participant (same physical transaction, `REQUIRED`) has marked it rollback-only after an exception the outer code caught. Spring rolls back and tells the caller its "commit" didn't happen. Fix with `REQUIRES_NEW` / `NESTED` for the inner unit or don't swallow the failure.

Q37. What happens if you catch an exception inside a `@Transactional` method?

The interceptor never sees it, so it commits whatever was done before — including partial writes. Catch only if you can genuinely recover, or call `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()` , or rethrow.

Q38. Does a transaction propagate to `@Async` methods or other threads?

No. Transaction resources are `ThreadLocal` -bound, so the new thread has no transaction (and no persistence context). `@Async` methods that need one must declare their own `@Transactional` , and should receive ids rather than managed entities.

Q39. `@Async` + `@Transactional` on the same method — what happens?

The async interceptor runs first (the call is handed to an executor), then on the executor thread the transaction interceptor starts a new transaction. It works, but the transaction is independent of the caller's — the caller may not have committed yet, so the async method may not see the caller's data. Combine with `@TransactionalEventListener(AFTER_COMMIT)` to start async work only after commit, as our `NotificationService.onAlertRaised` does.

Q40. How does `@Async` work, and what are its pitfalls?

`AsyncAnnotationBeanPostProcessor` proxies the bean; the interceptor submits the call to a `TaskExecutor` and returns immediately. Pitfalls: self-invocation runs synchronously, exceptions from `void` methods are only logged by `AsyncUncaughtExceptionHandler`, `ThreadLocal`s (security context, MDC, request attributes) are not propagated without a `TaskDecorator`, an unbounded queue can hide overload, and it requires `@EnableAsync`.

Q41. How does `@Cacheable` work, and why might it not cache?

`CacheInterceptor` computes a key from the arguments, returns a cached value on a hit without invoking the method, otherwise calls it and stores the result. It doesn't cache on self-invocation, without `@EnableCaching`, when `unless` / `condition` exclude it, or when keys differ unexpectedly (mutable arguments, missing `equals`). The default `ConcurrentMapCacheManager` never evicts.

Q42. How is `@PreAuthorize` implemented?

`@EnableMethodSecurity` registers advisors with `AuthorizationManagerBeforeMethodInterceptor`, which evaluates the SpEL expression against the current `Authentication` before invoking the method and throws `AccessDeniedException` if it fails. It runs before the transaction interceptor, so denied calls never open a transaction.

Q43. How do you do tenant-aware method security?

Expose a bean and call it from SpEL. "Ours is `@PreAuthorize("@orgAuth.can('journals.post')")`: `OrgAuth` is a bean named `orgAuth` that resolves the workspace of the current request (org code in the URL or `X-Org-Code`) and checks the caller's permission in that workspace, because Keycloak realm roles can't express per-workspace owners and admins."

Q44. What does `readOnly = true` actually do?

It's a hint: `JpaTransactionManager` sets Hibernate's flush mode to MANUAL (no dirty checking or flush), the JDBC connection gets `setReadOnly(true)` (drivers may optimise; PostgreSQL issues `SET TRANSACTION READ ONLY`, rejecting writes), and routing data sources can send it to a replica. It does not by itself prevent writes in every database.

Q45. Where should `@Transactional` go — controller, service, or repository?

On service methods representing a use case. Repositories already have their own (Spring Data's `SimpleJpaRepository` is `@Transactional(readOnly = true)` with write methods overridden). Controllers would tie the transaction to the HTTP lifecycle, including serialization, and encourage long transactions.

Q46. Why are long transactions dangerous?

Each holds a pooled connection, row locks and a growing persistence context for its whole duration. External HTTP calls or file processing inside a transaction can exhaust the connection pool and block other writers. In PostgreSQL long transactions also hold back vacuum. Keep I/O outside, and use `TransactionTemplate` around just the database part.

Q47. How do you check at runtime whether a transaction is active?

`TransactionSynchronizationManager.isActualTransactionActive()` and `getCurrentTransactionName()`; or `TRACE` logging for `org.springframework.transaction.interceptor` shows every "Getting transaction for [...]" and "Completing transaction". In a stack trace, look for `TransactionInterceptor.invoke`.

Q48. How do you tell whether a bean is a proxy, and get the real class?

`AopUtils.isAopProxy`, `isCglibProxy`, `isJdkDynamicProxy`; `AopUtils.getTargetClass(bean)` or `ClassUtils.getUserClass(bean)` for the class; cast to `Advised` to list advisors; `AopTestUtils.getTargetObject` in tests.

Q49. Why does `bean.getClass().getAnnotation(X.class)` return null for a proxied bean?

`getClass()` returns the generated CGLIB subclass, and annotations aren't inherited unless meta-annotated `@Inherited`. Use `AnnotationUtils.findAnnotation(AopUtils.getTargetClass(bean), X.class)` or `MergedAnnotations`.

Q50. How do proxies affect `equals` and `hashCode`?

A JDK proxy's `equals` considers two proxies equal when they have equal configuration (interfaces, advisors, target); otherwise it delegates to the target if the target implements `equals`. A CGLIB proxy delegates to the target's `equals`. Comparing a proxy with its raw target can be asymmetric, so avoid mixing raw and proxied references.

Q51. Is `@Transactional` on an interface honoured?

In Spring 6 yes, for both JDK and CGLIB proxies — the attribute source also searches interface methods. Spring's documentation still recommends annotating concrete classes, because interface annotations are invisible to AspectJ weaving and not inherited by Java rules.

Q52. What is the performance cost of proxies?

A proxy hop plus a cached interceptor chain: nanoseconds to a microsecond or so per call — negligible compared with I/O. It matters only for extremely hot, tiny methods or for broad pointcuts with runtime checks (`args` , `this` , `target`) on every bean. Scope pointcuts with `within` .

Q53. How does Spring choose the transaction manager when there are two databases?

By the `transactionManager` (or `value`) attribute: `@Transactional("billingTx")` . Without it Spring uses the single `PlatformTransactionManager` bean, or the `@Primary` one; with two and no primary it fails. Writing to two databases in one method gives two independent transactions — not atomic without XA.

Q54. What are `TransactionSynchronization` callbacks used for?

To run code at transaction phases: `beforeCommit` (Hibernate flushes), `beforeCompletion` , `afterCommit` , `afterCompletion` (resource cleanup). `@TransactionalEventListener` is built on them. Register manually with `TransactionSynchronizationManager.registerSynchronization` for "do this only if the transaction commits".

Q55. Design question: add per-tenant timing metrics to all service methods without touching them.

An `@Around` aspect on `within(com.ryfnflow.crm..service..*) && execution(public * *(..))` , using Micrometer `Timer` tagged by class, method and outcome — but take the tenant from a bounded source, not raw org ids, to avoid tag-cardinality explosion (or record tenant only in traces). Order it outermost so it measures transaction time too, and exclude health and scheduler beans. Or use `@Observed` with `ObservedAspect` and let tracing carry the tenant.

Q56. Tricky output: what does this print?

```
@Service class S {
    @Transactional public void a() { b(); }
    @Transactional(propagation = Propagation.REQUIRES_NEW) public void b() {
        System.out.println(TransactionSynchronizationManager.getCurrentTransactionName());
    }
}
s.a();
```

`S.a` (the fully qualified method name of `a`) — `b()` is self-invoked, so no new transaction starts and `b` runs inside `a` 's transaction. Called from another bean, it would print `S.b` .

Q57. Tricky: `@Transactional` method throws `IOException` after saving an entity. Is it saved?

Yes — `IOException` is checked, so the default rule commits. Add `rollbackFor = IOException.class` (or `Exception.class`) if the save must be undone.

Q58. Tricky: why is the transaction not rolled back when `@Transactional` is on a method of a class that `implements` a JDK-proxied interface, but the method isn't in the interface?

With a JDK proxy only interface methods go through the proxy; a method not declared on the interface can't even be called through the bean reference without casting to the class (which fails). If it's called internally from an interface method, it's self-invocation. Either declare it on the interface or use CGLIB (Boot's default).

Q59. What does `REQUIRES_NEW` do, and what is its hidden cost?

It suspends the current transaction (unbinding its resources from the thread) and starts an independent physical transaction that commits or rolls back on its own. The cost is a second pooled connection held while the first is suspended — under load a pool can deadlock waiting for itself. And it only works through the proxy.

Q60. `REQUIRES_NEW` vs `NESTED` ?

`REQUIRES_NEW` is a separate transaction: its commit survives an outer rollback, and it can't see the outer transaction's uncommitted changes. `NESTED` is a savepoint in the same transaction: rolling it back undoes only the nested part, but an outer rollback undoes everything. `NESTED` needs savepoint support (JDBC transaction manager); `JpaTransactionManager` supports it only for JDBC-level savepoints.

Q61. What does `MANDATORY` buy you?

A guarantee that the method is never the transaction boundary: called without an active transaction it throws `IllegalTransactionStateException` . Useful for repository-like helpers that must always be part of a caller's unit of work.

Q62. When do you choose `TransactionTemplate` over `@Transactional`?

When the transaction is only part of the method — claiming a row and then doing long work outside, looping batches that each commit, or retry loops — or where self-invocation would otherwise bite. "Our `RunWorker.claim()` and `OutboxRelay.relay()` both use it: the claim commits in milliseconds and the reconciliation runs outside any transaction."

Q63. How does `@Transactional` work in WebFlux?

With a `ReactiveTransactionManager` (R2DBC) the interceptor wraps the returned `Mono` / `Flux`, and transaction state is stored in the Reactor `Context` instead of a `ThreadLocal`, because the pipeline may switch threads. Blocking JPA calls inside a reactive chain don't participate. `TransactionalOperator` is the programmatic form.

Q64. Do virtual threads change how proxies and transactions behave?

No for request handling: one virtual thread per request keeps `ThreadLocal`-bound transactions and security working. Watch `synchronized` around blocking I/O (pins the carrier on Java 21), and remember `@Async` tasks still start without the caller's transaction or context unless a `TaskDecorator` copies the context.

Q65. How would you explain proxies to a junior who says "my `@Transactional` doesn't work"?

"The annotation does nothing by itself. Spring replaces your bean with a stand-in that opens the transaction and then calls your real object. Only calls that come *through the stand-in* get a transaction. So check three things: is the object a Spring bean, is the call coming from another bean (not `this`), and is the method public and not final. Then turn on `TRACE` logging for `org.springframework.transaction.interceptor` and look for 'Getting transaction for'."

17. Spring Boot Internals

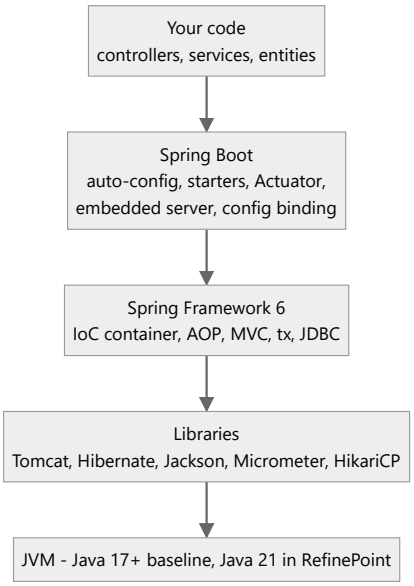
A senior interview rarely asks "what is Spring Boot?" and stops. The follow-ups are "what exactly happens between `main()` and the first request?", "how does Boot decide to create a `DataSource`?", "your property is ignored in production, how do you find out why?", and "how did you share configuration across your services?". This chapter walks the startup path, the auto-configuration engine, configuration binding, Actuator, packaging and the Boot 2 → 3 migration at the level where you can answer those follow-ups from memory, and uses RefinePoint's shared `ryfn-platform` starter as the worked example.

17.1 What Boot adds over plain Spring

Spring Framework gives you the IoC container, AOP, transactions, MVC and data access. It does not decide *which* beans you need. Before Boot, every project wrote the same `DataSource`, `EntityManagerFactory`, `DispatcherServlet` and `ObjectMapper` configuration, deployed a WAR to an external Tomcat, and pinned compatible library versions by hand.

Boot adds five things on top of the framework:

Boot feature	Problem it removes	Mechanism
Auto-configuration	Boilerplate bean definitions	`@Conditional` configuration classes listed in `AutoConfiguration.imports`
Starters	Picking compatible dependency sets	Dependency-only POMs + `spring-boot-dependencies` BOM
Embedded server	External container, WAR deployment	`ServletWebServerFactory` creates Tomcat/Jetty/Undertow inside the JVM
Externalized configuration	Hard-coded settings per environment	Ordered `PropertySource`s, profiles, `@ConfigurationProperties` binding
Production features	Ad-hoc health checks and metrics	Actuator endpoints, Micrometer, health groups, graceful shutdown



The key insight interviewers want: **Boot is not a different framework.** It is a set of `@Configuration` classes and a launcher. Everything it creates is an ordinary bean you could have declared yourself, and anything you declare wins over what Boot would have created (the "back-off" principle).

17.2 `@SpringBootApplication` decomposed

```
@Target(ElementType.TYPE) @Retention(RetentionPolicy.RUNTIME) @Documented @Inherited
@SpringBootConfiguration                    // = @Configuration, marks THE boot config (tests find it)
@EnableAutoConfiguration                    // = @AutoConfigurationPackage + @Import(AutoConfigurationImportSelector)
@ComponentScan(excludeFilters = {
    @Filter(type = FilterType.CUSTOM, classes = TypeExcludeFilter.class),
    @Filter(type = FilterType.CUSTOM, classes = AutoConfigurationExcludeFilter.class) })
public @interface SpringBootApplication { ... }
```

- **`@SpringBootConfiguration`** is `@Configuration` with one extra meaning: `@SpringBootTest` and slice tests search *up* the package tree for it to find the application's configuration. There must be exactly one.
- **`@EnableAutoConfiguration`** imports `AutoConfigurationImportSelector`, a `DeferredImportSelector`. "Deferred" is the important word: it runs *after* all your `@Configuration` classes are parsed, which is why `@ConditionalOnMissingBean` in an auto-configuration can see your beans and back off.
- **`@AutoConfigurationPackage`** registers the main class's package in a bean called `AutoConfigurationPackages`. JPA entity scanning and Spring Data repository scanning use it, which is why entities outside the main package are not found unless you add `@EntityScan`.
- **`@ComponentScan`** with no `basePackages` scans the package of the annotated class and below. `AutoConfigurationExcludeFilter` stops auto-configuration classes from being picked up twice if they happen to sit in a scanned package.

> **Trap:** putting the main class in the default package (no `package` statement) makes component scanning walk the entire classpath, including every jar. Startup becomes very slow or fails. Always put the main class in a root package such as `com.ryfnflow.crm.recon`.

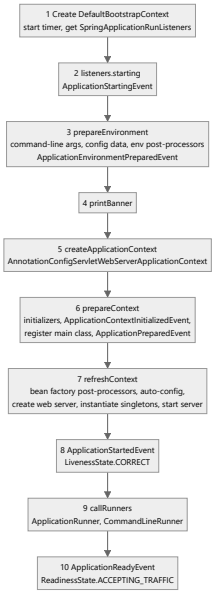
17.3 `SpringApplication.run()` step by step

`SpringApplication.run(App.class, args)` does two things: constructs a `SpringApplication`, then calls `run(args)`.

Construction (`new SpringApplication(primarySources)`):

1. Deduce the `WebApplicationType` from the classpath: `SERVLET` if `jakarta.servlet.Servlet` and `ConfigurableWebApplicationContext` are present, `REACTIVE` if only WebFlux's `DispatcherHandler` is present, otherwise `NONE`.
2. Load `BootstrapRegistryInitializer`s, `ApplicationContextInitializer`s and `ApplicationListener`s from `META-INF/spring.factories` across the classpath.
3. Deduce the main application class from the stack trace (used for logging and as the default source).

`run(args)`:



Step by step, with what matters in each:

1. **Bootstrap context.** A small `DefaultBootstrapContext` lives only until the `ApplicationContext` is ready. Config data loaders (for example a config-server client) register expensive objects here so they can be reused while loading configuration. Run listeners are loaded; the main one is `EventPublishingRunListener`, which turns each phase into an event.
2. **Starting.** `ApplicationStartingEvent` is published. Logging is initialised very early by `LoggingApplicationListener` reacting to this event.
3. **Environment.** An `ApplicationServletEnvironment` is created, command-line arguments are added as a property source, and `ApplicationEnvironmentPreparedEvent` triggers `EnvironmentPostProcessorApplicationListener`, which runs every `EnvironmentPostProcessor`. The most important one is `ConfigDataEnvironmentPostProcessor`, which loads `application.yml`, profile files and `spring.config.import` locations. `spring.main.*` properties are then bound onto the `SpringApplication` itself.
4. **Banner.** Printed to console/log unless `spring.main.banner-mode=off`.
5. **Context creation.** The type follows the web application type: `AnnotationConfigServletWebServerApplicationContext` for MVC, `AnnotationConfigReactiveWebServerApplicationContext` for WebFlux, `AnnotationConfigApplicationContext` otherwise.
6. **Prepare.** `ApplicationContextInitializer`s run, `ApplicationContextInitializedEvent` fires, the bootstrap context closes, the startup info line is logged ("Starting ReconApplication using Java 21..."), the `ApplicationArguments` singleton is registered, bean-definition overriding is set to *disallowed* (default since Boot 2.1), and the main class is registered as a bean definition. `ApplicationPreparedEvent` fires — bean definitions from sources exist, but nothing has been refreshed.
7. **Refresh** — the heart of the framework (`AbstractApplicationContext.refresh()`):
   - `invokeBeanFactoryPostProcessors`: `ConfigurationClassPostProcessor` parses your configuration, runs component scanning, processes `@Import`, and *last* the deferred `AutoConfigurationImportSelector`, evaluating conditions and registering auto-configured bean definitions.
   - `registerBeanPostProcessors`: autowiring, `@PostConstruct`, AOP proxy creators, `@ConfigurationProperties` binding post-processor.
   - `onRefresh`: the web context **creates** the embedded server (`ServletWebServerFactory.getWebServer`), initialising the servlet context but not yet accepting connections.
   - `finishBeanFactoryInitialization`: all non-lazy singletons are instantiated, injected, initialised and proxied.
   - `finishRefresh`: the lifecycle processor starts `SmartLifecycle` beans — including `WebServerStartStopLifecycle`, which **starts** the connector (port opens, `ServletWebServerInitializedEvent`) — then `ContextRefreshedEvent` is published. A JVM shutdown hook is registered.
8. **Started.** "Started ReconApplication in 4.2 seconds" is logged, `ApplicationStartedEvent` and `AvailabilityChangeEvent(LivenessState.CORRECT)` fire.
9. **Runners.** All `ApplicationRunner` and `CommandLineRunner` beans run, sorted by `@Order`, on the main thread. An exception here fails startup.

10. **Ready.** `ApplicationReadyEvent` and `AvailabilityChangeEvent(ReadinessState.ACCEPTING_TRAFFIC)` fire. The readiness probe turns green only now.

If anything throws, `ApplicationFailedEvent` is published, `FailureAnalyzer`s turn known exceptions into readable reports ("Port 8104 was already in use", "Parameter 0 of constructor required a bean of type ... that could not be found"), and the context is closed.

Event order summary (memorise it; it is a classic question):

#	Event	Context exists?	Typical use
1	`ApplicationStartingEvent`	No	Very early setup
2	`ApplicationEnvironmentPreparedEvent`	No	Inspect/modify environment
3	`ApplicationContextInitializedEvent`	Yes, no bean definitions loaded	Context tweaks
4	`ApplicationPreparedEvent`	Yes, definitions loaded, not refreshed	Rare
5	`WebServerInitializedEvent`	Refreshing	Learn the actual port (`server.port=0`)
6	`ContextRefreshedEvent`	Refreshed	Framework internals
7	`ApplicationStartedEvent` + Liveness CORRECT	Ready, runners not run	—
8	`ApplicationReadyEvent` + Readiness ACCEPTING_TRAFFIC	Fully ready	Warm caches, start consumers
—	`ApplicationFailedEvent`	Maybe	Alerting

Trap: a `@Component` with `@EventListener(ApplicationStartingEvent.class)` never fires. Events 1–4 are published before the context can contain your beans. Listeners for those events must be registered in `META-INF/spring.factories` under `org.springframework.context.ApplicationListener` or via `SpringApplication.addListeners(...)`.

17.4 The auto-configuration mechanism

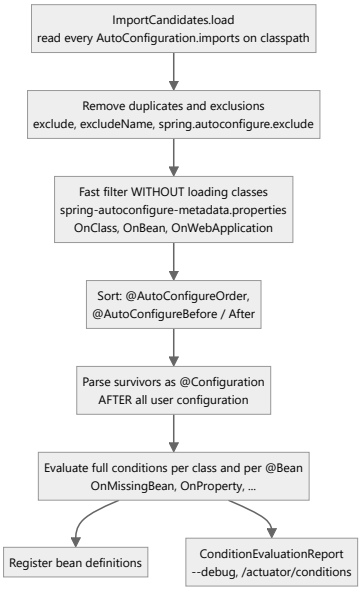
Where the candidates come from

Since Boot 2.7, auto-configuration classes are listed one per line in:

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

Boot 3.0 **removed** support for the old `org.springframework.boot.autoconfigure.EnableAutoConfiguration=` key in `META-INF/spring.factories`. `spring.factories` itself still exists for other extension points: `EnvironmentPostProcessor`, `ApplicationListener`, `FailureAnalyzer`, `ApplicationContextInitializer`, `TemplateAvailabilityProvider`. A library that only uses `spring.factories` for auto-configuration silently does nothing on Boot 3 — a very common migration bug.

`spring-boot-autoconfigure` alone lists about 150 classes. Evaluating every condition on every class by loading it would be slow, so Boot does it in two phases:



The metadata file `META-INF/spring-autoconfigure-metadata.properties` is generated at build time by `spring-boot-autoconfigure-processor`. It holds entries like `...DataSourceAutoConfiguration.ConditionalOnClass=javax.sql.DataSource,...EmbeddedDatabaseType`, so the filter can drop a class whose required classes are absent without even loading it.

The `@Conditional` family

Annotation	Matches when	Example
`@ConditionalOnClass` / `OnMissingClass`	Class is / is not on the classpath	Jackson config only if `ObjectMapper` exists
`@ConditionalOnBean` / `OnMissingBean`	A bean of that type/name exists / does not	Default `ObjectMapper` only if you did not define one
`@ConditionalOnSingleCandidate`	Exactly one bean, or one `@Primary`	`JdbcTemplate` needs a single `DataSource`
`@ConditionalOnProperty`	Property has a value (`havingValue` , `matchIfMissing`)	`rp.kafka.enabled=true`
`@ConditionalOnResource`	A resource exists	`classpath:logback.xml`
`@ConditionalOnWebApplication(type=SERVLET)` / `OnNotWebApplication`	Web type	MVC-only config
`@ConditionalOnExpression`	SpEL is true	Complex property combinations
`@ConditionalOnJava`	JVM version range	Version-specific features
`@ConditionalOnThreading(VIRTUAL)`	Virtual threads enabled (Boot 3.2)	Virtual-thread executors
`@ConditionalOnCloudPlatform`	Kubernetes, Cloud Foundry, ...	Probes on k8s

Under the hood every one is a `Condition` (most extend `SpringBootCondition` , which adds logging to the report). You can write your own and combine them with `AnyNestedCondition` , `AllNestedConditions` or `NoneNestedConditions` .

Class conditions on a `@Bean` method are dangerous. If a `@Bean` method's *return type* refers to a missing class, the configuration class itself fails to load when Spring introspects its methods. Put `@ConditionalOnClass` at class level, or on a nested static `@Configuration` class.

`@ConditionalOnMissingBean` only sees bean definitions registered so far. It is reliable in auto-configuration (which runs after user configuration) and unreliable in your own `@Configuration` classes, where order of parsing is not guaranteed.

Ordering

`@AutoConfiguration(after = DataSourceAutoConfiguration.class)` (or `@AutoConfigureAfter` / `@AutoConfigureBefore` , and `@AutoConfigureOrder` for a numeric order) control the order in which *auto-configurations* are parsed. They have no effect on user `@Configuration` classes and do **not** control bean *instantiation* order — that comes from dependencies. Ordering matters for conditions: a `@ConditionalOnBean(DataSource.class)` only sees the `DataSource` if `DataSourceAutoConfiguration` was processed first.

Debugging and excluding

```
java -jar recon.jar --debug          # or debug=true in application.yml
```

prints the **CONDITIONS EVALUATION REPORT**:

```
Positive matches:
   DataSourceAutoConfiguration matched:
      - @ConditionalOnClass found required classes 'javax.sql.DataSource', ... (OnClassCondition)
Negative matches:
   MongoAutoConfiguration:
      Did not match:
         - @ConditionalOnClass did not find required class 'com.mongodb.client.MongoClient' (OnClassCondition)
Exclusions:
   org.springframework.boot.autoconfigure.security.servlet.UserDetailsServiceAutoConfiguration
Unconditional classes:
   org.springframework.boot.autoconfigure.context.ConfigurationPropertiesAutoConfiguration
```

At runtime the same report is at `/actuator/conditions` ; `/actuator/beans` lists every bean with its dependencies and `/actuator/configprops` shows bound property values (sanitised).

Excluding:

```
@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class, KafkaAutoConfiguration.class })
```

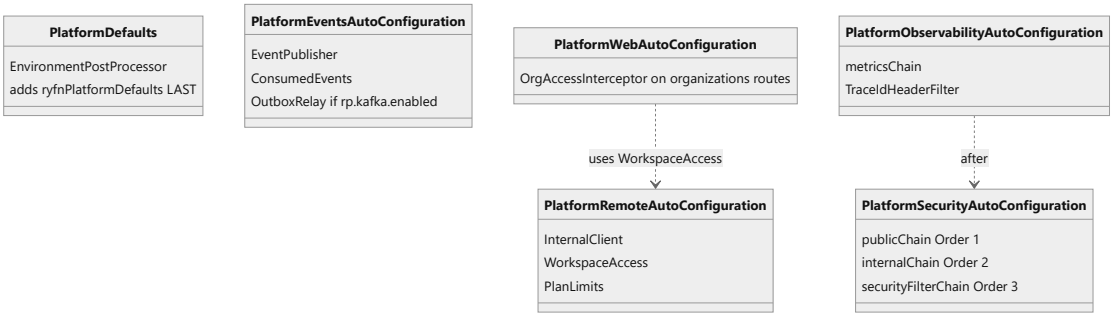
```
spring:
  autoconfigure:
    exclude:
      - org.springframework.boot.autoconfigure.security.servlet.UserDetailsServiceAutoConfiguration
```

The property form is better when only some environments need the exclusion (put it in a profile document).

17.5 Writing a custom starter — RefinePoint's `ryfn-platform`

The official layout for a starter named `acme` is two modules: `acme-spring-boot-autoconfigure` (the configuration classes) and `acme-spring-boot-starter` (an empty jar whose POM pulls in the autoconfigure module plus the libraries it needs). Third-party starters must **not** start with `spring-boot-` — that prefix is reserved for Spring's own. Small teams often ship one jar that is both, and that is what we did.

> **RefinePoint:** all 13 services depend on one shared library, `ryfn-platform`. Its `AutoConfiguration.imports` lists six classes: `PlatformSecurityAutoConfiguration`, `PlatformRemoteAutoConfiguration`, `PlatformEventsAutoConfiguration`, `PlatformWebAutoConfiguration`, `PlatformOpenApiAutoConfiguration` and `PlatformObservabilityAutoConfiguration`. Adding the dependency gives a service Keycloak JWT + API-key security, the `InternalClient` for service-to-service calls, the transactional outbox and Kafka relay, the `OrgAccessInterceptor` for org-scoped routes, Swagger and tracing — without one line of configuration in the service.



The real code shows the patterns interviewers probe:

```
@AutoConfiguration(after = { DataSourceAutoConfiguration.class, HibernateJpaAutoConfiguration.class,
                             KafkaAutoConfiguration.class, FlywayAutoConfiguration.class })
public class PlatformEventsAutoConfiguration {

    @Bean @ConditionalOnMissingBean          // a service may replace it
    public EventPublisher eventPublisher(EventStore store, ObjectMapper mapper /* ... */) { ... }

    @Bean @ConditionalOnMissingBean
    public ConsumedEvents consumedEvents(EventStore store, ObjectMapper mapper) { ... }

    @Bean
    @ConditionalOnProperty(name = "rp.kafka.enabled", havingValue = "true", matchIfMissing = true)
    public OutboxRelay outboxRelay(EventStore store, KafkaTemplate<String, String> kafka, TopicNames topics /* ... */) { ... }
}
```

1. **`after = ...`** guarantees the `DataSource`, `JdbcTemplate` and `KafkaTemplate` definitions exist before our conditions and beans that need them are evaluated.
2. **`@ConditionalOnMissingBean`** on every default bean: a service can override any piece by declaring its own bean.
3. **`@ConditionalOnProperty(matchIfMissing = true)`**: with `KAFKA_ENABLED=false` (tests, or one service running alone on a laptop) the relay is not created and events simply wait in `outbox_events`.

Shared defaults come from an `EnvironmentPostProcessor`, registered in `META-INF/spring.factories` (the one place Boot 3 still uses that file for us):

```
org.springframework.boot.env.EnvironmentPostProcessor=com.ryfnflow.crm.common.autoconfig.PlatformDefaults
```

```
public class PlatformDefaults implements EnvironmentPostProcessor {
    @Override
    public void postProcessEnvironment(ConfigurableEnvironment env, SpringApplication app) {
        Map<String, Object> d = new LinkedHashMap<>();
        d.put("spring.datasource.url",
            "jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:non_prd_db}?currentSchema=${rp.schema}");
        d.put("spring.jpa.open-in-view", "false");
        d.put("spring.jpa.hibernate.ddl-auto", "validate");
        d.put("management.endpoints.web.exposure.include", "health,info,metrics,prometheus");
        d.put("management.endpoint.health.probes.enabled", "true");
        d.put("spring.kafka.producer.acks", "all");
        // ... about 70 defaults
        env.getPropertySources().addLast(new MapPropertySource("ryfnPlatformDefaults", d));
    }
}
```

`addLast` makes these the **lowest-priority** source, so a service's own `application.yml`, an environment variable or a command-line argument always wins. Each service then needs only `spring.application.name`, `rp.schema` and its port.

> **RefinePoint:** `PlatformSecurityAutoConfiguration` declares the `ApiKeyAuthenticationFilter` as a bean so it can be injected into the security chains — but Boot automatically registers *every* `Filter` bean with the servlet container too, so it would run twice (once in the main filter chain, once inside Spring Security). We add a `FilterRegistrationBean` with `setEnabled(false)` for each such filter. That is a real bug custom starters hit.

> **Trap:** the platform jar is copied *inside* each service's fat jar at build time. After changing `ryfn-platform`, every service must be rebuilt with `mvn clean package`; a service that was not rebuilt keeps running the old platform code, even though the source looks right.

Testing an auto-configuration without starting an app uses `ApplicationContextRunner`:

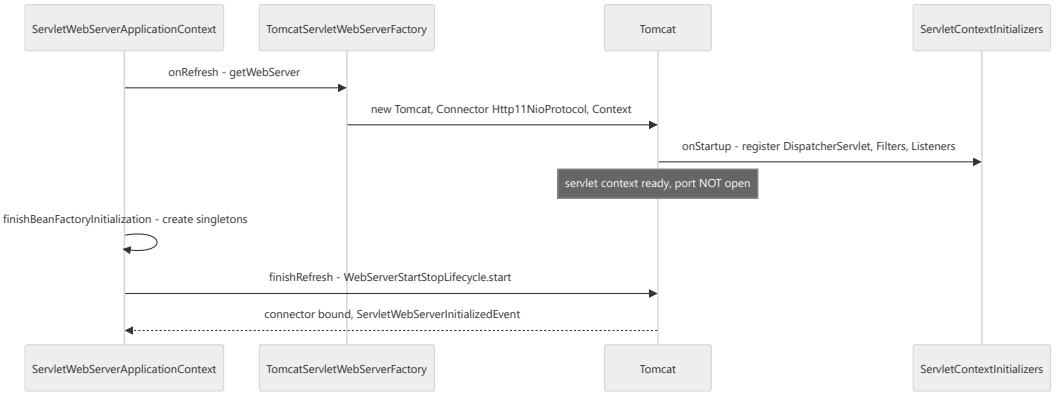
```
new ApplicationContextRunner()
    .withConfiguration(AutoConfigurations.of(PlatformEventsAutoConfiguration.class))
    .withPropertyValues("rp.kafka.enabled=false")
    .run(ctx -> assertThat(ctx).doesNotHaveBean(OutboxRelay.class));
```

17.6 Starters and dependency management

A starter is a POM with dependencies and no code: `spring-boot-starter-web` pulls `spring-boot-starter` (core, logging, YAML), `spring-boot-starter-json` (Jackson), `spring-boot-starter-tomcat`, and `spring-webmvc`. Versions come from the `spring-boot-dependencies` BOM, inherited through `spring-boot-starter-parent` or imported with `<scope>import</scope>` in `dependencyManagement`. The parent also configures resource filtering, compiler `-parameters` (needed for constructor binding and `@PathVariable` without names), and the `spring-boot-maven-plugin`.

To override a managed version, set the property the BOM uses: `<jackson-bom.version>2.18.2</jackson-bom.version>`. Overriding one artifact's version directly breaks alignment with its siblings.

17.7 The embedded server



Boot does not use `web.xml` or the Servlet container's `ServletContainerInitializer` scanning. It registers servlets and filters through `ServletContextInitializer` beans: `DispatcherServletRegistrationBean`, every `FilterRegistrationBean`, and any raw `Filter` / `Servlet` bean wrapped automatically. `server.*` properties are applied by `WebServerFactoryCustomizer` beans; you can add your own.

Switching server: exclude `spring-boot-starter-tomcat` from `spring-boot-starter-web` and add `spring-boot-starter-jetty` (or `-undertow`). The factory auto-configuration's `@ConditionalOnClass` picks whichever is present. (Boot 4 drops Undertow, so Jetty is the safer alternative.)

Tomcat threading model (NIO connector):

```
clients ──► Acceptor thread ──► Poller thread(s) (NIO selector)
                                        │ socket readable
                                        ▼
                    Executor: http-nio-8104-exec-1 .. exec-200
                    (one worker per in-flight request, blocking servlet API)
Pending connections beyond max-connections wait in the OS backlog (accept-count)
```

Property	Default	Meaning
`server.tomcat.threads.max`	200	Max worker threads = max concurrent requests being processed
`server.tomcat.threads.min-spare`	10	Threads kept alive when idle
`server.tomcat.max-connections`	8192	Open connections NIO will keep (most idle, keep-alive)
`server.tomcat.accept-count`	100	OS backlog queue when max-connections is reached
`server.tomcat.connection-timeout`	60s (Tomcat's built-in default; the 20s you may remember is from a standalone `server.xml`)	Time to wait for the request line after connect
`spring.threads.virtual.enabled`	false	Boot 3.2+: Tomcat uses a virtual thread per request

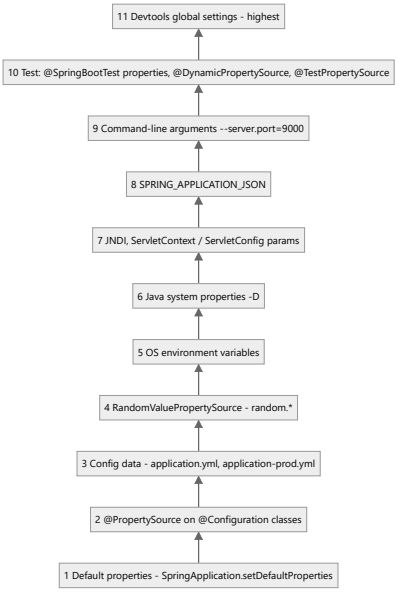
Tuning rules: the thread count should match what downstream resources can absorb. With a Hikari pool of 10 connections, 200 threads all doing DB work mostly wait on the pool (`connection-timeout` 30 s) — raising threads does not help. With virtual threads, the thread cap disappears, so the **connection pool and downstream limits become the only backpressure**; size them deliberately and add bulkheads/semaphores for slow dependencies.

> **RefinePoint:** `PlatformDefaults` sets `DB_POOL_SIZE` to 5 per instance by default, which keeps total connections modest across 13 services on one PostgreSQL. Run capacity scales by adding run-service instances, not by adding Tomcat threads.

17.8 Externalized configuration

Precedence order

Later sources override earlier ones (Boot 3 documentation order, lowest first):



Within config data, the order (lowest first) is: `application.yml` inside the jar → `application-{profile}.yml` inside the jar → `application.yml` outside the jar → `application-{profile}.yml` outside the jar. Default search locations are `classpath:/`, `classpath:/config/`, `file:./`, `file:./config/`, `file:./config/*/`, later ones winning. If both `.properties` and `.yml` exist in the same location, `.properties` wins.

> **Trap:** `@PropertySource` files are added during refresh — too late for `logging.*`, `spring.main.*` and `spring.config.*`, which are read before refresh. Use `spring.config.import` instead.

Relaxed binding maps `spring.datasource.hikari.maximum-pool-size`, `spring.datasource.hikari.maximumPoolSize` and `SPRING_DATASOURCE_HIKARI_MAXIMUMPOOLSIZE` to the same property. For environment variables: replace `.` with `_`, remove `-`, uppercase; list index `rp.security.public-paths[0]` becomes `RP_SECURITY_PUBLICPATHS_0`. `@Value` supports relaxed binding only when you write the canonical kebab-case form.

Placeholders with defaults (`${DB_HOST:localhost}`) resolve against the whole environment at the time the property is read, which is how `PlatformDefaults` can reference environment variables that each deployment sets.

Profiles and profile groups

```yaml
spring:
  application.name: run-service
  profiles:
    group:
      prod: "prodDb,prodKafka,json-logs"     # activating prod activates all three
---
spring.config.activate.on-profile: local
rp.kafka.enabled: false
logging.level.com.ryfnflow: DEBUG
---
spring.config.activate.on-profile: prod
server.tomcat.threads.max: 100
```

- Activate with `--spring.profiles.active=prod` or `SPRING_PROFILES_ACTIVE=prod`; `spring.profiles.default` applies when none is active (default `default`).
- Since Boot 2.4, `spring.profiles.active` and `spring.profiles.include` **cannot** appear in a profile-specific document (one activated with `on-profile`); use groups instead. This prevents profile activation cycles.
- `@Profile("!prod")` on a bean or configuration class; `Environment.acceptsProfiles(Profiles.of("prod | staging"))` in code.

Keep profiles for *environment shape* (which beans exist), not for every value; values belong in environment variables or a secret store. Twelve-factor style: one artifact, configuration from the environment.

`spring.config.import`

```yaml
spring:
  config:
    import:
      - optional:file:./.env[.properties]     # local secrets, if present
      - optional:configtree:/run/secrets/     # Docker/Kubernetes secret files: file name = property name
      - optional:configserver:http://config:8888
```

Imports are processed by `ConfigDataEnvironmentPostProcessor` before the rest of the configuration is resolved, so they can affect profiles and logging. `optional:` prevents startup failure when the location is missing.

`@ConfigurationProperties`

```java
@Validated
@ConfigurationProperties(prefix = "app.storage")
public record StorageProperties(
        @NotBlank String provider,                // s3 | azure | local
        @NotNull Duration timeout,                // "60s" converts automatically
        @DefaultValue("4") @Min(1) int maxConcurrent,
        DataSize maxUpload) { }                   // "20MB"

@SpringBootApplication
@ConfigurationPropertiesScan                      // or @EnableConfigurationProperties(StorageProperties.class)
public class App { }
```

How it works: `ConfigurationPropertiesBindingPostProcessor` uses the `Binder` to walk `ConfigurationPropertySource`s (a relaxed-name view of the environment), converting via the `ConversionService` (`Duration`, `DataSize`, `Period`, enums case-insensitive, comma lists). Records and classes with a single constructor use **constructor binding** automatically in Boot 3 (`@ConstructorBinding` is needed only to pick one of several constructors). JavaBean binding uses setters.

With `@Validated` and a Jakarta Validation provider on the classpath, invalid configuration **fails startup** with a clear report — far better than discovering a blank API key at the first call. Add `spring-boot-configuration-processor` (annotation processor) to generate `META-INF/spring-configuration-metadata.json`, which gives IDE auto-completion and documentation for your properties.

	`@Value`	`@ConfigurationProperties`
Relaxed binding	Limited	Full
Type-safe groups, nested objects, lists, maps	No	Yes
Validation	No	`@Validated`
SpEL	Yes (`#{...}`)	No
IDE metadata	No	Yes
Best for	One-off value	Any group of settings

17.9 Actuator

Actuator adds operational endpoints under `/actuator`. Since Boot 2.5 only `health` is exposed over HTTP by default. JMX is a separate switch (`spring.jmx.enabled`) and is off by default.

Endpoint	Purpose
`health`	Aggregated status of `HealthIndicator`s; liveness/readiness groups
`info`	Build info, git info, custom `info.*`
`metrics`, `prometheus`	Micrometer meters; Prometheus scrape format
`env`, `configprops`	Property sources and bound values (sanitised; values hidden by default in Boot 3)
`beans`, `conditions`, `mappings`	Beans, auto-config report, request mappings
`loggers`	Read and **change** log levels at runtime
`threaddump`, `heapdump`	Diagnostics (protect heapdump: it contains secrets)
`startup`	Startup steps (with `BufferingApplicationStartup`)
`scheduledtasks`, `caches`, `flyway`, `httpexchanges`, `sbom`	Specific subsystems
`shutdown`	Graceful shutdown over HTTP; disabled by default

Health and health groups

`HealthEndpoint` collects every `HealthIndicator` bean (`db` via a validation query, `diskSpace`, `kafka`-related, `redis`, `ping`) and aggregates with a `StatusAggregator` (DOWN > OUT_OF_SERVICE > UP > UNKNOWN). Overall DOWN maps to HTTP 503.

```yaml
management:
  endpoint:
    health:
      probes.enabled: true          # /actuator/health/liveness and /readiness (auto on Kubernetes)
      show-details: when-authorized
      group:
        readiness:
          include: readinessState, db
```

Liveness must not include external dependencies. If the database blips and liveness checks the database, Kubernetes restarts every pod at once — turning a database hiccup into a full outage. Liveness = "is this JVM broken?"; readiness = "should I receive traffic?" (DB and critical downstreams can go here).

Custom indicator:

```java
@Component("tempSpace")
class TempSpaceHealthIndicator implements HealthIndicator {
    private final Path tempDir;
    TempSpaceHealthIndicator(@Value("${app.temp-dir}") Path tempDir) { this.tempDir = tempDir; }

    @Override public Health health() {
        long freeMb = tempDir.toFile().getUsableSpace() / 1_048_576;
        return freeMb > 2048
            ? Health.up().withDetail("tempFreeMb", freeMb).build()
            : Health.down().withDetail("tempFreeMb", freeMb).withDetail("reason", "spill space low").build();
    }
}
```

Metrics with Micrometer

Micrometer is a facade (like SLF4J for metrics). Boot auto-configures a composite `MeterRegistry` plus one registry per backend on the classpath (`micrometer-registry-prometheus` , OTLP, Datadog). Instrumentation out of the box: `http.server.requests` , `http.client.requests` (for `RestClient` / `WebClient` built from Boot's builders), JVM memory/GC/threads, Hikari pool, Tomcat sessions, Kafka clients, executor and cache metrics.

```java
@Service
class MatchMetrics {
    private final Counter breaks;
    private final Timer runTimer;
    MatchMetrics(MeterRegistry registry) {
        breaks = Counter.builder("recon.breaks").tag("type", "amount").register(registry);
        runTimer = Timer.builder("recon.run.duration").publishPercentileHistogram().register(registry);
    }
    <T> T timeRun(Supplier<T> run) { return runTimer.record(run); }
}
```

Boot 3 introduced the **Observation API**: one `Observation` produces both a timer metric and a trace span, and Micrometer Tracing replaced Spring Cloud Sleuth. Trace and span ids are put in the logging MDC automatically.

> **Trap:** high-cardinality tags (user id, org code, raw URL with ids) create one time series per value and can take down Prometheus. Use templated URIs (Boot does: `uri=/api/v1/organizations/{code}/runs/{id}`) and bounded tag values.

> **RefinePoint:** `PlatformDefaults` tags every metric with `application` and `environment` , enables percentile histograms for server and client requests, samples 10% of traces, and exports spans over OTLP only when `OTEL_EXPORTER_OTLP_ENDPOINT` is set. `PlatformObservabilityAutoConfiguration` adds a dedicated security chain that lets Prometheus scrape `/actuator/prometheus` with a metrics token, and a filter that returns the trace id in an `X-Trace-Id` response header so a support ticket can quote it.

Securing Actuator

Options, usually combined: expose only what you need (`management.endpoints.web.exposure.include`), put management on a separate port not routed by the gateway (`management.server.port=9001`), and secure the endpoints with a dedicated `SecurityFilterChain` using `EndpointRequest.toAnyEndpoint()` . Never expose `env` , `heapdump` or `configprops` publicly — `heapdump` contains every secret in memory.

17.10 Logging

Boot uses Commons Logging internally and **Logback via SLF4J** by default (`spring-boot-starter-logging`); swap to Log4j2 by excluding it and adding `spring-boot-starter-log4j2` .

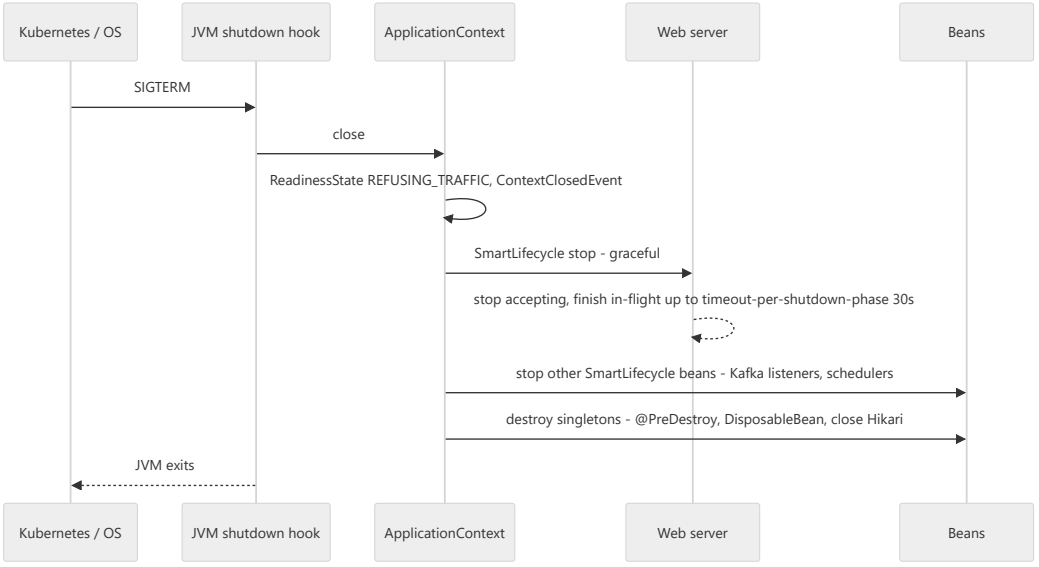
- Levels: `logging.level.root=INFO` , `logging.level.org.hibernate.SQL=DEBUG` ; groups: `logging.group.recon=com.ryfnflow.crm.recon,com.ryfnflow.engine` .
- File: `logging.file.name` , rolling with `logging.logback.rollingpolicy.max-file-size` .
- `logback-spring.xml` (not `logback.xml`) lets Boot initialise it and enables `<springProfile name="prod">` and `<springProperty>` .
- **Structured logging (Boot 3.4):** `logging.structured.format.console=ecs` (or `logstash` , `gelf`) writes one JSON object per line. RefinePoint switches this on with `LOG_FORMAT=json` .
- **Change level at runtime** without restart:

```
curl -X POST localhost:8104/actuator/loggers/com.ryfnflow.crm.recon \
     -H 'Content-Type: application/json' -d '{"configuredLevel":"DEBUG"}'
```

This changes only the instance you call; in a multi-instance deployment you call each instance (or use a config-refresh mechanism).

17.11 Graceful shutdown

Boot 3.4 enables **graceful shutdown by default** for all embedded servers (`server.shutdown=graceful` ; in earlier versions you had to set it). The sequence on `SIGTERM` :



`spring.lifecycle.timeout-per-shutdown-phase` (default 30 s) bounds each lifecycle phase. On Kubernetes, the pod is removed from endpoints *asynchronously*, so add a `preStop` sleep of a few seconds to avoid new requests arriving after the server stopped accepting. Background work must also be graceful: a DB-queue worker should stop claiming new rows and let the current one finish or release it.

> **RefinePoint:** run-service workers claim a run with `FOR UPDATE SKIP LOCKED` and commit the claim (`status = 'RUNNING'` , `instance` , `heartbeat_at`), so no lock is held while the run executes. On a normal stop, `RunWorker` 's `@PreDestroy` puts that instance's runs back to QUEUED for another instance. If the JVM is killed, `@PreDestroy` never runs: the 15-second heartbeat goes stale and a reaper on another instance fails any run silent for two minutes, so no run stays RUNNING forever. Design for the kill, not only for the polite stop.

17.12 Fat jar layout and layered jars

```
recon-service.jar
├── META-INF/MANIFEST.MF
│     Main-Class: org.springframework.boot.loader.launch.JarLauncher   (Boot 3.2+ package)
│     Start-Class: com.ryfnflow.crm.recon.ReconApplication
│     Spring-Boot-Classes: BOOT-INF/classes/
│     Spring-Boot-Lib: BOOT-INF/lib/
├── org/springframework/boot/loader/...     ← the launcher, stored uncompressed at the root
├── BOOT-INF/classes/                       ← your classes and application.yml
├── BOOT-INF/lib/                           ← every dependency jar, nested, NOT unpacked
│     ryfn-platform-1.0.jar, spring-webmvc-6.2.x.jar, ...
├── BOOT-INF/classpath.idx                  ← jar order on the classpath
└── BOOT-INF/layers.idx                     ← which file goes in which Docker layer
```

The JVM cannot load classes from a jar inside a jar. `JarLauncher` creates a class loader with a custom `jar:nested:` URL handler that reads nested entries directly (they are stored uncompressed), then calls `Start-Class.main()` reflectively on a new thread context class loader.

Layered jars split the content into layers ordered by change frequency: `dependencies` , `spring-boot-loader` , `snapshot-dependencies` , `application` . In a Dockerfile:

```
java -Djarmode=tools -jar app.jar extract --layers --launcher --destination extracted   # Boot 3.3+
```

then `COPY` each layer separately. Code changes rebuild only the small `application` layer; the 80 MB of dependencies stay cached in the registry and on nodes. Cloud Native Buildpacks (`mvn spring-boot:build-image`) do this automatically.

17.13 Boot 2 → 3 migration

Area	Boot 2.x	Boot 3.x
Java baseline	8 / 11	**17** (21 recommended)
Jakarta EE	`javax.*`	`jakarta.*` (Servlet 6, JPA 3.1, Validation 3)
Spring Framework	5.3	6.x
Hibernate	5.x	6.x (new SQM query engine, sequence naming changes)
Auto-config registration	`spring.factories`	`AutoConfiguration.imports` only
Security	`WebSecurityConfigurerAdapter` (deprecated 5.7)	`SecurityFilterChain` beans, lambda DSL, `requestMatchers`
Tracing	Spring Cloud Sleuth	Micrometer Tracing + Observation API
URL matching	`/runs/` matched `/runs`	Trailing slash match disabled
Errors	Custom bodies	`ProblemDetail` (RFC 7807/9457) support
HTTP clients	`RestTemplate`, `WebClient`	+ `RestClient` (3.2), HTTP interfaces
Native	Experimental (Spring Native)	AOT + GraalVM native built in

Migration path that works: upgrade to the latest 2.7 first and remove deprecation warnings, add `spring-boot-properties-migrator` temporarily (it logs renamed properties at startup), move to Java 17, then switch to 3.x and run the OpenRewrite `UpgradeSpringBoot_3_x` recipe for `javax` → `jakarta`. Check every third-party library for a Jakarta-compatible version — that is usually the long pole.

17.14 AOT, native images and CRaC

Spring AOT runs at build time: it starts the application context *in analysis mode* with the build classpath, evaluates conditions, and generates Java source for bean definitions (plain `new` and method calls instead of reflection), plus `RuntimeHints` (reflection, resources, proxies, serialization) for GraalVM.

GraalVM native image compiles the application into a platform binary under a **closed-world assumption**: everything reachable must be known at build time.

	JVM	Native image
Startup	Seconds	Tens of milliseconds
Memory	Higher	Much lower
Peak throughput	Best (JIT)	Lower without PGO
Build time	Fast	Minutes, memory-hungry
Reflection/dynamic proxies	Free	Must be declared (hints)
`@Profile` / `@ConditionalOnProperty`	Evaluated at runtime	Fixed at build time

Native fits serverless, CLIs and scale-to-zero. Long-running services under steady load (like reconciliation workers) are better on the JVM. Middle-ground options: **CDS/AppCDS** (Boot 3.3 supports extracting and training a class-data archive, roughly 30–40% faster startup), **AOT on the JVM** (`-Dspring.aot.enabled=true`) and **CRaC** (Boot 3.2: checkpoint a warmed JVM and restore it).

17.15 DevTools and startup optimisation

DevTools (scope `developmentOnly`/optional, excluded from the fat jar) uses two class loaders: a *base* loader for jars that do not change and a *restart* loader for your classes. On a class change, only the restart loader is thrown away, so a restart takes about a second. It also disables template caching, enables LiveReload and applies developer-friendly defaults. It is disabled automatically when running from a fat jar.

Startup time checklist:

1. Measure first: `BufferingApplicationStartup` + `/actuator/startup`, or the `--debug` report.
2. Exclude auto-configurations you do not need; narrow component scanning.
3. `spring.main.lazy-initialization=true` for dev/test (in production it hides wiring errors until the first request and shifts latency to that request).
4. Avoid heavy work in `@PostConstruct` — move warm-up to `ApplicationReadyEvent` or async.
5. Hibernate: `ddl-auto=validate` (not `update`), avoid scanning huge packages; Flyway validation is fast but migrations on startup are not.
6. CDS archive, AOT, or native image for the remaining gap.
7. JVM flags: `-XX:TieredStopAtLevel=1` only for local dev.

17.16 Interview questions and answers

Q1. What is Spring Boot and how is it different from Spring Framework?

Spring Boot is an opinionated layer on Spring Framework that auto-configures beans from the classpath, provides starters, an embedded server and production features. The framework provides the container, AOP, MVC and data access; Boot decides *which* framework beans to create and with what defaults. Everything Boot creates is a normal bean you can replace — that is the key difference from a "black box" framework.

Q2. What does `@SpringBootApplication` do?

It combines `@SpringBootConfiguration` (a `@Configuration` that tests can find), `@EnableAutoConfiguration` (imports the deferred `AutoConfigurationImportSelector` and registers the auto-configuration package) and `@ComponentScan` of the main class's package with filters that skip auto-configuration classes. You can use the three annotations separately when you need different scanning.

Q3. Walk me through what happens when `SpringApplication.run()` is called.

Construction deduces the web type and loads initializers and listeners from `spring.factories`. `run()` then creates the bootstrap context, publishes `ApplicationStartingEvent`, prepares the environment (command-line args, config data, `EnvironmentPostProcessor`s), prints the banner, creates the right `ApplicationContext`, runs initializers, registers the main class, refreshes the context (bean factory post-processors including auto-configuration, bean post-processors, web server creation, singleton instantiation, web server start), publishes `ApplicationStartedEvent`, calls runners and finally publishes `ApplicationReadyEvent`. Mention readiness turning green only at the end — it shows you understand why probes matter.

Q4. In what order are Spring Boot application events published?

`ApplicationStartingEvent` → `ApplicationEnvironmentPreparedEvent` → `ApplicationContextInitializedEvent` → `ApplicationPreparedEvent` → (during refresh) `WebServerInitializedEvent` and `ContextRefreshedEvent` → `ApplicationStartedEvent` + `LivenessState.CORRECT` → runners → `ApplicationReadyEvent` + `ReadinessState.ACCEPTING_TRAFFIC`. `ApplicationFailedEvent` on failure. The first four happen before your beans exist.

Q5. Why does my `@EventListener(ApplicationEnvironmentPreparedEvent.class)` bean never fire?

Because the event is published before the context is refreshed, so the listener bean does not exist yet. Register the listener in `META-INF/spring.factories` (`org.springframework.context.ApplicationListener=...`) or with `springApplication.addListeners(...)` before `run()`. For changing the environment, an `EnvironmentPostProcessor` is the cleaner tool.

Q6. How does auto-configuration work internally?

`@EnableAutoConfiguration` imports `AutoConfigurationImportSelector`, a `DeferredImportSelector`. After user configuration is parsed, it reads all `META-INF/spring/...AutoConfiguration.imports` files, removes exclusions, pre-filters with build-time metadata (`OnClass`, `OnBean`, `OnWebApplication`) without loading classes, sorts by `@AutoConfigureOrder`/before/after, and hands the survivors to `ConfigurationClassPostProcessor`. Each class and `@Bean` method is then guarded by `@Conditional` annotations, so beans appear only when the classpath, properties and existing beans call for them. The outcome is recorded in the `ConditionEvaluationReport`.

Q7. What changed between `spring.factories` and `AutoConfiguration.imports`?

Boot 2.7 introduced the `AutoConfiguration.imports` file and the `@AutoConfiguration` annotation; Boot 3.0 removed the `EnableAutoConfiguration` key from `spring.factories`. The new file is one class per line, simpler to read and merge. `spring.factories` still registers other extension points such as `EnvironmentPostProcessor` and `FailureAnalyzer` — RefinePoint uses both files in `ryfn-platform`.

Q8. What is `@AutoConfiguration` and how is it different from `@Configuration`?

It is a meta-annotation of `@Configuration(proxyBeanMethods = false)` with `before`/`after`/`beforeName`/`afterName` attributes for ordering. `proxyBeanMethods = false` means no CGLIB subclass: calling one `@Bean` method from another creates a new object instead of returning the singleton, so auto-configurations inject dependencies through method parameters. It also signals that the class must be loaded via the imports file, not component scanning.

Q9. How does `@ConditionalOnMissingBean` let me override Boot's beans?

User configuration is processed first because the auto-configuration selector is deferred. When Boot reaches, say, `JacksonAutoConfiguration`, your `ObjectMapper` definition already exists, so the condition fails and Boot's bean is skipped. It matches on type by default (and optionally name or annotation). It is only reliable in auto-configuration classes; in your own `@Configuration` classes the parsing order is not guaranteed.

Q10. Name the most important `@Conditional` annotations.

`@ConditionalOnClass`/`OnMissingClass`, `@ConditionalOnBean`/`OnMissingBean`, `@ConditionalOnSingleCandidate`, `@ConditionalOnProperty`, `@ConditionalOnResource`, `@ConditionalOnWebApplication`/`OnNotWebApplication`, `@ConditionalOnExpression`, `@ConditionalOnJava`, `@ConditionalOnCloudPlatform` and `@ConditionalOnThreading` (Boot 3.2, virtual threads). All are built on Spring's `Condition` interface, which you can implement for custom logic.

Q11. `@ConditionalOnProperty(name="x.enabled", havingValue="true", matchIfMissing=true)` — what are the cases?

Property absent → matches (because of `matchIfMissing`). Property `true` → matches. Property `false` or any other value → does not match. Without `havingValue`, any value except `false` matches. RefinePoint's `OutboxRelay` uses exactly this with `rp.kafka.enabled`, so Kafka is on unless explicitly turned off.

Q12. Why should `@ConditionalOnClass` not be used on a `@Bean` method whose return type is the conditional class?

When Spring introspects the configuration class it loads method signatures; if the return type class is missing, introspection throws `NoClassDefFoundError` before the condition is evaluated. Put the condition at class level or on a nested static configuration class, which isolates the reference.

Q13. How do you find out why an auto-configuration did or did not apply?

Start with `--debug` (or `debug=true`) to print the Conditions Evaluation Report with positive matches, negative matches with reasons, exclusions and unconditional classes. At runtime use `/actuator/conditions`, plus `/actuator/beans` to see which bean won and `/actuator/configprops` / `env` to check the properties that drove the conditions. In tests, `ApplicationContextRunner` reproduces the scenario in milliseconds.

Q14. How do you disable a specific auto-configuration?

`@SpringBootApplication(exclude = X.class)` or `excludeName` for classes not on the classpath, or the `spring.autoconfigure.exclude` property, which can be profile-specific. Excluding is preferable to defining a dummy bean, which leaves the rest of the auto-configuration active and confusing.

Q15. How do you control auto-configuration ordering, and what does it not control?

`@AutoConfiguration(before/after)`, `@AutoConfigureBefore/After` and `@AutoConfigureOrder` control the order in which auto-configurations are parsed, which matters for `@ConditionalOnBean`. They do not order user configurations and do not decide bean instantiation order — dependencies do (`@DependsOn` if there is no injection relationship).

Q16. How would you write a custom Spring Boot starter?

Create an autoconfigure module with `@AutoConfiguration` classes guarded by conditions, a `@ConfigurationProperties` class with the configuration processor for metadata, register the classes in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, use `@ConditionalOnMissingBean` on every bean so applications can override, and test with `ApplicationContextRunner`. Then create a starter POM that depends on the autoconfigure module and required libraries. Name it `xyz-spring-boot-starter`.

Q17. Tell me about a starter you built.

> **RefinePoint:** "We have 13 services and every one needs the same security, tenancy checks, outbox, Kafka settings, observability and OpenAPI. I built `ryfn-platform`, a shared library with six auto-configurations. Security defines three ordered `SecurityFilterChain`s: public paths (health, docs and the service's own `rp.security.public-paths`), `/internal/**` with a shared internal token, and everything else with a Keycloak JWT or an `X-API-Key`. `PlatformWebAutoConfiguration` registers `OrgAccessInterceptor` on `/api/v1/organizations/*/**` so no controller can forget membership and role checks. Defaults such as the datasource URL with a per-service schema, `open-in-view=false`, Kafka `acks=all` and Actuator exposure come from an `EnvironmentPostProcessor` added with lowest priority. A new service is a `main` class, its schema name and its business code."

Q18. What is an `EnvironmentPostProcessor` and when do you use it?

A hook, registered in `spring.factories`, that can add or modify property sources after the environment is prepared and before the context is created. Use it for defaults shared by many applications, decrypting properties, or deriving properties from others (RefinePoint builds the Kafka SASL JAAS string from `KAFKA_USERNAME` / `KAFKA_PASSWORD`). Use `addLast` for defaults that applications can override, `addFirst` for values that must win.

Q19. A filter bean in your starter runs twice per request. Why?

Boot auto-registers every `jakarta.servlet.Filter` bean with the servlet container. If the same filter is also added to a Spring Security chain with `addFilterBefore`, it runs once in the container chain and once in the security chain. Fix: declare a `FilterRegistrationBean` for it with `setEnabled(false)`, or do not make it a bean. Extending `OncePerRequestFilter` only prevents re-execution within the *same* filter instance's attribute, which helps but is not the real fix.

Q20. What is a starter and how does dependency management work?

A starter is a POM that groups dependencies for one capability, with no code. Versions come from the `spring-boot-dependencies` BOM, inherited through `spring-boot-starter-parent` or imported in `dependencyManagement`. Override a version via the BOM property (`<postgresql.version>`) so aligned modules move together.

Q21. How does the embedded Tomcat start?

During `onRefresh`, `ServletWebServerApplicationContext` asks the `ServletWebServerFactory` bean for a `WebServer`; the Tomcat factory creates `Tomcat`, a connector and a context and runs `ServletContextInitializer` beans (dispatcher servlet, filters, listeners). The connector is started later, in `finishRefresh`, by `WebServerStartStopLifecycle`, after all singletons exist — so the port opens only when the app can serve requests. `ServletWebServerInitializedEvent` then carries the real port.

Q22. How do you switch from Tomcat to Jetty or Undertow?

Exclude `spring-boot-starter-tomcat` from `spring-boot-starter-web` and add `spring-boot-starter-jetty`. The `ServletWebServerFactoryAutoConfiguration` imports factory configurations each guarded by `@ConditionalOnClass` of the server classes, so whichever server is on the classpath wins. Undertow is not supported in Boot 4.

Q23. How do you tune Tomcat for high concurrency?

Size `server.tomcat.threads.max` (200 default), `max-connections` (8192) and `accept-count` (100) together with the database pool and downstream limits; threads beyond what the pool can serve only queue on the pool. Set connection and keep-alive timeouts, enable response compression only where useful, and measure with `tomcat.threads.busy` and Hikari `pending` metrics. On Java 21, `spring.threads.virtual.enabled=true` removes the thread cap for I/O-bound services; then protect downstreams explicitly.

Q24. What changes when you enable virtual threads in Boot 3.2+?

Tomcat (and Jetty) run each request on a new virtual thread; `@Async`, `@Scheduled` executors, Kafka listener containers and others use virtual threads where Boot configures them. Blocking calls unmount the virtual thread instead of holding a platform thread. Watch for pinning on `synchronized` blocks around I/O (fixed in JDK 24, still present on 21), `ThreadLocal` heavy caches, and the fact that thread pools no longer limit concurrency.

Q25. Explain the property source precedence order.

From lowest to highest: default properties, `@PropertySource`, config data files (jar then external, base then profile-specific), `random.*`, OS environment variables, Java system properties, JNDI and servlet parameters, `SPRING_APPLICATION_JSON`, command-line arguments, then test-specific sources and DevTools global settings. A good one-liner: "the closer to the deployment, the higher the priority".

Q26. A property set in `application.yml` is ignored in production. How do you debug?

Check `/actuator/env/<property.name>` — it shows every property source that defines it and which one wins. Common causes: an environment variable with the relaxed name overriding it (`SERVER_PORT` set by the platform), an external `application.yml` next to the jar, a profile document overriding it, a typo or wrong prefix (the configuration processor's metadata catches this in the IDE), or `@PropertySource` being too late for that property.

Q27. How do environment variables map to properties?

Relaxed binding: uppercase, `.` → `_`, remove dashes: `spring.datasource.hikari.maximum-pool-size` → `SPRING_DATASOURCE_HIKARI_MAXIMUMPOOLSIZE`. List elements use `_0_` style indices: `RP_SECURITY_PUBLICPATHS_0`. Map keys keep their case only with bracket notation in properties files, not in env vars.

Q28. What are profiles and profile groups?

Profiles are named sets of configuration and beans activated per environment (`spring.profiles.active`). Groups let one name activate several: `spring.profiles.group.prod=prodDb,prodKafka`. Profile-specific documents use `spring.config.activate.on-profile`; they cannot themselves activate profiles. Use `@Profile` on beans that should exist only in some environments, like a fake payment gateway for local.

Q29. What is `spring.config.import` used for?

Loading additional configuration early in config-data processing: extra files (`optional:file:./.env[.properties]`), directory trees of secrets (`configtree:/run/secrets/`), Spring Cloud Config (`configserver:`), Vault or Consul. Because it runs in config-data processing, imported values can affect profiles, logging and anything else, unlike `@PropertySource`.

Q30. `@Value` vs `@ConfigurationProperties`?

`@ConfigurationProperties` binds a prefix into a typed, validated, immutable object with full relaxed binding and IDE metadata — use it for any group of settings. `@Value` injects a single value and supports SpEL, but scatters keys across the code, cannot validate and does not support relaxed binding fully. In code review I convert anything with more than one or two related `@Value`s into a properties record.

Q31. How does `@ConfigurationProperties` binding work and how do you validate it?

`ConfigurationPropertiesBindingPostProcessor` uses the `Binder` over relaxed `ConfigurationPropertySource`s and the `ConversionService`; records and single-constructor classes bind through the constructor, others through setters. Add `@Validated` and Jakarta constraints (`@NotBlank`, `@Min`, `@Valid` on nested objects); a violation fails startup with a report listing each invalid property. Fail-fast configuration is a production safety net.

Q32. How do you bind durations and sizes?

Use `java.time.Duration` and `org.springframework.util.unit.DataSize` fields; values like `30s`, `500ms`, `2h`, `10MB` convert automatically. A plain number uses the default unit (milliseconds for `Duration`, bytes for `DataSize`), which you can change with `@DurationUnit` / `@DataSizeUnit`. This avoids the classic `timeout=30` "seconds or millis?" bug.

Q33. What does Actuator provide out of the box, and what is exposed by default?

Health, info, metrics, Prometheus, env, configprops, beans, conditions, mappings, loggers, thread dump, heap dump, startup, scheduled tasks, caches, Flyway, HTTP exchanges, SBOM and shutdown. Over HTTP only `health` is exposed by default; everything else must be listed in `management.endpoints.web.exposure.include`. `shutdown` is also disabled until enabled explicitly.

Q34. How do liveness and readiness probes work in Boot?

Boot tracks `LivenessState` and `ReadinessState` via `ApplicationAvailability`, updated by lifecycle events. With probes enabled (automatic on Kubernetes), `/actuator/health/liveness` and `/actuator/health/readiness` are health groups. Readiness becomes ACCEPTING_TRAFFIC only after runners finish and becomes REFUSING_TRAFFIC at shutdown. You can publish `AvailabilityChangeEvent` yourself, for example to refuse traffic while a large cache reloads.

Q35. Should the database be part of the liveness probe?

No. Liveness failure restarts the pod; if the database is down, restarting every instance does not fix it and causes a thundering-herd restart. Put critical dependencies in readiness (or nowhere, and rely on circuit breakers), and keep liveness about the process itself (deadlock, out of memory).

Q36. How do you write a custom health indicator?

Implement `HealthIndicator` (or `ReactiveHealthIndicator`) as a bean; the bean name minus `HealthIndicator` becomes the component name. Return `Health.up()` / `down()` with details. Keep it fast and side-effect-free — health is called frequently; cache expensive checks. Add it to a group with `management.endpoint.health.group.readiness.include`.

Q37. How do you add custom metrics?

Inject `MeterRegistry` and register `Counter`, `Timer`, `Gauge`, `DistributionSummary` or `LongTaskTimer`, or use `@Timed` / `@Counted` (with the aspect bean) or the Observation API (`Observation.createNotStarted("recon.run", registry).observe(...)`) to get a metric and a span together. Keep tag values bounded, add common tags (application, environment) once via `management.metrics.tags.*`, and enable histograms for latency SLOs.

Q38. How do you secure Actuator?

Expose only what is needed, run management on a separate internal port if possible, and secure with a dedicated `SecurityFilterChain` matching `EndpointRequest.toAnyEndpoint()`. Health can be public with `show-details: when-authorized`; Prometheus is scraped with a token or from an internal network; `env`, `configprops` and `heapdump` should never be reachable from the internet. Boot 3 sanitises all `env` / `configprops` values by default (`show-values: never`).

Q39. How do you change a log level in production without a restart?

`POST /actuator/loggers/{logger}` with `{"configuredLevel":"DEBUG"}`, then reset with `null`. It is per instance and not persistent. For fleet-wide changes, use a config refresh mechanism or restart with `LOGGING_LEVEL_COM_RYFNFLOW=DEBUG`. Turn it back off — debug logging on a hot path can halve throughput.

Q40. What is the difference between `logback.xml` and `logback-spring.xml`?

`logback.xml` is loaded by Logback itself before Boot initialises logging, so it cannot use Boot extensions. `logback-spring.xml` is loaded by Boot's `LoggingSystem` and supports `<springProfile>` and `<springProperty>`. Prefer properties (`logging.*`, structured logging in 3.4) and add a Spring XML file only for appenders properties cannot express.

Q41. How does graceful shutdown work?

On `SIGTERM` the JVM shutdown hook closes the context: readiness flips to REFUSING_TRAFFIC, the web server stops accepting new connections and waits for in-flight requests up to `spring.lifecycle.timeout-per-shutdown-phase` (30 s), then other `SmartLifecycle` beans stop (listeners, schedulers), then singletons are destroyed (`@PreDestroy`, pools closed). Boot 3.4 makes `server.shutdown=graceful` the default. Add a `preStop` sleep on Kubernetes and make background workers stop claiming new work.

Q42. What is inside a Spring Boot fat jar and how does it run?

The manifest's `Main-Class` is `JarLauncher` (package `org.springframework.boot.loader.launch` since 3.2) and `Start-Class` is your main class. Your classes are in `BOOT-INF/classes`, dependencies are nested jars in `BOOT-INF/lib`, plus `classpath.idx` and `layers.idx`. The launcher builds a class loader that reads nested jars directly via a custom URL handler and invokes your `main`. The jars are stored uncompressed so they can be read in place.

Q43. What are layered jars and why use them?

A fat jar split into layers by change frequency (`dependencies`, `spring-boot-loader`, `snapshot-dependencies`, `application`) recorded in `layers.idx`. Extract with `-Djarmode=tools extract --layers` and copy each layer into its own Docker layer. A code change rebuilds and pushes only the few-MB application layer, so builds, pushes and pulls are much faster.

Q44. What were the main challenges migrating Boot 2 to 3?

Java 17 baseline; `javax` → `jakarta` namespace across code and every library (third-party libraries without Jakarta versions block you); `spring.factories` auto-configuration no longer loaded; Spring Security 6 removed `WebSecurityConfigurerAdapter` and `antMatchers`; Hibernate 6 query and ID-generation changes; Sleuth replaced by Micrometer Tracing; trailing-slash matching off; property renames. Go via 2.7, use `spring-boot-properties-migrator` and OpenRewrite, and run the full integration test suite.

Q45. What is Spring AOT and how does it relate to GraalVM native image?

Spring AOT runs during the build, evaluates the context with the build-time classpath and generates source code for bean definitions plus runtime hints. Native image then compiles the app ahead of time under a closed-world assumption, using those hints for reflection, proxies and resources. The result starts in milliseconds and uses far less memory, but conditions and profiles are frozen at build time and peak throughput is usually lower than a JIT-warmed JVM.

Q46. When would you not use a native image?

For long-running, throughput-heavy services (reconciliation engines, batch workers) where startup time is irrelevant and JIT optimisation wins; for apps relying on runtime classpath changes, dynamic class generation or plugins loaded via `ServiceLoader` from external jars (RefinePoint's connector plugins); and when build-time cost slows the team down. CDS or CRaC give much of the startup gain with fewer constraints.

Q47. What does DevTools do and why is it excluded from production?

It restarts the app quickly using a separate restart class loader for your classes, disables caches, triggers LiveReload and can support remote updates. It is excluded from fat jars (`developmentOnly` /optional) and disables itself when running from a jar, because restart class loaders and remote update endpoints are a security and stability risk. A known side effect in dev: two class loaders can cause `ClassCastException` for classes deserialized by libraries loaded in the base loader.

Q48. How would you reduce startup time of a Boot service?

Measure with `/actuator/startup` or the startup log first. Then exclude unused auto-configurations, narrow scanning, move warm-up work off the startup path, use `ddl-auto=validate` , avoid eager remote calls in `@PostConstruct` , and consider CDS, AOT on the JVM, CRaC or native. Lazy initialisation helps in tests and dev, but in production it defers errors and first-request latency.

Q49. What is `ApplicationRunner` vs `CommandLineRunner` , and what is a better alternative for warm-up?

Both run after the context starts and before `ApplicationReadyEvent` ; `ApplicationRunner` receives parsed `ApplicationArguments` , `CommandLineRunner` the raw `String[]` . They block startup and their exceptions fail the app — good for one-off jobs or mandatory checks. For warm-up that should not fail startup or delay readiness, listen to `ApplicationReadyEvent` and run asynchronously.

Q50. What does bean definition overriding being disabled mean?

Since Boot 2.1, registering two bean definitions with the same name fails startup (`BeanDefinitionOverrideException`) instead of silently replacing the first. You can re-enable it with `spring.main.allow-bean-definition-overriding=true` , but the right fix is a distinct name, `@ConditionalOnMissingBean` , or `@Primary` .

Q51. Your service fails with "Parameter 0 of constructor in X required a bean of type Y that could not be found". How do you approach it?

That message comes from a `FailureAnalyzer` . Check whether Y is in a scanned package (main class package and below), whether an auto-configuration that would create it was skipped (look in the conditions report for Y's auto-configuration and why it did not match), whether a profile or property disabled it, or whether a starter is missing from the classpath. In RefinePoint, for example, `OutboxRelay` exists only when `rp.kafka.enabled` is true, so a class injecting it directly would fail exactly like this under `KAFKA_ENABLED=false` .

Q52. How does Boot decide between a servlet and a reactive application?

`WebApplicationType.deduceFromClasspath()` : if `DispatcherHandler` (WebFlux) is present and `DispatcherServlet` and Jersey are absent → REACTIVE; if servlet classes are present → SERVLET; else NONE. With both MVC and WebFlux on the classpath, SERVLET wins (you can still use `WebClient`). Override with `spring.main.web-application-type` .

Q53. How do you test that your auto-configuration backs off correctly?

`ApplicationContextRunner` with `withConfiguration(AutoConfigurations.of(...))` , `withUserConfiguration(MyOverride.class)` and `withPropertyValues(...)` ; assert with `assertThat(context).hasSingleBean(...)` , `doesNotHaveBean(...)` or `hasFailed()` . Use `FilteredClassLoader` to simulate a missing class for `@ConditionalOnClass` cases. Tests run in milliseconds and cover each condition branch.

Q54. How do you share common configuration across many microservices?

Options: a shared starter with auto-configurations and an `EnvironmentPostProcessor` for defaults (RefinePoint's choice, versioned with the code), a config server or Kubernetes ConfigMaps for environment values, and secrets in a vault/secret store via `configtree` or config-data loaders. Keep secrets out of the jar; keep defaults overridable; document every property with configuration metadata.

Q55. What do you check before taking a Boot service to production?

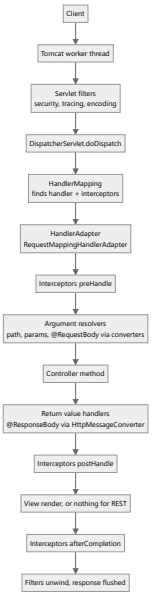
Graceful shutdown and probes configured; Actuator exposure locked down; metrics with common tags and latency histograms; structured logs with trace ids; `open-in-view=false` ; `ddl-auto=validate` with Flyway owning the schema; connection pool sized to the database limit; timeouts on every outbound client; configuration validated at startup; layered jar or buildpack image; JVM container settings (`-XX:MaxRAMPercentage`). RefinePoint's `PlatformDefaults` encodes most of this list so no service can forget it.

18. Spring MVC & REST APIs

Senior REST interviews split into two halves. The first is mechanics: what happens between the socket and your controller, why an interceptor cannot change a `@ResponseBody`, why an exception thrown in a filter skips `@ControllerAdvice`. The second is design: which status code, how to page a million rows, how to make `POST` safe to retry. This chapter covers both, then shows how RefinePoint applies them across 13 services.

18.1 The request flow

Tomcat parses the request, runs the servlet filter chain, then calls one servlet: `DispatcherServlet`, the **front controller**. Everything after that is Spring's `doDispatch` method.



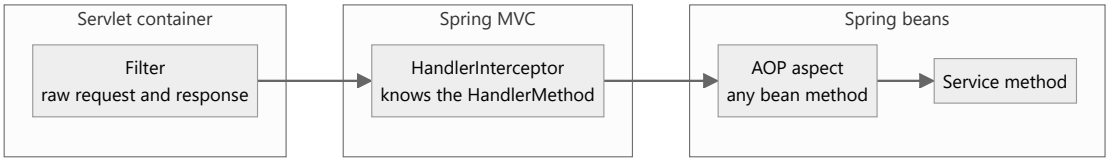
The steps in `doDispatch`, in order:

1. `checkMultipart` wraps multipart requests.
2. `getHandler` asks each `HandlerMapping` in order. `RequestMappingHandlerMapping` holds a registry of every `@RequestMapping` built at startup and matches path, method, `consumes`, `produces`, params and headers. It returns a `HandlerExecutionChain`: the `HandlerMethod` plus matching interceptors. No match leads to the resource handler and then a 404.
3. `getHandlerAdapter` picks the adapter that can invoke this handler type.
4. `applyPreHandle` runs interceptors in order; returning `false` stops the request.
5. `ha.handle` resolves arguments, invokes your method, and processes the return value. **For `@ResponseBody`, the body is serialised and written here.**
6. `applyPostHandle` runs in reverse order.
7. `processDispatchResult` handles any exception via `HandlerExceptionResolver`s, renders a view if there is one, and calls `afterCompletion`.

> **Trap:** `postHandle` cannot reliably change a REST response. The body was already written in step 5, and the response may be committed. To modify bodies use `ResponseBodyAdvice`; to add headers use a filter, or `preHandle`.

Matching since Boot 2.6 uses `PathPatternParser` (parsed patterns, faster than `AntPathMatcher`). `**` is allowed only at the end of a pattern. Boot 3 no longer treats `/runs/` as `/runs`.

18.2 Filters vs interceptors vs AOP



	Filter	Interceptor	AOP aspect
Level	Servlet API	DispatcherServlet	Bean proxy
Runs for	Every request, including static and error paths	Only requests mapped by MVC	Any proxied call, HTTP or not
Sees	Request and response, can wrap them	Request, response, handler	Method args and return value
Can replace the request/response object	Yes, by wrapping	No	No
Exceptions reach `@ControllerAdvice`	**No**	Yes	Yes, if under a controller call
Typical use	Auth, CORS, tracing, compression	Tenant and role checks, per-route logic	Transactions, caching, metrics

Rule of thumb: pick the outermost layer that has the information you need. Authentication goes in a filter because it must run before anything else; a check that needs the matched route belongs in an interceptor; a business rule belongs in the service.

> **RefinePoint:** `OrgAccessInterceptor` is a `HandlerInterceptor` registered by `PlatformWebAutoConfiguration` on `/api/v1/organizations/*/**`. GET and HEAD need any role in the workspace, writes need at least MEMBER, and a locked workspace takes only what `WorkspaceLocks` allows. Because it is in the shared library, no controller in any service can forget the check. Stricter rules stay on the controller as `@PreAuthorize`. It is an interceptor rather than a filter because its exceptions must reach `GlobalExceptionHandler` and become our normal JSON error.

18.3 Controllers, mapping and arguments

`@RestController` is `@Controller` + `@ResponseBody` on every method: return values go through message converters rather than being treated as view names.

```
@RestController
@RequestMapping("/api/v1/organizations/{orgCode}/notifications/inbox")
class InboxApi {
    @GetMapping
    ApiResponse<List<Notification>> mine(@PathVariable String orgCode,
                                         @RequestParam(defaultValue = "50") int limit,
                                         @RequestHeader(value = "Accept-Language", required = false) String lang,
                                         @AuthenticationPrincipal Jwt jwt) { ... }
}
```

Each parameter is handled by a `HandlerMethodArgumentResolver`, the first whose `supportsParameter` returns true:

Annotation / type	Source	Notes
`@PathVariable`	URI template	`-parameters` compile flag needed if the name is omitted (Boot's parent sets it)
`@RequestParam`	Query or form	Required by default; missing gives 400
`@RequestHeader`, `@CookieValue`	Header, cookie	
`@RequestBody`	Body via converter	Needs `Content-Type` a converter accepts, else 415
`@RequestPart`	One multipart part	Can be a JSON DTO, unlike `@RequestParam`
`@ModelAttribute` (implicit for POJOs)	Query and form fields bound to setters	Mass-assignment risk
`Pageable`, `Sort`	`page`, `size`, `sort` params	Registered by Spring Data web support
`Authentication`, `Principal`	Security context	

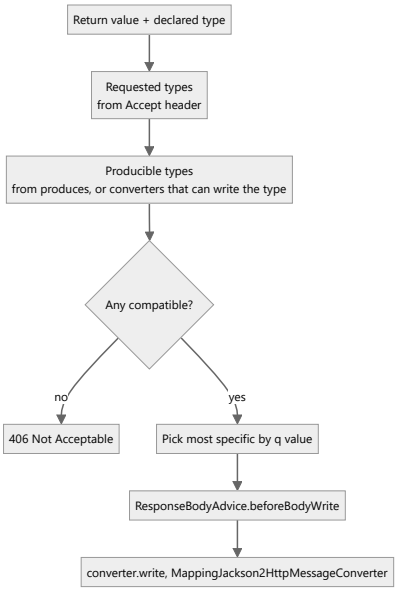
A custom resolver gives a clean signature for a repeated need:

```
record OrgContext(UUID orgId, OrgRole role) { }

class OrgContextResolver implements HandlerMethodArgumentResolver {
    public boolean supportsParameter(MethodParameter p) { return p.getParameterType() == OrgContext.class; }
    public Object resolveArgument(MethodParameter p, ModelAndViewContainer m, NativeWebRequest req, WebDataBinderFactory f) {
        String code = req.getHeader("X-Org-Code");
        return lookup(code);   // generic sketch, registered through WebMvcConfigurer.addArgumentResolvers
    }
}
```

18.4 HttpMessageConverters, Jackson and content negotiation

`@RequestBody` and `@ResponseBody` are handled by `RequestResponseBodyMethodProcessor`, which picks an `HttpMessageConverter` by Java type and media type.



For a request, the `Content-Type` selects the converter (`Content-Type: application/xml` with no XML converter gives 415). For a response, `Accept` drives negotiation. Path-extension negotiation (`/runs.json`) was removed for security reasons; query-parameter negotiation is opt-in.

Boot's `JacksonAutoConfiguration` builds one `ObjectMapper` from `spring.jackson.*` and all `Jackson2ObjectMapperBuilderCustomizer` beans, with defaults different from plain Jackson: `FAIL_ON_UNKNOWN_PROPERTIES=false` , `WRITE_DATES_AS_TIMESTAMPS=false` , and the `JavaTimeModule` registered.

> **Trap:** declaring `@Bean ObjectMapper om() { return new ObjectMapper(); }` backs off Boot's mapper entirely. `Instant` fields then fail or serialise oddly, and unknown JSON fields start throwing 400s. Use a `Jackson2ObjectMapperBuilderCustomizer` or `spring.jackson.*` to adjust rather than replace.

Common settings: `@JsonInclude(NON_NULL)` , `@JsonIgnoreProperties(ignoreUnknown = true)` on DTOs read from third parties, `@JsonProperty(access = WRITE_ONLY)` for passwords, `BigDecimal` kept as a string or plain number for money (never `double`).

18.5 Validation

Add `spring-boot-starter-validation` (it stopped being part of the web starter in Boot 2.3), which brings Hibernate Validator.

```
public record CreateRuleRequest(
        @NotBlank @Size(max = 200) String name,
        @NotNull @DecimalMin("0.00") BigDecimal tolerance,
        @Valid @NotEmpty List<FieldPair> fields) { }   // @Valid cascades into elements

@PostMapping
ApiResponse<RuleDto> create(@PathVariable String orgCode, @Valid @RequestBody CreateRuleRequest body) { ... }
```

Where	Trigger	Exception thrown
`@Valid @RequestBody`	Argument resolver	`MethodArgumentNotValidException`
`@ModelAttribute` params	Binding	`BindException` / `MethodArgumentNotValidException` (Boot 3)
Constraints directly on `@RequestParam` , `@PathVariable`	Built-in method validation (Spring 6.1)	`HandlerMethodValidationException`
`@Validated` on a service class	AOP proxy (`MethodValidationPostProcessor`)	`ConstraintViolationException`

`@Valid` vs `@Validated` : `@Valid` is the Jakarta annotation and cascades into nested objects. `@Validated` is Spring's: it supports **groups** and, on a class, turns on method validation through a proxy. Because it is a proxy, self-invocation is not validated.

Groups let one DTO have per-operation rules:

```
interface OnCreate { }
interface OnUpdate { }
record RuleDto(@Null(groups = OnCreate.class) @NotNull(groups = OnUpdate.class) UUID id, @NotBlank String name) { }

@PostMapping ApiResponse<RuleDto> create(@Validated(OnCreate.class) @RequestBody RuleDto dto) { ... }
```

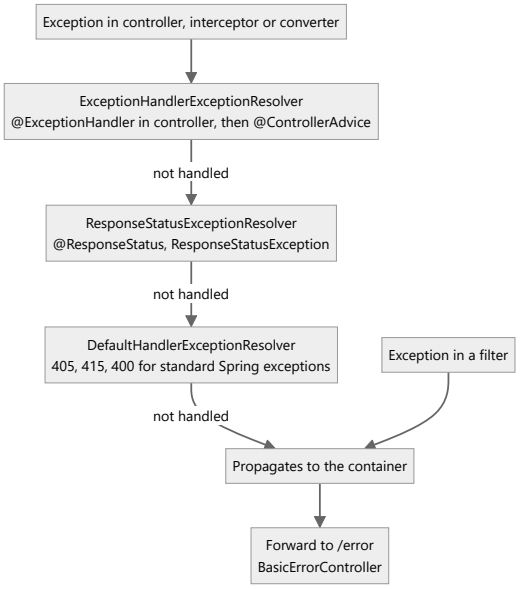
Note that a constraint with no `groups` belongs to `Default` , which is *not* checked when you name another group, unless your group extends `Default` . Separate request records are usually clearer than groups.

Custom constraint:

```
@Target(FIELD) @Retention(RUNTIME) @Constraint(validatedBy = CurrencyCodeValidator.class)
public @interface CurrencyCode {
    String message() default "must be an ISO 4217 code";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
class CurrencyCodeValidator implements ConstraintValidator<CurrencyCode, String> {
    public boolean isValid(String v, ConstraintValidatorContext c) {
        return v == null || Currency.getAvailableCurrencies().stream().anyMatch(x -> x.getCurrencyCode().equals(v));
    }
}
```

Validators return `true` for null by convention, so that `@NotNull` stays a separate, composable rule. Validators are Spring beans and may inject dependencies.

18.6 Global exception handling



The closest matching exception type wins, and a local `@ExceptionHandler` in the controller beats any `@ControllerAdvice`. Advices can be ordered with `@Order` and limited with `basePackages`, `assignableTypes` or `annotations`.

Filter exceptions skip this. Spring Security translates its own in `ExceptionTranslationFilter`; for your own filters, either write the response directly or inject the `HandlerExceptionResolver` bean named `handlerExceptionResolver` and delegate to it.

ProblemDetail (RFC 7807, updated by RFC 9457) is the standard error body, supported since Spring 6:

```
@RestControllerAdvice
class Errors extends ResponseEntityExceptionHandler {   // handles standard MVC exceptions
    @ExceptionHandler(PlanLimitExceededException.class)
    ProblemDetail limit(PlanLimitExceededException e) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.PAYMENT_REQUIRED, e.getMessage());
        pd.setType(URI.create("https://example.com/errors/plan-limit"));
        pd.setProperty("limit", e.limit());
        return pd;      // Content-Type: application/problem+json
    }
}
```

Setting `spring.mvc.problemdetails.enabled=true` makes Boot register such an advice for standard exceptions.

> **RefinePoint:** we use our own envelope, not ProblemDetail, so the React client has one shape to handle on success and failure. `ApiResponse<T>` carries `success`, `data`, `message`, `errors` and `meta` (`timestamp`, `requestId`). `GlobalExceptionHandler` in `ryfn-platform` maps exceptions centrally: `EntityNotFoundException` to 404, validation and `IllegalArgumentException` to 400, `AccessDeniedException` to 403, `ConflictException` to 409, `RateLimitExceededException` to 429, `PlanLimitExceededException` to 402, `WorkspaceLockedException` to 423. One detail worth telling: a request with `Accept: text/event-stream` cannot carry a JSON error, and trying to write one fails the handler itself. So a `forClient` helper returns the status alone for those.

Envelope vs ProblemDetail: an envelope gives clients a uniform parse and room for metadata. Its costs are that it duplicates what HTTP status already says, and generic tools do not understand it. ProblemDetail is the standard and fits public APIs. Either is defensible; being consistent matters more. Never send stack traces, SQL or class names to the client (`server.error.include-stacktrace=never`, which is the default).

18.7 REST design

REST constraints: client-server, stateless, cacheable, uniform interface, layered system, code on demand (optional). Most "REST" APIs are Richardson level 2: resources + HTTP verbs + status codes. Level 3 adds hypermedia.

Resources are nouns: `GET /organizations/acme/runs/42`, not `/getRun?id=42`. For actions that do not map to CRUD, either model the result as a resource (`POST /runs/42/exports` creates an export) or accept a verb sub-resource (`POST /runs/42/retry`).

Method	Safe	Idempotent	Cacheable	Typical use	Success
GET	Yes	Yes	Yes	Read	200
HEAD	Yes	Yes	Yes	Headers only	200
OPTIONS	Yes	Yes	No	Capabilities, CORS preflight	204
POST	No	**No**	Rarely	Create, process	201 + `Location`, or 202
PUT	No	Yes	No	Replace whole resource, or create at a known id	200, 204, 201
PATCH	No	**Not guaranteed**	No	Partial update	200, 204
DELETE	No	Yes	No	Remove	204 (or 404 on repeat)

Safe means no intended state change. **Idempotent** means N identical requests leave the *server state* the same as one, though the response may differ (a second DELETE can return 404).

Code	Meaning	Use for
200 / 201 / 202 / 204	OK / created / accepted for async / no body	202 + status URL for long jobs
301 / 302 / 304 / 307 / 308	Redirects / not modified	307, 308 keep the method
400	Malformed or invalid input	Validation failure (some teams use 422)
401	Not authenticated	Missing or bad token
403	Authenticated, not allowed	Wrong role
404	Not found	Also to hide existence across tenants
405 / 406 / 415	Wrong method / no acceptable representation / unsupported body type	
409	Conflict with current state	Duplicate, illegal state transition
412 / 428	`If-Match` failed / precondition required	Optimistic concurrency
422	Well-formed but semantically invalid	
429	Rate limited	With `Retry-After`
500 / 502 / 503 / 504	Bug / bad upstream / overloaded / upstream timeout	Clients may retry 503, 504

Pagination, sorting, filtering

Style	Request	Pros	Cons
Offset	`?page=3&size=50`	Jump to any page, simple	DB scans and discards `OFFSET` rows; rows shift under concurrent inserts
Keyset	`?after=2026-10-01T12:00:00Z,42&size=50`	Stable and fast at any depth with an index	No random page jump
Cursor	`?cursor=opaque`	Keyset hidden; can change internally	Opaque, cannot share a page number

Return page metadata either in the body or in headers (`Link`, `X-Total-Count`). A total count on a big table is expensive; prefer `Slice` semantics or an approximate count there. Cap `size` on the server (`spring.data.web.pageable.max-page-size`). Whitelist sort fields: an arbitrary `sort` can hit an unindexed column and take the database down.

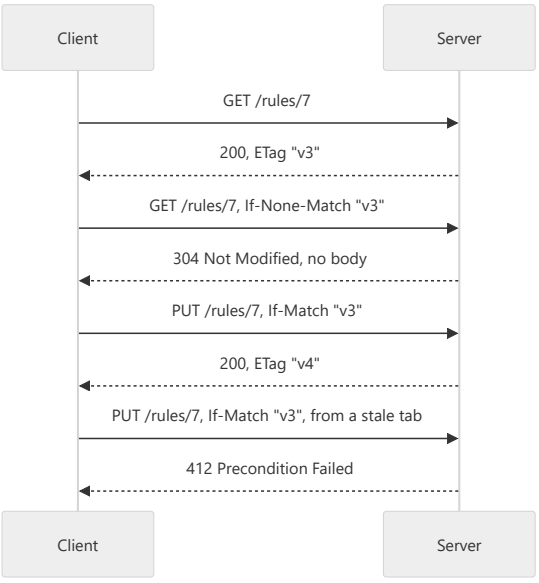
Versioning strategies

Strategy	Example	Pros	Cons
URI	`/api/v1/runs`	Obvious, routable, cache-friendly	Purists object; version lives in every link
Header	`X-API-Version: 2`	Clean URLs	Invisible, harder to test and cache
Media type	`Accept: application/vnd.rp.v2+json`	"Most RESTful"	Heavy for clients and tools
Query param	`?v=2`	Easy	Messy; caching issues

Prefer additive evolution (new optional fields; clients ignore unknowns) and version only for breaking changes. Publish a deprecation window with `Deprecation` and `Sunset` headers.

HATEOAS embeds links so clients discover actions (`"_links": {"retry": {"href": ...}}`). It suits long-lived public APIs with diverse clients. Most internal APIs skip it because the client is deployed with the server and the contract lives in OpenAPI.

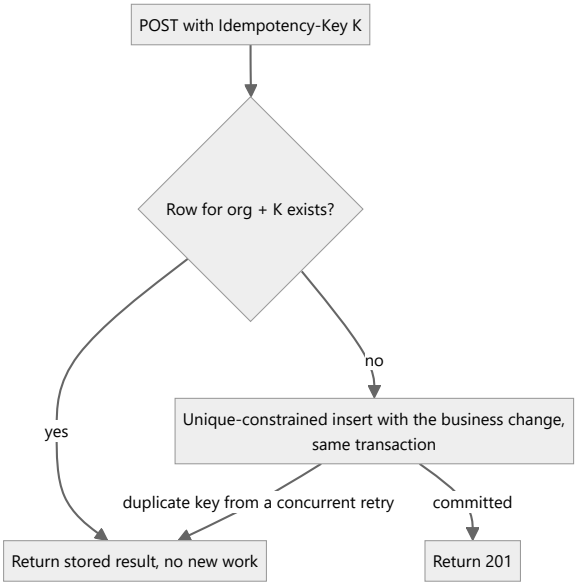
ETags and conditional requests



`ShallowEtagHeaderFilter` hashes the rendered body: it saves bandwidth, but the server still does all the work. A deep ETag comes from a version column and can skip the work: `if (request.checkNotModified(etag)) return null;`. `If-Match` gives optimistic concurrency over HTTP, preventing the lost update when two users edit one resource.

Idempotency keys for POST

`POST` is not idempotent, but networks fail after the server commits. The client cannot tell, and a retry creates a duplicate payment or run. The client generates a unique key per logical operation and resends it on retry.



Points to raise: the unique constraint is what makes it correct, not a check-then-insert; scope the key by tenant; store the response so the retry gets the same answer; reject a reused key with a different body; expire keys after a retention period.

> **RefinePoint:** `recon_runs` has an `idempotency_key` column, and `ReconciliationExecutor` calls `findByOrgIdAndIdempotencyKey` before starting a run, returning the existing run for a repeated request. The same thinking sits behind Kafka consumers: `ConsumedEvents.firstTime(consumer, eventId)` is an `INSERT ... ON CONFLICT DO NOTHING`, since at-least-once delivery guarantees repeats.

18.8 File upload, download and streaming

Upload. `MultipartFile` is backed by `StandardServletMultipartResolver`. Parts larger than `file-size-threshold` go to a temp file. Limits default to `spring.servlet.multipart.max-file-size=1MB` and `max-request-size=10MB`; exceeding them raises `MaxUploadSizeExceededException`, which should map to 413. Validate the real content rather than the claimed type, never trust the client's file name for a path, and scan if users share files. For large files, a presigned URL straight to object storage keeps application threads free.

Download. Three choices:

Return type	Memory	Runs on	Use for
`byte[]`	Whole file on heap	Request thread	Tiny files only
`Resource` (`FileSystemResource`)	Streamed; supports `Range`	Request thread	Existing files, resume support
`StreamingResponseBody`	Streamed as you write	MVC async executor	Generated content: CSV, ZIP

```
@GetMapping(BASE + "/export")
public ResponseEntity<StreamingResponseBody> export() {
    return ResponseEntity.ok()
        .contentType(MediaType.parseMediaType("application/zip"))
        .header(HttpHeaders.CONTENT_DISPOSITION, ContentDisposition.attachment().filename(name).build().toString())
        .header(HttpHeaders.CACHE_CONTROL, "private, no-store")
        .body(accounts::export);       // writes to the OutputStream on another thread
}
```

> **RefinePoint:** that is close to the real `AccountController` in identity-service, which streams a person's data as a ZIP so its size does not touch the heap. Reconciliation result exports are heavier still, so `ExportWorker` in run-service prepares them in the background from a `FOR UPDATE SKIP LOCKED` queue, and the user downloads the finished file. For large output, an async job beats a long HTTP request.

Traps with `StreamingResponseBody`: the lambda runs after the controller returned, on another thread. So the `@Transactional` scope has ended, `ThreadLocal` context such as `SecurityContext` needs propagating, and `spring.mvc.async.request-timeout` can cut the stream. A failure midway cannot change the status (it is already committed), so the client sees a truncated file. The default async executor should be replaced by a bounded one in production.

Server-Sent Events in MVC use `SseEmitter`: one long-lived connection, server-to-client only.

> **RefinePoint:** `InboxController` in alert-notify-service streams new notifications with `SseEmitter`: a `hello` event with the unread count, then a `notification` event for each new one, plus periodic comment pings to keep the connection open. It sets `X-Accel-Buffering: no` so nginx passes each event on at once, and removes the emitter on completion, timeout and error so the map of open streams does not leak.

18.9 CORS

The browser (not the server) enforces the same-origin policy. For a non-simple request it first sends a **preflight** `OPTIONS` with `Origin` and `Access-Control-Request-Method`; the server answers with `Access-Control-Allow-*`, and only then is the real request sent. Curl and server-to-server calls ignore CORS entirely, so **CORS is not an authorisation mechanism**.

Configuration points: `@CrossOrigin`, `WebMvcConfigurer.addCorsMappings`, or a `CorsConfigurationSource`. With Spring Security, enable `http.cors(...)`, otherwise the preflight has no token and gets a 401 before MVC ever sees it. `allowCredentials(true)` cannot be combined with `allowedOrigins("*")`; list the origins, or use `allowedOriginPatterns`.

> **RefinePoint:** CORS is the gateway's job. A service answers browsers directly only when `rp.security.allowed-origins` is set; otherwise `PlatformSecurityAutoConfiguration` disables CORS in that service. One place to configure, and no double `Access-Control-Allow-Origin` headers, which browsers reject.

18.10 Outbound HTTP clients

	`RestTemplate`	`WebClient`	`RestClient`	`@HttpExchange`
Since	Spring 3	Spring 5	Spring 6.1 / Boot 3.2	Spring 6
Model	Blocking	Non-blocking (usable blocking)	Blocking, fluent	Declarative, on top of a client
API	Template methods	Fluent, `Mono` / `Flux`	Fluent, same style as WebClient	Annotated interface
Dependency	spring-web	spring-webflux + reactor	spring-web	spring-web
Status	Maintenance, no new features	Active	**Default for new blocking code**	Active

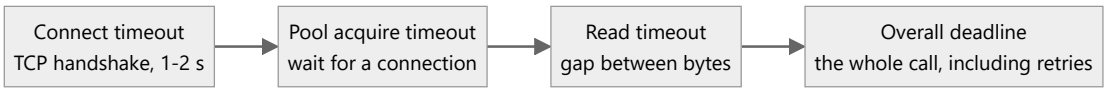
```
RestClient client = builder.baseUrl("https://api.example.com")
    .defaultStatusHandler(HttpStatusCode::isError, (req, res) -> { throw new UpstreamException(res.getStatusCode()); })
    .build();
Rate rate = client.get().uri("/rates/{pair}", "EURUSD").retrieve().body(Rate.class);

public interface RateApi {                      // HTTP interface
    @GetExchange("/rates/{pair}") Rate rate(@PathVariable String pair);
}
RateApi api = HttpServiceProxyFactory.builderFor(RestClientAdapter.create(client)).build().createClient(RateApi.class);
```

Inject Boot's auto-configured `RestClient.Builder`: it carries the Observation registry, so calls emit `http.client.requests` metrics and propagate traces.

RefinePoint: `InternalClient` is a `RestClient` over `JdkClientHttpRequestFactory`, one per target service, with a 2 s connect timeout and a 10 s read timeout by default. It adds `X-Internal-Token` and `X-Internal-Source`, can forward the caller's credentials (`asCaller`) or call as the service itself (`system`), and turns non-2xx responses into `RemoteCallException` through a default status handler, which `GlobalExceptionHandler` maps back into an HTTP error.

Timeouts and retries



- **Every outbound call needs a timeout.** The JDK and `RestTemplate` defaults can wait forever, and one slow dependency exhausts your thread pool.
- A read timeout is not a total timeout: a trickling server can keep a call alive indefinitely. Enforce a deadline if it matters.
- **Retry only what is safe:** idempotent methods, or POST with an idempotency key; on connection errors, 502, 503, 504, 429; not on 400, 401, 403, 404.
- **Exponential backoff with jitter**, otherwise all clients retry in sync and create a retry storm. Honour `Retry-After`.
- **Retries multiply.** Three layers with three attempts each make 27 calls on the bottom service. Retry at one layer only.
- Pair with a **circuit breaker** (Resilience4j) so a dead dependency fails fast, and a **bulkhead** so it cannot consume every thread.

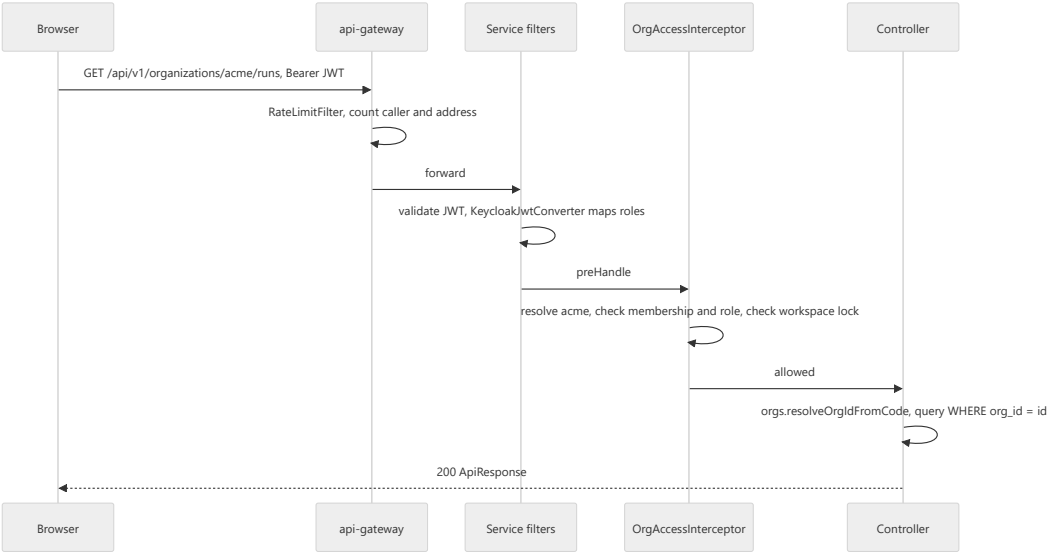
18.11 OpenAPI, security basics and multi-tenant paths

springdoc-openapi generates the OpenAPI 3 document at `/v3/api-docs` from your mappings, with Swagger UI at `/swagger-ui.html`. `@Operation`, `@Schema` and `@Tag` add documentation. Code-first keeps docs in step with code; contract-first (write the YAML, generate stubs) fits when several teams must agree before building. Commit the spec and diff it in CI to catch breaking changes.

RefinePoint: `PlatformOpenApiAutoConfiguration` gives every service Swagger. The docs paths are in the always-public list of the security chain, and Swagger UI signs in against Keycloak using authorization code with PKCE (`springdoc.swagger-ui.oauth.use-pkce-with-authorization-code-grant=true`).

API security checklist: TLS only; short-lived JWTs validated for signature, issuer, audience and expiry; authorise on every request and every object (**IDOR/BOLA** is the top API risk: `/runs/42` must check that run 42 belongs to the caller's tenant); request size limits; no sensitive data in URLs because they end up in logs; DTOs rather than entities on the wire, which prevents mass assignment and accidental exposure; rate limits; audit logs.

Multi-tenant paths



RefinePoint: most routes carry the workspace in the path, `/api/v1/organizations/{orgCode}/...`, so it is visible in logs and links and the interceptor can check it uniformly. Routes about "the workspace open in the session" use the `X-Org-Code` header instead, read by `OrgContextService`. Either way, the code from the client is only a *claim*: it is resolved against the caller's membership on the server, and every query then filters on the resolved `org_id`.

18.12 Rate limiting

Algorithm	Behaviour	Trade-off
Fixed window	Count per minute	Simple; allows a burst of double the limit at the boundary
Sliding window	Rolling count	Accurate; stores more
Token bucket	Refill at rate R, capacity B	Allows bursts up to B, smooth average
Leaky bucket	Constant outflow	Smooths traffic, adds queueing

Answer with 429, `Retry-After`, and ideally `X-RateLimit-Limit` / `Remaining`. In-memory counters limit per instance, so N instances allow N times the limit; a shared limit needs Redis with an atomic script. Key by API key or user first and IP second, since many users can share a NAT address.

> **RefinePoint:** `RateLimitFilter` in api-gateway is an in-memory token bucket in a `ConcurrentHashMap`: 300 per minute per API key, 900 per user, 1800 per IP address, all configurable. Past the limit it returns 429 with `Retry-After`, and every answer carries `X-RateLimit-Limit` and `X-RateLimit-Remaining`. CORS preflights, health checks and Paddle webhooks are not counted. Inside services, `SlidingWindowLimiter` protects specific expensive operations. Both are per instance; that is a stated trade-off, a safe floor rather than an exact global limit.

18.13 Testing with MockMvc

Test type	Starts	Speed	Use for
`@WebMvcTest(Controller.class)`	MVC slice, no server; services mocked with `@MockitoBean`	Fast	Mapping, validation, status codes, JSON shape
`@SpringBootTest` + `@AutoConfigureMockMvc`	Whole context, no server	Medium	Full stack, real security
`@SpringBootTest(webEnvironment = RANDOM_PORT)`	Real Tomcat	Slowest	Real HTTP, filters, streaming
`@RestClientTest`	Client slice + `MockRestServiceServer`	Fast	Outbound clients

```
mvc.perform(post("/api/v1/alert-rules")
        .header("X-Org-Code", "team")
        .with(jwt())                          // spring-security-test: fake authenticated JWT
        .contentType(MediaType.APPLICATION_JSON)
        .content(rule))
   .andExpect(status().isOk())
   .andExpect(jsonPath("$.success").value(true));
```

MockMvc calls `DispatcherServlet` directly, without a socket. Filters run if registered, but anything async needs `asyncDispatch`, and real connector behaviour (`max-http-header-size`, chunking) is not exercised. Spring 6.2 adds `MockMvcTester` with AssertJ-style assertions; Boot 3.4 replaces `@MockBean` with `@MockitoBean`.

18.14 Interview questions and answers

Q1. Explain the Spring MVC request flow.

The container runs the filter chain, then `DispatcherServlet.doDispatch` finds a handler chain via `HandlerMapping`, picks a `HandlerAdapter`, runs interceptor `preHandle`, resolves arguments, invokes the controller, writes the return value through a converter for `@ResponseBody`, runs `postHandle`, handles exceptions through resolvers or renders a view, runs `afterCompletion`, and the filters unwind. Saying the body is written *before* `postHandle` marks you as someone who has debugged this.

Q2. What is the front controller pattern?

One entry point receives all requests and delegates to handlers, centralising routing, error handling and cross-cutting behaviour. `DispatcherServlet` is Spring's implementation, and it finds its collaborators (mappings, adapters, resolvers) as beans, so each is replaceable.

Q3. How is the `DispatcherServlet` registered in Boot if there is no `web.xml`?

`DispatcherServletAutoConfiguration` creates the servlet bean and a `DispatcherServletRegistrationBean`, a `ServletContextInitializer` that the embedded server runs at startup, mapped to `spring.mvc.servlet.path` (default `/`). `load-on-startup` defaults to -1, so it initialises on the first request; setting it to 1 removes that first-call latency.

Q4. `@Controller` vs `@RestController`?

`@RestController` is `@Controller` + `@ResponseBody`. With plain `@Controller` a returned `String` is a view name, resolved by a `ViewResolver`. With `@RestController` it goes through a converter and is written as the body. You can mix by putting `@ResponseBody` on individual methods.

Q5. Filter vs interceptor vs aspect: when each?

A filter is servlet-level and sees every request, including those MVC never maps, and can wrap the request or response. An interceptor is MVC-level and knows the handler method. An aspect wraps any bean method regardless of HTTP. Use filters for authentication and tracing, interceptors for route-aware checks like tenant access, and aspects for transactions and metrics.

Q6. What order do interceptor callbacks run in?

With interceptors A then B: `A.preHandle, B.preHandle, handler, B.postHandle, A.postHandle, B.afterCompletion, A.afterCompletion`. If `B.preHandle` returns false, the handler and `postHandle`s are skipped, and only `A.afterCompletion` runs, because `afterCompletion` runs only for interceptors whose `preHandle` returned true. On an exception, `postHandle` is skipped but `afterCompletion` still runs.

Q7. Why doesn't my exception thrown in a servlet filter reach `@ControllerAdvice`?

Because filters run outside `DispatcherServlet`, and exception resolvers live inside it. The exception goes to the container, which forwards to `/error`. Write the error response in the filter, or inject `handlerExceptionResolver` and delegate to it.

Q8. How do you read the request body twice, for example to log it?

The servlet input stream can be read once. Wrap the request in a filter with `ContentCachingRequestWrapper`, which caches bytes as they are read; note that it fills only *after* the body is consumed downstream, so log in the filter after `chain.doFilter`. Limit the size, and redact sensitive data. For responses use `ContentCachingResponseWrapper` and call `copyBodyToResponse()`, or the client gets an empty body.

Q9. How does Spring decide which converter to use?

It looks at the body's Java type and media type. For requests, the `Content-Type` selects a converter whose `canRead` is true, else 415. For responses, it intersects `Accept` with what converters can produce for the return type (narrowed by `produces`), picks the most specific by quality, else 406. Boot's default for objects is `MappingJackson2HttpMessageConverter`.

Q10. Why would a controller return 406 though the JSON is fine?

The client's `Accept` has nothing the server can produce: `Accept: application/xml` with no XML converter, or `produces = "application/json"` with `Accept: text/csv`. It also occurs when an error handler tries to write JSON to a client that accepts only `text/event-stream`, which is exactly the case RefinePoint's `forClient` handles.

Q11. How do you customise Jackson without losing Boot's defaults?

Use `spring.jackson.*` properties or a `Jackson2ObjectMapperBuilderCustomizer` bean; both feed the builder Boot's `ObjectMapper` is made from. Defining your own `ObjectMapper` bean disables that. For a one-off, use a dedicated mapper for that use, not the primary bean.

Q12. `@RequestParam` vs `@PathVariable` vs `@RequestBody`?

Path variables identify a resource, query parameters filter or modify a read, and the body carries a representation for create or update. A missing required param gives 400. `@RequestBody` needs a converter and a compatible `Content-Type`.

Q13. `@Valid` vs `@Validated`?

`@Valid` is standard Jakarta, triggers validation and cascades into nested objects. `@Validated` is Spring's: it adds validation groups and, at class level, enables method validation via an AOP proxy. Use `@Valid` on nested fields and request bodies, `@Validated` for groups or on services.

Q14. Which exception does validation throw?

`@Valid @RequestBody` gives `MethodArgumentNotValidException`. Constraints directly on controller method parameters give `HandlerMethodValidationException` from Spring 6.1's built-in method validation. Class-level `@Validated` on a service gives `ConstraintViolationException`. Handling only the first is a common gap, which lets the others surface as 500s or inconsistent bodies.

Q15. Why is my `@NotNull` on a nested object not validated?

Validation does not cascade without `@Valid` on the field. `List<@Valid Item> items` or `@Valid List<Item> items` both cascade into elements.

Q16. How do you write a custom validator, and why return true for null?

Define a constraint annotation with `message`, `groups` and `payload`, plus a `ConstraintValidator` implementation. Return true on null so nullability stays a separate decision via `@NotNull`, making constraints composable. For rules across fields, put the annotation on the class.

Q17. How does `@ControllerAdvice` work? What if two handlers match?

At startup `ExceptionHandlerExceptionResolver` collects advice beans. On an exception it checks the controller's own `@ExceptionHandler` methods first, then advices in `@Order` order; within one class the closest match in the exception hierarchy wins. Causes are also checked. Advice can be scoped with `basePackages`, `assignableTypes` or `annotations`.

Q18. What is ProblemDetail? Would you use it?

A standard JSON error body (`type`, `title`, `status`, `detail`, `instance` plus extensions) from RFC 7807, updated by RFC 9457, with media type `application/problem+json`. Spring 6 has `ProblemDetail` and `ErrorResponse`, and Boot enables it through `spring.mvc.problemdetails.enabled`. I would use it for a public API. In RefinePoint we kept an `ApiResponse` envelope because one first-party client gets the same parse on success and failure, and I would call that a deliberate trade-off.

Q19. Should a 200 response ever contain an error?

No. Status codes drive retries, caches, gateways, monitoring and alerting. A `200` with `{"success": false}` makes error rates invisible to all of them. An envelope is fine as long as the status matches, as ours does.

Q20. PUT vs PATCH vs POST?

POST creates or processes, and the server chooses the URI. PUT replaces the full resource at a client-known URI and is idempotent. PATCH partially updates and is not guaranteed idempotent. `{"op":"increment"}` is a non-idempotent PATCH; JSON Merge Patch (RFC 7386), where null means delete, is usually idempotent. With a partial DTO you must distinguish "absent" from "null", which plain records cannot: use `JsonNode`, `Optional` fields or Merge Patch.

Q21. Is DELETE idempotent if the second call returns 404?

Yes. Idempotency concerns server state, not the response. After one or many deletes the resource is gone. Some APIs return 204 on repeats for friendliness; both are valid.

Q22. 401 vs 403? 400 vs 422? 404 vs 403 for another tenant's resource?

401 means we do not know who you are, 403 means we know and you may not. 400 is malformed, 422 is well-formed but semantically invalid; many teams use 400 for both. For another tenant's resource return 404, so that the API does not reveal the id exists.

Q23. How do you model a long-running operation?

`POST` returns 202 with a `Location` to a status resource; the client polls or receives a webhook or event. RefinePoint runs work this way: a run is created QUEUED and a `RunWorker` claims it, while the status lives on the run itself. Holding an HTTP request open for minutes would tie up threads and fail on every proxy timeout.

Q24. Offset vs keyset pagination?

Offset is simple and allows page jumps, but the database reads and discards every skipped row, and concurrent inserts shift pages. Keyset (`WHERE (created_at, id) < (?, ?) ORDER BY created_at DESC, id DESC LIMIT 50`) stays fast at any depth with a composite index and is stable, but cannot jump to page 500. Use keyset for feeds, exports and big tables.

Q25. How would you design filtering and sorting?

Simple query params for common fields (`?status=FAILED&from=...`). Whitelist filter and sort fields, map them to indexed columns, cap page size. A generic filter language is a large surface that can hit unindexed columns, so add one only if needed. Never concatenate field names into SQL.

Q26. How do you version an API? What is a breaking change?

URI versioning for most teams, because it is visible and routable. Breaking changes: removing or renaming a field, changing a type or meaning, adding a required input, tightening validation, changing status codes. Adding an optional field or endpoint is not, provided clients ignore unknown fields (Boot's Jackson default). Run old and new side by side with a deprecation window and usage metrics.

Q27. What is HATEOAS and do you use it?

Hypermedia as the engine of application state: responses include links to the available next actions. It decouples clients from URL structure, but it costs effort and few clients actually navigate by links. I would consider it for a long-lived public API; for first-party clients, OpenAPI is enough.

Q28. What are ETags used for?

Conditional GET: send `If-None-Match`, get 304, save bandwidth. Optimistic concurrency: `If-Match` on PUT gets 412 if someone changed the resource. `ShallowEtagHeaderFilter` saves bandwidth only, since it computes the response before comparing. A version-based ETag can skip the work too.

Q29. How do you make POST idempotent?

A client-generated `Idempotency-Key`, stored with a unique constraint scoped by tenant in the same transaction as the business change. On repeat, return the stored result. The constraint decides the race between concurrent retries; a check-then-insert alone does not. Also compare a hash of the body, to reject a reused key with a different payload.

Q30. What happens if the client retries while the first request is still in progress?

Without coordination, both run. With a unique constraint, the second blocks or fails on insert. Then it can return 409 ("in progress"), wait for the first result, or return the stored row. Payment APIs typically return 409 for in-flight duplicates and let the client retry later.

Q31. How do you handle a 2 GB file upload?

Do not buffer it: stream it with a multipart temp file, or better, give the client a presigned URL to object storage so the app never handles the bytes. Then process asynchronously via a queue, with checksums and resumable uploads for reliability. Raise `spring.servlet.multipart.*`, the gateway limit and the proxy limit together, because a request must pass all three.

Q32. `StreamingResponseBody` vs returning `Resource`?

`Resource` suits an existing file: the converter streams it, and handles `Range` for resume. `StreamingResponseBody` suits generated content written incrementally; it runs on the MVC async executor. Neither loads everything onto the heap, unlike `byte[]`.

Q33. What can go wrong while streaming a response?

The status is already committed, so a mid-stream error appears as a truncated file: add a trailer record or checksum if integrity matters. Transaction and persistence context have ended by the time the lambda runs. `ThreadLocal`s (security, tenant) are not there unless propagated. Async timeouts cut long streams. Proxies may buffer unless told otherwise.

Q34. How does CORS work and why does a preflight get 401?

For non-simple requests the browser sends `OPTIONS` first, with no `Authorization` header. If the security chain runs before CORS handling, it rejects the preflight with 401, and the browser reports a CORS error. Enabling `http.cors(...)` makes Spring Security's `CorsFilter` answer it first. In RefinePoint the gateway handles CORS, and the rate limiter deliberately does not count preflights.

Q35. Is CORS a security feature for my API?

It protects *browser users* from other sites reading responses with their credentials. It does not protect your API: any non-browser client ignores it. Authentication and authorisation still have to be enforced.

Q36. Is CSRF protection needed for a REST API?

Only if the browser sends credentials automatically, as with session cookies. With a bearer token in the `Authorization` header, a malicious site cannot attach it, so CSRF protection is usually disabled for such stateless APIs, as in our security chains. If you keep tokens in cookies, you need CSRF protection and `SameSite`.

Q37. RestTemplate vs WebClient vs RestClient?

`RestTemplate` is blocking and in maintenance. `WebClient` is non-blocking, needs the Reactor dependency, and is right for reactive stacks or heavy fan-out. `RestClient` (Spring 6.1) is synchronous with a fluent API: my default for MVC services, particularly with virtual threads. Using `WebClient` with `.block()` everywhere pays for Reactor and gains nothing.

Q38. What are HTTP interfaces?

A declarative client: annotate an interface with `@GetExchange` or `@PostExchange`, and `HttpServiceProxyFactory` creates a proxy backed by `RestClient`, `WebClient` or `RestTemplate`. It is Feign's model without the extra dependency. It suits typed clients for stable APIs; a hand-written wrapper suits complex error handling.

Q39. What timeouts do you set on an outbound call?

Connect (1–2 s), read (based on the dependency's p99, well under the caller's own deadline), connection pool acquire, and an overall deadline if a trickling response matters. Our `InternalClient` defaults to 2 s connect and 10 s read. Missing timeouts are the most common cause of cascading failure: threads pile up waiting on one slow dependency.

Q40. When is retrying harmful?

On non-idempotent calls without a key, since it produces duplicates. On overload, since retries add load (retry storm). When layered, since attempts multiply. And when the total time exceeds the caller's deadline, because the work is wasted. Use backoff with jitter, a retry budget, one retry layer and a circuit breaker.

Q41. A service's latency spiked and all threads are busy. How do you diagnose it?

Check metrics first: `tomcat.threads.busy`, `hikaricp.connections.pending`, `http.client.requests` per dependency, GC pauses. Take a thread dump: threads parked in `HikariPool.getConnection` mean a pool bottleneck or slow queries, threads in socket read mean a slow dependency without a timeout, many on one monitor mean lock contention. Mitigate with timeouts and bulkheads, then fix the root cause. Raising `threads.max` usually moves the queue rather than removing it.

Q42. How do you expose a file-processing API without tying up Tomcat threads?

Accept the upload, save it, enqueue, return 202. Workers process from the queue, and the client polls or gets notified. MVC's `Callable`, `DeferredResult` or `SseEmitter` free the container thread while waiting for a result. On virtual threads the thread cost is low, but a queue still gives you retries, durability and backpressure.

Q43. `DeferredResult` vs `Callable` vs `SseEmitter`?

`Callable` runs your code on the MVC async executor. `DeferredResult` is completed later by any thread, such as a Kafka listener. `SseEmitter` and `ResponseBodyEmitter` send many items over one connection. In each, the container thread goes back to the pool until there is a result to write, and the request is dispatched again to finish.

Q44. SSE vs WebSocket vs polling?

Polling is simple and wasteful. SSE is one-way over plain HTTP, reconnects automatically, and goes through proxies when buffering is off. WebSocket is two-way and needs more infrastructure. Notifications fit SSE: RefinePoint's inbox stream is a `text/event-stream` with `SseEmitter`. Each open stream is a held connection, so plan capacity, and with multiple instances the event must reach whichever instance holds the connection.

Q45. How do you enforce tenant isolation at the API layer?

Resolve the tenant from a claim in the request (path or header) against the authenticated user's memberships, on the server, in one central place. Then pass the resolved `org_id` into every query. RefinePoint does this with `OrgAccessInterceptor` plus `orgs.resolveOrgIdFromCode(orgCode)`, and `BaseOrgEntity` makes every tenant row carry `org_id`. The classic bug is a repository `findById(id)` that ignores the tenant; use `findByOrgIdAndId`, and return 404 otherwise.

Q46. Path vs header for the tenant?

A path is visible in logs, bookmarkable, cacheable per tenant, and lets a route-based interceptor check it uniformly. A header keeps URLs clean, suits a "current workspace" session, and needs care in caches (`Vary`). RefinePoint mostly uses the path, with `X-Org-Code` for session-scoped routes. Neither is trusted on its own.

Q47. How do you protect against IDOR?

Authorise each object, not just the endpoint. Scope every lookup by tenant and owner, use non-guessable ids (we use UUIDs) as defence in depth rather than as the control, and write tests in which user A requests user B's id and expects 404.

Q48. Why not return JPA entities from controllers?

It exposes the schema and couples the API to the database. Lazy relations trigger extra queries or `LazyInitializationException` during serialisation, and `@RequestBody` entities invite mass assignment (a client sets `orgId` or `role`). DTOs or records give a stable contract.

Q49. How do you design rate limiting across multiple gateway instances?

In-memory buckets are per instance, so N instances allow N times the limit. For an exact limit, use Redis with an atomic Lua script or a dedicated rate-limit service, adding a network hop and a dependency that needs a fail-open decision. RefinePoint's gateway is in-memory by choice: simple, no dependency, a safe floor. I would add Redis if a contractual per-customer limit required it.

Q50. Token bucket vs sliding window?

Token bucket allows controlled bursts and is cheap: two numbers per key, computed lazily on each request from elapsed time. Sliding window is more exact in its count but stores more. Fixed window is simplest, but lets twice the limit through at a boundary. Our gateway uses a token bucket refilled evenly over a minute; our in-service `SlidingWindowLimiter` is used where a strict rolling count matters.

Q51. What should a 429 response include?

`Retry-After`, a machine-readable error, and rate headers so clients can self-throttle. RefinePoint's body uses the same envelope with `errors: ["RATE_LIMITED"]`. Clients should back off with jitter.

Q52. `@WebMvcTest` vs `@SpringBootTest`?

`@WebMvcTest` loads only the MVC slice with mocked collaborators: quick, for mapping, validation, serialisation and security rules. `@SpringBootTest` loads everything: for integration through to a real database (with Testcontainers). Keep many slice tests and fewer full tests. Each distinct context configuration costs a new context, so reuse them via the test context cache.

Q53. What does MockMvc not test?

The real network and container: connector limits, header size, chunked behaviour, TLS, and exact error dispatch to `/error`. Async needs `asyncDispatch`, and streaming behaviour differs. Use a `RANDOM_PORT` test for those.

Q54. Why does a trailing slash now return 404 after upgrading to Boot 3?

Spring 6 removed trailing-slash matching by default, so `/runs/` no longer matches `/runs`. Fix clients, or add explicit mappings, or (temporarily) add a filter that redirects. Avoid re-enabling it permanently: differing path interpretation between security rules and handlers has been a source of bypass bugs.

Q55. How do you handle date and money in JSON?

Dates: ISO 8601 with offset (`Instant`, `OffsetDateTime`), UTC on the wire, and let the client localise. RefinePoint stores timestamps in UTC, and the UI shows each person's time zone. Money: `BigDecimal` plus a currency code, never `double`, and consider serialising as a string, since JavaScript numbers are doubles and silently lose precision on large or precise amounts.

Q56. What is content negotiation and how does it interact with error handling?

Choosing a representation from `Accept` and `produces`. If a handler says `produces = "text/csv"` and throws, Spring removes the producible attribute so the error handler can write JSON (and if the client accepts only CSV, it may fail with 406). So design errors around what your clients accept, and test the error path, including for SSE and file endpoints.

Q57. What makes a good REST API, in your view?

Predictable resource names, correct status codes, consistent errors, pagination everywhere, idempotency for retries, additive evolution, an OpenAPI contract tested in CI, authorisation on each object, timeouts and limits, and traces and metrics per route. Consistency beats cleverness. RefinePoint enforces most of this through `ryfn-platform`, so each new service inherits it without having to remember it.

Q58. How does the gateway add value over putting everything in each service?

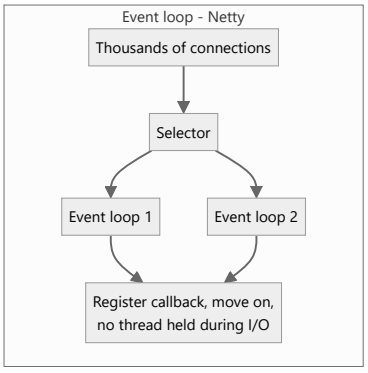
It gives one entry point for routing, TLS, CORS and coarse rate limiting, so services need not repeat them, and clients need only one host. It must stay thin: business logic belongs in services. Authorisation still happens in the services too: RefinePoint's gateway reads the JWT subject only to count requests and does not verify the signature, leaving verification to each service, so nothing depends on the gateway alone.

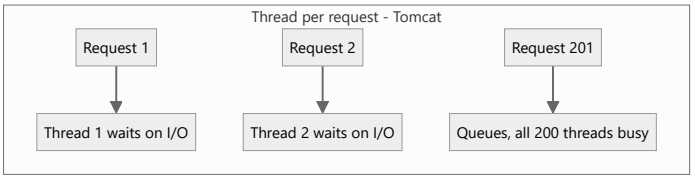
19. Reactive Programming & Spring WebFlux

The interview question has shifted. It used to be "explain `Mono` and `Flux`". Since Java 21 it is "you have virtual threads now, so why would you still use WebFlux?". To answer well you need to know what reactive really gives you (backpressure, composition, fewer threads), what it costs (debugging, context, a coloured API), and how the machinery works: who subscribes, which thread runs what, and why one blocking call can stall thousands of requests. RefinePoint is a good honest case: 12 services are Spring MVC, and exactly one, the gateway, is reactive.

19.1 Why reactive

In thread-per-request, each in-flight request holds a thread for its whole life, including time spent waiting on I/O. A service that spends 95% of a request waiting on the network uses a 1 MB-stack platform thread to do nothing.





Reactive programming is a declarative style for asynchronous data streams with **backpressure**. You describe a pipeline; when data arrives, the framework runs the next step on whatever thread is available. Benefits:

- **Fewer threads under high concurrency**, especially many slow or idle connections (gateways, streaming, fan-out).
- **Backpressure**: a slow consumer can limit a fast producer rather than be overwhelmed.
- **Composition**: timeouts, retries, parallel calls and fallbacks are one operator each.

Costs, which a senior answer names unprompted:

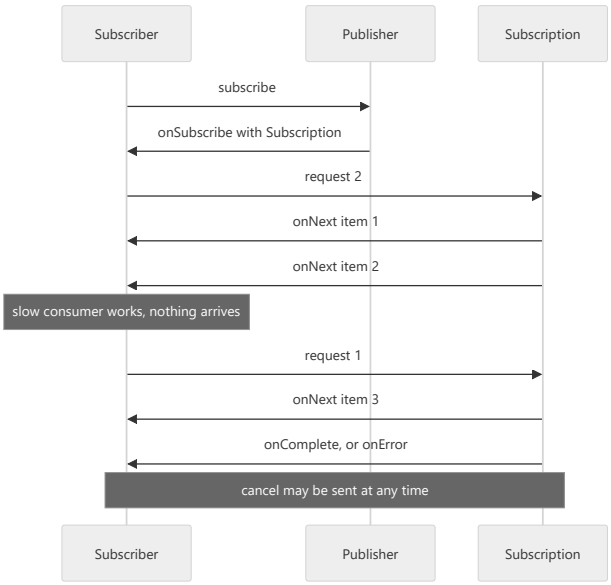
- Stack traces become useless without tooling.
- `ThreadLocal` context (MDC, security, transactions) does not work the normal way.
- One blocking call on an event loop stalls every request on that loop.
- JDBC, JPA and most existing libraries are blocking.
- Harder to read, write, review and hire for.
- No throughput gain for CPU-bound work.

Reactive does not make a single request faster. It improves *capacity under concurrency*.

19.2 Reactive Streams: the specification

Reactive Streams is a small spec of four interfaces, adopted into the JDK as `java.util.concurrent.Flow` in Java 9. Reactor, RxJava and Akka Streams all implement it.

```
public interface Publisher<T>  { void subscribe(Subscriber<? super T> s); }
public interface Subscriber<T> {
    void onSubscribe(Subscription s);
    void onNext(T t);
    void onError(Throwable t);
    void onComplete();
}
public interface Subscription  { void request(long n); void cancel(); }
public interface Processor<T, R> extends Subscriber<T>, Publisher<R> { }
```



Key rules:

- **Nothing happens until `subscribe`.** A pipeline is only a description.
- **Demand flows upstream, data flows downstream.** A publisher must not emit more than the total requested. That is backpressure: pull-based flow control over a push-based model.
- Signal order: `onSubscribe onNext* (onError | onComplete)?`. At most one terminal signal, and signals to one subscriber are serial (never concurrent).
- `onError` and `onComplete` are terminal; nothing follows them.

Backpressure in practice

```
source ⟶ map ⟶ flatMap ⟶ subscriber
       ⟵ request(32) ⟵ request(256) ⟵ request(unbounded)
```

Each operator translates demand. `flatMap` requests 256 inner publishers by default; each inner prefetches 32. `subscribe(consumer)` requests `Long.MAX_VALUE`: no backpressure at all from that subscriber.

If a source cannot slow down (mouse clicks, a hot sink, a Kafka broker you must keep up with), you choose an overflow strategy:

Operator	When demand is lacking
`onBackpressureBuffer(n)`	Buffer up to n, then error or apply a strategy
`onBackpressureDrop()`	Discard new items
`onBackpressureLatest()`	Keep only the most recent
`onBackpressureError()`	Signal `OverflowException`
`limitRate(n)`	Rewrite demand into batches of n

Over HTTP, backpressure maps onto **TCP flow control**: when the client reads slowly, the socket buffer fills, Netty stops writing, and demand stops propagating. Across a boundary that has no flow control (a plain message broker pushing at you, for instance), it stops being automatic.

19.3 Project Reactor: `Mono` and `Flux`

`Mono<T>` is 0 or 1 item; `Flux<T>` is 0 to N. Both are publishers, and Reactor is the library behind WebFlux.

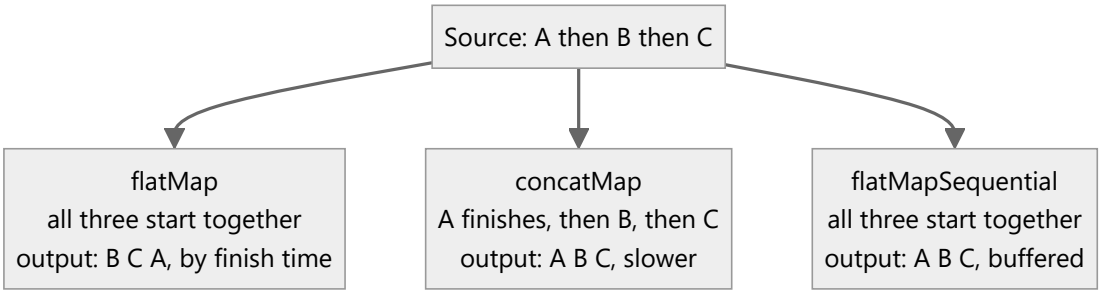
```
Mono<String> a = Mono.just("x");                  // value computed NOW, at assembly
Mono<String> b = Mono.fromCallable(this::load);   // computed at each subscribe
Mono<String> c = Mono.defer(() -> Mono.just(load())); // whole Mono built at each subscribe
Flux<Integer> d = Flux.range(1, 5);
Flux<Long> e = Flux.interval(Duration.ofSeconds(1));
```

Assembly time vs subscription time. Building a pipeline is assembly; it constructs a chain of operator objects. `subscribe` walks back up the chain, and data then flows down. Each call to an operator returns a **new** publisher, and the original is unchanged, so this is a classic bug:

```
Flux<Integer> f = Flux.just(1, 2, 3);
f.map(x -> x * 2);      // result discarded: no effect at all
f.subscribe(System.out::println);   // prints 1 2 3
```

Operators you must know

Operator	Purpose	Ordering / concurrency
`map`	Sync 1:1 transformation	Same thread, keeps order
`flatMap`	Async: each item to a publisher, merged	**Concurrent, order not kept**, default 256
`concatMap`	Async, one at a time	Sequential, keeps order
`flatMapSequential`	Async, concurrent, **output in source order**	Buffers early finishers
`zip`	Combine latest from each, pairwise	Waits for all; ends when any ends
`merge`	Interleave as they arrive	Eager subscribe to all
`concat`	One after another	Lazy subscribe to next
`switchIfEmpty`	Fallback publisher if empty	Argument built at assembly!
`defaultIfEmpty`	Fallback value if empty	
`onErrorResume`	Fallback publisher on error	Can filter by type
`onErrorReturn`	Fallback value on error	
`onErrorMap`	Translate exception	
`retryWhen`	Resubscribe with a policy	Backoff, jitter, filter
`timeout`	Error if no signal in time	Cancels upstream
`doOnNext`, `doOnError`, `doFinally`	Side effects	Do not change items
`collectList`	Flux to `Mono<List>`	Buffers everything
`buffer`, `window`	Batches	



The `flatMap` decision is asked constantly. Pick `map` when the function is synchronous and returns a value; pick `flatMap` when it returns a `Mono` / `Flux`. Using `map` with an async function gives you `Mono<Mono<T>>`, and the inner never subscribes, so the call silently never happens.

> **RefinePoint:** the gateway's `StatusController` builds the public status page with real operators. Each component's check is `http.get().uri(part.url()).retrieve().toBodilessEntity().timeout(TIMEOUT).map(...).onErrorResume(e -> Mono.just(down))`, and the page is `Flux.fromIterable(parts).flatMapSequential(this::check).collectList()`. `flatMapSequential` is exactly right: services are checked concurrently, so one slow service does not delay the others, yet the page lists them in a stable order. `onErrorResume` converts a failure into a "down" value, so one dead service cannot fail the whole page.

Retry with backoff:

```
webClient.get().uri("/rates").retrieve().bodyToMono(Rate.class)
    .timeout(Duration.ofSeconds(2))
    .retryWhen(Retry.backoff(3, Duration.ofMillis(200))
        .jitter(0.5)
        .filter(e -> e instanceof TimeoutException || e instanceof WebClientResponseException.ServiceUnavailable)
        .onRetryExhaustedThrow((spec, signal) -> signal.failure()))
    .onErrorResume(WebClientResponseException.NotFound.class, e -> Mono.empty());
```

`retry` works by **resubscribing** upstream. On a cold HTTP Mono it sends a new request. Put `timeout` *before* `retryWhen` for a per-attempt timeout; put another *after* for an overall deadline. `Retry.backoff` by default also retries everything, so always add a filter (no retrying a 400).

> **Trap:** `switchIfEmpty(Mono.just(expensive()))` or `switchIfEmpty(repo.save(x))` builds its argument at assembly, on every request, even when the upstream is not empty. If the argument's *construction* does work (a method that has side effects before returning its `Mono`), it runs every time. Wrap with `switchIfEmpty(Mono.defer(() -> repo.save(x)))`. The same applies to `Mono.just(load())`.

19.4 Cold vs hot

	Cold	Hot
Data starts	At each subscribe, per subscriber	Regardless of subscribers
Late subscriber	Sees everything from the start	Sees only what comes next
Examples	`WebClient` call, `Flux.range`, R2DBC query	`Sinks.many()`, `Flux.interval` shared, UI events

Two subscribers to a cold Mono make two HTTP calls. Convert when needed:

```
Mono<Rate> once = client.fetch().cache();                    // first result replayed to all
Flux<Tick> shared = ticks.share();                           // publish + refCount: multicast while any subscriber
Flux<Tick> replay = ticks.replay(10).autoConnect(1);         // late subscribers get last 10

Sinks.Many<Event> sink = Sinks.many().multicast().onBackpressureBuffer();  // push from imperative code
sink.tryEmitNext(event);
Flux<Event> events = sink.asFlux();
```

19.5 Schedulers: `publishOn` vs `subscribeOn`

Reactor is **concurrency-agnostic**: by default everything runs on the thread that called `subscribe`, or the thread that emitted the signal. Operators do not imply new threads, except time-based ones (`delayElements`, `timeout`, `interval`, which default to `parallel`).

Scheduler	Threads	For
`Schedulers.parallel()`	CPU cores	Non-blocking, CPU work
`Schedulers.boundedElastic()`	10 × cores, with a task queue per thread	**Wrapping blocking calls**
`Schedulers.single()`	1	Serial work
`Schedulers.immediate()`	Current	No hop
`Schedulers.fromExecutorService(...)`	Yours	A dedicated pool



- `publishOn` changes the thread for signals going **downstream** from where it is written. Multiple `publishOn`s each apply below themselves.
- `subscribeOn` changes the thread on which the subscription travels up and the source runs. Its position does not matter. With several, **the one closest to the source wins**, but only until a `publishOn` switches again.

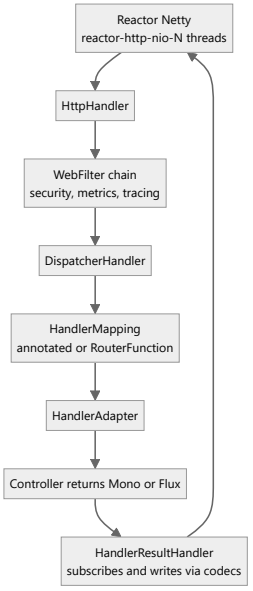
```
Flux.range(1, 2)
    .map(i -> log("A", i))
    .publishOn(Schedulers.parallel())
    .map(i -> log("B", i))
    .subscribeOn(Schedulers.boundedElastic())
    .subscribe();
// A on boundedElastic-1, B on parallel-1
```

Wrap blocking calls like this:

```
Mono<Report> report = Mono.fromCallable(() -> legacyJdbc.load(id))   // blocking
    .subscribeOn(Schedulers.boundedElastic());                        // moved off the event loop
```

Reactor 3.6 on JDK 21 can run `boundedElastic` on virtual threads (`-Dreactor.schedulers.defaultBoundedElasticOnVirtualThreads=true`).

19.6 The WebFlux stack and the Netty event loop



The names are parallel to MVC: `DispatcherHandler`, `WebFilter`, `ServerWebExchange` (rather than request/response objects) and `HttpMessageReader` / `Writer` codecs. Annotated controllers look the same; the alternative is functional `RouterFunction` + `HandlerFunction`.

Who subscribes? You do not. The framework subscribes to what your controller returned. Calling `subscribe()` yourself inside a handler creates a detached pipeline that is unrelated to the request, its errors and its context. That is a common bug.

Event loop model. Reactor Netty runs an event loop group: `reactor.netty.ioWorkerCount` threads, default `max(cores, 4)`. Each channel is pinned to one event loop for life, and each event loop processes I/O events and tasks from a queue in order.

```
Event loop reactor-http-nio-2:
  ├─ socket A readable   → decode → run pipeline → WebClient.send → return
  ├─ socket B writable   → write next buffer      → return
  ├─ socket C (backend reply for A) → continue A's pipeline → write → return
  └─ if ANY of these blocks for 200 ms, every channel on this loop waits 200 ms
```

Blocking in reactive, and BlockHound

Never block an event loop. With 8 event loops, 8 blocked calls stop the service. Symptoms: latency spikes across unrelated endpoints, while CPU is low and thread dumps show `reactor-http-nio-*` in `socket read` or `Thread.sleep`.

Common accidental blockers: JDBC, `RestTemplate`, file I/O, `Thread.sleep`, `.block()`, and logging to a slow synchronous appender.

Calling `.block()` on a non-blocking thread fails fast with `IllegalStateException: block()/blockFirst()/blockLast() are blocking, which is not supported in thread reactor-http-nio-2`. For the rest, **BlockHound** instruments JDK blocking methods and throws if they are called on a thread marked non-blocking. Install it in tests:

```
@BeforeAll static void setup() { BlockHound.install(); }
```

BlockHound detects calls such as sleep, socket and file I/O. It does not detect long CPU work, and a tight `synchronized` block is generally acceptable.

> **RefinePoint:** `RateLimitFilter.Bucket` has short `synchronized` methods that do a few double-precision operations. That is fine on an event loop: critical sections take nanoseconds and do no I/O. The bucket map is a `ConcurrentHashMap`, and idle entries are removed periodically so it cannot grow forever. What would be wrong there is a Redis or JDBC call without a reactive driver.

19.7 WebClient

```
HttpClient netty = HttpClient.create(ConnectionProvider.builder("rates")
        .maxConnections(100).pendingAcquireTimeout(Duration.ofSeconds(2))
        .maxIdleTime(Duration.ofSeconds(30)).build())
    .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 2000)
    .responseTimeout(Duration.ofSeconds(5));

WebClient client = builder.clientConnector(new ReactorClientHttpConnector(netty))
    .baseUrl("https://api.example.com")
    .codecs(c -> c.defaultCodecs().maxInMemorySize(2 * 1024 * 1024))
    .build();

Mono<Dashboard> dashboard = Mono.zip(
        client.get().uri("/a").retrieve().bodyToMono(A.class),
```

```
        client.get().uri("/b").retrieve().bodyToMono(B.class))
    .map(t -> new Dashboard(t.getT1(), t.getT2()));   // concurrent, total time = the slower one
```

Points interviewers check:

- `retrieve()` maps 4xx/5xx to `WebClientResponseException` and is the default choice. `exchangeToMono` gives response access but **you must consume or release the body**, or the connection leaks.
- The default codec buffer is **256 KB**; larger bodies raise `DataBufferLimitException`. Raise it, or stream.
- `maxIdleTime` should be shorter than the server's or load balancer's idle timeout, or you get `PrematureCloseException` from reusing dead connections.
- `Mono.zip` with one empty source completes empty and cancels the rest: that surprises people. `.defaultIfEmpty` or `Optional` wrapping avoids it.
- Build from the injected `WebClient.Builder`, to get tracing and metrics.

> **RefinePoint:** the gateway's `ApiDocsController` fetches each service's OpenAPI document with `WebClient`, `timeout(5 s)`, a 5-minute in-memory cache, and `onErrorResume` that returns the last good copy, or a 503 if there is none. It raises `maxInMemorySize` because API documents exceed the default. It is a clean, small example of graceful degradation in one reactive pipeline.

19.8 R2DBC

JDBC is blocking by design, so a truly reactive stack needs **R2DBC**: a non-blocking SPI with drivers for PostgreSQL, MySQL, SQL Server and others. Spring Data R2DBC provides `ReactiveCrudRepository`, `R2dbcEntityTemplate` and `DatabaseClient`.

```
interface RunRepo extends ReactiveCrudRepository<Run, UUID> {
    Flux<Run> findByOrgId(UUID orgId);
}
Mono<Run> saved = tx.transactional(repo.save(run));   // TransactionalOperator; @Transactional also works
```

Limitations that matter when deciding:

- **It is not an ORM.** No lazy loading, no dirty checking, no relationships, no second-level cache; closer to Spring Data JDBC.
- Transactions are tied to the Reactor Context, not a `ThreadLocal`, so they need a `ReactiveTransactionManager`.
- Flyway and Liquibase still use JDBC for migrations.
- Fewer tools and less community experience than JDBC, and the database's connection limit is unchanged: reactive removes thread waits, not database capacity.

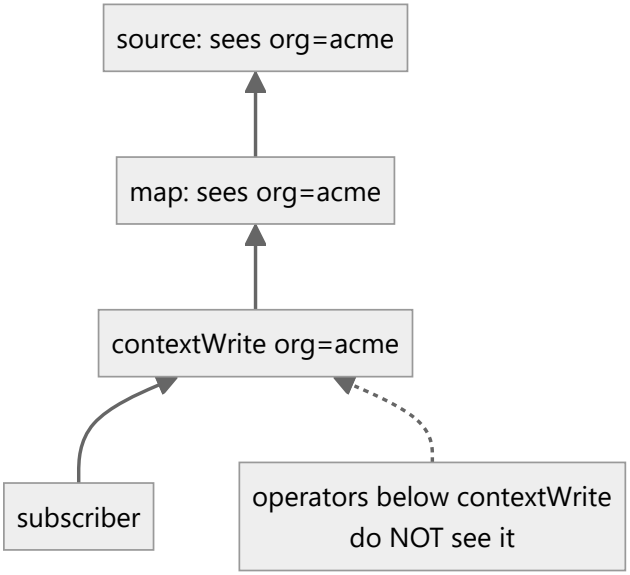
Wrapping JDBC in `boundedElastic` is the pragmatic alternative; it works, but you keep a thread per query, and pool sizes set the limit.

19.9 Context propagation

`ThreadLocal` relies on a request staying on one thread. In a reactive pipeline it hops between threads, so `MDC`, `SecurityContextHolder` and `TransactionSynchronizationManager` no longer work the normal way.

Reactor Context is an immutable key-value map attached to the subscription, travelling from subscriber up.

```
Mono<String> greeting = Mono.deferContextual(ctx -> Mono.just("org=" + ctx.get("org")))
    .contextWrite(Context.of("org", "acme"));        // visible to operators ABOVE this line
```



Because the context travels at subscribe time, from the bottom up, `contextWrite` affects what is **above** it. Putting it at the top of a chain does nothing for operators beneath. Spring Security uses this as `ReactiveSecurityContextHolder`, and reactive transactions do the same.

Bridging to ThreadLocals: Micrometer's `context-propagation` library defines `ThreadLocalAccessor`s. `Hooks.enableAutomaticContextPropagation()` (Reactor 3.5.3+), or `spring.reactor.context-propagation=auto` in Boot 3.2+, restores registered ThreadLocals around each operator, so log lines keep their `traceId` in MDC. The cost is some overhead per operator.

19.10 Testing and debugging

```
StepVerifier.create(service.matchAll(orgId))
    .expectNext(first)
    .expectNextMatches(r -> r.rate() > 0.9)
    .verifyComplete();

StepVerifier.withVirtualTime(() -> Mono.delay(Duration.ofHours(1)).map(x -> "done"))
    .expectSubscription()
    .thenAwait(Duration.ofHours(1))      // no real wait
    .expectNext("done")
    .verifyComplete();

StepVerifier.create(Flux.range(1, 100), 0)  // demand 0: test backpressure
    .thenRequest(2).expectNext(1, 2).thenCancel().verify();
```

`TestPublisher` emits signals on demand and can misbehave deliberately, `PublisherProbe` checks that a fallback branch was actually subscribed, and `WebTestClient` tests endpoints. With `withVirtualTime` the publisher must be created *inside* the supplier, otherwise real schedulers are already captured.

Debugging. A stack trace shows Reactor's scheduling frames, not your assembly code.

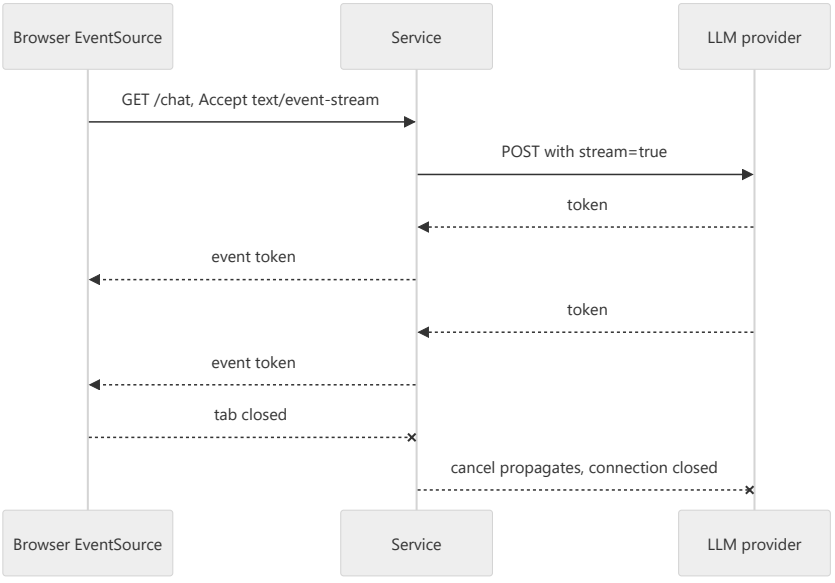
Tool	Cost	How
`checkpoint("loading rates")`	Low; add at key points	Adds description to traces
`log()`	Noisy	Prints signals: `onSubscribe`, `request`, `onNext`
`Hooks.onOperatorDebug()`	**High**: captures a stack at every assembly	Dev only
`ReactorDebugAgent` (reactor-tools)	Low: bytecode instrumentation at load time	OK in production
`name().tag().tap(Micrometer.metrics(registry))`	Low	Pipeline metrics

Also common is "my pipeline does nothing": the usual causes are no subscriber, a discarded operator result, an empty source hitting a `flatMap`, or an `onErrorResume` swallowing the error. Check with `log()`.

19.11 SSE and streaming, e.g. LLM tokens

Reactive suits streaming well, since a stream is simply a `Flux`:

```
@GetMapping(value = "/chat", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
Flux<ServerSentEvent<String>> chat(@RequestParam String prompt) {
    return llm.post().uri("/v1/chat").bodyValue(request(prompt))
        .retrieve().bodyToFlux(String.class)        // provider's token stream
        .map(token -> ServerSentEvent.builder(token).event("token").build())
        .timeout(Duration.ofSeconds(30))            // max gap between tokens
        .onErrorResume(e -> Flux.just(ServerSentEvent.builder("failed").event("error").build()));
}
```



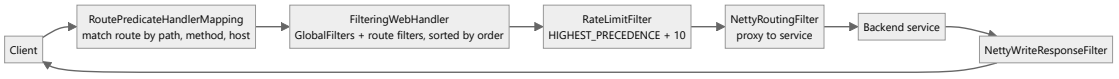
The best feature is **cancellation**: when the browser disconnects, `cancel` travels up and the provider call stops, so you stop paying for tokens no one reads. In MVC you must wire this by hand with `SseEmitter.onCompletion`.

Practical points: disable proxy buffering (nginx `X-Accel-Buffering: no`), send heartbeats so idle connections stay up, tag errors as an event because the HTTP status is already sent, and do not hold a DB connection during the stream. MVC controllers can also return `Flux` and Spring adapts it, so you do not need a full WebFlux app to stream.

> **RefinePoint:** we built this differently, and that is a fair thing to say. `LlmClient.complete` is a blocking call returning a whole `LlmResponse` (`OpenAiCompatibleClient` sends `"stream": false`), because our AI features produce structured JSON (`JsonExtractor` parses a reconciliation draft), which is useless half-finished. Live updates that do exist, such as the notification inbox, use MVC's `SseEmitter`. If I added token streaming to the help assistant, I would return a `Flux` from an MVC controller or use `SseEmitter` on a virtual thread instead of converting ai-service to WebFlux.

19.12 Spring Cloud Gateway is WebFlux

Spring Cloud Gateway (reactive variant) runs on Netty and WebFlux, which fits a gateway: tens of thousands of mostly-waiting connections, with no business logic and no JDBC.



A `GlobalFilter` returns `Mono<Void>`. Code before `chain.filter` is "pre", and `.then(...)` after it is "post":

```
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
    long start = System.nanoTime();
    return chain.filter(exchange)
        .then(Mono.fromRunnable(() -> record(exchange, System.nanoTime() - start)));
}
```

To reject, set the status and write the body without calling the chain.

> **RefinePoint:** `RateLimitFilter implements GlobalFilter, Ordered` runs at `HIGHEST_PRECEDENCE + 10`. It skips OPTIONS, `/actuator` and `/api/webhooks/`, then takes one token from the caller's bucket (API key digest, or the JWT `sub`) and from the IP bucket. If it is under the limit it returns `chain.filter(exchange)`; otherwise it sets 429, `Retry-After`, and returns `response.writeWith(Mono.just(buffer))` with our standard JSON error. It decodes the JWT payload only to find `sub`, without checking the signature, because services verify tokens. A forged `sub` buys nothing, since the IP bucket is still counted. `MetricsGuard`, a `WebFilter`, protects the gateway's `/actuator/prometheus` with a bearer token.

There is also **Spring Cloud Gateway Server MVC**, a servlet variant for teams that prefer MVC, and it pairs well with virtual threads.

19.13 WebFlux vs MVC + virtual threads

Virtual threads (Java 21) remove the main reason most teams adopted reactive: the cost of a thread waiting on I/O. Write blocking code; the JVM unmounts the virtual thread at each blocking call, and the carrier thread goes on to other work.

Criterion	MVC + virtual threads	WebFlux
Programming style	Plain imperative	Declarative pipelines
Stack traces, debugging	Normal	Need tooling
`ThreadLocal`, MDC, `@Transactional`, JPA	Work as usual	Need Context / R2DBC
Existing blocking libraries	Fine	Must be isolated
Many concurrent I/O waits	Excellent	Excellent
Backpressure in the stream	Not built in; use semaphores	**Built in**
Streaming, cancellation	Possible, manual	**First class**
Composing fan-out, timeouts, retries	Structured concurrency (preview in 21) or Resilience4j	**Operators**
Memory per connection	Low, some per stack	Lowest
Team hiring and maintenance	Easy	Harder
Blocking the wrong thing	Pinning (`synchronized` on 21)	Stalls the event loop

My decision rule:

- **New CRUD or business service with JDBC/JPA:** MVC, virtual threads if I/O concurrency is high. 90% of services.
- **Gateway, proxy, streaming fan-out, huge numbers of idle connections:** WebFlux.
- **Pipeline where backpressure and cancellation are the actual requirement:** Reactor, possibly just the pipeline in an MVC service.
- **Team already expert and stack already reactive:** stay; a rewrite buys little.
- **CPU-bound work:** neither helps. Use a bounded pool of platform threads.

> **RefinePoint:** 12 services are MVC with JPA and `JdbcTemplate`; the reactive one is api-gateway, because Spring Cloud Gateway is built on WebFlux, and it does only I/O. The run engine is CPU- and disk-bound work on DuckDB, so reactive would not help at all; capacity is managed by worker slots and queue claims. Mixing paradigms cost us nothing, because services only meet over HTTP.

19.14 Interview questions and answers

Q1. What is reactive programming?

A declarative style for asynchronous streams of data with non-blocking backpressure. You describe a pipeline of transformations; execution is triggered by subscription and driven by data arrival rather than by a thread walking through statements. It serves capacity under I/O concurrency, not single-request speed.

Q2. What are the four Reactive Streams interfaces?

`Publisher` (emits), `Subscriber` (receives via `onSubscribe`, `onNext`, `onError`, `onComplete`), `Subscription` (`request(n)`, `cancel`), and `Processor` (both). Included in the JDK as `java.util.concurrent.Flow` since Java 9. The spec defines interoperability; it is too low-level to use directly, which is what Reactor is for.

Q3. What is backpressure?

A way for a consumer to tell a producer how much it can handle: the subscriber calls `request(n)`, and the publisher must not exceed the total demand. Without it, a fast producer forces unbounded buffering and eventually an out-of-memory error. Over HTTP it maps to TCP flow control.

Q4. What if the source cannot honour backpressure?

Choose an overflow strategy explicitly: `onBackpressureBuffer` (bounded), `Drop`, `Latest` or `Error`. Hot sources such as sensor feeds cannot slow down, so you must decide whether to lose data, buffer it, or fail. Dropping is fine for latest-value gauges and wrong for financial events.

Q5. `Mono` vs `Flux`? Why have `Mono` at all?

`Mono` is 0..1, `Flux` is 0..N. `Mono` conveys single-result semantics in the type, and some operators exist only there or are optimised. `Mono<Void>` represents completion with no value, as in a `GlobalFilter`.

Q6. What does this print?

```
Flux<Integer> f = Flux.just(1, 2, 3).map(x -> { System.out.println("m" + x); return x; });
System.out.println("built");
```

Only `built`. Nothing runs without a subscriber. If `f.subscribe()` is called twice, `m1 m2 m3` prints twice, because it is a cold publisher.

Q7. `map` vs `flatMap`?

`map` is a synchronous 1:1 function returning a value. `flatMap` takes a function returning a publisher, subscribes to the inner publishers concurrently and merges their results. Use `map` for computation, and `flatMap` for any async call. Using `map` with an async function produces a nested `Mono<Mono<T>>`, and the inner never runs.

Q8. `flatMap` vs `concatMap` vs `flatMapSequential`?

`flatMap` is concurrent (default 256) with no order guarantee. `concatMap` runs one at a time, keeps order and is slowest. `flatMapSequential` runs concurrently yet emits in source order by buffering, so a slow first element holds back the output and uses memory. RefinePoint's status page uses `flatMapSequential` for concurrent checks in stable order.

Q9. How do you limit concurrency of outbound calls?

`flatMap(fn, concurrency)`, for example `flatMap(this::call, 10)`. Otherwise a `Flux` of 10,000 ids launches 256 calls at once and exhausts the connection pool, or overwhelms the downstream. It is a one-argument bulkhead, one of reactive's nicest features.

Q10. `zip` vs `merge` vs `concat`?

`zip` pairs one element from each source, waits for all, and ends when any ends. `merge` interleaves as items arrive, subscribing to all eagerly. `concat` subscribes to the next only when the previous ends. Use `zip` to combine parallel calls into one result; `merge` for fan-in; `concat` for ordered fallback sequences.

Q11. What happens if one source in `Mono.zip` is empty?

The result completes empty and the other sources are cancelled, with no error. So a dashboard with one empty section silently returns nothing. Use `defaultIfEmpty`, or map to `Optional` with `.map(Optional::of).defaultIfEmpty(Optional.empty())`.

Q12. What is wrong with `switchIfEmpty(callService())`?

`callService()` is evaluated at assembly time on every request, whether or not the upstream is empty. If the method does work before returning its publisher, that work happens regardless. Wrap it in `Mono.defer(() -> callService())`. Same for `Mono.just(load())`, where `load` always runs immediately.

Q13. `Mono.just` vs `Mono.fromCallable` vs `Mono.defer`?

`just` holds an already-computed value, and if the value is null it throws. `fromCallable` runs the callable per subscription and treats a null return as empty. `defer` builds a whole new publisher per subscription. For blocking or costly work, use `fromCallable` with `subscribeOn(boundedElastic)`.

Q14. How do errors work in a pipeline?

An exception in an operator becomes an `onError` signal, which skips remaining `map`-style operators until an error operator or the subscriber. The sequence terminates. `onErrorResume` swaps in a fallback, `onErrorReturn` a value, `onErrorMap` translates, `retry` resubscribes. `onErrorContinue` drops an element and continues, but it has confusing semantics; avoid it unless you understand which upstream operators support it.

Q15. What happens if I `subscribe()` with no error handler?

On an error, Reactor raises `ErrorCallbackNotImplemented` to the global `Hooks.onErrorDropped`-style logging. The pipeline ends, and only a log line shows it. In a server handler you should not subscribe anyway; return the publisher.

Q16. Should I ever call `subscribe()` in a WebFlux handler?

Almost never. The framework subscribes to the returned publisher, tying it to the request lifecycle, context, cancellation and error handling. A manual `subscribe` is fire-and-forget: no request context, no error path, and it continues after the client disconnects. Compose with `flatMap` / `then`. For deliberate background work, use a managed queue.

Q17. Explain `publishOn` vs `subscribeOn`.

`publishOn` moves execution for operators downstream of it. `subscribeOn` chooses the thread on which subscription and source emission run, regardless of where it is written. Rule: `subscribeOn` for a blocking source, `publishOn` to shift later work elsewhere.

Q18. What thread does each `map` run on?

```
Mono.just(1)
    .map(a -> a + 1)
    .publishOn(Schedulers.parallel())
    .map(b -> b + 1)
    .subscribeOn(Schedulers.boundedElastic())
    .map(c -> c + 1)
    .subscribe();
```

First `map` on `boundedElastic`, second and third on `parallel`. `subscribeOn` moved the source and everything up to the `publishOn`; `publishOn` moved everything after it, regardless of `subscribeOn` being written later.

Q19. What happens with two `subscribeOn`s?

The one closest to the source controls the source's thread. The other only affects the brief subscription path above it. Library authors use this to guarantee where their own source runs.

Q20. Which scheduler for blocking calls and why not `parallel`?

`boundedElastic`: its threads are meant to block, capped at 10 × cores with task queues. `parallel` has only one thread per core, so blocking one wastes a fixed fraction of capacity, and blocking calls on those threads are also rejected by `block()` checks.

Q21. What is the event loop model?

A small number of threads (about one per core) each run a loop over a selector, processing ready I/O events and queued tasks, never waiting. Each connection is pinned to one loop. Handlers do small pieces of work and register a continuation for I/O. Concurrency comes from interleaving rather than from thread count, so thousands of connections use only a few threads.

Q22. How many threads does a WebFlux app use?

Reactor Netty I/O workers default to `max(cores, 4)`. Add `parallel` (one per core) and `boundedElastic` when used. A thread dump shows `reactor-http-nio-N`. The WebClient shares the same event loops by default, so one service's slow work also affects its own outbound calls.

Q23. WebFlux latency spikes across all endpoints, CPU is low. What do you check?

A blocked event loop. Take a thread dump and look at `reactor-http-nio-*` threads stuck in socket read, `Thread.sleep`, JDBC or lock waits. Confirm with BlockHound in tests and `reactor.netty` metrics. Check for hidden blockers, like synchronous logging, DNS resolution, or a "reactive" client that blocks internally. Fix by removing the blocking call, or moving it to `boundedElastic`.

Q24. What is BlockHound?

A Java agent that instruments blocking JDK methods and throws if they are called on non-blocking threads. Install it in tests, with `allowBlockingCallsInside` for known-safe cases. It catches accidental blocking early, though it cannot catch slow CPU work on an event loop. Note that on JDK 13+ it needs `-XX:+AllowRedefinitionToAddDeleteMethods`.

Q25. Why does `.block()` throw in WebFlux?

Reactor marks event-loop threads as `NonBlocking`, and `block()` checks and throws `IllegalStateException` to prevent the deadlock in which a thread waits for a result that only it could deliver. Blocking is allowed on `boundedElastic` or on non-Reactor threads, but it is usually a design smell.

Q26. Cold vs hot publishers?

A cold publisher starts its work for each subscriber, so every subscriber gets all data and two subscribers make two HTTP calls. A hot one emits regardless of subscribers, and late subscribers miss earlier items. `share`, `publish`, `cache` and `replay` convert; `Sinks` creates a hot source.

Q27. Two subscribers to a `WebClient` Mono: how many HTTP calls?

Two, because it is cold. Use `.cache()` if the result should be shared. A sneaky version: `Mono<User> u = client.getUser(); Mono.zip(u.map(a), u.map(b))` makes two calls, and `cache()` fixes that. A `cache()` with no TTL holds its value as long as the reference lives.

Q28. How do you implement a retry with backoff, and what mistakes do people make?

`retryWhen(Retry.backoff(3, Duration.ofMillis(200)).jitter(0.5).filter(isTransient))`. Mistakes: no filter, so 400s retry; retrying non-idempotent POSTs; `timeout` in the wrong place (per attempt or overall); no jitter, which makes synchronised storms; and hiding the original error, which `onRetryExhaustedThrow` fixes.

Q29. How does `timeout` work?

If no signal arrives in the duration, it cancels upstream and emits `TimeoutException`. On a `Flux`, it is the time *between* items, plus the first, not total duration. It runs on `parallel`. Cancellation closes the HTTP connection, so the work on the far side may still complete.

Q30. What does cancellation do, and why does it matter?

`cancel` travels upstream through every operator, closing connections and stopping sources. If a client disconnects, the whole pipeline stops, saving DB and API costs. Cancel can be seen with `doOnCancel`, and `doFinally` handles all outcomes. Do not assume a cancelled DB write rolled back without checking how the driver treats it.

Q31. How does WebClient differ from RestTemplate under load?

`RestTemplate` holds a thread per call, so 1000 concurrent calls need 1000 threads. `WebClient` uses a few event loops and a bounded connection pool, so 1000 calls wait on sockets, not threads, and pending acquires are limited. With virtual threads, `RestClient` closes much of the gap, and is simpler for a blocking service.

Q32. What are WebClient's common production problems?

`DataBufferLimitException` (default 256 KB buffer); `PrematureCloseException` (idle connection closed by the server; set `maxIdleTime`); `PoolAcquireTimeoutException` (too few connections, or a leak); leaks from `exchangeToMono` without consuming the body; no timeouts by default; and building a new `WebClient` or `HttpClient` per call, which loses the pool. RefinePoint's gateway keeps one `WebClient` per controller and sets explicit timeouts.

Q33. What is R2DBC, and is it worth it?

A non-blocking database driver SPI. It removes thread waits on the DB in an already reactive stack, with Spring Data R2DBC on top. But it is no ORM, and the database's capacity limits are the same. If you need JPA, stay on MVC. With virtual threads, JDBC's blocking is cheap, which weakens the case further.

Q34. Can I use JPA with WebFlux?

Technically, by wrapping each call in `fromCallable().subscribeOn(boundedElastic)`, but you get the worst of both. Lazy loading and `@Transactional` depend on `ThreadLocal`s, so transactions and sessions do not span operators, and your concurrency is limited by the elastic pool and JDBC pool. If most of the service is JPA, use MVC.

Q35. How do transactions work in reactive?

`@Transactional` on a method returning `Mono` / `Flux` uses `ReactiveTransactionManager` and keeps the connection in the Reactor Context, committing on complete, rolling back on error, and rolling back on cancel by default. `TransactionalOperator` is the programmatic form. A subscriber must exist: returning the publisher is what matters, because without a subscribe nothing happens, not even the transaction.

Q36. Why doesn't `ThreadLocal` work, and what replaces it?

A pipeline hops threads, so a value set at one operator is absent at the next, and can leak to another request on that thread. Reactor Context, an immutable map bound to the subscription, replaces it. Micrometer's context-propagation bridges to ThreadLocals when libraries need them.

Q37. Why must `contextWrite` come after the operators that read it?

Context is populated during subscription, which flows from the bottom of the chain to the top. So `contextWrite` is visible to operators above it. People who write it at the top see `NoSuchElementException` or empty context. In web apps a `WebFilter` adds it around `chain.filter`, which puts it at the outermost level, visible to all handlers.

Q38. How do you log a trace id in a reactive app?

Enable automatic propagation (`spring.reactor.context-propagation=auto`, Boot 3.2+) with Micrometer Tracing, so MDC is restored per operator and normal logging gets `traceId`. Before that, people used a `doOnEach` helper that read from Context and set MDC by hand.

Q39. How do you unit test a `Flux`?

`StepVerifier.create(flux).expectNext(...).verifyComplete()`, `expectError`, `thenCancel`. `withVirtualTime` for delays without real waits. An initial demand of 0 plus `thenRequest` for backpressure tests. `TestPublisher` for controlled or badly-behaved sources. `PublisherProbe` to assert that a fallback was subscribed to.

Q40. How do you debug a reactive stack trace?

`checkpoint("description")` at boundaries for a cheap marker. `ReactorDebugAgent.init()` for assembly info in production, at low cost. `Hooks.onOperatorDebug()` in dev only, because it captures a stack at every operator. `log()` to see signals. Name pipelines and export metrics to see where time goes.

Q41. A pipeline never emits and never errors. How do you approach it?

Add `log()` at several points to see which signals pass. Candidates: nobody subscribed, a result was discarded, the source is empty so `flatMap` never fires, `request` was never made, a connection pool is exhausted, or an `onErrorResume` returned `Mono.empty()`. Add a `timeout` so a hang becomes a visible error.

Q42. How do you stream LLM tokens to a browser?

Use `WebClient.bodyToFlux` on the provider's stream, map to `ServerSentEvent`, return `Flux` with `text/event-stream`. Disable proxy buffering, heartbeat, set a per-token `timeout`, and send errors as an event. Client disconnect propagates cancel, which stops the upstream call. For structured output that must parse whole, streaming adds little: that is why RefinePoint's `LlmClient` returns the full response.

Q43. `SseEmitter` vs `Flux<ServerSentEvent>`?

`SseEmitter` is MVC's imperative API: you call `send` from any thread and must handle completion, timeout and error callbacks and clean up your registry. `Flux` is declarative and propagates cancellation automatically, and works from WebFlux or from an MVC controller. RefinePoint's inbox uses `SseEmitter` because a Kafka-driven service pushes to a registry of open connections, and it fits the MVC stack.

Q44. Why is Spring Cloud Gateway built on WebFlux?

A gateway is nearly pure I/O: accept, route, proxy, stream both ways, with many concurrent and long-lived connections. A non-blocking model keeps memory and threads low, and streaming bodies pass through with backpressure. The rule that follows: never put blocking code in a gateway filter.

Q45. How do pre and post filters work in a `GlobalFilter`?

Work before `chain.filter(exchange)` is pre-filter; `chain.filter(exchange).then(Mono.fromRunnable(...))` is post-filter, once downstream completes. Filters run by `Ordered` value, lowest first, with post logic in reverse. Our rate limiter is at `HIGHEST_PRECEDENCE + 10`, so it rejects a request before anything else is spent on it.

Q46. How would you add rate limiting to Spring Cloud Gateway?

Use the built-in `RequestRateLimiter` filter with Redis for a shared limit, or a custom `GlobalFilter` for no dependency. RefinePoint chose the second: token buckets in a `ConcurrentHashMap` with API-key, user and IP keys, evicting idle entries, and returning 429 with `Retry-After`. The trade-off: each gateway instance counts separately.

Q47. Is `synchronized` allowed on the event loop?

Yes, for very short critical sections with no I/O. It does stall that loop if contended, so keep it to a few instructions. `RateLimitFilter.Bucket` does exactly that. A `ConcurrentHashMap` is fine too, though avoid expensive `computeIfAbsent` functions, since they hold a bin lock while running.

Q48. Are virtual threads replacing WebFlux?

For most services, yes: the main motivation, cheap waiting, is now available with plain blocking code, with normal stack traces and `ThreadLocal`s. WebFlux keeps its place where backpressure, cancellation and stream composition are the requirement: gateways, streaming, or large fan-out pipelines. Do not rewrite a working reactive service only for that reason.

Q49. What do virtual threads not give you that Reactor does?

Backpressure as a protocol, cancellation that propagates automatically, and declarative stream operators (`buffer`, `window`, `zip`, `retryWhen`). Thread limits also vanish, so you need explicit semaphores to protect downstreams. Structured concurrency helps with fan-out and cancellation, though it was still a preview feature in Java 21.

Q50. What about the reverse: what does Reactor not give you that virtual threads do?

Readable sequential code, normal debugging, stack traces and profilers, full compatibility with JDBC/JPA and `ThreadLocal`-based libraries, and no "function colouring", where everything that calls reactive code must become reactive too.

Q51. Would you use WebFlux for a new service? Give your decision process.

I would first ask where time goes and what the concurrency looks like. Mostly DB CRUD on JPA: MVC. High I/O concurrency with blocking libraries: MVC with virtual threads. Proxying, streaming, thousands of idle connections or stream processing with backpressure: WebFlux. CPU-bound: neither; bound the pool. I also count team experience, since a reactive codebase nobody can debug is a production risk.

Q52. Does reactive reduce the load on the database?

No. It only reduces threads waiting on it. The DB still has the same number of connections, locks and I/O. Unlimited concurrency in the app can make the DB worse, so you still need pool limits and bounded `flatMap` concurrency.

Q53. Can a reactive app have higher latency than blocking?

Yes. At low concurrency, operator overhead and thread hops add latency compared with a direct call. An imperative app is often as fast or faster until concurrency exhausts its thread pool. Reactive's advantage is holding latency steady at high concurrency, not reducing it at low concurrency.

Q54. How do you call a blocking library from WebFlux safely?

`Mono.fromCallable(() -> blocking()).subscribeOn(Schedulers.boundedElastic())`. Consider a dedicated bounded scheduler so one slow library cannot exhaust the shared elastic pool. Add `timeout`, but note that it cannot interrupt the underlying blocking call unless that call is interruptible.

Q55. How is `Flux.interval` or `delayElements` different in threading?

They emit from a timer on the `parallel` scheduler, even if you called from another thread, so downstream operators run there unless you `publishOn`. Blocking in such a pipeline blocks a parallel thread. It surprises people who expect their calling thread.

Q56. What is `Mono<Void>` and why does `then()` matter?

A signal of completion or error with no value. `then()` ignores elements and completes when upstream completes; `then(other)` continues with another publisher. Use it to sequence side effects, as in `repo.save(x).then(notify(x))`. Remember `doOnNext(x -> send(x))` with a `Mono`-returning `send` never subscribes to it: use `flatMap` for async side effects.

Q57. What is `flatMap`'s prefetch and why do you see 32 and 256?

Queues.SMALL_BUFFER_SIZE (256) is the default `flatMap` concurrency, and XS_BUFFER_SIZE (32) the default prefetch per inner source. They explain why a `Flux` subscribed with `flatMap` takes 256 items upfront, and why `log()` shows `request(256)`. They can be tuned per operator, and by system properties.

Q58. How do you convert `Flux<DataBuffer>` for large downloads and avoid leaks?

Pass the `Flux<DataBuffer>` straight through, `exchange.getResponse().writeWith(flux)`, or use `DataBufferUtils.write` to a file. Buffers are reference-counted Netty memory, so any operator that drops one without releasing it leaks direct memory; use `DataBufferUtils.release`, and enable Netty's leak detector during tests. Never `collectList` a large body, since that defeats streaming.

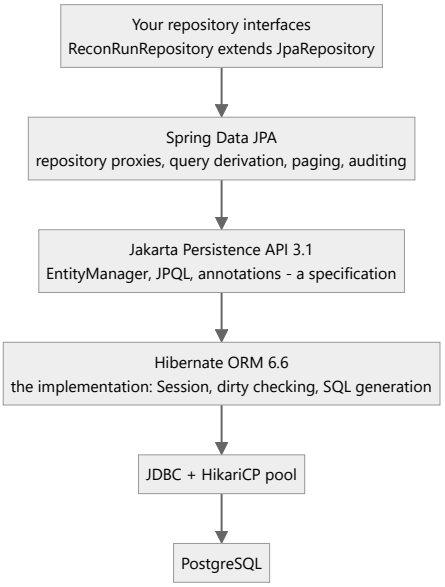
Q59. How did you use reactive in your project?

RefinePoint: "Only where it earns its keep. The gateway is Spring Cloud Gateway, so reactive by nature. I wrote a `RateLimitFilter` as a `GlobalFilter` with in-memory token buckets, and small `WebClient` pipelines for the status page and API docs, using `flatMapSequential`, `timeout` and `onErrorResume` so one dead service degrades a page rather than breaking it. The other services are MVC with JPA and `JdbcTemplate`, since they are transactional and CPU-heavy, where reactive would add complexity and no capacity. Live notifications use `SseEmitter`."

20. Spring Data JPA & Hibernate Internals

JPA questions are where senior candidates are most often caught out, because JPA looks simple until production: a page that issues 400 queries, an update that silently does nothing, a `LazyInitializationException` in a JSON serializer, a batch insert that turns out to be 100,000 round trips. Interviewers probe the **persistence context**, because almost every JPA surprise is explained by it. This chapter builds that model first, then covers mapping, fetching, locking and multi-tenancy, and closes with when to stop using JPA, which RefinePoint does deliberately on its hot paths.

20.1 JPA vs Hibernate vs Spring Data JPA



Layer	What it is	You use it for
JPA (`jakarta.persistence`)	Specification: interfaces and annotations	Portable mapping and `EntityManager` API
Hibernate	The most common JPA provider, plus its own extras	`@BatchSize` , `@JdbcTypeCode` , `@SoftDelete` , filters, statistics
Spring Data JPA	Abstraction over JPA	Repositories without implementations, paging, Specifications, auditing

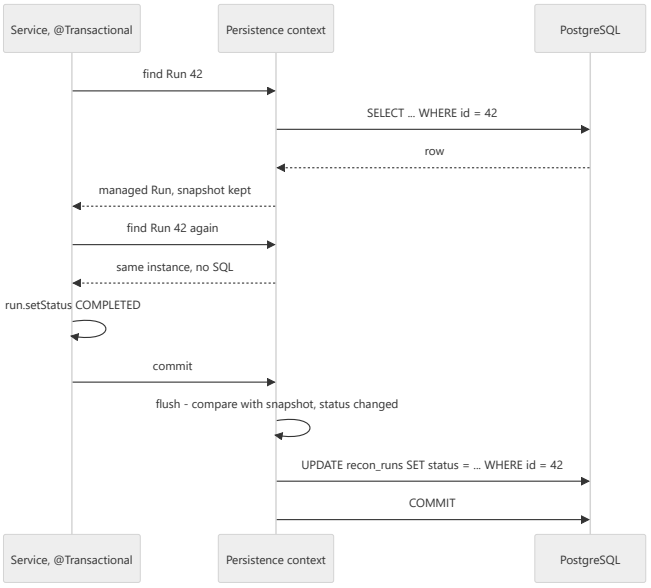
`Session` extends `EntityManager` in Hibernate 6, so `entityManager.unwrap(Session.class)` gives you the native API when you need it. Boot 3.4 ships Hibernate 6.6.

20.2 The persistence context

The **persistence context** is a map of `(entity type, id) → managed instance` plus a snapshot of each instance's state as it was loaded. One `EntityManager` owns one persistence context. In Spring, the transaction-scoped `EntityManager` you inject is a shared proxy that binds the real one to the current transaction (through `TransactionSynchronizationManager`), so one transaction means one persistence context.

It gives three things:

1. **First-level cache / identity guarantee.** `em.find(Run.class, id)` twice in one transaction returns the *same object* and runs one SELECT. Within a context, one row maps to exactly one Java object.
2. **Automatic dirty checking.** At flush, Hibernate compares each managed entity with its snapshot and generates UPDATEs for what changed. You do not call `save` to update a managed entity.
3. **Write-behind.** INSERTs, UPDATEs and DELETEs are queued in an action queue and sent at flush, which allows ordering and JDBC batching.



Flush and flush modes

Flush means "synchronise the queued changes to the database"; it is not commit. Flush happens:

Trigger	Mode `AUTO` (default)	Mode `COMMIT`
Transaction commit	Yes	Yes
Before a JPQL/HQL query whose tables have pending changes	Yes	No
Before a native SQL query	Yes, when bootstrapped through JPA (Hibernate cannot tell which tables it touches)	No
Explicit `em.flush()`	Yes	Yes

Hibernate's native API adds `MANUAL` (only explicit flush) and `ALWAYS`.

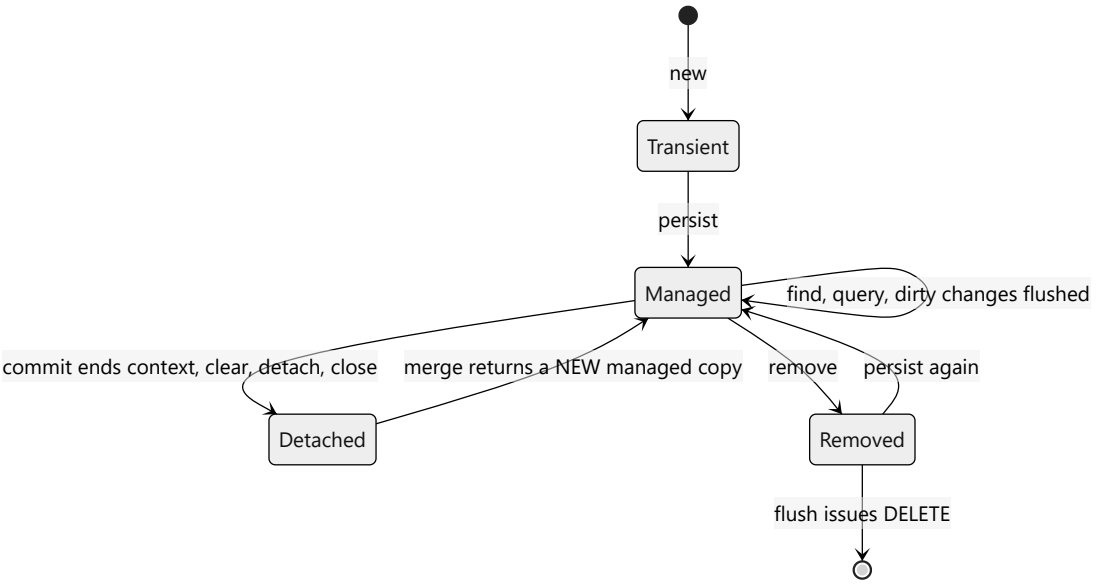
> **Trap:** "I set a field, but the native `SELECT` in the same transaction did not see it." If the flush mode is `COMMIT`, or the query goes through `JdbcTemplate` (a different API that knows nothing about the persistence context), pending changes are invisible. `JdbcTemplate` uses the same connection inside a Spring transaction, but Hibernate does not flush for it. Call `em.flush()` before mixing.

Dirty checking cost

Each managed entity costs memory for the instance plus its snapshot, and each flush compares every managed entity, field by field. Loading 100,000 entities in one transaction makes every flush slow and the heap large. Remedies:

- `@Transactional(readOnly = true)`: Spring sets the Hibernate session to read-only and flush mode `MANUAL`, so no snapshots are kept and no dirty check runs.
- Projections (DTOs) instead of entities for reads.
- In batch jobs, `flush()` and `clear()` every N rows.
- Bytecode enhancement with dirty tracking, so Hibernate records which fields changed instead of comparing.

20.3 Entity lifecycle



State	In persistence context?	Has DB row?	Changes saved automatically?
Transient	No	No	No
Managed	Yes	Yes, or will at flush	**Yes**
Detached	No	Yes	No
Removed	Yes, scheduled for delete	Until flush	Delete at flush

`EntityManager` operations

Operation	Does	Edge cases
`persist(e)`	Makes a transient instance managed; INSERT at flush (immediately with `IDENTITY`)	On a detached instance: `EntityExistsException` / "detached entity passed to persist"
`merge(e)`	Copies state of `e` onto a managed instance (loaded by SELECT if needed) and **returns that instance**	`e` itself stays detached; keep using the return value. Hibernate 6.6: merging a detached entity with a generated id that no longer exists throws an optimistic lock exception instead of inserting
`find(cls, id)`	Context first, then SELECT; `null` if absent	
`getReference(cls, id)`	Returns an uninitialised **proxy** without SQL	`EntityNotFoundException` on first access if absent. Ideal for setting a foreign key
`remove(e)`	Schedules DELETE	Must be managed; detached gives `IllegalArgumentException`
`detach(e)`, `clear()`	Evicts one or all	Pending unflushed changes to them are lost
`refresh(e)`	Reloads state from DB, overwriting changes	
`flush()`	Executes queued SQL	Inside the transaction; still rollback-able

What Spring Data's `save` really does

```
@Transactional
public <S extends T> S save(S entity) {          // SimpleJpaRepository
    if (entityInformation.isNew(entity)) { em.persist(entity); return entity; }
    else { return em.merge(entity); }
}
```

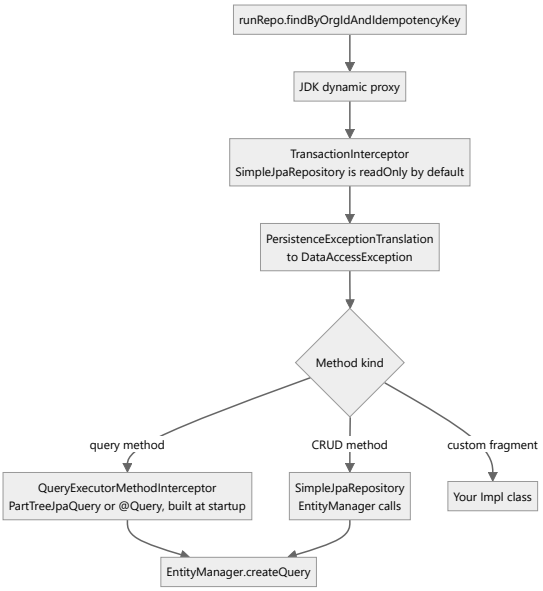
`isNew` is true if the id is `null` (or a primitive 0), or if a wrapper `@Version` field is `null`, or as decided by `Persistable.isNew()`. Consequences:

- An entity with an id **assigned by the application** (`id = UUID.randomUUID()` in the constructor) is "not new", so `save` calls `merge`, which issues a SELECT first to see if the row exists. Implement `Persistable` or add a `@Version` field to avoid it.
- Calling `save` on a managed entity is harmless but useless; dirty checking already handles it.
- Always use the **return value** of `save`; with merge it is a different object.

> **RefinePoint:** `BaseEntity` uses `@Id @GeneratedValue(strategy = GenerationType.UUID)`, so the id is `null` until persist, `isNew` is true, and `save` takes the cheap `persist` path. The UUID is generated in the JVM, so unlike `IDENTITY` no round trip is needed to learn it.

20.4 Spring Data repositories

At startup, `@EnableJpaRepositories` (auto-configured by Boot from the `AutoConfigurationPackages` base package) scans for repository interfaces and registers a `JpaRepositoryFactoryBean` for each.



Query derivation parses method names with `PartTree` at startup: `findTop30ByOrgIdOrderByStartedAtDesc`, `countByOrgIdAndStatusIn`, `existsBy...`, `deleteBy...`. A reference to a property that does not exist **fails application startup**, which is a nice safety net. Keywords: `And`, `Or`, `Between`, `LessThan`, `In`, `IsNull`, `Like`, `StartingWith`, `IgnoreCase`, `OrderBy`, `Top` / `First`, `Distinct`.

> **RefinePoint:** `ReconRunRepository` shows the full range: `Page<ReconRun> findByOrgIdAndProcessIdOrderByStartedAtDesc(UUID orgId, UUID processId, Pageable pageable)`, `findByOrgIdAndIdempotencyKey`, `findFirstByProcessIdAndStatusInOrderByRunNumberDesc` (is a run already going?), and `countByOrgIdAndQueuedAtGreaterThanEqualAndStatusNot`, which counts a workspace's runs for plan limits. When a name gets past readability, like "latest completed run per process", it becomes an `@Query` JPQL with a correlated subquery.

`@Query`:

```
@Query("""
    SELECT r FROM ReconRun r
    WHERE r.orgId = :orgId AND r.status = 'COMPLETED' AND r.resultTable IS NOT NULL
      AND r.completedAt = (SELECT max(x.completedAt) FROM ReconRun x
                           WHERE x.orgId = :orgId AND x.processId = r.processId AND x.status = 'COMPLETED')
    """)
List<ReconRun> findLatestCompletedPerProcess(@Param("orgId") UUID orgId);

@Query(value = "SELECT * FROM recon_runs WHERE org_id = :org AND status = 'FAILED'",
       countQuery = "SELECT count(*) FROM recon_runs WHERE org_id = :org AND status = 'FAILED'",
       nativeQuery = true)
Page<ReconRun> failed(@Param("org") UUID org, Pageable p);

@Modifying(clearAutomatically = true, flushAutomatically = true)
@Query("DELETE FROM ErrorRecord r WHERE r.datasourceId = :ds AND r.status = 'OPEN'")
int deleteOpen(@Param("ds") UUID datasourceId);
```

JPQL is validated at startup; native SQL is not. **Bulk `@Modifying` queries bypass the persistence context**: managed entities in memory keep stale values, `@Version` is not incremented, cascades and lifecycle callbacks do not run. `clearAutomatically` clears the context afterwards, and `flushAutomatically` flushes pending changes first.

Projections

Kind	Example	SQL selects	Notes
Entity	`List<ReconRun>`	All columns	Managed, dirty checked
Closed interface	`interface RunSummary { UUID getId(); String getStatus(); }`	**Only those columns**	Proxy-backed
Open interface	`@Value("#{target.matched + target.unmatched}")`	All columns	SpEL needs the whole entity
DTO / record	`record RunRow(UUID id, String status)`	Constructor args	Derived queries support records; JPQL uses `SELECT new com.x.RunRow(r.id, r.status)`
Dynamic	`<T> List<T> findByOrgId(UUID org, Class<T> type)`	Depends on type	Caller chooses
Tuple	`List<Tuple>`	As written	Flexible, untyped

DTO projections are the most effective single performance improvement for read endpoints: no snapshots, no dirty checking, no lazy loading surprises, fewer columns.

Specifications and Criteria

For dynamic filters (any combination of optional criteria), concatenating JPQL strings is error-prone and an injection risk. `JpaSpecificationExecutor<T>` takes composable predicates built on the Criteria API:

```
static Specification<ReconRun> inOrg(UUID org)        { return (r, q, cb) -> cb.equal(r.get("orgId"), org); }
static Specification<ReconRun> status(String s)        { return s == null ? null : (r, q, cb) -> cb.equal(r.get("status"), s); }
static Specification<ReconRun> since(LocalDateTime t)  { return t == null ? null : (r, q, cb) -> cb.greaterThanOrEqualTo(r.get("startedAt"), t); }

Page<ReconRun> page = repo.findAll(Specification.where(inOrg(org)).and(status(s)).and(since(t)), pageable);
```

`null` specifications are ignored, which makes optional filters simple. The JPA static metamodel (`ReconRun_.status`) makes them type-safe. Querydsl and jOOQ are alternatives with more readable syntax.

20.5 Pagination

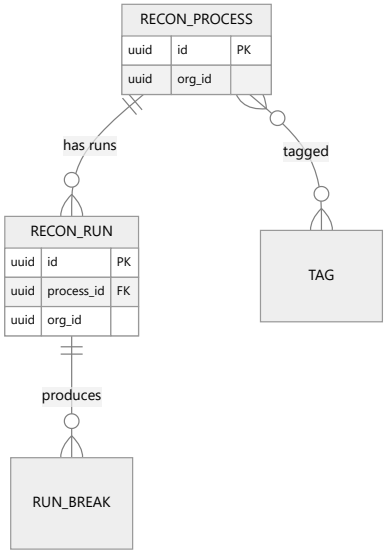
`Page<T>` runs the content query *and* a `count` query; `Slice<T>` fetches `size + 1` rows to know if there is a next page, without counting; `List<T>` with a `Pageable` does neither.

	Page	Slice	Keyset (Window / ScrollPosition)
Total count	Yes, extra query	No	No
Deep pages	`OFFSET` scans skipped rows	Same	Constant cost with an index
Stable while inserting	No	No	Yes
UI fit	Numbered pages	"Load more"	Infinite scroll, exports, APIs

```
-- offset: page 2000 reads and throws away 100,000 rows
SELECT * FROM recon_runs WHERE org_id = ? ORDER BY started_at DESC, id DESC LIMIT 50 OFFSET 100000;
-- keyset: an index range scan from where the last page ended
SELECT * FROM recon_runs WHERE org_id = ? AND (started_at, id) < (?, ?) ORDER BY started_at DESC, id DESC LIMIT 50;
```

Spring Data 3.1 added `Window<T> findFirst50ByOrgIdOrderByStartedAtDescIdDesc(UUID org, ScrollPosition pos)` with `ScrollPosition.keyset()` . Always include a unique tie-breaker (`id`) in the sort, or rows with equal timestamps get skipped or duplicated between pages. For a `Page` over a complex `@Query` , provide `countQuery` explicitly; the derived count of a query with joins can be wrong or very slow.

20.6 Relationships and mapping



Owning side is the side whose mapping controls the foreign key column. In a bidirectional `@OneToMany` / `@ManyToOne`, the `@ManyToOne` side (the table with the FK) owns it; the collection side says `mappedBy` and is ignored when writing.

```
@Entity class ReconProcess extends BaseOrgEntity {
    @OneToMany(mappedBy = "process", cascade = CascadeType.ALL, orphanRemoval = true)
    private Set<FieldMapping> mappings = new HashSet<>();

    public void addMapping(FieldMapping m)    { mappings.add(m); m.setProcess(this); }   // keep both sides in sync
    public void removeMapping(FieldMapping m) { mappings.remove(m); m.setProcess(null); }
}
@Entity class FieldMapping extends BaseOrgEntity {
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "process_id")
    private ReconProcess process;              // OWNING side: this writes process_id
}
```

Rules worth stating in an interview:

- Adding to the inverse collection only (`process.getMappings().add(m)`) without setting `m.process` writes **nothing** to the FK. Use helper methods that sync both sides.
- **Unidirectional `@OneToMany` without `@JoinColumn`** creates a join table, and updating it can delete and reinsert all rows. Prefer bidirectional or unidirectional `@ManyToOne`.
- **Cascade** propagates operations (`PERSIST` , `MERGE` , `REMOVE` , `ALL`) from parent to child. Cascade on `@ManyToOne` towards a parent is almost always wrong: deleting a child would delete the parent.
- **`orphanRemoval = true`** deletes a child removed from the collection. `CascadeType.REMOVE` only deletes children when the *parent* is deleted.
- **`@ManyToMany`** : use a `Set` (with a `List` , Hibernate deletes and reinserts all join rows on change), and as soon as the link needs attributes (who added it, when), replace it with an explicit join entity and two `@ManyToOne` s.
- `equals` / `hashCode` on entities in `Set` s must be stable across persist (see chapter 1, Q31).

> **RefinePoint:** we map almost no associations. Entities hold foreign keys as plain values (`ReconRun.processId` is a `UUID` , not a `@ManyToOne ReconProcess`), and code loads what it needs explicitly. Each service owns its own schema and aggregates are small, so object graphs add little and cost us N+1 risks, cascades and lazy loading. Referencing other aggregates by id is also standard DDD advice, and it is the only option across service boundaries anyway.

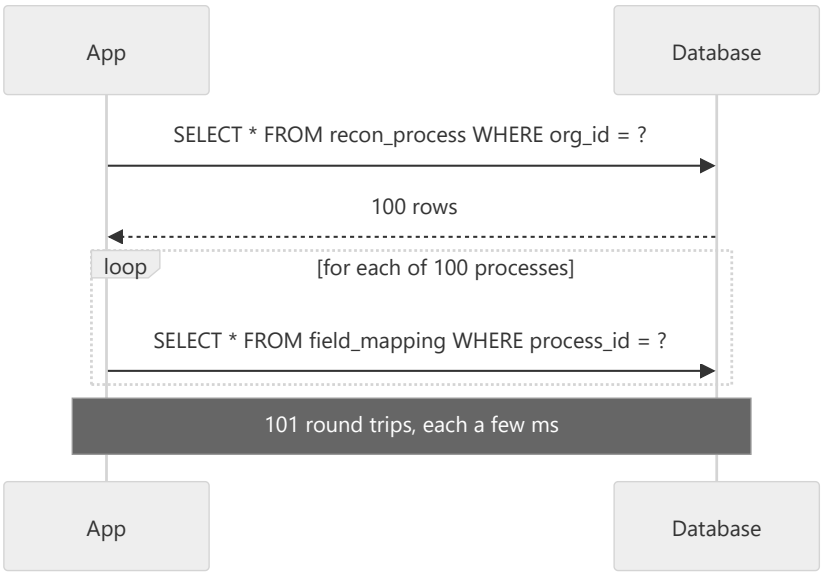
20.7 Fetching and the N+1 problem

Association	JPA default fetch
`@ManyToOne` , `@OneToOne`	**EAGER**
`@OneToMany` , `@ManyToMany`	LAZY

Make every association `LAZY` and fetch explicitly per use case. EAGER cannot be turned off for one query, and with JPQL it still causes N+1, because the query loads the parents and Hibernate then fetches each eager association separately.

N+1: one query loads N parents, then accessing a lazy association on each parent issues one more query each.

```
List<ReconProcess> processes = repo.findByOrgId(org);          // 1 query
for (ReconProcess p : processes) p.getMappings().size();        // + N queries
```



Diagnosis: enable `hibernate.generate_statistics` or log SQL (`logging.level.org.hibernate.SQL=DEBUG`, `org.hibernate.orm.jdbc.bind=TRACE` for parameters in Hibernate 6), look for repeated identical statements per request, count queries per request in integration tests (datasource-proxy or Hypersistence's `SQLStatementCountValidator`), and use APM traces that show 100 spans of the same SQL.

Fixes:

Fix	How	Good for	Watch out
`JOIN FETCH`	`SELECT p FROM ReconProcess p JOIN FETCH p.mappings WHERE p.orgId = :org`	One use case, one collection	Duplicated parents (Hibernate 6 deduplicates entities automatically); pagination applied **in memory** (warning HHH90003004)
`@EntityGraph`	`@EntityGraph(attributePaths = "mappings")` `List<ReconProcess> findByOrgId(UUID org)`	Derived queries	Same pagination caveat
Batch fetching	`@BatchSize(size = 50)` or `hibernate.default_batch_fetch_size=50`	Global safety net	N/50 + 1 queries instead of 1
`@Fetch(FetchMode.SUBSELECT)`	Second query with the original query as subselect	Collections of all loaded parents	Re-runs the parent query
DTO projection	`SELECT new ...` with explicit joins	Read endpoints	Not managed
Two queries	Load parents page, then children `WHERE process_id IN (:ids)`	Pagination + collections	Manual assembly

Fetching two `List` collections in one query throws `MultipleBagFetchException`; even with `Set`s it produces a **Cartesian product** (10 mappings × 10 rules = 100 rows per process). Fetch one collection per query and batch the rest.

LazyInitializationException and open-in-view

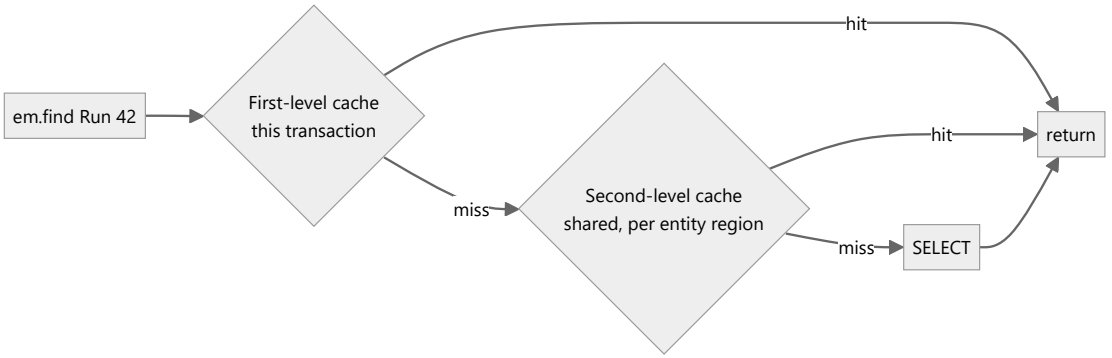
`LazyInitializationException: could not initialize proxy – no Session` happens when an uninitialised proxy or collection is touched after its persistence context closed: typically in a controller or the JSON serializer after the `@Transactional` service returned.

Open Session in View (OSIV), `spring.jpa.open-in-view`, is **true by default** in Boot (with a startup warning). `OpenEntityManagerInViewInterceptor` keeps the `EntityManager` open for the whole web request, so lazy loading works in the view. The price: a DB connection may be held during rendering and slow serialisation, and lazy loads during serialisation run outside any transaction, silently generating N+1 queries in the JSON layer.

Fix properly: turn it off, fetch what each use case needs inside the service (fetch join, entity graph, DTO), and return DTOs.

> **RefinePoint:** `PlatformDefaults` sets `spring.jpa.open-in-view=false` for all 13 services. A lazy access outside the service layer fails loudly in development instead of turning into hidden queries in production.

20.8 Second-level cache and query cache



The **second-level cache** is shared across sessions (and, with a distributed provider, across instances). Enable it with a JCache provider (Ehcache 3, Caffeine via JCache, Hazelcast, Infinispan): `hibernate.cache.use_second_level_cache=true`, a region factory, and `@Cacheable` + `@org.hibernate.annotations.Cache(usage = READ_WRITE)` on entities. It caches entity state by id (dehydrated, not object references). Concurrency strategies: `READ_ONLY` (immutable reference data), `NONSTRICT_READ_WRITE`, `READ_WRITE` (soft locks), `TRANSACTIONAL`.

The **query cache** stores *ids* of a query's results and is invalidated whenever **any** row in a table involved changes. For frequently written tables it adds overhead and gives almost nothing. It only helps alongside the entity cache.

Use the L2 cache for read-mostly reference data. Pitfalls: bulk updates and native SQL bypass it (Hibernate invalidates whole regions on native writes it cannot analyse), stale data across instances with local caches, memory, and changes made by other applications.

> **RefinePoint:** we do not enable the Hibernate L2 cache. Services run as several instances, and most hot data (runs, breaks) is written constantly. For the few read-mostly values (remote API key checks, workspace lookups across services), `ryfn-platform` has a small `TtlCache` with explicit expiry, where staleness is a stated, bounded choice.

20.9 ID generation

Strategy	How the id is produced	JDBC batch inserts	Notes
`IDENTITY`	DB auto-increment at INSERT	**Disabled** for inserts	Hibernate must execute the INSERT immediately at `persist` to learn the id
`SEQUENCE`	`nextval` before insert	Yes	`allocationSize` (default 50) with the **pooled optimizer**: one `nextval` gives 50 ids in memory
`TABLE`	Emulated sequence in a table	Yes	Row lock contention; avoid
`UUID` (JPA 3.1)	Generated in the JVM	Yes	No round trip; 16 bytes
`AUTO`	Provider chooses		Hibernate 6: `SEQUENCE` for numbers, UUID generator for `UUID`

```
Pooled optimizer, allocationSize = 50, INCREMENT BY 50 in the DB sequence:
  nextval → 101   app hands out 52..101 from memory
  nextval → 151   app hands out 102..151
  Multiple instances each get their own block → no collisions, gaps are normal
```

The DB sequence's `INCREMENT BY` must match `allocationSize`, or ids collide or Hibernate fails validation. Hibernate 6 also changed the default sequence name to one per entity (`<entity>_seq`) instead of a shared `hibernate_sequence`, a classic Boot 3 migration surprise.

UUID trade-offs: random UUIDv4 keys insert into random positions of a B-tree index, causing page splits and poor cache locality on large tables; they are also 16 bytes against 8 for `bigint`. Time-ordered UUIDv7 keeps the generate-anywhere benefit with sequential locality. Benefits of UUIDs: ids can be created before persist, safe across services and in URLs (not enumerable), and easy to merge between environments.

20.10 Batch inserts and updates

```yaml
spring:
  jpa:
    properties:
      hibernate:
        jdbc.batch_size: 50
        order_inserts: true          # group INSERTs by entity so batches are not broken
        order_updates: true
        jdbc.batch_versioned_data: true
  datasource:
    url: jdbc:postgresql://host/db?reWriteBatchedInserts=true   # driver rewrites to multi-row INSERT
```

```java
@Transactional
void importRows(List<Row> rows) {
    for (int i = 0; i < rows.size(); i++) {
        em.persist(toEntity(rows.get(i)));
        if (i % 50 == 49) { em.flush(); em.clear(); }    // bound memory and dirty-check cost
```

```
    }
}
```

What breaks batching: `IDENTITY` ids, interleaving entity types without `order_inserts`, a flush triggered by a query in the loop, and `saveAll` on entities with assigned ids (each `merge` SELECTs first). Even working, JPA batching still builds one entity and snapshot per row; for real bulk loads, `JdbcTemplate.batchUpdate` or PostgreSQL `COPY` is several times faster.

20.11 Locking

Optimistic: `@Version`

```
@Version private long version;
```

Every UPDATE becomes `UPDATE ... SET ..., version = 8 WHERE id = ? AND version = 7`. Zero rows updated means someone else changed it: Hibernate throws `OptimisticLockException`, which Spring translates to `ObjectOptimisticLockingFailureException`. Handle it by retrying the whole read-modify-write, or by returning 409 to the user. It prevents **lost updates** without holding locks, best when conflicts are rare. Over HTTP it pairs with `ETag` / `If-Match` (chapter 18).

Pessimistic

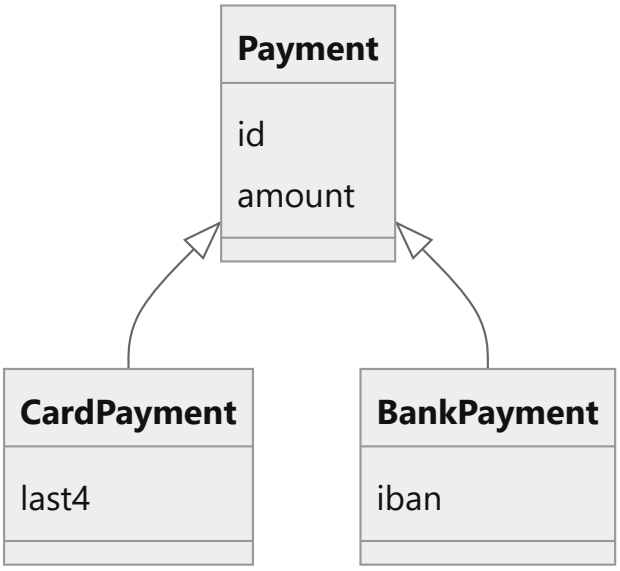
```
@Lock(LockModeType.PESSIMISTIC_WRITE)                                  // SELECT ... FOR UPDATE
@QueryHints(@QueryHint(name = "jakarta.persistence.lock.timeout", value = "3000"))
@Query("select s from OrgSubscription s where lower(s.pendingEmail) = lower(:email) and s.orgId is null")
List<OrgSubscription> lockPendingByEmail(@Param("email") String email);
```

Mode	PostgreSQL SQL	Use
`PESSIMISTIC_READ`	`FOR SHARE`	Others may read-lock, nobody may write
`PESSIMISTIC_WRITE`	`FOR UPDATE`	Exclusive until commit
`PESSIMISTIC_FORCE_INCREMENT`	`FOR UPDATE` + version bump	
Lock timeout `0` / `-2` (Hibernate)	`NOWAIT` / `SKIP LOCKED`	Fail fast / queue workers

Locks are held until the transaction ends, so keep those transactions short, lock rows in a consistent order to avoid deadlocks, and never hold a row lock across a remote call.

> **RefinePoint:** billing-service's `OrgSubscriptionRepository.lockPendingByEmail` is `@Lock(PESSIMISTIC_WRITE)`: the welcome page and sign-up can both try to claim a pending Paddle purchase at the same moment, and the lock makes sure the purchase is applied, or cancelled and refunded, exactly once. The work queues use `FOR UPDATE SKIP LOCKED` through `JdbcTemplate`, because there the point is that concurrent workers *skip* locked rows instead of waiting.

20.12 Inheritance mapping



Strategy	Tables	Polymorphic query	Pros	Cons
`SINGLE_TABLE` (default)	One, with a discriminator column	One SELECT, no joins	Fastest	Subclass columns must be nullable; wide table
`JOINED`	Parent + one per subclass	Joins (outer joins for polymorphic)	Normalised, NOT NULL possible	Join cost grows with depth
`TABLE_PER_CLASS`	One per concrete class, full columns	`UNION ALL`	No joins for concrete queries	Polymorphic queries slow; no `IDENTITY`
`@MappedSuperclass`	No table for parent	**Not possible**	Share columns and mapping only	Parent is not an entity

`@MappedSuperclass` is for *code reuse of mapping*, not a type hierarchy you query. You cannot write `SELECT b FROM BaseEntity b` or have a `@ManyToOne BaseEntity`.

> **RefinePoint:** `BaseEntity` (`id` as a generated UUID, `createdAt` with `@CreationTimestamp`, `updatedAt` with `@UpdateTimestamp`) and `BaseOrgEntity` (adds `org_id NOT NULL`) are `@MappedSuperclass`es. Every tenant table gets the same columns from one place, and nobody ever needs "all entities of any type" polymorphically.

20.13 Auditing, soft delete and JSON columns

Auditing. Two ways to fill timestamps:

	Hibernate	Spring Data
Annotations	`@CreationTimestamp`, `@UpdateTimestamp`	`@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`
Setup	None	`@EnableJpaAuditing` + `@EntityListeners(AuditingEntityListener.class)`
"Who"	No	`AuditorAware<T>` bean, from the security context

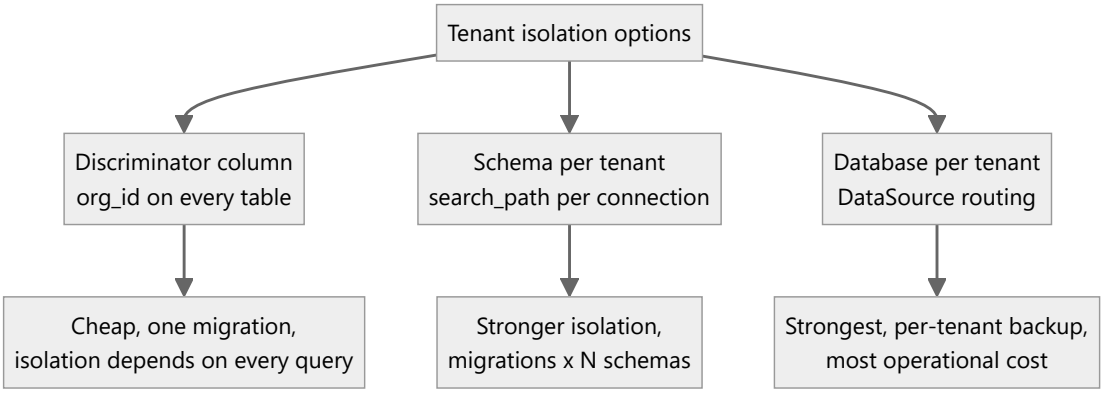
Neither keeps history. For a history of changes, use Hibernate Envers (an `_AUD` table per entity) or domain audit events.

> **RefinePoint:** entities use Hibernate's timestamps through `BaseEntity`, with `hibernate.jdbc.time_zone=UTC` so stored times are UTC. *Who did what* is a business record, not a column: services emit audit events (`AuditTrail`, via the outbox) that audit-service stores, which is what an auditor of a financial system actually asks for.

Soft delete. Hibernate 6.4+ has `@SoftDelete` (a boolean or timestamp column, filtered automatically). Before that: `@SQLDelete(sql = "UPDATE x SET deleted = true WHERE id = ?")` plus `@SQLRestriction("deleted = false")` (6.3, replacing `@Where`). Consequences: unique constraints must become partial indexes (`WHERE deleted = false`), every index and native query must remember the flag, and data grows. Consider an archive table, or event history, instead.

JSON columns. `@JdbcTypeCode(SqlTypes.JSON)` on a `Map` or record maps it to PostgreSQL `jsonb` without custom types in Hibernate 6. RefinePoint's `AuditLog` and `PaymentOrder` use exactly that.

20.14 Multi-tenancy approaches



	Discriminator (`org_id`)	Schema per tenant	Database per tenant
Isolation	Logical; one missed filter leaks data	Strong	Strongest
Cost per tenant	Near zero	Medium	High
Migrations	Once	N times	N times
Cross-tenant reporting	Easy	Harder	Hardest
Noisy neighbours	Shared	Shared	Separate
Hibernate support	`@TenantId` (6.0+), filters	`MultiTenantConnectionProvider` + `CurrentTenantIdentifierResolver`	Same, or `AbstractRoutingDataSource`
Fits	SaaS with many small tenants	Tens to hundreds of tenants with compliance needs	Few large or regulated tenants

Enforcing a discriminator automatically:

- **Hibernate** `@TenantId`: a field filled from `CurrentTenantIdentifierResolver`, added to every query of that entity.
- **Hibernate filters**: `@FilterDef(name = "org", parameters = @ParamDef(name = "orgId", type = UUID.class))` + `@Filter(name = "org", condition = "org_id = :orgId")`, enabled per session (`session.enableFilter("org").setParameter(...)`). Filters do **not** apply to `find` by id or native SQL.
- **PostgreSQL row-level security**: policies `USING (org_id = current_setting('app.org')::uuid)`, enforced by the database even for native SQL. Requires setting the variable on every connection checkout, and testing that pooling cannot leak it.

> **RefinePoint:** we use the discriminator column. `BaseOrgEntity` makes `org_id` mandatory, `OrgAccessInterceptor` resolves the workspace code to an org id after checking membership, and repositories take the org id explicitly: `findByOrgIdAndProcessIdOrderByStartedAtDesc`, `findByOrgIdAndIdempotencyKey`. Explicit parameters are visible in code review; a method without `OrgId` in its name stands out. Separately, each *service* owns one PostgreSQL schema (`currentSchema=${rp.schema}`, `hibernate.default_schema`), which is service isolation, not tenant isolation.

20.15 When to use JdbcTemplate instead

JPA is good at loading an aggregate, changing it and saving it. It is the wrong tool for set-based work.

Use JPA for	Use `JdbcTemplate` / `JdbcClient` / jOOQ for
CRUD on aggregates with invariants	Bulk insert, upsert (`ON CONFLICT`), `COPY`
Dirty checking, cascades, optimistic locking	Queue claims with `FOR UPDATE SKIP LOCKED` and `RETURNING`
Paged lists with filters	Reporting, window functions, CTEs
Moderate data volumes	Millions of rows, streaming result sets
	Dynamic tables (names decided at runtime)

```
jdbc.batchUpdate("""
    INSERT INTO lookup_rows (id, table_id, key, range_from, range_to, row_values, position)
    VALUES (?, ?, ?, ?, ?, ?::jsonb, ?)
    ON CONFLICT (table_id, key) WHERE key IS NOT NULL DO UPDATE SET row_values = EXCLUDED.row_values
    """, batch);
```

> **RefinePoint:** we use both in the same services, in the same transactions. JPA repositories cover configuration and entities: processes, runs, subscriptions, settings. `JdbcTemplate` covers the hot paths: `RunWorker` claims runs with `SELECT ... FOR UPDATE OF r SKIP LOCKED` and marks them RUNNING with a heartbeat; a reaper resets stale runs with `UPDATE ... WHERE id IN (SELECT ... FOR UPDATE SKIP LOCKED) RETURNING id`; lookup-service imports rows with a batched `ON CONFLICT` upsert; `BreakEvents` and data-quality checks insert with `batchUpdate`; consumers dedupe with `INSERT ... ON CONFLICT DO NOTHING`. Reconciliation results themselves live in result tables processed by DuckDB, which no ORM could map.

Spring 6.1's `JdbcClient` is a fluent wrapper: `jdbc.sql("SELECT ... WHERE id = :id").param("id", id).query(Run.class).optional()`.

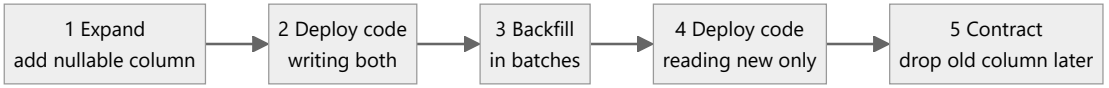
20.16 Flyway migrations

Flyway applies versioned SQL scripts in order and records each in `flyway_schema_history` with a checksum.

```
db/migration/
  V1__baseline.sql             versioned: runs once, in order
  V2__break_management.sql
  V14__run_queue.sql
  R__views.sql                 repeatable: re-runs when its checksum changes, after versioned ones
```

Boot runs Flyway **before** the `EntityManagerFactory` is created (the factory depends on the Flyway initialiser), so `ddl-auto=validate` checks the migrated schema. At startup Flyway validates checksums of applied scripts: editing a script that has already run fails startup, deliberately. Never edit an applied migration; add a new one. (`flyway repair` realigns checksums when a change was intentional, for example a comment-only edit.)

Zero-downtime changes use expand/contract, because old and new versions run side by side during a rolling deploy:



Renaming a column in one migration breaks the old instances still running. Large backfills should not run inside the startup migration (they hold locks and delay startup); `CREATE INDEX CONCURRENTLY` cannot run in a transaction, so it needs its own non-transactional migration.

> **RefinePoint:** every service owns one schema and its migrations. `PlatformDefaults` sets `spring.flyway.schemas=${rp.schema}`, `create-schemas=true` and `ddl-auto=validate`, so Hibernate never changes the schema; it only checks that mappings match. Each service starts from `V1__baseline.sql` and applies its numbered changes in order (run-service is past V14 already).

20.17 Interview questions and answers

Q1. What is the difference between JPA, Hibernate and Spring Data JPA?

JPA is the specification: annotations plus the `EntityManager` API. Hibernate is an implementation of it with many extras. Spring Data JPA sits on top and generates repository implementations, query derivation, paging and auditing. You can use JPA without Spring Data, but Spring Data JPA cannot work without a JPA provider.

Q2. What is the persistence context?

The set of managed entity instances for one `EntityManager`, keyed by type and id, with a snapshot of each. It guarantees one object per row per context, serves repeated lookups without SQL, detects changes at flush (dirty checking), and queues writes. In Spring, it is normally scoped to the transaction.

Q3. What is the first-level cache? Can it be disabled?

It is the persistence context itself. It cannot be disabled, because it is how JPA tracks identity. It is per transaction and not shared, so it does not cause staleness across requests. In long batch transactions it grows, so `clear()` it periodically.

Q4. What does this print?

```
@Transactional void test(UUID id) {
    ReconRun a = repo.findById(id).orElseThrow();
    ReconRun b = repo.findById(id).orElseThrow();
    System.out.println(a == b);
}
```

`true`, with one SELECT. Both lookups hit the same persistence context. Without `@Transactional` (and with OSIV off), each repository call runs in its own transaction, so it prints `false` and issues two SELECTs.

Q5. I changed an entity's field and never called save. Why was it updated?

It was managed: loaded inside a transaction. At commit, dirty checking compared it with its snapshot and issued an UPDATE. That is by design. To avoid it, change a detached copy or a DTO, use `readOnly = true` transactions, or do not mutate entities you only meant to read.

Q6. Why did my update not get saved?

Usually the entity was detached (loaded in a different transaction, or no transaction at all), so nothing tracked it. Or the method was not transactional because of self-invocation, or it was `readOnly`, or an exception rolled back the transaction, or the update was a `@Modifying` query whose effect was then overwritten by a later flush of stale managed state. Check the transaction boundaries first.

Q7. When does Hibernate flush?

At commit, before a JPQL query that touches tables with pending changes (in `AUTO` mode), before native queries when bootstrapped via JPA, and on explicit `flush()`. Flush sends SQL; commit makes it durable. A flushed change can still be rolled back.

Q8. What are the entity states?

Transient (new, unknown to JPA), managed (in a persistence context and tracked), detached (has an id and row, but no longer tracked), removed (scheduled for delete). `persist` makes transient managed, `merge` copies detached onto managed, closing the context detaches, `remove` marks it removed.

Q9. `persist` vs `merge`?

`persist` makes the given instance managed; it is for new entities and returns nothing. `merge` copies the state of the given instance onto a managed instance, loading it if needed, and returns that managed one; the argument stays detached. Using `merge` for new entities costs an extra SELECT; using `persist` on a detached entity throws.

Q10. What does Spring Data's `save()` do?

It calls `persist` if `isNew` (id null, or version null, or `Persistable.isNew()`), else `merge`, and returns the result. So with application-assigned ids, every `save` is a merge with a SELECT first. Always use the return value.

Q11. `findById` vs `getReferenceById`?

`findById` hits the context then the DB, and returns `Optional` (empty if absent). `getReferenceById` returns a lazy proxy with no SQL; the first access to a non-id field loads it, and throws `EntityNotFoundException` if the row is missing. Use the reference to set a foreign key without loading the parent: `child.setProcess(repo.getReferenceById(processId))`.

Q12. How are Spring Data repository implementations created?

At startup, repository interfaces are found by scanning, and for each a factory bean creates a JDK proxy whose target is `SimpleJpaRepository` (CRUD), plus an interceptor that executes query methods. Query methods are parsed into `PartTree` or `@Query` objects at startup, and fail startup if invalid. Custom fragments (`XxxRepositoryCustom` plus `Impl`) are mixed in. The proxy adds transactions and exception translation.

Q13. Are repository methods transactional?

Yes. `SimpleJpaRepository` is `@Transactional(readOnly = true)` at class level, with `@Transactional` on write methods. Derived query methods you declare inherit the read-only default. For several calls to share one context and commit atomically, the service method must be `@Transactional` itself.

Q14. How does query derivation fail?

At startup, with `PropertyReferenceException` or a query creation error, when a method name refers to a property that does not exist. That is a benefit: renaming a field breaks the build or startup, not production. Long names are a readability smell; past three or four criteria, use `@Query` or a Specification.

Q15. What does `@Modifying` do, and what are the pitfalls?

It marks an `@Query` as an INSERT, UPDATE or DELETE executed with `executeUpdate`. It bypasses the persistence context: managed entities keep stale values, and later flushes can overwrite the bulk change. `@Version`, cascades and entity callbacks are skipped. Use `clearAutomatically` and `flushAutomatically`, and it must run in a writable transaction.

Q16. JPQL vs native query?

JPQL is written against entities, validated at startup, portable, and understood by Hibernate (flush decisions, caches). Native SQL gives full database features (window functions, `ON CONFLICT`, `SKIP LOCKED`, `jsonb` operators), but is not validated, needs a manual count query for paging, and can confuse the cache. Use native where it adds something you need.

Q17. What projection types does Spring Data support?

Closed interface projections (select only the getters' columns), open interface projections (SpEL, so the full entity is loaded), class/record DTOs (constructor expressions), dynamic projections (a `Class<T>` parameter) and `Tuple`. For read-heavy endpoints, DTO and closed projections avoid entity overhead.

Q18. How do you write a dynamic search with optional filters?

`JpaSpecificationExecutor` with small composable `Specification`s, where each returns null when its filter is absent; or Querydsl, or the Criteria API directly. Avoid string concatenation of JPQL. Whitelist sortable fields and make sure filter columns are indexed.

Q19. `Page` vs `Slice`?

`Page` runs an extra count query to give total elements and pages. `Slice` fetches one extra row to know whether a next page exists, and is cheaper on big tables. For deep paging, use keyset (`Window`, `ScrollPosition`) instead of either.

Q20. Why is deep offset pagination slow?

`OFFSET 100000` makes the database produce and discard 100,000 rows before returning 50. Keyset pagination (`WHERE (started_at, id) < (?, ?) ORDER BY started_at DESC, id DESC LIMIT 50`) uses an index range scan and costs the same at any depth. Include a unique tie-breaker.

Q21. What is the owning side of a relationship?

The side whose mapping writes the foreign key: the `@ManyToOne` side, or the side with `@JoinColumn` / `@JoinTable`. The `mappedBy` side is inverse and ignored for writes. Changing only the inverse collection does not change the database.

Q22. Why do we need helper methods for bidirectional associations?

Because both sides are Java references that JPA does not sync for you. If you add a child to the parent's collection without setting the child's parent field, the in-memory graph is inconsistent and the FK is not written. `addMapping` / `removeMapping` set both sides in one place.

Q23. `CascadeType.REMOVE` vs `orphanRemoval`?

Cascade REMOVE deletes children when the parent is removed. `orphanRemoval` also deletes a child as soon as it is removed from the parent's collection, or reassigned. Use `orphanRemoval` for true composition (the child cannot exist alone). Never cascade from child to parent.

Q24. What are the problems with `@ManyToMany`?

With a `List` (a bag), Hibernate deletes all join rows and reinserts them on every change. Extra attributes on the link cannot be mapped. Cascading REMOVE can delete shared entities. Use a `Set`, and replace with an explicit join entity once the link has its own data.

Q25. Default fetch types, and what should you use?

`@ManyToOne` / `@OneToOne` are EAGER by default; collections are LAZY. Make everything LAZY, then fetch explicitly per use case, because an EAGER association cannot be made lazy for a single query and causes extra queries in JPQL.

Q26. What is the N+1 problem?

One query loads N rows, then N more queries load an association for each. It hides in loops, in mappers and in JSON serialisation. It is invisible at 10 rows and severe at 10,000, since every round trip costs network latency.

Q27. How do you detect N+1 in an application?

Log SQL in dev (`org.hibernate.SQL=DEBUG`) and look for repeated statements, enable Hibernate statistics, assert query counts in integration tests, and use APM traces in production, which show many identical queries in one request. Also watch for slow endpoints whose DB time far exceeds a single query's.

Q28. How do you fix N+1? Compare the options.

`JOIN FETCH` or `@EntityGraph` load in one query for a given use case. `@BatchSize` / `default_batch_fetch_size` turns N queries into N/size queries as a global safety net. `SUBSELECT` fetch loads all children with one extra query. DTO projections select exactly what the view needs. For paginated parents with collections: page the parent ids, then fetch children with `IN`.

Q29. What happens if you `JOIN FETCH` a collection with pagination?

Hibernate cannot apply `LIMIT` to joined rows without cutting a parent's children, so it fetches **all** rows and paginates in memory, logging HHH90003004. On a big table this is an out-of-memory risk. You can make it fail instead with `hibernate.query.fail_on_pagination_over_collection_fetch=true`. Fix by paging ids first.

Q30. What is `MultipleBagFetchException`?

Thrown when you fetch-join two `List` (bag) collections in one query, because Hibernate cannot tell duplicates apart in a Cartesian product. Switching to `Set` stops the exception but keeps the Cartesian explosion. Fetch one collection per query and batch-fetch the others.

Q31. What causes `LazyInitializationException`, and how do you fix it properly?

Touching an uninitialised lazy proxy or collection after the persistence context closed, usually in the web layer or serializer. Proper fixes: fetch needed data inside the transactional service (fetch join, entity graph, DTO) and return DTOs. Workarounds to avoid: OSIV, `enable_lazy_load_no_trans` (a new session and connection per lazy load), and making associations EAGER.

Q32. What is open-in-view and why disable it?

`spring.jpa.open-in-view=true` (Boot's default) keeps the `EntityManager` open for the whole HTTP request, so lazy loading works in controllers and serializers. It hides N+1 queries in serialisation, holds a connection longer, and blurs where data access happens. RefinePoint disables it platform-wide in `PlatformDefaults`.

Q33. What is the second-level cache, and when would you use it?

A shared cache of entity state by id across sessions, with pluggable providers and per-entity concurrency strategies. Use it for read-mostly reference data looked up by id. Avoid it for frequently written data and when other systems write to the database, and use a distributed or invalidating provider with multiple instances.

Q34. Why is the query cache rarely worth it?

It stores result ids keyed by query and parameters, and it is invalidated whenever any table in the query changes. On tables with regular writes, it is invalidated constantly and adds overhead. It also needs the entity cache, otherwise each cached id becomes a separate SELECT, which is worse than the original query.

Q35. Why does `IDENTITY` disable batch inserts?

Hibernate must know the id as soon as an entity becomes managed, and with `IDENTITY` only the INSERT itself produces it. So each `persist` executes its INSERT immediately and individually. `SEQUENCE` (with a pooled allocation) or a UUID gets ids without inserting, so INSERTs can be queued and batched.

Q36. How does the pooled sequence optimizer work?

With `allocationSize = 50` and the sequence `INCREMENT BY 50`, one `nextval` reserves a block of 50 ids that the JVM hands out from memory. Each instance reserves its own blocks, so ids never collide and only one round trip happens per 50 inserts. Gaps are normal after restarts. A mismatch between allocation size and the increment causes duplicates or validation failure.

Q37. UUID vs bigint primary keys?

UUIDs can be generated anywhere (before persist, across services, offline), are not enumerable in URLs, and merge cleanly across environments. They cost 16 bytes against 8, and random v4 values fragment B-tree indexes on large tables. Use UUIDv7 for time-ordered locality. RefinePoint uses `GenerationType.UUID` everywhere, which fits the many services and public URLs.

Q38. How do you make batch inserts fast with Hibernate?

`hibernate.jdbc.batch_size` (30–100), `order_inserts` / `order_updates`, a non-IDENTITY id, `reWriteBatchedInserts=true` for PostgreSQL, and `flush()` + `clear()` every batch to bound the context. Avoid `saveAll` with assigned ids. For millions of rows use `JdbcTemplate.batchUpdate` or `COPY`.

Q39. Optimistic vs pessimistic locking?

Optimistic (`@Version`) detects conflicts at write time without holding locks: high concurrency, rare conflicts, user edits, and it works across requests via a version or ETag. Pessimistic (`SELECT ... FOR UPDATE`) blocks others until commit: high contention or must-not-fail critical sections such as claiming a payment. Pessimistic needs short transactions and consistent lock ordering.

Q40. What happens on an optimistic lock failure, and how do you handle it?

The UPDATE matches zero rows because the version changed, so Hibernate throws `OptimisticLockException` (Spring: `ObjectOptimisticLockingFailureException`), and the transaction is marked for rollback. Retry the whole unit of work in a new transaction (Spring Retry or a loop), or return 409 to the user to reload. Retrying only the save inside the same transaction does not work.

Q41. How do you implement a job queue with JPA?

Native or `@Lock(PESSIMISTIC_WRITE)` with lock timeout `-2` (SKIP LOCKED in Hibernate) to claim rows, then commit the claim straightaway so the lock is not held during processing. In RefinePoint we do it with `JdbcTemplate`: `SELECT ... LIMIT 1 FOR UPDATE SKIP LOCKED` in a short `TransactionTemplate`, then `UPDATE status = 'RUNNING', instance = ?, heartbeat_at = now()`. A reaper re-handles rows whose heartbeat went stale.

Q42. What inheritance strategies exist, and which do you choose?

`SINGLE_TABLE` is fastest with nullable subclass columns; `JOINED` is normalised with joins; `TABLE_PER_CLASS` uses `UNION`s for polymorphic queries. Default to `SINGLE_TABLE` for small hierarchies queried polymorphically, `JOINED` when subclasses differ a lot and integrity matters. If you never query polymorphically, use `@MappedSuperclass`, or composition.

Q43. `@MappedSuperclass` vs `@Entity` inheritance?

`@MappedSuperclass` shares mapped fields with subclasses but is not an entity: it has no table, cannot be queried, and cannot be a relationship target. Entity inheritance creates a polymorphic type you can query and reference. RefinePoint's `BaseEntity` / `BaseOrgEntity` are mapped superclasses, because we want the same columns everywhere, not a queryable hierarchy.

Q44. How do you add auditing columns?

Either Hibernate's `@CreationTimestamp` / `@UpdateTimestamp` (no setup), or Spring Data's `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy` with `@EnableJpaAuditing`, `AuditingEntityListener` and an `AuditorAware` bean. Neither covers bulk updates. For full change history, use Envers or domain audit events; RefinePoint does the latter through audit-service.

Q45. How do you implement soft delete, and what does it cost?

Hibernate 6.4's `@SoftDelete`, or `@SQLDelete` + `@SQLRestriction` before that. Costs: unique constraints become partial indexes, native queries must remember the filter, related entities need consistent rules, and tables grow. An archive table, or keeping deletions as events, is often cleaner.

Q46. What multi-tenancy approaches does Hibernate support?

Discriminator (`@TenantId` in Hibernate 6, or filters), schema per tenant and database per tenant (both through `MultiTenantConnectionProvider` plus `CurrentTenantIdentifierResolver`). Choose by tenant count, isolation and compliance needs, and operational cost. Many SaaS products use a discriminator, optionally backed by PostgreSQL row-level security.

Q47. How did you implement multi-tenancy?

> **RefinePoint:** "A discriminator column. Every tenant entity extends `BaseOrgEntity`, so `org_id` is `NOT NULL` by construction. The request's workspace code is resolved to an org id by `OrgAccessInterceptor` after a membership check, and repositories take it explicitly, like `findByOrgIdAndIdempotencyKey`, so a query without a tenant is visible in review. I chose explicit parameters over Hibernate filters because filters do not apply to `find` by id or native SQL, and a lot of our hot-path SQL is native through `JdbcTemplate`."

Q48. What are the risks of Hibernate filters for tenancy?

They must be enabled on every session (one missing place leaks data), they do not apply to `find` / `getReference` by id, to native queries, or to bulk operations in all cases, and they are easy to forget in background jobs that run without a request. Defence in depth: row-level security in PostgreSQL, plus tests that try cross-tenant access.

Q49. When would you use JdbcTemplate instead of JPA?

For set-based or database-specific work: bulk inserts and upserts, `SKIP LOCKED` queue claims, reporting queries with CTEs and window functions, streaming large results, and dynamic table names. JPA stays for aggregates with invariants and moderate volume. They mix safely in one Spring transaction; just flush the persistence context before JDBC reads entity tables you changed.

Q50. How do JPA and JdbcTemplate share a transaction?

`JpaTransactionManager` binds the JDBC connection of the `EntityManager` to the transaction, and `JdbcTemplate` obtains its connection via `DataSourceUtils`, which returns that same bound connection. So both commit or roll back together. Hibernate does not know what JDBC changed, though: entities already loaded may be stale, and pending JPA changes are not visible to JDBC until a flush.

Q51. How do Flyway and Hibernate cooperate at startup?

Boot runs Flyway migrations before building the `EntityManagerFactory`, then Hibernate's `ddl-auto=validate` checks that mappings match the migrated schema and fails startup if not. Never use `ddl-auto=update` in shared environments: it cannot drop or rename, and it changes schemas without review. RefinePoint sets `validate` in the platform defaults.

Q52. Someone edited an applied migration and the service won't start. What happened and what now?

Flyway validates checksums at startup and found a mismatch with `flyway_schema_history`. If the change was meant to alter the schema, revert the file and add a new versioned migration. If it was cosmetic, run `flyway repair` to update the checksum. Applied migrations are immutable history.

Q53. How do you rename a column with zero downtime?

Expand/contract: add the new column, deploy code that writes both and reads the old, backfill in batches, deploy code that reads the new, and later drop the old column. A one-step rename breaks the old instances still running during a rolling deploy.

Q54. A list endpoint got slow as data grew. Walk me through your investigation.

Measure first: count queries per request and time per query from traces. N+1 shows as many identical queries, so fix with fetch joins, batching or a DTO. One slow query: run `EXPLAIN ANALYZE`, check for a missing index on the filter and sort columns (`org_id, started_at`), deep `OFFSET`, a `count(*)` behind `Page`. Huge result sets: paginate, project fewer columns, and use `readOnly` transactions. Then load-test the fix.

Q55. What does `@Transactional(readOnly = true)` do for JPA?

Spring sets the Hibernate session to read-only and the flush mode to `MANUAL`, so no snapshots are kept, no dirty checking runs, and no accidental updates happen. The JDBC connection is also marked read-only, which some drivers and routing data sources use to send queries to replicas. It is a real optimisation for large reads, not only documentation.

Q56. How does Hibernate map a JSON column?

In Hibernate 6, `@JdbcTypeCode(SqlTypes.JSON)` on a `Map`, `List` or record field maps it to `json` / `jsonb` using the configured JSON format mapper (Jackson if present). RefinePoint's `AuditLog` details and billing's `PaymentOrder` use it. For querying inside JSON, native SQL with `jsonb` operators and a GIN index works best.

Q57. Why can a JPA entity's `equals` / `hashCode` break a `Set`?

If they use the generated id, the hash changes from null to a value at persist, so the entity is "lost" in its hash bucket. If they use all fields, lazy proxies and mutable fields cause the same problem. Use a stable business key, or id-based `equals` with a constant `hashCode`, and compare classes with Hibernate-proxy awareness.

Q58. How do you stream a large result set without running out of memory?

Use a `Stream<T>` repository method inside a read-only transaction with a fetch size hint (`@QueryHints(@QueryHint(name = HINT_FETCH_SIZE, value = "1000"))`), process each row, and `detach` or `clear` as you go. PostgreSQL only streams with autocommit off and a fetch size set. Or use `JdbcTemplate` with a `RowCallbackHandler` , which avoids entity overhead entirely.

Q59. What changed for JPA when moving to Boot 3?

`javax.persistence` became `jakarta.persistence` , Hibernate 5 became 6 (new SQM query parser, stricter HQL, `@Where` replaced by `@SQLRestriction` , per-entity default sequence names, `@Type` changes replaced by `@JdbcTypeCode`), and query result deduplication for fetch joins became automatic. Boot 3.4 brings Hibernate 6.6, which also refuses to merge a detached entity whose generated id no longer exists. Run the integration suite against a real PostgreSQL (Testcontainers) when upgrading.

Q60. How would you explain your data-access design in an interview?

> **RefinePoint:** "Every service owns one PostgreSQL schema, migrated by Flyway with `ddl-auto=validate` . Entities extend `BaseOrgEntity` , so each has a UUID id, UTC timestamps and `org_id` . We map associations as ids rather than object graphs, so there are no cascades or N+1 surprises, and open-in-view is off platform-wide. Spring Data repositories cover CRUD and paging with org-scoped derived queries. Billing claims use a `PESSIMISTIC_WRITE` lock. Everything set-based or hot goes through `JdbcTemplate` : `SKIP LOCKED` work queues, `ON CONFLICT` upserts, batched inserts and idempotent consumer inserts. Each tool is used where it is strongest."

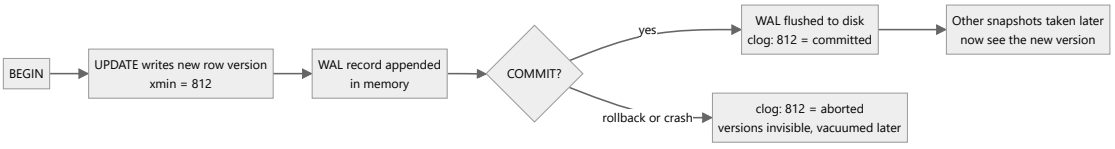
21. Transactions in Depth

Transactions are where senior interviews separate people who have *used* `@Transactional` from people who know what it does. The questions go deep fast: which connection does the inner method use, why did the outer method throw `UnexpectedRollbackException`, why did the pool run dry under load, how do you keep a database write and a Kafka message consistent without XA. This chapter goes from ACID down to the `ThreadLocal` that holds your connection, then up to sagas and the transactional outbox we run in production.

21.1 ACID — what each letter really costs

Letter	Promise	How PostgreSQL delivers it	What breaks it in application code
Atomicity	All statements commit or none do	Every row version carries the creating transaction id (`xmin`); an aborted transaction's versions are simply invisible	Doing non-transactional side effects (HTTP call, Kafka send, file write) inside the transaction
Consistency	Constraints hold before and after	`NOT NULL` , `CHECK` , `UNIQUE` , `FOREIGN KEY` , deferred constraints checked at commit	Invariants kept only in Java ("never more than 5 seats") without a constraint or a lock
Isolation	Concurrent transactions do not see each other's half-done work	MVCC snapshots + row locks + predicate locks (SERIALIZABLE)	Choosing READ COMMITTED and writing read-modify-write logic
Durability	Committed data survives a crash	Write-ahead log (WAL) flushed (`fsync`) at commit, replayed on recovery	`synchronous_commit = off` (trades last few ms of commits for speed)

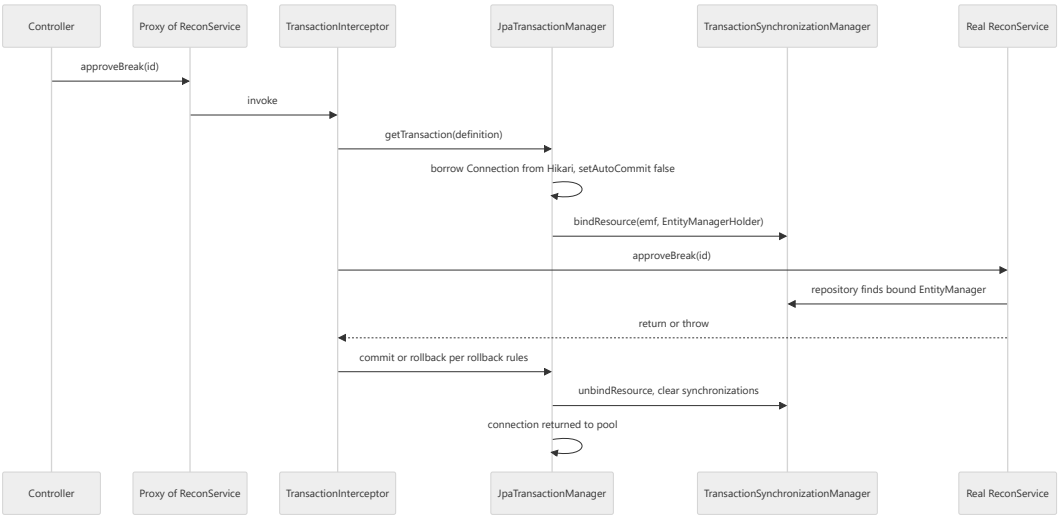
The **C** is the odd one out: atomicity, isolation and durability are database guarantees; consistency is a joint responsibility. The database enforces declared constraints; your code enforces everything else, and it can only do so correctly if A, I and D hold.



PostgreSQL never overwrites a row in place: an `UPDATE` writes a new tuple version and marks the old one dead (`xmax`). Rollback is therefore nearly free (just mark the transaction aborted in the commit log), and readers never block writers. The price is **vacuum**: dead versions must be cleaned up, and a long-running transaction prevents that cleanup because its snapshot might still need old versions. This is one of several reasons long transactions are harmful (§21.9).

21.2 How a Spring local transaction actually works

`@Transactional` is AOP. The bean you inject is a proxy (JDK dynamic proxy for interfaces, CGLIB subclass otherwise; Boot defaults to CGLIB). The proxy's advice is `TransactionInterceptor`, which delegates to a `PlatformTransactionManager`.



Key internals:

1. **`TransactionDefinition`** — propagation, isolation, timeout, readOnly, name. Built from the annotation by `AnnotationTransactionAttributeSource` (it caches attributes per method).
2. **`PlatformTransactionManager`** — three methods: `getTransaction(def)` , `commit(status)` , `rollback(status)` . Implementations: `DataSourceTransactionManager` / `JdbcTransactionManager` (plain JDBC, `JdbcTemplate`), `JpaTransactionManager` (JPA, also covers `JdbcTemplate` on the same `DataSource`), `JtaTransactionManager` (XA), `KafkaTransactionManager` . Reactive code uses `ReactiveTransactionManager` (`R2dbcTransactionManager`) and the Reactor context instead of thread locals.
3. **`TransactionSynchronizationManager`** — a set of `ThreadLocal` s: `resources` (a map from `DataSource` / `EntityManagerFactory` to the holder of the current connection/EntityManager), `synchronizations` (callbacks like `afterCommit`), current transaction name, read-only flag, isolation level and

`actualTransactionActive`.

4. **Resource lookup** — `JdbcTemplate` calls `DataSourceUtils.getConnection(ds)`, which first checks the thread-local map. If a transaction bound a connection, you get *that* connection; otherwise a fresh auto-commit connection. Spring Data repositories use a shared `EntityManager` proxy that does the same lookup. That is how two repositories called from one service method share one database transaction without passing anything around.

```
Thread "http-nio-8080-exec-7"
 └── ThreadLocal resources
       ├── HikariDataSource@1a2b      → ConnectionHolder(conn#4, autoCommit=false)
       └── EntityManagerFactory@9f0e  → EntityManagerHolder(em, tx active)
 └── ThreadLocal synchronizations     → [TransactionalEventListener adapters, afterCommit callbacks]
 └── currentTransactionReadOnly       → false
```

Consequences you must be able to state: the transaction is **bound to the thread**. Code on another thread (an `@Async` method, a `CompletableFuture.supplyAsync`, a parallel stream, a new virtual thread) does not see it.

Proxy rules that follow from AOP

- **Self-invocation bypasses the proxy**: `this.save()` inside the same bean calls the real method, so its `@Transactional` is ignored.
- Annotate **public** methods. Since Spring 6.0, protected and package-private methods are also intercepted on CGLIB proxies, but private methods and `final` methods never are (CGLIB cannot override them).
- `@Transactional` on an interface method works only with interface-based proxies; put it on the class.
- The proxy wraps the *bean*, so calling the method on an object you created with `new` gets no transaction.

21.3 Propagation — all seven levels

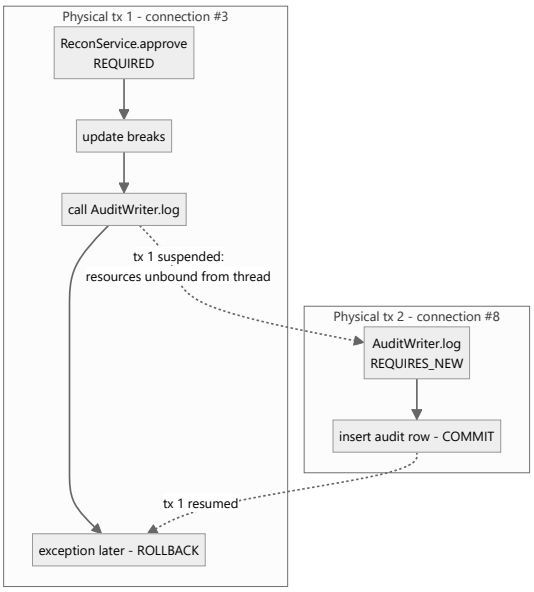
Propagation answers: "a transactional method is called; is there already a transaction on this thread, and what should I do about it?"

Propagation	Existing transaction present	No transaction present	Typical use
`REQUIRED` (default)	Join it	Start a new one	Almost everything
`REQUIRES_NEW`	**Suspend** it, start an independent one (new connection)	Start a new one	Audit/error logging that must survive the caller's rollback; claiming a work item
`NESTED`	Create a **savepoint** inside it	Behave like REQUIRED	Optional sub-step whose failure should roll back only itself
`SUPPORTS`	Join it	Run non-transactionally	Read helpers that do not care
`NOT_SUPPORTED`	Suspend it, run without	Run without	Long non-DB work called from a transactional method (e.g., calling an LLM)
`MANDATORY`	Join it	Throw `IllegalTransactionStateException`	Repository-like methods that must never run alone (e.g., `EventPublisher.publish`)
`NEVER`	Throw	Run without	Guard: method must not be called inside a transaction

REQUIRED: one physical transaction, many logical scopes

Each `@Transactional` method creates a *logical* transaction scope, but with REQUIRED they all map to one *physical* transaction. Only the outermost scope commits. If any inner scope fails, it cannot roll back alone: it sets the shared transaction's **rollback-only** flag. That is the root of `UnexpectedRollbackException` (§21.6).

REQUIRES_NEW: a second connection



Suspension unbinds the outer resources from the thread-local map and stores them in a `SuspendedResourcesHolder`; the inner method gets a new connection, commits, and the outer resources are re-bound. Use cases:

- **Audit or failure logs** that must persist even if the business transaction rolls back ("user tried to approve and failed").
- **Claiming work** so the claim is visible immediately to other workers.
- **Sequence/number allocation** that must not be held locked for the outer transaction's duration.

Costs: two connections per thread at the same time (pool deadlock risk, §21.9), the inner transaction cannot see the outer one's uncommitted rows, and if the inner one updates a row the outer one has locked, **the thread deadlocks with itself** (inner waits for the outer's lock; the outer waits for the inner to return).

> **RefinePoint:** we chose the opposite of REQUIRES_NEW for audit. `AuditTrail.record(...)` writes the entry to the service's outbox in the **same** transaction as the change, and Kafka delivers it to audit-service. The audit log can never claim something that was rolled back, and an audit-service outage never blocks the action. The trade-off: a rolled-back attempt leaves no audit entry. If you must also record failed attempts, that is when REQUIRES_NEW earns its place. Know both patterns and be able to say which question each answers ("what changed" vs "who tried").

NESTED: savepoints

`NESTED` issues `SAVEPOINT sp1` on the *same* connection. If the nested method throws, Spring runs `ROLLBACK TO SAVEPOINT sp1` and the outer transaction continues; if it succeeds, `RELEASE SAVEPOINT`. Nothing is durable until the outer transaction commits, which is the difference from REQUIRES_NEW.

```
@Transactional
public ImportResult importRows(List<Row> rows) {
    for (Row row : rows) {
        try { rowImporter.importOne(row); }          // NESTED: a bad row rolls back only itself
        catch (DataIntegrityViolationException e) { result.reject(row, e); }
    }
    return result;                                   // good rows commit together
}

@Transactional(propagation = Propagation.NESTED)
public void importOne(Row row) { ... }
```

Support: `DataSourceTransactionManager` / `JdbcTransactionManager` yes (JDBC 3 savepoints). `JpaTransactionManager` supports it only for JDBC access through the same connection: the database rolls back to the savepoint but Hibernate's persistence context still holds the entities you changed, so mixing NESTED with JPA entity changes is unreliable. `JtaTransactionManager` does not support it. In PostgreSQL a failed statement aborts the whole transaction (`current transaction is aborted, commands ignored until end of transaction block`), so savepoints are the only way to continue after an error inside one transaction.

21.4 Isolation levels and the anomalies they prevent

The anomalies, with timelines

Dirty read — reading another transaction's uncommitted write.

```
T1: UPDATE acct SET bal = 0 WHERE id=1     (not committed)
T2:                                         SELECT bal → 0   ← dirty
T1: ROLLBACK                                T2 acted on a value that never existed
```

Non-repeatable read — the same row read twice gives different values because someone committed in between.

```
T1: SELECT status FROM run WHERE id=7 → QUEUED
T2:                                    UPDATE run SET status='RUNNING' WHERE id=7; COMMIT
T1: SELECT status FROM run WHERE id=7 → RUNNING   ← changed under T1
```

Phantom read — the same *predicate* returns a different set of rows.

```
T1: SELECT count(*) FROM breaks WHERE run_id=7 → 10
T2:                                    INSERT INTO breaks (run_id ...) VALUES (7 ...); COMMIT
T1: SELECT count(*) FROM breaks WHERE run_id=7 → 11   ← phantom row
```

Lost update — two read-modify-write cycles overlap; one write overwrites the other.

```
T1: SELECT bal → 100                T2: SELECT bal → 100
T1: UPDATE bal = 100 − 30 → 70      T2: UPDATE bal = 100 − 50 → 50 (waits for T1's row lock)
T1: COMMIT                          T2: proceeds, writes 50; COMMIT  ← T1's −30 is lost
```

Write skew — two transactions read an overlapping set, each writes a *different* row, and together they violate an invariant neither violated alone.

```
Rule: at least one approver must stay on duty
T1: SELECT count(*) WHERE on_duty → 2     T2: SELECT count(*) WHERE on_duty → 2
T1: UPDATE approver SET on_duty=false WHERE id=A
                                          T2: UPDATE approver SET on_duty=false WHERE id=B
T1: COMMIT                                T2: COMMIT   → zero approvers on duty
```

No row was written by both, so row locks never conflict. Only SERIALIZABLE (or explicit locking of the rows read) prevents it.

The matrix

Level	Dirty read	Non-repeatable	Phantom	Lost update	Write skew
READ UNCOMMITTED (SQL std)	possible	possible	possible	possible	possible
READ COMMITTED	prevented	possible	possible	possible	possible
REPEATABLE READ (SQL std)	prevented	prevented	possible	prevented*	possible
REPEATABLE READ (PostgreSQL)	prevented	prevented	**prevented**	prevented (error 40001)	possible
SERIALIZABLE	prevented	prevented	prevented	prevented	prevented

*Depends on implementation. MySQL InnoDB REPEATABLE READ (its default) uses next-key/gap locks for locking reads but plain `SELECT` reads a snapshot, so lost updates are still possible there without `FOR UPDATE`.

PostgreSQL specifics you should quote

- **Default is READ COMMITTED**. `READ UNCOMMITTED` is accepted but behaves as READ COMMITTED — PostgreSQL never shows dirty data.
- In READ COMMITTED each **statement** gets a fresh snapshot. Two `SELECT`s in one transaction can disagree.
- When an `UPDATE` / `DELETE` / `SELECT FOR UPDATE` finds a row that a concurrent transaction is changing, it **waits** for that transaction. If it commits, PostgreSQL re-reads the *newest* version and re-checks the `WHERE` clause on it (the "EvalPlanQual" recheck), then updates that version. That is why `UPDATE acct SET bal = bal - 30` is safe under READ COMMITTED while read-then-write in Java is not.
- **REPEATABLE READ** is snapshot isolation: one snapshot for the whole transaction, taken at the first statement. If you try to update a row someone else changed after your snapshot, you get `ERROR: could not serialize access due to concurrent update` (SQLSTATE `40001`). The application must **retry the whole transaction**.
- **SERIALIZABLE** is Serializable Snapshot Isolation (SSI): snapshot isolation plus tracking of read/write dependencies (SIRead predicate locks). It aborts one transaction of a dangerous cycle with `40001`. Overhead is moderate; the real cost is that you need retry logic everywhere.

```java
@Retryable(retryFor = CannotAcquireLockException.class, maxAttempts = 3, backoff = @Backoff(delay = 50, multiplier = 2))
@Transactional(isolation = Isolation.SERIALIZABLE)
public void goOffDuty(UUID approverId) { ... }
```

Spring translates `40001` into `CannotAcquireLockException` / `PessimisticLockingFailureException` subclasses (both `TransientDataAccessException`). The retry must wrap the **outside** of the transaction — a retry inside a rolled-back transaction is useless. With Spring Retry, put `@Retryable` on the same method and make sure the retry advice runs before the transaction advice (it does by default because `@EnableRetry` uses a higher precedence), or call the transactional method from a separate retrying bean.

> **Interview tip:** when asked "which isolation level do you use?", the senior answer is: "READ COMMITTED (the PostgreSQL default) plus targeted locking — atomic `UPDATE ... SET x = x + 1`, conditional updates, `SELECT ... FOR UPDATE` on the rows that guard an invariant, and unique constraints for 'only one' rules. `@Version` optimistic locking fits user edits with long think time. I would not raise the global isolation level; I would raise it per transaction where an invariant spans rows."

21.5 readOnly, timeout and rollback rules

`readOnly = true`

It is a **hint** whose effect depends on the transaction manager and driver:

Layer	Effect
Spring	Sets `TransactionSynchronizationManager.isCurrentTransactionReadOnly()` — usable for routing to a read replica (`AbstractRoutingDataSource` + `LazyConnectionDataSourceProxy`)
`JpaTransactionManager` + Hibernate	Session flush mode set to `MANUAL` and session default read-only: no dirty-checking snapshots kept, no flush at commit — less memory and CPU for big reads
JDBC	`Connection.setReadOnly(true)` ; with `JdbcTransactionManager.setEnforceReadOnly(true)` Spring also executes `SET TRANSACTION READ ONLY`
PostgreSQL	A read-only transaction rejects writes (`cannot execute UPDATE in a read-only transaction`) and can use cheaper paths for SERIALIZABLE (`READ ONLY DEFERRABLE` never aborts)

Common pattern: `@Transactional(readOnly = true)` on the service class, `@Transactional` on write methods. Spring Data's `SimpleJpaRepository` does exactly this.

> **Trap:** `readOnly` on an outer REQUIRED transaction is inherited by inner methods. An inner `@Transactional` write method joining a read-only transaction will not flush its JPA changes (flush mode MANUAL) — changes silently disappear. Spring logs nothing by default.

`timeout`

`@Transactional(timeout = 30)` sets a deadline in seconds. It is **not** a thread interrupt: Spring checks the deadline when a statement is prepared (`JdbcTemplate` applies `Statement.setQueryTimeout` with the remaining time, JPA applies the query timeout hint), and throws `TransactionTimedOutException` if the deadline has passed. A loop that spends 5 minutes in Java code between queries will not be stopped mid-loop. For a database-side guarantee, set PostgreSQL `statement_timeout` , `lock_timeout` and `idle_in_transaction_session_timeout` .

Rollback rules

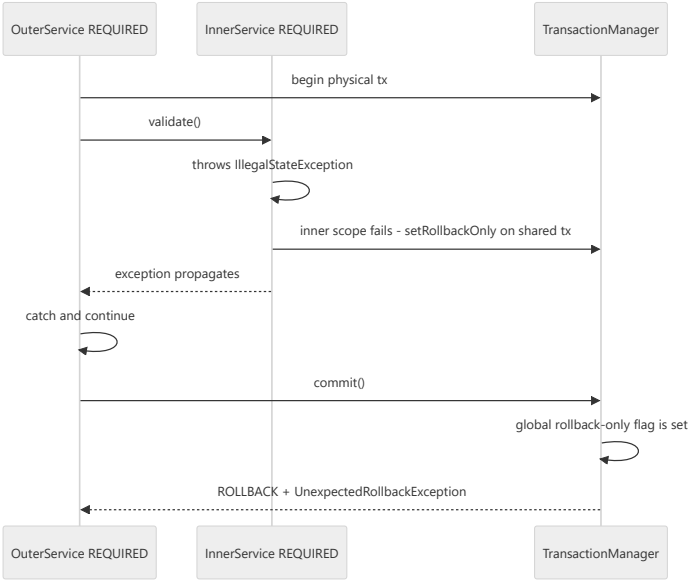
Default: roll back on **unchecked** exceptions (`RuntimeException`) and `Error` ; **commit** on checked exceptions. Historical reason: EJB treated checked exceptions as "business outcomes".

```
@Transactional(rollbackFor = Exception.class)                // roll back on checked too
@Transactional(noRollbackFor = DuplicateBreakException.class) // keep the tx despite this runtime exception
```

Spring Framework 6.2 (Boot 3.4) adds a global switch: `@EnableTransactionManagement(rollbackOn = RollbackOn.ALL_EXCEPTIONS)` . Rollback rules match by exception type and its subclasses; the string form (`rollbackForClassName`) matches by name pattern.

The decision is made when the exception **leaves the proxy**. An exception caught inside the method never reaches the interceptor, so the transaction commits — unless an inner REQUIRED method already marked it rollback-only.

21.6 Exceptions and the rollback-only trap



The inner method did not start the physical transaction, so it cannot roll it back; it can only vote. The outer method swallowed the exception and asked to commit, which contradicts the vote, so Spring rolls back and throws `UnexpectedRollbackException: Transaction silently rolled back because it has been marked as rollback-only`.

Fixes, depending on intent:

1. The inner failure is really fatal → do not catch it.
2. The inner step is optional → make it `REQUIRES_NEW` (independent outcome) or `NESTED` (savepoint), or `noRollbackFor` that exception.
3. The inner method should not be transactional at all (pure validation) → remove `@Transactional` from it, so throwing does not touch the transaction.

You can also mark rollback without throwing: `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`, or `status.setRollbackOnly()` inside `TransactionTemplate`.

The full list of "my @Transactional did nothing" causes

Cause	Why
Self-invocation (`this.method()`)	Proxy bypassed
Private / final method	Proxy cannot intercept
Bean created with `new` or not a Spring bean	No proxy
Checked exception thrown	Default rules commit
Exception caught inside the method	Interceptor never sees it
Wrong transaction manager (two `DataSource` s)	Transaction on DS A, query on DS B (auto-commit)
Work done on another thread	Thread-local binding
MySQL MyISAM table	Engine is not transactional
`@Transactional` from `jakarta.transaction` vs Spring's	Both work in Spring, but `jakarta` has no `readOnly` / `isolation` / `propagation` nuances beyond `TxType`

21.7 Programmatic transactions

Declarative is the default; programmatic is better when the transaction boundary is **smaller than a method** or **chosen at runtime** (loops, schedulers, batches).

```
// TransactionTemplate: callback style, uses the same PlatformTransactionManager
Integer sent = tx.execute(status -> sendBatch());            // commit on return, rollback on RuntimeException
tx.executeWithoutResult(status -> { if (bad) status.setRollbackOnly(); });

// Custom settings per template
var requiresNew = new TransactionTemplate(txManager);
requiresNew.setPropagationBehavior(TransactionDefinition.PROPAGATION_REQUIRES_NEW);
requiresNew.setTimeout(10);
```

RefinePoint: `OutboxRelay` (in ryfn-platform) is a `@Scheduled` method that loops: `do { sent = tx.execute(status -> sendBatch()); } while (sent == BATCH);`. Each batch of 100 is its own short transaction: lock rows with `FOR UPDATE SKIP LOCKED`, send each to Kafka, mark it sent, commit. If we had put `@Transactional` on `relay()`, one transaction would cover the whole backlog, keep thousands of rows locked, and lose all progress on failure. `JobScheduler.claim()` and `RunWorker.claim()` use `TransactionTemplate` the same way: a few-millisecond transaction to claim work, and the long work itself runs **outside** any transaction.

Lowest level: `PlatformTransactionManager` directly (`getTransaction` / `commit` / `rollback` in try/finally) — only in framework code.

21.8 Transaction-bound events

```
@Transactional
public void complete(UUID runId) {
    run.complete();
    events.publishEvent(new RunCompleted(runId));   // ApplicationEventPublisher
}

@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onCompleted(RunCompleted e) { notifier.email(e); }
```

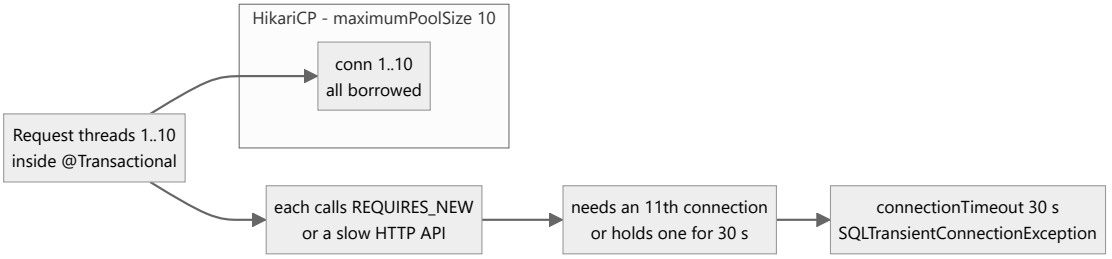
Phase	Runs	Use for
`BEFORE_COMMIT`	During `commit()`, before the database commit, inside the transaction	Last-moment writes/validation that must be atomic with the transaction
`AFTER_COMMIT` (default)	After a successful commit	Emails, cache eviction, WebSocket pushes
`AFTER_ROLLBACK`	After rollback	Compensation logging, metrics
`AFTER_COMPLETION`	After either	Cleanup

Internals: publishing inside a transaction registers a `TransactionSynchronization` in the thread-local list; the transaction manager calls `beforeCommit`, `afterCommit`, `afterCompletion` on it. If there is **no** active transaction, the event is dropped unless `fallbackExecution = true`.

Pitfalls:

- In `AFTER_COMMIT`, the old transaction's resources are still bound but already committed. A write with plain REQUIRED joins a finished transaction and is **never committed**. Spring 6.1+ refuses to start if a `@TransactionalEventListener` method is annotated `@Transactional` with anything other than `REQUIRES_NEW` or `NOT_SUPPORTED`.
- `AFTER_COMMIT` is not durable: if the JVM dies after the commit and before the listener runs, the email is never sent. For anything that must happen, use the outbox (§21.11).
- Listeners run synchronously on the same thread unless you add `@Async`.

21.9 Transactions, threads, virtual threads and pool exhaustion



Why a transaction pins a connection: from `BEGIN` to `COMMIT` the connection belongs to that thread. Anything slow inside the method — an HTTP call, an LLM request, generating a report, waiting on a lock — keeps it borrowed. With 10 connections and 200 request threads, ten slow transactions stop the whole service; every other request waits `connectionTimeout` (30 s by default) and fails with `Connection is not available, request timed out after 30000ms`.

Rules to follow:

1. **No remote calls inside a transaction.** Fetch first, transact briefly, call afterwards (or use the outbox).
2. **Short claim transactions, long work outside** (the run-queue pattern).
3. `spring.jpa.open-in-view=false` (ryfn-platform's `PlatformDefaults` sets it for every RefinePoint service) — Open Session In View keeps the persistence context, and with Hibernate's default connection handling the connection too, until the view is rendered.
4. Size the pool for the database, not for threads: a few dozen connections per database server, divided across instances. HikariCP's formula `connections = cores * 2 + effective spindles` is a starting point.
5. Set `idle_in_transaction_session_timeout` in PostgreSQL to kill sessions a crashed or buggy app left open.
6. REQUIRES_NEW needs two connections per thread: if all N threads are in the outer transaction and all need an inner one, nobody can proceed — a **pool deadlock** until timeout.

`@Async` **and executors:** the task runs on another thread, outside the caller's transaction. If the caller passes an entity id and the async task reads it before the caller commits, the task does not find the row. Publish async work from `AFTER_COMMIT` or the outbox.

Virtual threads (Java 21): `TransactionSynchronizationManager` uses `ThreadLocal`, and each virtual thread has its own thread locals, so declarative transactions work unchanged on virtual threads (`spring.threads.virtual.enabled=true`). Two caveats: (a) virtual threads make it cheap to have 10,000 concurrent requests but the pool still has 10 connections, so requests now queue on `getConnection()` instead of on the Tomcat thread pool — limit concurrency with a semaphore or by pool size; (b) in Java 21 a virtual thread blocked inside a `synchronized` block **pins** its carrier thread; older JDBC drivers and pools used `synchronized` heavily. PostgreSQL JDBC 42.6+ and HikariCP 5 replaced most of it with `ReentrantLock`, and JDK 24 (JEP 491) removes pinning for `synchronized`.

Reactive: WebFlux + R2DBC keeps transaction state in the Reactor `Context`, not thread locals; `@Transactional` works with `R2dbcTransactionManager` because the operators carry the context across threads.

21.10 Locking: optimistic vs pessimistic

	Optimistic	Pessimistic
Idea	Detect conflict at write time	Prevent conflict by locking at read time
Mechanism	`@Version` column: `UPDATE ... SET version = 6 WHERE id = ? AND version = 5`	`SELECT ... FOR UPDATE` (row lock held until commit)
On conflict	0 rows updated → `OptimisticLockException` → retry or tell the user	Second transaction waits (or fails with NOWAIT / skips with SKIP LOCKED)
Best for	Low contention, long "think time" (user edits a form for 5 minutes)	High contention on few rows, short transactions, queues
Cost	Retry logic, wasted work on conflict	Blocking, deadlock risk, held connections

```
@Entity
class Invoice {
    @Version private long version;    // Hibernate adds "AND version = ?" and increments it
}

public interface TaskRepository extends JpaRepository<Task, UUID> {
```

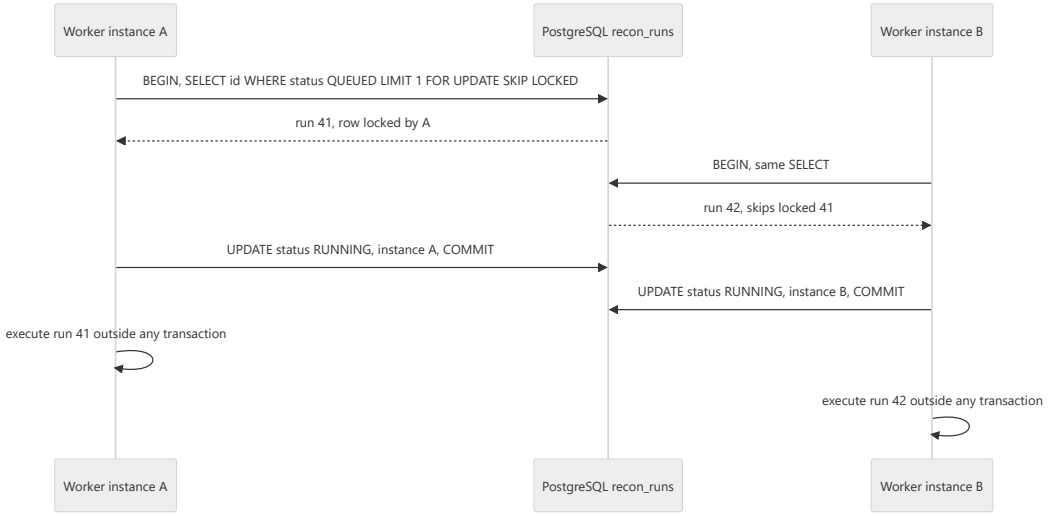
```
    @Lock(LockModeType.PESSIMISTIC_WRITE)                    // SELECT ... FOR NO KEY UPDATE / FOR UPDATE
    @QueryHints(@QueryHint(name = "jakarta.persistence.lock.timeout", value = "-2"))  // -2 = SKIP LOCKED, 0 = NOWAIT
    @Query("select t from Task t where t.status = 'QUEUED' order by t.createdAt")
    List<Task> claimNext(Pageable one);
}
```

For REST APIs, optimistic locking extends to HTTP: return the version as an `ETag`, require `If-Match` on `PUT`, answer `412 Precondition Failed` on mismatch. This stops two finance users silently overwriting each other's rule edits.

FOR UPDATE, NOWAIT and SKIP LOCKED

Clause	Behaviour when the row is locked by someone else
`FOR UPDATE`	Wait until the other transaction ends (bounded by `lock_timeout`)
`FOR UPDATE NOWAIT`	Fail immediately: `ERROR 55P03 could not obtain lock on row`
`FOR UPDATE SKIP LOCKED`	Pretend the row is not there; return the next unlocked rows

PostgreSQL also has weaker row locks: `FOR NO KEY UPDATE` (what a normal `UPDATE` takes when key columns do not change; does not block foreign-key checks), `FOR SHARE` and `FOR KEY SHARE` (taken by FK checks).



RefinePoint: run-service, job-service (`JobScheduler`), the export worker and the datasource refresh worker are all PostgreSQL work queues. `RunWorker.claim()` runs in a `TransactionTemplate`: it takes `pg_advisory_xact_lock(hashtext('run-queue'))` so per-workspace concurrency caps are checked by one claimer at a time (the claim lasts milliseconds), selects one queued run `FOR UPDATE OF r SKIP LOCKED`, marks it RUNNING with the instance name and a heartbeat, and commits. The run itself executes outside the transaction. A reaper finds runs whose heartbeat is older than two minutes with `UPDATE ... WHERE id IN (SELECT ... FOR UPDATE SKIP LOCKED) RETURNING id`, so only one instance fails each orphaned run. Scaling is "start another instance" — no leader election, no ZooKeeper.

Deadlocks

Two transactions lock rows in opposite order. PostgreSQL detects the cycle after `deadlock_timeout` (1 s) and aborts one with `40P01 deadlock detected`; Spring maps it to `DeadlockLoserDataAccessException` (deprecated alias) / `CannotAcquireLockException`. Prevention: lock rows in a consistent order (`ORDER BY id ... FOR UPDATE`), keep transactions short, avoid lock upgrades (read with `FOR SHARE` then update), retry the loser.

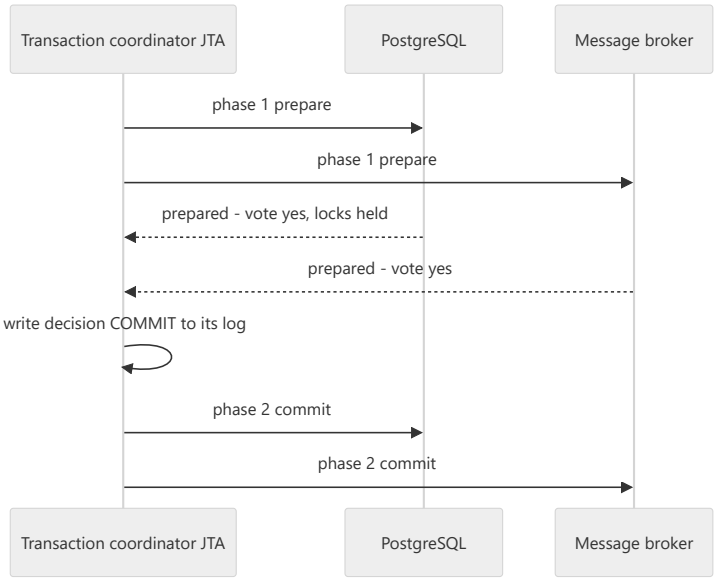
21.11 Distributed transactions: the dual-write problem, 2PC, sagas and the outbox

The dual-write problem

```
@Transactional
public void completeRun(UUID id) {
    runRepo.markCompleted(id);
    kafka.send("run.completed", id.toString(), json);   // not part of the DB transaction
}
```

Four failure windows: Kafka send succeeds and DB commit fails (consumers act on a run that is not completed); DB commits and the process dies before send (event lost); send times out but was actually written (unknown); send after commit fails (event lost). There is no ordering of two independent writes that is safe. You need either one transaction spanning both systems (2PC) or a design where only one system is written transactionally (outbox).

Two-phase commit (XA)



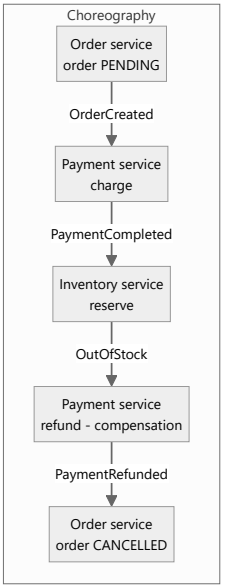
Why we avoid it in microservices:

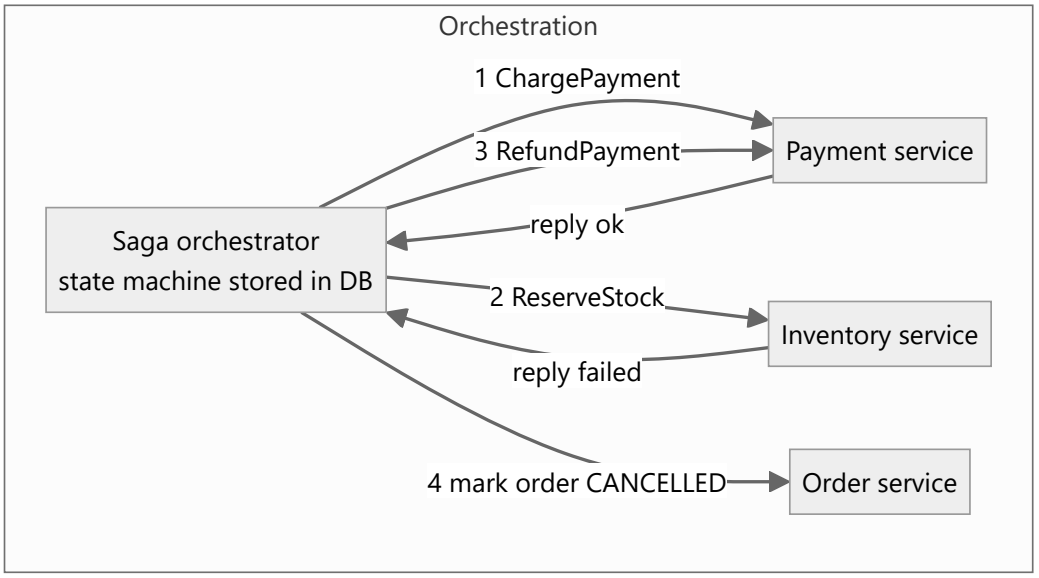
- **Blocking**: a participant that voted yes must hold locks until it hears the decision; if the coordinator dies, those rows stay locked ("in-doubt" transactions) until manual recovery.
- **Availability**: the transaction succeeds only if every participant is up — availability multiplies down.
- **Support**: Kafka does not support XA; many cloud databases and HTTP APIs cannot participate at all. PostgreSQL supports it (`PREPARE TRANSACTION`) but ships with `max_prepared_transactions = 0`.
- **Latency and coupling**: extra round trips, shared coordinator, tight runtime coupling between services that should be independent.

2PC is still valid inside one boundary (an app server writing to two databases you own, using Atomikos or Narayana with `JtaTransactionManager`).

Sagas

A saga is a sequence of local transactions, each publishing an event or command that triggers the next; on failure, earlier steps are undone by **compensating transactions**. You give up isolation (other transactions see intermediate states) to gain availability.

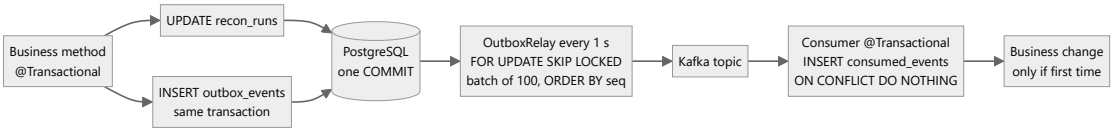




	Choreography	Orchestration
Control	Each service reacts to events	One component tells each service what to do
Coupling	Services know event types only	Orchestrator knows every participant
Visibility	Flow is implicit, hard to see	Flow is explicit in one state machine
Good for	2–4 steps, simple flows	Many steps, branching, timeouts, human approvals
Tools	Kafka topics	Temporal, Camunda, a hand-written state table

Design rules: each step is a local ACID transaction; compensations must be idempotent and must be possible (you cannot "un-send" an email — send it last, the "pivot transaction" idea); use **semantic locks** (status `PENDING`) so other flows know a record is mid-saga; every message is published with the outbox.

The transactional outbox



The service writes the event as a row **in the same local transaction** as the business change. Atomicity is now a database guarantee: both or neither. A separate relay reads unsent rows and publishes them. Two relay styles:

- **Polling publisher** (what we use): a scheduled query. Simple, no extra infrastructure, adds up to one polling interval of latency.
- **Transaction log tailing / CDC**: Debezium reads the PostgreSQL WAL through a logical replication slot and publishes rows. Lower latency, no polling load, but another moving part to operate (and a replication slot that retains WAL if Debezium stops).

> **RefinePoint:** `EventPublisher.publish(topic, key, type, payload)` builds an `EventEnvelope` (event id UUID, type, source service, timestamp, payload) and inserts it into `outbox_events` using the caller's transaction. `OutboxRelay` sends rows oldest first (`ORDER BY seq`), waits for each Kafka ack (`.get(10, SECONDS)`), sets `sent_at`, and on the first failure records `attempts` / `last_error` and **stops the batch** so later events never overtake an earlier one. Because a commit can fail after Kafka acknowledged, or a relay can crash between send and update, delivery is **at least once**. Every consumer calls `ConsumedEvents.firstTime(consumer, eventId)` — an `INSERT INTO consumed_events (consumer, event_id) VALUES (?, ?) ON CONFLICT DO NOTHING` that returns whether one row was inserted — *inside its own business transaction*, so "record that I handled it" and "do the work" commit or roll back together. A purge job deletes sent events after 7 days and consumed ids after 30.

Idempotency

At-least-once delivery + idempotent consumer = effectively exactly-once processing. Ways to make an operation idempotent:

Technique	Example
Dedup table keyed by message id	`consumed_events (consumer, event_id)` primary key
Natural idempotence	`UPDATE run SET status = 'COMPLETED' WHERE id = ? AND status = 'RUNNING'` — second call changes 0 rows
Unique business key	`UNIQUE (org_id, paddle_subscription_id)` + upsert
Idempotency-Key header on APIs	Store key + request hash + response; replay the stored response for repeats
Version check	Apply event only if `event.version > row.version`

Kafka's own "exactly-once semantics" (idempotent producer + transactions + `read_committed` consumers) covers Kafka-to-Kafka pipelines. It does not include your database, which is why the outbox and dedup table are still needed.

21.12 Interview questions and answers

Q1. What is a transaction and what does ACID mean?

A transaction is a unit of work that the database executes as all-or-nothing, isolated from concurrent work and durable once committed. Atomicity: all statements commit or none do. Consistency: declared constraints hold before and after. Isolation: concurrent transactions behave (to a chosen degree) as if run one after another. Durability: a committed change survives a crash because the WAL is flushed at commit. Add the senior nuance: consistency is shared with the application — the database only enforces the rules you declared.

Q2. How does PostgreSQL implement atomicity and isolation without blocking readers?

MVCC. Each row version records the transaction that created it (`xmin`) and the one that deleted or replaced it (`xmax`). A snapshot is a list of which transactions were committed when it was taken; a reader sees only versions created by committed-before-snapshot transactions. Writers create new versions, so readers never wait for writers. Rollback just marks the transaction aborted in the commit log. Vacuum later removes dead versions.

Q3. What happens internally when a method annotated with `@Transactional` is called?

The call hits the Spring proxy, whose `TransactionInterceptor` reads the method's transaction attributes, asks the `PlatformTransactionManager` for a transaction (borrowing a connection, `setAutoCommit(false)` , binding it to the thread through `TransactionSynchronizationManager`), invokes the real method, then commits or rolls back according to the rollback rules, runs synchronization callbacks, unbinds the resources and returns the connection to the pool. Repositories and `JdbcTemplate` find the bound connection through the thread-local map.

Q4. Which `PlatformTransactionManager` does Spring Boot configure?

With `spring-boot-starter-data-jpa` : `JpaTransactionManager` (which also exposes the JDBC connection, so `JdbcTemplate` on the same `DataSource` joins the same transaction). With only JDBC: `JdbcTransactionManager` (a `DataSourceTransactionManager` that also translates commit exceptions). With JTA libraries on the classpath: `JtaTransactionManager` . With R2DBC: `R2dbcTransactionManager` . If you define several, mark one `@Primary` or name it in `@Transactional("ordersTx")` .

Q5. What is `TransactionSynchronizationManager` ?

The thread-local registry of the current transaction: bound resources (DataSource → `ConnectionHolder` , EntityManagerFactory → `EntityManagerHolder`), registered `TransactionSynchronization` callbacks, the transaction name, read-only flag, isolation level and whether a real transaction is active. You rarely call it directly, but `isActualTransactionActive()` and `registerSynchronization(...)` (for an `afterCommit` hook) are useful in framework code.

Q6. Why does `@Transactional` not work when I call the method from the same class?

Because `this.method()` calls the target object directly, not the proxy, so the interceptor never runs. Fixes: move the method to another bean (cleanest), inject the bean into itself lazily (`@Lazy` self-reference), use `TransactionTemplate` around the code, or use AspectJ weaving mode (`@EnableTransactionManagement(mode = ASPECTJ)`) which weaves the advice into the class bytecode.

Q7. Does `@Transactional` work on private methods?

No. Proxies (JDK or CGLIB) cannot intercept private methods; CGLIB also cannot override `final` methods. Since Spring 6.0, protected and package-private methods are intercepted on class-based proxies, but public remains the convention. AspectJ weaving works on any method.

Q8. Should `@Transactional` go on the controller, service or repository?

The service layer — the transaction boundary is the use case. Controllers should not hold transactions (they do HTTP and serialization work and would keep connections longer); repositories alone are too small (each call would be a separate transaction). Spring Data repositories already have `@Transactional(readOnly = true)` on reads and `@Transactional` on writes, which join the service's transaction.

Q9. Explain the seven propagation levels in one breath each.

REQUIRED: join or create. REQUIRES_NEW: always create, suspending any current one. NESTED: savepoint inside the current one, or create. SUPPORTS: join if present, else run without. NOT_SUPPORTED: suspend and run without. MANDATORY: must join, else exception. NEVER: must not be in one, else exception.

Q10. What is the difference between REQUIRES_NEW and NESTED?

REQUIRES_NEW is a separate physical transaction on a separate connection: it commits independently and survives the outer rollback; it cannot see the outer one's uncommitted data. NESTED is a savepoint on the same connection: its rollback undoes only its work, but nothing commits until the outer transaction commits, so an outer rollback undoes the nested work too, and it sees everything the outer transaction did.

Q11. Give real use cases for REQUIRES_NEW.

Audit or security logs that must persist even if the business operation fails ("login failed", "approval rejected by rule"); persisting a failure status for a job before rethrowing; allocating numbers from a counter table without holding the counter lock for the whole outer transaction; claiming a queue item so it is visible immediately. Each one should be short and must not touch rows the outer transaction has locked.

Q12. What can go wrong with REQUIRES_NEW?

Pool deadlock: each thread needs two connections; under load all connections are held by outer transactions and nobody gets an inner one. Self-deadlock: the inner transaction updates a row the outer one locked and waits until `lock_timeout`. Inconsistency: the inner commit survives an outer rollback, which may be wrong for business data. And it is silently ignored under self-invocation, like every other attribute.

Q13. When does NESTED not work?

With `JtaTransactionManager` (no savepoints), with `JpaTransactionManager` when you rely on entity changes (the database rolls back to the savepoint but the persistence context keeps the modified entities and may flush them later), and with drivers that lack savepoint support. With `JdbcTransactionManager` and plain SQL it is reliable.

Q14. What happens if an inner REQUIRED method throws and the outer method catches the exception?

The inner scope marks the shared transaction rollback-only. The outer method continues and returns normally, the interceptor tries to commit, sees the flag, rolls back and throws `UnexpectedRollbackException`. The caller sees an exception from a method that "succeeded". Fix by not catching, or by making the inner call REQUIRES_NEW or NESTED, or by removing `@Transactional` from the inner method if it only validates.

Q15. Which exceptions trigger rollback by default, and why are checked exceptions different?

`RuntimeException` and `Error` roll back; checked exceptions commit. The convention comes from EJB: checked exceptions were "application exceptions" (expected outcomes such as `InsufficientFundsException`), unchecked were system failures. Use `rollbackFor = Exception.class`, or in Spring 6.2 the global `rollbackOn = RollbackOn.ALL_EXCEPTIONS` on `@EnableTransactionManagement`, if your codebase uses checked exceptions for failures.

Q16. Predict the outcome.

```
@Transactional
public void save(Run r) throws IOException {
    repo.save(r);
    throw new IOException("disk full");
}
```

The run is **committed** and the caller gets `IOException`. Checked exception, default rules. A frequent production bug in file-import code.

Q17. Predict the outcome.

```
@Transactional
public void approveAll(List<UUID> ids) {
    for (UUID id : ids) {
        try { approver.approve(id); }          // approve() is @Transactional (REQUIRED) and throws for one id
        catch (RuntimeException e) { log.warn("skip {}", id); }
    }
}
```

Nothing is approved and `UnexpectedRollbackException` is thrown at the end. The first failure marked the whole transaction rollback-only. To skip bad items, give each item its own transaction (REQUIRES_NEW, or a `TransactionTemplate` per item called from a non-transactional loop) or use NESTED with JDBC.

Q18. What does `readOnly = true` actually do?

It is a hint. Spring exposes it as a thread-local flag (useful for routing to replicas). With Hibernate it sets flush mode MANUAL and read-only sessions, so no dirty-check snapshots are kept and nothing is flushed — real memory and CPU savings for large reads. The JDBC connection is set read-only, and with `enforceReadOnly` Spring issues `SET TRANSACTION READ ONLY`, after which PostgreSQL rejects writes. It does not lock anything.

Q19. How would you route read-only transactions to a replica?

Use an `AbstractRoutingDataSource` whose `determineCurrentLookupKey()` returns `replica` when `TransactionSynchronizationManager.isCurrentTransactionReadOnly()` is true, wrapped in a `LazyConnectionDataSourceProxy` so the physical connection is fetched only at the first statement — after the transaction attributes are known. Warn about replication lag: a user who just saved and immediately reads may not see the change (read-your-writes), so pin those reads to the primary.

Q20. How does the transaction `timeout` work, and why did my long loop not time out?

Spring stores a deadline and checks it whenever a statement is created through Spring-managed resources (it sets the JDBC query timeout to the remaining time and throws `TransactionTimedOutException` once exceeded). It never interrupts Java code, so CPU work or an HTTP call between statements is not stopped. For

hard limits use PostgreSQL `statement_timeout` and `idle_in_transaction_session_timeout`, and timeouts on the HTTP client itself.

Q21. What isolation level does PostgreSQL use by default, and what does it mean in practice?

READ COMMITTED. Every statement sees a snapshot of data committed before that statement started; two reads in one transaction may differ; rows you update are re-checked against the newest committed version if someone changed them concurrently. Dirty reads never happen in PostgreSQL at any level.

Q22. Explain dirty read, non-repeatable read and phantom read.

Dirty: reading uncommitted data of another transaction. Non-repeatable: reading the same row twice and getting different committed values. Phantom: running the same predicate query twice and getting a different set of rows because of inserts or deletes committed in between. Non-repeatable is about one row's content; phantom is about the membership of a result set.

Q23. What is a lost update and how do you prevent it?

Two transactions read the same value, each computes a new value in the application, and the later write overwrites the earlier one. Prevent with: an atomic SQL update (`SET balance = balance - :amt`), a pessimistic lock (`SELECT ... FOR UPDATE` before reading), optimistic locking (`@Version`), or REPEATABLE READ in PostgreSQL (the second writer gets a serialization error and retries).

Q24. What is write skew? Which isolation level prevents it?

Two transactions read an overlapping set of rows, then each updates a different row so that a combined invariant breaks (two approvers both go off duty; two bookings for the same room inserted after both checked "no overlap"). Snapshot isolation (PostgreSQL REPEATABLE READ) allows it because there is no write-write conflict. SERIALIZABLE (SSI) prevents it; so do explicit locks on the rows that represent the invariant, or a constraint (an exclusion constraint for overlapping ranges).

Q25. How does PostgreSQL REPEATABLE READ differ from the SQL standard and from MySQL?

The standard allows phantoms at REPEATABLE READ; PostgreSQL does not, because it uses one snapshot for the whole transaction. On a write-write conflict PostgreSQL raises `40001` instead of overwriting. MySQL InnoDB's REPEATABLE READ (its default) gives consistent snapshots for plain reads, but locking reads and updates see the latest data and use gap locks; lost updates are possible with plain read-then-write.

Q26. How does SERIALIZABLE work in PostgreSQL and what must the application do?

It uses Serializable Snapshot Isolation: transactions run on snapshots as in REPEATABLE READ, while PostgreSQL tracks read-write dependencies with predicate (SIRead) locks; when it detects a pattern that could produce a non-serializable result, it aborts one transaction with SQLSTATE `40001`. The application must retry the whole transaction, outside the transactional method. Declaring `READ ONLY DEFERRABLE` for long reports avoids aborts for them.

Q27. Why can't you retry inside the `@Transactional` method?

After a serialization failure or deadlock, PostgreSQL has aborted the transaction; every further statement fails with "current transaction is aborted". The retry must start a new transaction, so it belongs in a layer above the proxy: a retrying caller, `@Retryable` ordered outside the transaction advice, or a loop around `TransactionTemplate.execute`.

Q28. When would you change the isolation level of a single method?

When an invariant spans several rows and you cannot express it as a constraint or a single atomic statement — for example "total allocated must not exceed budget" across many allocation rows. Options: SERIALIZABLE with retry for that method, or lock the parent budget row with `FOR UPDATE` and stay at READ COMMITTED. The second is usually simpler and more predictable.

Q29. Can `@Transactional(isolation = ...)` change the level in the middle of an existing transaction?

No. Isolation applies when a transaction starts. A REQUIRED method joining an existing transaction runs at the outer level; Spring ignores the inner setting (with `validateExistingTransaction = true` on the manager it throws instead). Use REQUIRES_NEW if the inner work truly needs a different level.

Q30. What is the difference between optimistic and pessimistic locking? When do you use each?

Optimistic: no lock; a version column checked at update time; conflict detected late and handled by retry or by asking the user. Use for low contention and long user think time. Pessimistic: lock the row when reading (`FOR UPDATE`); conflicts wait. Use for hot rows with short transactions, counters, queues. Many systems use both: optimistic for UI edits, pessimistic for background workers.

Q31. How does `@Version` work in Hibernate?

Hibernate adds `AND version = :old` to the `UPDATE` and sets `version = old + 1`. If the update count is 0, someone changed or deleted the row: Hibernate throws `OptimisticLockException` (Spring: `ObjectOptimisticLockingFailureException`). Version can be numeric or a timestamp; numeric is safer. `LockModeType.OPTIMISTIC_FORCE_INCREMENT` bumps the version even without changes, useful to version an aggregate root when a child changes.

Q32. How do you implement optimistic locking across a REST API?

Return the version as an `ETag` (or a `version` field) on GET; the client sends it back in `If-Match` on PUT/PATCH; the service compares before applying (or sets it on the entity so Hibernate checks it) and returns `412 Precondition Failed` or `409 Conflict` on mismatch. Without this, two users editing the same reconciliation rules in separate tabs silently overwrite each other — the version check inside one transaction alone does not help because each request loads fresh data.

Q33. Explain `SELECT ... FOR UPDATE`, `NOWAIT` and `SKIP LOCKED`.

`FOR UPDATE` locks the selected rows until the transaction ends; other lockers wait. `NOWAIT` fails immediately with a lock error instead of waiting — good for user actions where you would rather say "someone else is editing". `SKIP LOCKED` silently skips rows locked by others — ideal for work queues where any free item will do. `SKIP LOCKED` gives an inconsistent view by design, so never use it for reports.

Q34. How do you build a job queue on PostgreSQL with multiple instances?

Table of jobs with a status. Each worker, in a short transaction: `SELECT id FROM jobs WHERE status = 'QUEUED' ORDER BY created_at LIMIT n FOR UPDATE SKIP LOCKED`, update them to `RUNNING` with its instance id and a heartbeat, commit. Run the work outside the transaction. On completion update the status. A reaper requeues or fails rows whose heartbeat is stale. Add an index on `(status, created_at)`; a partial index `WHERE status = 'QUEUED'` is even better.

> **RefinePoint:** this is exactly run-service and job-service. `JobScheduler` claims as many due jobs as it has free workers (`... AND running_execution_id IS NULL AND (next_run_at <= now() OR retry_at <= now()) ... FOR UPDATE SKIP LOCKED`), marks each running, and moves its next due time; a watchdog handles timeouts and stale instances; failed attempts retry with doubling backoff. We scale by adding instances.

Q35. Why not do the actual work inside the claim transaction?

The row lock and the connection would be held for the whole run — minutes for a large reconciliation. That pins a pool connection per running job, holds back vacuum, and if the instance dies the lock disappears but nobody can tell whether the work half-happened. Short claim + status column + heartbeat gives visibility and recovery.

Q36. What causes deadlocks and how do you handle them?

Two transactions each hold a lock the other needs, usually because they lock the same rows in different orders. PostgreSQL detects the cycle (after `deadlock_timeout`, default 1 s) and aborts one with `40P01`. Handle: lock in a consistent order (sort ids, `ORDER BY id FOR UPDATE`), keep transactions short, avoid read-then-upgrade patterns, and retry the aborted transaction. Monitor with `log_lock_waits = on`.

Q37. What is the difference between a row lock and an advisory lock?

Row locks protect rows and are taken by `UPDATE`, `DELETE` and `FOR UPDATE`. Advisory locks are application-defined locks on an arbitrary 64-bit key (`pg_advisory_xact_lock(key)` released at transaction end, `pg_advisory_lock` released at session end or explicitly). Use them to serialize an operation that does not map to a single row — RefinePoint's `RunWorker` takes `pg_advisory_xact_lock(hashtext('run-queue'))` so only one claimer at a time evaluates per-workspace concurrency caps. Prefer the transaction-scoped form; session-scoped locks leak with pooled connections.

Q38. How does `TransactionTemplate` compare to `@Transactional`?

Same transaction manager and semantics; different style. `@Transactional` is declarative, method-sized and proxy-based (self-invocation pitfalls). `TransactionTemplate` is explicit, can wrap any block, can be called in a loop to make many small transactions, and its settings can be chosen at runtime. Use it in schedulers, batch loops and infrastructure code; use the annotation for services.

Q39. Why does RefinePoint's `OutboxRelay` use `TransactionTemplate` instead of `@Transactional`?

Because the transaction must be per batch, not per invocation. `relay()` loops calling `tx.execute(status -> sendBatch())` until a batch comes back smaller than 100. Each batch locks at most 100 rows briefly and commits its progress; a failure loses at most one batch's `sent_at` updates (those events are re-sent — at-least-once). One method-level transaction would lock the whole backlog and roll back all progress on any failure.

Q40. What are the phases of `@TransactionalEventListener` and their pitfalls?

BEFORE_COMMIT, AFTER_COMMIT (default), AFTER_ROLLBACK, AFTER_COMPLETION. Pitfalls: events published without a transaction are dropped unless `fallbackExecution = true`; writes in AFTER_COMMIT need REQUIRES_NEW (Spring 6.1 enforces this at startup); AFTER_COMMIT is not durable — a crash between commit and listener loses the side effect; listeners are synchronous unless `@Async`, so a slow listener delays the HTTP response even though the data is already committed.

Q41. When would you use AFTER_COMMIT listeners instead of the outbox?

For side effects that are cheap to lose or that are local and best-effort: evicting a cache, pushing a WebSocket refresh, incrementing an in-memory metric. Anything another service depends on, or that money or compliance depends on, goes through the outbox.

Q42. Does a transaction propagate to an `@Async` method or a `CompletableFuture`?

No. The transaction is bound to the calling thread. The async method runs on a pool thread with no transaction (or its own, if annotated). It may also run before the caller commits, so it might not see the caller's data. Trigger async work after commit (AFTER_COMMIT listener or outbox) and pass ids, not managed entities — entities are tied to the caller's persistence context.

Q43. Do transactions work with virtual threads?

Yes. Thread locals are per virtual thread, so `TransactionSynchronizationManager` works unchanged. The bottleneck moves to the connection pool: thousands of virtual threads compete for a handful of connections, so add a concurrency limit and keep transactions short. On Java 21, watch for carrier pinning inside `synchronized` blocks in older drivers; current pgJDBC and HikariCP avoid most of it, and JDK 24 removes the problem for `synchronized`.

Q44. A service starts failing with "Connection is not available, request timed out after 30000ms". How do you investigate?

Hikari's pool is exhausted. Check Hikari metrics (`hikaricp_connections_active`, `hikaricp_connections_pending`, usage histograms) and enable `leakDetectionThreshold` to log stack traces of connections held too long. In PostgreSQL, `SELECT pid, state, xact_start, query FROM pg_stat_activity ORDER BY xact_start` shows long or `idle in transaction` sessions. Usual culprits: remote calls inside `@Transactional`, Open Session In View, REQUIRES_NEW under load, a lock wait (one blocked transaction holds a connection and everyone behind it queues), or a pool too small for instances × concurrency. Fix the cause; increasing the pool only moves the bottleneck to the database.

Q45. Why are long transactions harmful in PostgreSQL specifically?

They hold a connection, hold row locks (blocking writers), keep their snapshot's `xmin` horizon old so vacuum cannot remove dead tuples anywhere in the database (table and index bloat, slower queries), and raise the chance of serialization failures and deadlocks. An `idle in transaction` session from a buggy client is the worst case. Set `idle_in_transaction_session_timeout` and alert on transactions older than a few minutes.

Q46. What is Open Session In View and why disable it?

Spring Boot's default `spring.jpa.open-in-view=true` registers an interceptor that opens the `EntityManager` at the start of a web request and closes it at the end, so lazy associations can load during JSON serialization. It hides N+1 queries in the view layer and keeps the persistence context (and typically its connection) for the whole request, including slow serialization or remote calls. Disable it and fetch what the response needs inside the service (fetch joins, entity graphs, DTO projections).

Q47. What is the dual-write problem?

Writing to two systems (database and Kafka, database and a search index, database and another service's API) without a shared transaction. Any ordering leaves a window where one write succeeds and the other does not: events lost, or events describing changes that were rolled back. Solve it by writing to only one system transactionally (outbox, CDC) and propagating asynchronously and idempotently.

Q48. What is 2PC and why do microservices avoid it?

Two-phase commit: a coordinator asks every participant to prepare (durably promise to commit, holding locks), then tells all to commit if all voted yes. It gives atomicity across resources but is blocking (participants hold locks while waiting for the coordinator; a coordinator crash leaves in-doubt transactions), reduces availability (all participants must be up), adds latency, and many systems — Kafka, most SaaS APIs, many managed databases — cannot take part. Microservices prefer eventual consistency with sagas and outboxes.

Q49. Explain the saga pattern. Choreography vs orchestration?

A saga splits a business transaction into local transactions, each followed by an event or command; a failure triggers compensating transactions for completed steps. Choreography: services react to each other's events — low coupling, but the flow is implicit and hard to follow beyond a few steps. Orchestration: a central coordinator (a persisted state machine, or a workflow engine like Temporal or Camunda) sends commands and handles replies, timeouts and compensations — explicit and testable, at the cost of a component that knows every step.

Q50. What are compensating transactions and what makes them hard?

Business-level undo: refund a charge, release a reservation, mark an order cancelled. They are not rollbacks — the original change was visible and may have triggered other effects. They must be idempotent and retryable; some actions cannot be compensated (an email sent, cash dispensed), so order steps so that irreversible ones come last, and use semantic locks (status PENDING) to keep other processes from acting on half-finished data.

Q51. Describe the transactional outbox pattern end to end.

In the same local transaction as the business change, insert the event into an outbox table. A relay (poller or CDC) reads unsent events in order, publishes them to the broker, and marks them sent. Consumers deduplicate by event id because delivery is at least once. You get atomicity between state and event without 2PC, durability of events while the broker is down, and ordering per relay.

Q52. Tell me about the outbox in your project.

"Every RefinePoint service publishes through `EventPublisher`, which inserts an envelope into `outbox_events` using the caller's transaction — if the transaction rolls back, there is no event; if Kafka is down, events wait in the table. `OutboxRelay` runs every second on every instance, locks up to 100 oldest unsent rows with `FOR UPDATE SKIP LOCKED` so instances never double-send, sends each and waits for the ack, marks it sent, and on the first failure records the error and stops the batch so a later event in that batch cannot overtake an earlier one. Consumers call `ConsumedEvents.firstTime(consumer, eventId)` — an `INSERT ... ON CONFLICT DO NOTHING` into `consumed_events` — inside the handler's transaction, so duplicates are ignored atomically. A purge removes sent events after a week and consumed ids after a month. Trade-offs we accepted: up to about a second of latency, at-least-once delivery, and no strict global order — with several instances, relays can hold adjacent batches at the same moment, so consumers must not assume one."

Q53. Why must the dedup insert be in the same transaction as the business update?

If you record the event id in one transaction and do the work in another, a crash between them either loses the work (id recorded, work not done — the redelivery is ignored) or duplicates it (work done, id not recorded). In one transaction both happen or neither does, which is what makes at-least-once delivery safe.

Q54. Polling outbox vs Debezium CDC?

Polling: simple, no extra infrastructure, works with any database, adds polling latency and some query load; ordering and locking are your responsibility. CDC: reads the WAL through a logical replication slot, near-real-time, no polling, preserves commit order, but adds Kafka Connect and Debezium to operate, and a stopped connector makes the slot retain WAL until the disk fills. For a small team, polling is the pragmatic choice.

Q55. What is idempotency and how do you design idempotent APIs?

An operation is idempotent if repeating it has the same effect as doing it once. GET, PUT and DELETE are idempotent by HTTP semantics; POST is not. For POST (payments, run creation), accept an `Idempotency-Key` header, store it with a hash of the request and the response under a unique constraint, return the stored response for repeats, and reject the same key with a different body. Internally, prefer conditional updates (`WHERE status = 'RUNNING'`) and unique business keys so retries are harmless.

Q56. How do you handle a webhook (e.g., Paddle) that may be delivered twice or out of order?

Verify the signature, then deduplicate by the provider's event id (unique constraint), and apply state changes only if they move the record forward (compare the event's `occurred_at` or a version with what is stored). Respond 2xx quickly and process asynchronously if processing is slow, since providers retry on timeouts. This is the same idempotent-consumer idea as for Kafka.

Q57. Can Kafka transactions replace the outbox?

No. Kafka transactions make a consume-transform-produce loop atomic within Kafka (offsets and produced messages commit together). They cannot include a PostgreSQL commit. Spring's old `ChainedTransactionManager` (deprecated) committed the DB and Kafka one after the other, which still leaves a window. The outbox makes the database the single source of truth.

Q58. How do you test transactional behaviour?

Integration tests against a real PostgreSQL (Testcontainers), because H2 differs in locking and isolation. Test rollback paths explicitly (throw from a mocked collaborator and assert nothing was written). Be careful with `@Transactional` on test classes: Spring rolls the test transaction back at the end, which hides commit-time behaviour (deferred constraints are never checked, `AFTER_COMMIT` listeners never fire, `REQUIRES_NEW` writes are not rolled back). For concurrency, start two threads with a `CountDownLatch` and assert one gets a lock or version conflict.

Q59. Two `DataSource`s in one service — how do transactions work?

Each needs its own transaction manager; `@Transactional("reportingTx")` picks one. A method annotated for one does not cover statements on the other (those run in auto-commit). There is no atomicity across the two without JTA. Usually the right design avoids needing it: one database per service, or an outbox-style handoff between them.

Q60. "Our nightly import sometimes leaves half the rows and sometimes rolls everything back. Design it properly."

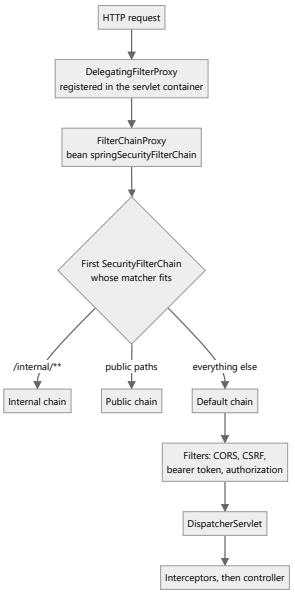
Split the work into chunks with their own short transactions (`TransactionTemplate` per chunk of 500–1000 rows); track progress in an import-run table (status, last committed chunk, counts, errors); make each chunk idempotent (upsert by business key, or delete-and-reinsert by import id) so a restart can resume; collect bad rows into an error table instead of failing the chunk (or use savepoints per row with JDBC); and publish "import completed" through the outbox only when the final status commits. Spring Batch implements this chunk-oriented pattern with a `JobRepository` if you want a framework.

22. Spring Security, OAuth2, JWT & Keycloak

A senior security interview rarely stops at "how do you configure JWT in Spring". It moves to where the filter sits, why a request got 401 rather than 403, what the signature on a token proves and does not prove, how you stop tenant A reading tenant B's data, and how you revoke something designed not to be revocable. This chapter builds from the servlet filter upward and ends with OWASP and production practice.

22.1 Architecture: from servlet container to your controller

Spring Security is a chain of servlet filters. The servlet container knows nothing about Spring beans, so a bridge is needed.



- `DelegatingFilterProxy` is a plain servlet `Filter` that looks up a Spring bean by name and delegates to it. It lets a bean take part in the servlet lifecycle.
- `FilterChainProxy` holds a list of `SecurityFilterChain`s. It picks the **first** one whose `securityMatcher` matches and runs only that chain, then clears the `SecurityContext` when finished. It also applies `HttpFirewall` (rejects `//`, encoded slashes, bad methods).
- `SecurityFilterChain` is an ordered list of filters for one kind of request.

The order of the filters that matter:

Order	Filter	Job
1	`SecurityContextHolderFilter`	Loads the context from the repository (session, or nothing when stateless)
2	`HeaderWriterFilter`	Security headers
3	`CorsFilter`	Answers preflight, adds CORS headers (must come before authentication, preflights carry no credentials)
4	`CsrfFilter`	Checks CSRF token on unsafe methods
5	`LogoutFilter`	Handles logout URL
6	`UsernamePasswordAuthenticationFilter`, `BearerTokenAuthenticationFilter`	Authentication mechanisms
7	`AnonymousAuthenticationFilter`	Puts an anonymous token in the context if nobody authenticated
8	`ExceptionTranslationFilter`	Turns security exceptions into 401 (`AuthenticationEntryPoint`) or 403 (`AccessDeniedHandler`)
9	`AuthorizationFilter`	Final URL rule check (`authorizeHttpRequests`)

401 vs 403: `AuthenticationException` or an access denial for an anonymous user produces 401, meaning "I don't know who you are". `AccessDeniedException` for an authenticated user produces 403, meaning "I know you, and you may not". Exceptions thrown from a controller or `@PreAuthorize` must pass back up through `ExceptionTranslationFilter`; a `@ControllerAdvice` that catches `Exception` will swallow them and turn 403 into 500.

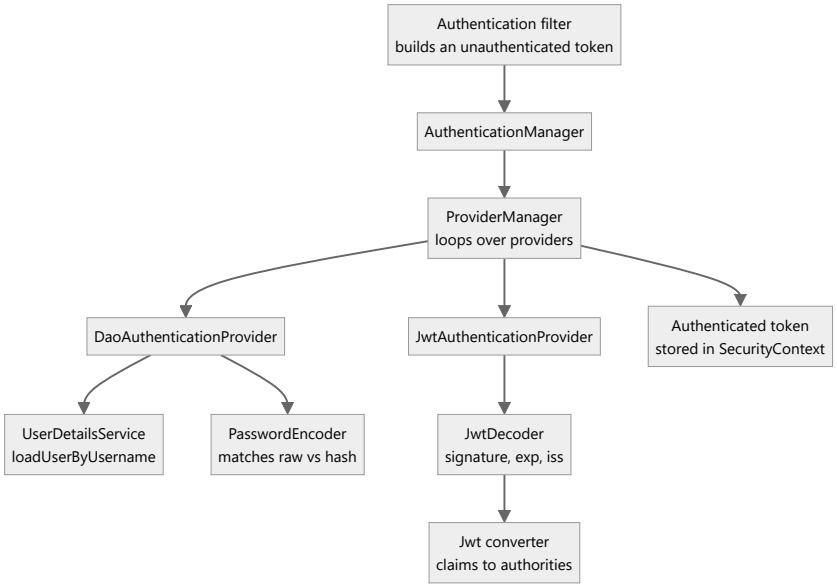
SecurityContextHolder

`SecurityContextHolder` holds a `SecurityContext` that holds an `Authentication` (principal, credentials, authorities). The storage strategy decides where:

Strategy	Behaviour
`MODE_THREADLOCAL` (default)	One context per thread
`MODE_INHERITABLETHREADLOCAL`	Child threads copy it at creation; unsafe with pools because a pooled thread keeps its old context
`MODE_GLOBAL`	One context for the JVM; desktop apps only

Because the default is a thread local, `@Async` methods and executors lose the user. Wrap the executor in `DelegatingSecurityContextAsyncTaskExecutor`, or use a `TaskDecorator` that copies and clears the context. WebFlux keeps it in the Reactor context (`ReactiveSecurityContextHolder`).

22.2 Authentication flow



- `AuthenticationManager` has one method: `authenticate(Authentication)`.
- `ProviderManager` is the usual implementation. It asks each `AuthenticationProvider` whose `supports(tokenClass)` is true; the first success wins. If none succeeds, it tries its parent manager, then throws. It erases credentials from the result.
- `DaoAuthenticationProvider` loads a `UserDetails` and compares the password through `PasswordEncoder`. To resist username enumeration by timing, it hashes the presented password even when the user does not exist.

Password storage

Never store plain, encrypted or fast-hashed (MD5, SHA-256) passwords. Use a slow, salted, adaptive function.

```
@Bean PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder(); // stores {bcrypt}$2a$10$...
}
```

A bcrypt string `$2a$10$N9qo8uLOickgx2ZMRZoMye...` contains the version, the cost (10 means 2^10 rounds) and a 22-character salt, followed by the hash. So no separate salt column is needed. Bcrypt only reads the first 72 bytes of input. OWASP currently prefers **Argon2id** (memory-hard, resists GPUs) with bcrypt acceptable for existing systems. `DelegatingPasswordEncoder` stores the algorithm id in `{...}`, so you can upgrade: verify with the old one, re-hash on next login (`UserDetailsPasswordService`). Tune the cost so one hash takes about 250 ms on your hardware.

> **RefinePoint:** we store no passwords at all. Keycloak owns credentials, password policy and MFA; our services only see tokens. Reducing what you must protect is the strongest design choice available.

22.3 Authorization

Two layers: URL rules at the filter level and method rules at the proxy level.

```
@Configuration
@EnableMethodSecurity
class SecurityConfig {
    @Bean
    SecurityFilterChain api(HttpSecurity http, KeycloakJwtConverter converter) throws Exception {
        http
            .cors(Customizer.withDefaults())
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health").permitAll()
                .requestMatchers(HttpMethod.POST, "/api/v1/admin/**").hasRole("admin")
```

```
                .anyRequest().authenticated())
            .oauth2ResourceServer(o -> o.jwt(j -> j.jwtAuthenticationConverter(converter)));
        return http.build();
    }
}
```

Boot 3 and Security 6 changes interviewers ask about: `WebSecurityConfigurerAdapter` is gone (declare `SecurityFilterChain` beans); `antMatchers` became `requestMatchers`; `authorizeRequests` became `authorizeHttpRequests` (uses `AuthorizationManager`); `@EnableGlobalMethodSecurity` became `@EnableMethodSecurity` with `prePostEnabled` on by default; lambda DSL replaces `.and()` chains; the security context is no longer saved automatically (`SecurityContextHolderFilter`).

`hasRole` vs `hasAuthority`: `hasRole('admin')` looks for the authority `ROLE_admin`; `hasAuthority('ROLE_admin')` is the exact string. A role is simply an authority with the prefix.

Method security

Annotation	When	Example
`@PreAuthorize`	Before the method, SpEL	`@PreAuthorize("hasRole('admin') or #userId == authentication.name")`
`@PostAuthorize`	After, can use `returnObject`	`@PostAuthorize("returnObject.ownerId == authentication.name")`
`@PreFilter` / `@PostFilter`	Filter collections	`@PostFilter("filterObject.public")` — loads then filters; use a query instead
`@Secured`, `@RolesAllowed`	Simple role lists	Older or JSR-250

Method security is AOP, so all the proxy rules apply: self-invocation skips the check. SpEL can call any bean with `@beanName.method(...)`, and that is how real permission logic is written. Annotations can be composed into meta-annotations.

```
@PreAuthorize("@orgAuth.isAdmin()")
@Retention(RUNTIME) @Target(METHOD)
public @interface CanManageOrganization {}

@PreAuthorize("@orgAuth.can('breaks.resolve')")
public void resolve(UUID breakId) { ... }
```

For object-level rules, the classic extension point is `PermissionEvaluator` with `hasPermission(#id, 'Break', 'resolve')`. A named bean like `@orgAuth` does the same job with less ceremony.

22.4 CSRF, CORS, sessions

CSRF exploits the browser attaching **cookies** automatically: a malicious page submits a form to your site and the browser adds the session cookie. Defence: a token the attacker's page cannot read (synchroniser token or double-submit cookie), and `SameSite=Lax/Strict` cookies. **Disable it only when the API authenticates with something browsers do not attach automatically**, such as `Authorization: Bearer` or `X-API-Key`. Keep it on for session cookies, and also if you put a JWT in a cookie, since the cookie brings the same problem back.

CORS relaxes the browser's same-origin policy. The browser sends a preflight `OPTIONS` for non-simple requests, and the server answers with allowed origin, methods and headers. CORS protects **users' browsers**, not your server: `curl` ignores it entirely, so it is not access control. `allowedOrigins("*")` with `allowCredentials(true)` is rejected by Spring and browsers. Configure it in Spring Security through `.cors(...)` so preflights pass before authentication.

Sessions vs stateless:

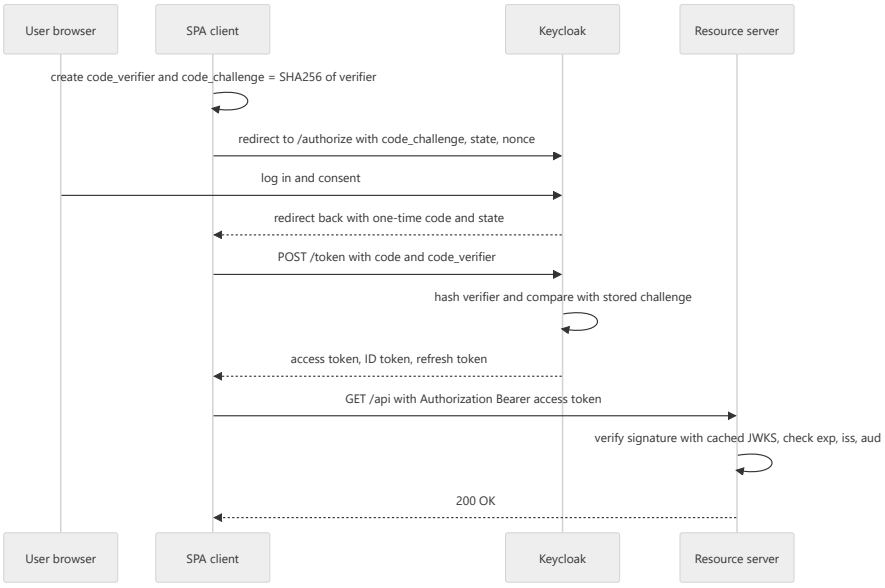
	Server session (`JSESSIONID`)	Stateless token (JWT)
State	On server (or Redis)	In the token
Revocation	Instant: delete the session	Hard: valid until expiry
Scale	Needs sticky sessions or shared store	Any instance can validate
Browser attack surface	CSRF	XSS if stored in JS-readable storage
Size	Small cookie	1–2 KB on every request

22.5 OAuth2 and OpenID Connect

OAuth2 is **delegated authorization**: letting a client act on a resource with limited rights, without giving it the user's password. Roles: **resource owner** (the user), **client** (the app), **authorization server** (Keycloak), **resource server** (our API).

Authorization code + PKCE

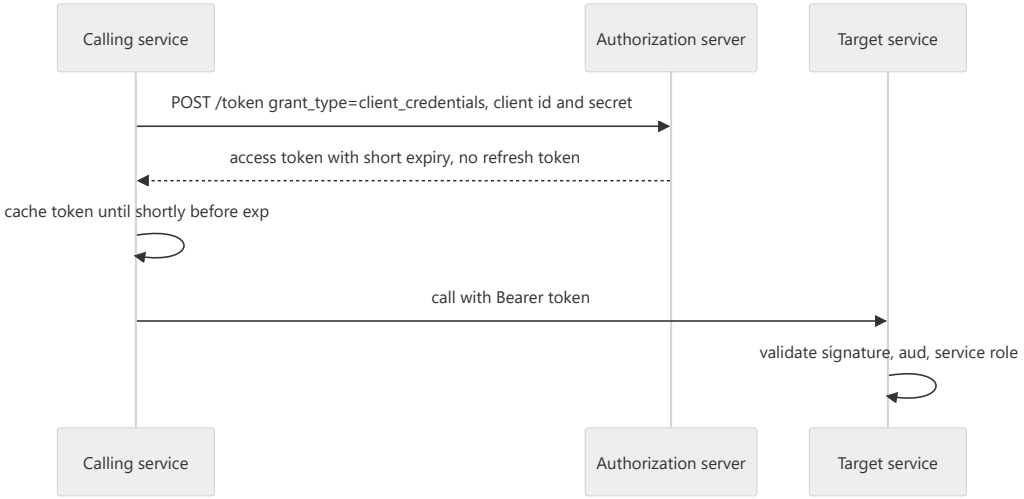
The default for every user-facing client, including SPAs and mobile apps.



PKCE defeats code interception: a stolen code is useless without the verifier, which never left the client. `state` stops login CSRF. `nonce` ties the ID token to this login and stops replay. OAuth 2.1 requires PKCE for all authorization-code clients.

Client credentials

No user. A machine authenticates as itself.



Other grants

Grant	Status	Note
Authorization code + PKCE	Use it	Users on any client type
Client credentials	Use it	Service to service
Device code	Use it	TVs, CLIs without a browser
Refresh token	Use it	Rotation: each use issues a new one and invalidates the old; reuse detection kills the family
Implicit	**Deprecated**	Tokens in URL fragments leak through history and referrers
Password (ROPC)	**Deprecated**	The client sees the password; no MFA

OIDC

OpenID Connect is an **authentication** layer on OAuth2. With `scope=openid` the token response also has an **ID token**: a JWT for the *client* saying who logged in (`sub`, `name`, `email`, `nonce`, `aud` = the client). The **access token** is for the *resource server*. Never send an ID token to an API as a credential, and never read user identity in the client from an access token. OIDC also standardises discovery (`/.well-known/openid-configuration`), the UserInfo endpoint, and JWKS.

22.6 JWT

```
eyJhbGciOiJSUzI1NiIsImtpZCI6ImFiYyJ9 . eyJzdWIiOiI0MiIsImV4cCI6MTcwMDAwMDAwMH0 . MEUCIQDf...
└──────────── header ──────────────┘   └──────────────── payload ────────────────┘   └ signature ┘
{"alg":"RS256","kid":"abc"}           {"sub":"42","iss":"https://sso/realms/rp",  sign(base64url(header)
                                       "aud":"account","exp":1700000000,                + "." +
                                       "realm_access":{"roles":["admin"]}}              base64url(payload))
```

It is **base64url-encoded, not encrypted**: anyone can read it. Signing gives integrity and authenticity, not secrecy. Never put secrets or personal data you would not show the user in it. JWE is the encrypted variant, rarely needed.

	HS256	RS256 / ES256
Key	One shared secret	Private key signs, public key verifies
Who can forge	Every service that can verify	Only the authorization server
Distribution	Secret copied everywhere	Public keys over JWKS
Use	One service issuing to itself	Anything with several services

JWKS (`/protocol/openid-connect/certs` in Keycloak) publishes public keys, each with a `kid`. The decoder reads `kid` from the header, finds the key in its cache, and refetches on an unknown `kid`. That is how rotation works without downtime: publish a new key, start signing with it, keep the old one until its tokens expire.

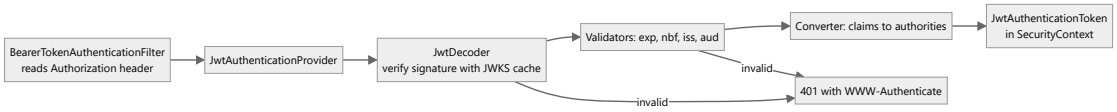
Attacks to name: `alg: none` (accept unsigned tokens); RS256-to-HS256 confusion (attacker signs with the public key as HMAC secret); weak HMAC secrets brute-forced offline; missing `aud` check, so a token for another API is accepted; `kid` or `jku` used to load attacker-controlled keys. Defend by pinning the expected algorithm and validating `iss`, `aud`, `exp`. Spring's `NimbusJwtDecoder` is keyed to an algorithm and validates `exp` / `nbf` with 60 s clock skew and `iss` when configured through `issuer-uri`; audience must be added.

The revocation problem: a signed JWT is valid until `exp`, because no one consults a server. Options, in order of use:

1. Short access token (5–15 minutes) + refresh token rotation: disabling the user stops refresh, so the exposure is bounded by the access token's life.
2. Deny-list of `jti`s in Redis, checked per request, for emergencies. This reintroduces a lookup, but only for a small set.
3. Token introspection (RFC 7662) or opaque tokens: a server call per request gives instant revocation at a latency and availability cost.
4. Per-user "tokens valid after" timestamp compared with `iat`.

Resource server configuration

```yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.example.com/realms/refinepoint   # discovery, iss validation
          jwk-set-uri: https://auth.example.com/realms/refinepoint/protocol/openid-connect/certs
          audiences: refinepoint
```



22.7 Keycloak

Concept	Meaning
Realm	An isolated space of users, clients, roles and keys
Client	An application. *Public* (SPA, no secret, PKCE) or *confidential* (has a secret; can have a service account for client credentials)
Realm role / client role	Global role vs a role scoped to one client; composite roles group others
Group	Set of users that inherits roles and attributes
Protocol mapper	Decides what goes into tokens: audience, attributes, role claims
Identity brokering	Log in through Google, Azure AD, or a corporate SAML/OIDC provider

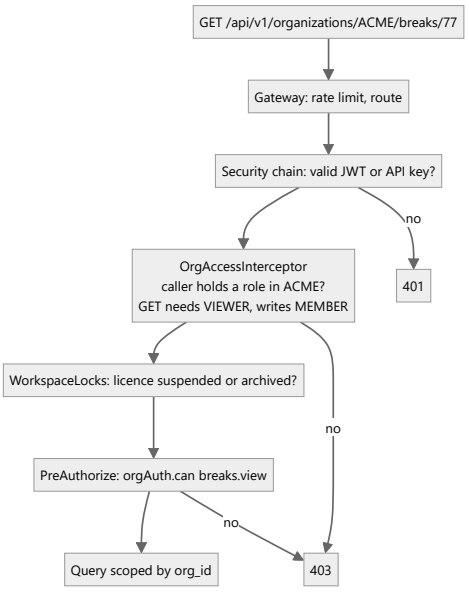
Roles appear at `realm_access.roles` and `resource_access.<client>.roles`. Spring knows neither by default: it maps `scope` to `SCOPE_*` authorities only. So you write a converter.

```java
@Component
public class KeycloakJwtConverter implements Converter<Jwt, AbstractAuthenticationToken> {
    @Override public AbstractAuthenticationToken convert(Jwt jwt) {
        return new JwtAuthenticationToken(jwt, extractRealmRoles(jwt));   // realm_access.roles → ROLE_x
    }
}
```

The old Keycloak Spring adapters are deprecated and removed. Use Spring's standard resource server support and keep services independent of Keycloak's libraries; to them it is just an OIDC provider.

22.8 Multi-tenant authorization

Being logged in says nothing about which workspace you may touch. **Broken access control across tenants is the top risk of any SaaS.**



> **RefinePoint:** the org code is in the URL (or `X-Org-Code` on older endpoints). Membership is enforced in **one place**, `OrgAccessInterceptor`, so no controller can forget it: reads need any role, writes need at least MEMBER. `OrgRole` is ordered `VIEWER, MEMBER, ADMIN, OWNER` with `atLeast(...)`. Finer rules use `@PreAuthorize("@orgAuth.can('breaks.resolve')")` against the `Permissions` catalogue, where workspaces can define custom roles and scope process permissions to a group. Realm roles cannot express this, because workspace owners hold no realm role at all. The access decision sits behind a `WorkspaceAccess` port: identity-service implements it over its tables, every other service uses `RemoteWorkspaceAccess`, which asks identity-service and caches the answer briefly.

IDOR (Insecure Direct Object Reference): `GET /organizations/ACME/breaks/77`, where break 77 belongs to another tenant. The interceptor passes because the caller really is in ACME. The fix is that **every query filters by tenant**: `findByIdAndOrgId(id, orgId)`, never `findById(id)`. `BaseOrgEntity` gives every tenant table an `org_id`. Additional depth: unguessable UUIDs (obscurity only, not a control), PostgreSQL Row-Level Security or a Hibernate `@Filter` / `@TenantId` as a safety net. Return 404, not 403, so a caller cannot probe which ids exist.

22.9 API keys, service-to-service auth, webhooks

API keys for machines. Random, high-entropy secret shown once, stored hashed.

> **RefinePoint:** `ApiKeyAuthenticationFilter` (before `BearerTokenAuthenticationFilter`) reads `X-API-Key`. Sending both a key and a bearer token gets a 400. `ApiKeyPaths` allows workspace endpoints only and forbids `/members`, `/api-keys`, `/billing`, `/export` and `/team`, so a leaked key cannot grant itself more access. A valid key becomes a synthetic `JwtAuthenticationToken` for the creator, limited to the key's workspace and role with **no realm roles**, so everything downstream works as for a person. Keys are stored as `key_hash` (SHA-256) plus a `key_prefix` such as `rpk_...` for recognising them in lists, with `expires_at`, `last_used_at` and `revoked_at`. A key's role can never exceed its creator's.

Why SHA-256 rather than bcrypt here: passwords are low-entropy, so hashes must be slow to resist guessing. A random 256-bit key cannot be guessed, and it is verified on every request, so a slow hash would only hurt latency.

Service-to-service options:

Option	How	Trade-off
Shared internal token	Header compared in constant time	Simple; one secret, no per-service identity
Client credentials	Each service gets a JWT from the authorization server	Per-service identity and scopes; token management
Forwarding the user token	Pass the bearer on	Keeps user identity; audience gets broad
mTLS	Both sides present certificates (often through a service mesh)	Strong identity at transport level; certificate operations

> **RefinePoint:** `/internal/**` is never routed by the gateway, and `InternalTokenFilter` requires `X-Internal-Token`, compared with `MessageDigest.isEqual`. `InternalClient` adds it. This is defence in depth on a private network. Client credentials or mTLS would be the upgrade if services stopped sharing a trusted host network.

Webhook signatures. A webhook URL is public, so anyone can POST to it. Verify an HMAC over the **raw** body before parsing.

```
// Paddle-Signature: ts=1700000000;h1=<hex> — several h1 values while a secret rotates
if (Math.abs(Instant.now().getEpochSecond() - signedAt) > 300) throw new SecurityException("replay");
String expected = hmacSha256(ts + ":" + payload);
if (signatures.stream().noneMatch(h -> constantTimeEquals(expected, h))) throw new SecurityException("bad signature");
```

The timestamp inside the signed data stops replays; constant-time comparison stops timing attacks; accepting multiple `h1` values allows secret rotation with no downtime.

22.10 OWASP Top 10 for Java developers

OWASP 2021	Java example	Defence
A01 Broken access control	IDOR, missing tenant filter, `@PreAuthorize` skipped by self-invocation	Deny by default, central checks, scoped queries
A02 Cryptographic failures	MD5 passwords, secrets in logs, HTTP	bcrypt/Argon2, TLS, AES-GCM
A03 Injection	String-concatenated SQL/JPQL, log injection, Log4Shell	Bind parameters, whitelist identifiers
A04 Insecure design	No rate limit on OTP, no tenant model	Threat modelling
A05 Misconfiguration	Actuator exposed, default passwords, verbose errors	Expose `health` only, generic errors
A06 Vulnerable components	Old Jackson, Log4j, Spring	SBOM, Dependabot, `dependency-check`
A07 Auth failures	Weak sessions, no MFA, no lockout	OIDC provider, MFA
A08 Integrity failures	Native deserialization, unsigned updates	Avoid `ObjectInputStream`, sign artefacts
A09 Logging failures	No audit of denials, tokens in logs	Audit trail, masking
A10 SSRF	Server fetches user URL → `169.254.169.254` cloud metadata	Allow-lists, block internal ranges

(The 2025 edition reorders these, folds SSRF into access control and adds supply-chain failures; the 2021 numbering is what most interviewers still quote.)

SQL injection. Parameter binding sends the value separately from the statement, so it is never parsed as SQL. Identifiers (`ORDER BY` columns) cannot be bound; whitelist them.

```
jdbc.queryForList("SELECT * FROM breaks WHERE org_id = ? AND status = ?", orgId, status); // safe
"... ORDER BY " + sortParam                                                               // injectable
```

JPQL string concatenation is equally injectable. Rule languages deserve the same care: compiling user rules to SQL must bind values and validate identifiers, as our `SqlCompiler` must for DuckDB.

XSS. Untrusted data rendered as HTML runs in the user's session. React escapes by default, except `dangerouslySetInnerHTML`. Add a strict CSP. For JSON APIs, return the correct `Content-Type` plus `X-Content-Type-Options: nosniff`.

SSRF.

> **RefinePoint:** users configure webhook URLs and SFTP or database hosts, which the server then contacts. `OutboundUrlValidator.requireSafe` demands HTTPS, refuses embedded credentials, resolves the host and rejects loopback, private, link-local (the cloud metadata range), multicast, carrier-grade NAT and reserved addresses. The class documents its residual risk, DNS rebinding between check and send: the sender disables redirects, and egress filtering on the host is the complete fix.

Deserialization. `ObjectInputStream.readObject` instantiates classes before you can check them, so gadget chains on the classpath give remote code execution. Avoid native serialization; use JSON. If forced, `ObjectInputFilter` (JEP 290) allow-lists classes. In Jackson never use `enableDefaultTyping` or `@JsonTypeInfo(use = CLASS)` on untrusted input; use `@JsonSubTypes` with explicit names. Disable DTDs in XML parsers to stop XXE.

Secrets management. Never in Git or images. Inject through environment or a secret store (Vault, AWS Secrets Manager, Kubernetes Secrets with encryption at rest), and rotate. Run gitleaks in CI. Treat a committed secret as compromised: rotate first, then clean history.

Rate limiting and brute force. Limit by IP and by identity. Add login lockout or progressive delay, and enable Keycloak's brute-force detection.

> **RefinePoint:** the gateway's `RateLimitFilter` keeps token buckets in a `ConcurrentHashMap`, per IP and per caller; a bucket holds a minute's requests and refills evenly; past it comes 429 with `Retry-After`. Sensitive endpoints add a `SlidingWindowLimiter`, such as SSO domain lookups at 30 per minute. Both count per instance, which is a safe floor, not an exact global limit. Health checks, preflights and Paddle webhooks are not counted. Address buckets use the connection's remote address, and `AuditTrail` reads only the `X-Real-IP` set by nginx; neither reads `X-Forwarded-For`, which a client can forge.

Security headers. Spring adds by default: `X-Content-Type-Options: nosniff`, `Cache-Control: no-store`, `X-Frame-Options: DENY`, `Strict-Transport-Security` (HTTPS only). Add yourself: `Content-Security-Policy`, `Referrer-Policy`, `Permissions-Policy`.

22.11 Testing security

```
@WebMvcTest(BreakController.class)
@Import(SecurityConfig.class)
class BreakControllerTest {
    @Test @WithMockUser(roles = "admin")
```

```
    void adminAllowed() throws Exception { mvc.perform(post("/x").with(csrf())).andExpect(status().isOk()); }

    @Test
    void jwtWithRole() throws Exception {
        mvc.perform(get("/x").with(jwt().authorities(new SimpleGrantedAuthority("ROLE_viewer"))))
           .andExpect(status().isForbidden());
    }

    @Test void anonymousRejected() throws Exception { mvc.perform(get("/x")).andExpect(status().isUnauthorized()); }
}
```

Test the **negative** cases: anonymous, wrong role, wrong tenant. Every tenant endpoint deserves a test that user A cannot read tenant B's id. `@WithSecurityContext` builds custom principals. Use `SecurityMockMvcRequestPostProcessors` for `jwt()` and `csrf()`.

22.12 Interview questions and answers

Q1. How does a request flow through Spring Security?

The container calls `DelegatingFilterProxy`, which delegates to the `FilterChainProxy` bean, which picks the first matching `SecurityFilterChain` and runs its filters: context load, headers, CORS, CSRF, authentication, anonymous, exception translation, authorization. Then the request reaches `DispatcherServlet`, interceptors and the controller.

Q2. Why is `DelegatingFilterProxy` needed?

The servlet container creates filters itself and knows nothing about Spring. The proxy is the container-managed filter; it looks up a Spring bean by name, so the real filter can use dependency injection and the application context lifecycle.

Q3. Can you have multiple `SecurityFilterChain`s?

Yes, and `FilterChainProxy` uses only the **first** match, so order them with `@Order`, specific matchers first. A chain without a `securityMatcher` matches everything and must be last. RefinePoint has three: public paths (permit all), `/internal/**` (internal token), and the default (authenticated).

Q4. What is the difference between 401 and 403?

401: not authenticated, or authentication failed; client should authenticate. 403: authenticated but not permitted. `ExceptionTranslationFilter` picks: `AuthenticationEntryPoint` gives 401, `AccessDeniedHandler` gives 403. An access denial for an anonymous user is treated as 401.

Q5. My `@PreAuthorize` failure returns 500. Why?

A `@ControllerAdvice` with `@ExceptionHandler(Exception.class)` caught `AccessDeniedException` before it reached `ExceptionTranslationFilter`. Rethrow it or add a specific handler returning 403.

Q6. How is `SecurityContextHolder` thread-safe?

By default each thread has its own context through a `ThreadLocal`, and `FilterChainProxy` clears it at the end of the request, so a pooled thread does not leak it to the next one.

Q7. How do you propagate the security context to `@Async` methods?

Wrap the executor with `DelegatingSecurityContextAsyncTaskExecutor`, or use a `TaskDecorator` that captures, sets and clears the context. Avoid `MODE_INHERITABLETHREADLOCAL` with pools: a thread inherits once at creation and keeps stale identity afterwards. New virtual threads also start without it.

Q8. Explain `AuthenticationManager`, `ProviderManager` and `AuthenticationProvider`.

`AuthenticationManager` is the interface. `ProviderManager` implements it by delegating to a list of `AuthenticationProvider`s, choosing by `supports()`, with an optional parent. Each provider handles one mechanism: password, JWT, and so on.

Q9. How do you add a custom authentication mechanism?

Write a filter (`OncePerRequestFilter`) that extracts the credential and either calls `AuthenticationManager` with a custom token and provider, or validates directly and sets the context. Insert with `addFilterBefore(filter, BearerTokenAuthenticationFilter.class)`. Do not also register it as a plain `@Component` without disabling servlet registration, or it runs twice. `ApiKeyAuthenticationFilter` is ours.

Q10. What is `OncePerRequestFilter`?

A base class that guarantees a filter runs once per request, even when the request goes through forwards or error dispatches, by marking it with a request attribute. Use it for authentication filters.

Q11. Why bcrypt and not SHA-256 for passwords?

SHA-256 is fast by design, so a GPU tries billions per second. Bcrypt is deliberately slow with a tunable cost and a built-in salt. Salting stops rainbow tables and makes equal passwords hash differently. Argon2id adds memory hardness.

Q12. How do you migrate from MD5 to bcrypt without forcing resets?

Wrap: store `bcrypt(md5(password))` for all rows now, marked as legacy. On next login verify the wrapped form, then re-hash the plain password with bcrypt. `DelegatingPasswordEncoder` with `{id}` prefixes makes the mix explicit.

Q13. `hasRole` vs `hasAuthority`?

`hasRole('X')` checks for authority `ROLE_X`; `hasAuthority('X')` checks the exact string. Mixing them up is the commonest cause of mysterious 403s. Use authorities for fine permissions (`SCOPE_read`), roles for coarse groups.

Q14. URL security vs method security?

URL rules are coarse, sit at the edge and also cover non-controller paths. Method rules are close to the business action and can inspect arguments. Use both: authenticated by default at the URL level, specifics on methods. Method security alone leaves gaps; URL rules alone miss object-level rules.

Q15. Why does `@PreAuthorize` not work on an internal call?

It is proxy-based. `this.method()` bypasses the proxy, so the check never runs and no error appears. Also it needs `@EnableMethodSecurity`, and the class must be a Spring bean.

Q16. What changed from Spring Security 5 to 6?

`WebSecurityConfigurerAdapter` removed; `SecurityFilterChain` beans. `requestMatchers` replaced `antMatchers`. `authorizeHttpRequests` with `AuthorizationManager`. `@EnableMethodSecurity`. Security context save is explicit. `jakarta.*` namespace. CSRF token handling is deferred and BREACH-protected by default. All dispatches are authorized, including `FORWARD` and `ERROR`.

Q17. When is it safe to disable CSRF?

When authentication comes from a credential the browser does not send automatically, such as a bearer header or API key. If a token lives in a cookie, or you use sessions, keep CSRF on. Non-browser-only APIs do not need it.

Q18. CORS vs CSRF?

CORS loosens the same-origin policy so that approved origins can read responses; it is enforced by the browser. CSRF is an attack in which another site triggers an authenticated request. CORS does not stop CSRF: simple cross-site form POSTs are sent without any preflight.

Q19. Why is CORS configured in Spring Security rather than only in MVC?

Preflight `OPTIONS` requests carry no credentials. If security filters run first they respond 401 and the browser reports a CORS error. `.cors(...)` installs `CorsFilter` ahead of authentication.

Q20. Explain OAuth2 roles.

Resource owner: the user who owns the data. Client: the application acting for them. Authorization server: authenticates and issues tokens. Resource server: API that accepts tokens. In RefinePoint: finance user, React app, Keycloak, our services.

Q21. OAuth2 vs OIDC vs SAML?

OAuth2 is delegated authorization, with no standard way to say who the user is. OIDC adds authentication on top with an ID token (JSON/JWT, modern apps). SAML is an older XML-based SSO standard, still common with enterprise identity providers. Keycloak can broker SAML into OIDC for our apps.

Q22. Walk through authorization code + PKCE.

The client makes a random verifier and sends its SHA-256 as a challenge in the authorize redirect. The user logs in; the authorization server returns a one-time code. The client posts the code and the verifier to the token endpoint. The server hashes the verifier, compares it, and issues tokens. Stealing the code alone gets nothing.

Q23. Why are implicit and password grants deprecated?

Implicit returns tokens in the URL, exposing them to history, logs and referrers, with no client authentication and no refresh. Password gives the client the user's password and rules out MFA and federation. Both are removed in OAuth 2.1; code + PKCE replaces both.

Q24. How should a SPA store tokens?

No storage is perfect. `localStorage` is readable by any XSS. Memory is safer but lost on reload (use silent renew). The strongest option is a **Backend-for-Frontend**: the server does OAuth and holds tokens, and the browser only has an `HttpOnly`, `Secure`, `SameSite` session cookie, which brings CSRF protection back into play.

Q25. Access token vs refresh token vs ID token?

Access: short-lived, for the API. Refresh: long-lived, sent only to the authorization server to get new tokens, stored securely, rotated. ID: for the client, describes the login, never sent to APIs.

Q26. What is refresh token rotation?

Each refresh returns a new refresh token and invalidates the old one. If an old one is presented again, it means theft, so the server revokes the entire family. This limits the damage of a stolen refresh token.

Q27. What is inside a JWT and is it encrypted?

Header (`alg`, `kid`), payload (claims) and signature, base64url-encoded and dot-separated. It is signed, not encrypted: anyone can decode it, but nobody can change it without breaking the signature.

Q28. RS256 vs HS256?

HS256 uses one shared secret, so every verifier can also forge. RS256 signs with a private key and verifies with a public key, so services can verify but never mint. Across microservices use asymmetric keys; ES256 gives smaller signatures.

Q29. How does a resource server verify a token without calling Keycloak on every request?

It fetches public keys once from JWKS and caches them. Verification is a local signature check plus claim checks (`exp`, `iss`, `aud`). It refetches when it sees an unknown `kid`, which is how key rotation works. If Keycloak is down, valid tokens still work as long as keys are cached.

Q30. How do you revoke a JWT?

You can't directly; design around it. Use short expiry with refresh rotation, a `jti` deny-list in Redis for urgent cases, a per-user valid-after timestamp, or introspection/opaque tokens where instant revocation matters more than latency. State the trade-off: revocation means bringing state back.

Q31. What claims must you validate?

Signature with a pinned algorithm, `exp`, `nbf`, `iss`, and `aud`. Spring validates `iss` and timestamps; `aud` needs `spring.security.oauth2.resourceserver.jwt.audiences` or a custom validator. Skipping `aud` allows a token issued for another API in the same realm to be used against yours.

Q32. What is the `alg: none` attack and the key confusion attack?

`none`: attacker removes the signature and sets `alg` to `none`; bad libraries accept it. Confusion: the server expects RS256, but the attacker signs with HS256 using the public key as the secret, and a naive library verifies with it. Fix both by fixing the accepted algorithm on the verifier side and never trusting the header's choice.

Q33. RefinePoint's API key token sets `alg: none`. Isn't that a vulnerability?

No, because that `Jwt` object is built in memory by `ApiKeyAuthenticationFilter` after the key is verified, and is never parsed from a request. The `alg: none` attack applies to a decoder that accepts tokens from outside; ours accepts only signed tokens through `NimbusJwtDecoder`. The synthetic token lets downstream code treat keys and users identically. It is a good example of understanding *where* a rule applies.

Q34. How does Keycloak map to Spring authorities?

Spring's default converter reads only `scope` / `scp` into `SCOPE_*`. Keycloak puts roles in `realm_access.roles`, so we supply a `Converter<Jwt, AbstractAuthenticationToken>`. Ours maps each realm role to `ROLE_<name>` and returns a `JwtAuthenticationToken`.

Q35. Realm roles or client roles?

Realm roles are global: one per platform concept. Client roles are per application, useful when several apps have separate permission sets. For a SaaS tenant model, neither is enough: tenant roles live in the application's own database, because one user can be OWNER in one workspace and VIEWER in another.

Q36. Why not put tenant roles in the JWT?

The token grows with memberships, role changes lag until the token expires, and the mapping couples Keycloak to business data. Looking up membership per request (cached briefly) gives near-immediate effect. We do that through `WorkspaceAccess`. The trade-off is a short staleness window from the cache.

Q37. How do you prevent IDOR in a multi-tenant system?

Check membership centrally, then scope **every** query with the tenant id from the verified context, never with an id from the client alone. Add safety nets (`@TenantId`, Row-Level Security), tests that try cross-tenant access, and 404 responses. UUIDs help against enumeration but are not authorization.

Q38. Why use an interceptor rather than a filter or `@PreAuthorize` for org access?

An interceptor runs after authentication, knows the mapped route, and applies automatically to every org-scoped path. A filter is earlier and coarser. Per-method annotations can be forgotten. A central default check plus stricter annotations is "secure by default"; forgetting an annotation then fails closed.

Q39. Can the client just change `X-Org-Code` to access another tenant?

Changing it is harmless if the server treats it as a claim to verify: `OrgAuth` and `WorkspaceAccess` check the caller's role in the named workspace. A header is input, not proof. The vulnerability appears only if code trusts it without checking membership.

Q40. How do you store API keys?

Generate with `SecureRandom`, show once, store a SHA-256 hash plus a short prefix for display, with `expires_at`, `revoked_at` and `last_used_at`. Scope them narrowly. Compare by hash lookup. Because keys are high-entropy, a fast hash is correct, and bcrypt would add latency per request for no security gain.

Q41. API key vs OAuth client credentials?

API keys are simple and long-lived, with no expiry by default and no standard scopes; they suit customers scripting against you. Client credentials give short-lived tokens, central revocation and scopes, but require the caller to implement a token flow. Many SaaS products offer keys first for adoption.

Q42. Compare options for service-to-service authentication.

Shared token: simplest; a leak compromises everyone. Client credentials: per-service identity with audit, needs an authorization server. Forwarded user token: preserves the user, but with confused-deputy risk if audiences are broad. mTLS: strong transport identity, usually with a mesh such as Istio. Mature setups combine mTLS for "which service" with a JWT for "which user".

Q43. Why compare secrets in constant time?

`String.equals` exits at the first different character, so response timing leaks how many leading characters were right; attackers can recover a secret byte by byte. `MessageDigest.isEqual` looks at all bytes.

Q44. How do you verify a webhook, and what are the common bugs?

Compute HMAC-SHA256 over the raw body with the shared secret and compare in constant time; check a signed timestamp for replays; dedupe by event id. Common bugs: hashing re-serialized JSON rather than raw bytes, `==` comparison, no replay window, no secret rotation support.

Q45. What is SSRF and how do you prevent it?

The attacker makes your server send requests to internal targets (`http://169.254.169.254/...`, localhost admin ports). Prevent by allowing only HTTPS, resolving and rejecting internal ranges, disabling redirects, and filtering egress at the network level. Resolve-then-connect leaves a DNS-rebinding window, so pin the resolved IP or rely on egress rules.

Q46. How does Java deserialization become RCE?

`readObject` runs code in classes on the classpath during reconstruction. Chains of existing "gadget" classes end up invoking something dangerous, before your cast ever happens. Avoid native serialization, set `ObjectInputFilter`, and avoid Jackson polymorphic default typing. Log4Shell was a related lesson: JNDI lookup from a log message loaded remote code.

Q47. How do you prevent SQL injection when the sort column comes from the client?

Bind parameters cannot stand for identifiers. Map the requested name through a whitelist (`Map.of("date", "created_at")`) and reject others. Spring Data's `Sort` needs the same care for native queries; `JpaSort.unsafe` is called unsafe for a reason.

Q48. Where do you keep secrets in a Spring Boot deployment?

Environment variables or mounted files from a secret manager (Vault, AWS Secrets Manager, Kubernetes Secrets). In code, reference `${KEYCLOAK_ADMIN_CLIENT_SECRET:}` with no real default. Do not log the environment; restrict actuator `env`. Plan rotation: for example Paddle signature checks accept several `h1` s during a change.

Q49. How do you protect login from brute force and credential stuffing?

Per-account and per-IP rate limits, progressive delay rather than permanent lockout (a lockout lets an attacker lock victims out), MFA, breached-password checks, identical errors for unknown user and wrong password, CAPTCHA after failures. With Keycloak, enable brute-force detection. At the gateway, per-IP limits cover

the rest.

Q50. Which client IP do you use for rate limiting?

One your own infrastructure establishes, never one the client can set. `X-Forwarded-For` is client-controlled unless every hop rewrites it, and trusting it gives an attacker a fresh rate-limit bucket per request. Our gateway buckets on the connection's remote address, and the audit trail reads `X-Real-IP` set by nginx. The catch: behind a proxy the raw remote address is the proxy's, so forwarded-header handling must trust only that proxy.

Q51. Why are in-memory rate limits acceptable, and when are they not?

Each instance counts alone, so with N instances the effective limit is up to N times higher. As abuse protection that is a fine floor and needs no extra infrastructure. For contractual quotas or billing use a shared store, such as Redis with an atomic Lua script. Our gateway's buckets are documented as per gateway, and plan limits are enforced separately (`PlanLimits`).

Q52. Which security headers matter and what do they do?

HSTS forces HTTPS. CSP restricts script sources, the main XSS mitigation. `X-Content-Type-Options: nosniff` stops MIME sniffing. `X-Frame-Options` / `frame-ancestors` stop clickjacking. `Referrer-Policy` limits URL leakage. `Cache-Control: no-store` keeps sensitive responses out of shared caches. An embedded app such as a marketplace iframe needs `frame-ancestors` opened for that host only.

Q53. How do you test security rules?

`@WebMvcTest` with the security config imported, `@WithMockUser` , and `.with(jwt())` / `.with(csrf())` post-processors. Prove the negatives: anonymous gets 401, wrong role 403, wrong tenant 404. Add a route inventory test asserting that every mapped endpoint requires authentication unless listed public, so a new endpoint cannot be accidentally open.

Q54. `@WithMockUser` passes but production returns 403. Why?

`@WithMockUser` bypasses token conversion, so a bad `Converter` or claim shape is never tested. Authorities may be `ROLE_admin` in the test but `SCOPE_x` in production. Use `jwt().jwt(j -> j.claim("realm_access", ...))` with the real converter, or a Testcontainers Keycloak integration test.

Q55. Design authentication and authorization for a new multi-tenant SaaS.

OIDC provider, authorization code + PKCE for the UI, stateless resource servers validating RS256 tokens with JWKS. Tenant in URL path; central interceptor for membership; role and permission table per tenant; `org_id` on every table and in every query; API keys hashed, scoped and excluded from admin paths; services authenticated on internal routes; webhook HMAC; gateway rate limits; outbox-backed audit trail. This is RefinePoint's shape, and each choice has a trade-off you can explain.

Q56. A JWT was leaked in logs. What do you do?

Treat it as compromised until expiry. If short-lived, wait or add its `jti` to a deny-list; if a refresh token leaked, revoke its session in Keycloak. Fix log masking, purge logs, check access by that token through the audit trail, and consider rotating signing keys only if a key itself leaked. Rotating the key logs everyone out.